Metric Markov cotype two of ℓ_1 with $C^2 = 3024$: a machine-checked proof

Abstract

For every stationary reversible Markov chain on n states and every $x_1, \dots, x_n \in \ell_1$, there are $y_1, \dots, y_n \in \ell_1$ satisfying the metric Markov cotype two inequality with constant $C^2 = 3024$. Equivalently, $N_2(\ell_1) \le 12\sqrt{21}$. The argument has three ingredients: the cut representation of finitely many points of ℓ_1, a cubic "flattening" map with a quartic potential, and a comparison between the geometric (resolvent) average and the Cesàro average of the transition matrices. The formal proof follows this outline in Lean 4 with Mathlib. It replaces the stopped martingale of the informal proof by deterministic resolvent identities and a telescoping summation. The final theorem keeps the reversibility hypothesis, but uses it only to derive stationarity.

Source	openai/math@fd4aeeb, family 332, OpenAIMath_332_MetricMarkovCotypeL1
Lean	leanprover/lean4:v4.23.0, Mathlib 37df177aaa
Machine checks	compiles; no `sorry`; statement matches target; module build
Axioms	`propext`, `Classical.choice`, `Quot.sound`
Independent review	FAITHFUL (claude-opus-5-5)
Agents	captain claude-opus-5-5; solvers aiprover; report claude-opus-5-5
Compute	314 AIProver sessions in 119 jobs, 379 session-hours; 95 GPU-hours on 2 x NVIDIA GH200 (H100, 96
Hosted models	28 calls to claude-opus-5-5, claude-sonnet-5-5, $13.38 up to the final review
Run	`srv_20261009_010221_OpenAIMath_332_MetricMarkovCotypeL1`

Contents

1 Statement **1**

2 Formalization **2**

3 Proof **4**
3.1 Overview 4
3.2 Lemmas 5
3.3 Proof of the theorem 65

4 Independent review **66**

A Complete Lean file **68**

1 Statement

Let ℓ_1 be the real Banach space of real sequences $x = (x(k))_{k \ge 1}$ with $\|x\|_1 = \sum_{k \ge 1} |x(k)| < \infty$. Write $\mathbb{N} = \{1, 2, \dots\}$ and $[n] = \{1, \dots, n\}$.

- A matrix $A = (a_{ij})_{i,j\in[n]}$ is *stochastic* if its entries are nonnegative and every row sums to one. - A *probability vector* π on $[n]$ has nonnegative entries summing to one; zero entries are allowed. - A is *reversible* with respect to π if $\pi_i a_{ij} = \pi_j a_{ji}$ for all i, j. - A^s denotes the s-th matrix power.

Theorem 1.1 (Metric Markov cotype two of ℓ_1; $C^2 = 3024$, i.e. $N_2(\ell_1) \le 12\sqrt{21}$)**.** *Let $n, t \in \mathbb{N}$, let π be a probability vector on $[n]$, and let A be a stochastic matrix on $[n]$ that is reversible with respect to π. Then for all $x_1, \dots, x_n \in \ell_1$ there exist $y_1, \dots, y_n \in \ell_1$ such that*

$$\sum_i \pi_i \|x_i - y_i\|_1^2 + t \sum_{i,j} \pi_i a_{ij} \|y_i - y_j\|_1^2 \le 3024 \sum_{i,j} \pi_i \Big(\frac{1}{t}\sum_{s=1}^{t} A^s\Big)_{ij} \|x_i - x_j\|_1^2.$$

The informal proof uses reversibility only through stationarity, $\pi A = \pi$.

2 Formalization

The formalization is written in Lean 4 on top of Mathlib. The relevant Mathlib objects are: - the space ℓ^p of p-summable sequences (`lp`); - real matrices indexed by $\operatorname{Fin} n = \{0, 1, \dots, n-1\}$ and their powers; - unconditional sums of series (`tsum`).

Notation and conventions. - We write $\operatorname{Fin} n = \{0, \dots, n-1\}$ with its natural order. It replaces the source's $[n] = \{1, \dots, n\}$; the relabelling is immaterial. - $\mathcal{P}(\operatorname{Fin} n)$ is the (finite) set of all subsets of $\operatorname{Fin} n$, including $\emptyset$ and $\operatorname{Fin} n$. - For a real sequence $(c_s)_{s\ge0}$ we write $\sum_{s\ge0} c_s$ for Lean's `tsum`. It is the sum of the series when the series is summable (for real series: absolutely convergent), and 0 otherwise. In every statement below in which such a series occurs, it is summable under the stated hypotheses; where summability is part of a lemma's conclusion, this is said explicitly. - For $t \in \mathbb{N}_0 = \{0, 1, 2, \dots\}$ we put

$$p_t = \frac{1}{t+1}, \qquad q_t = \frac{t}{t+1},$$

so that $p_t + q_t = 1$ and $0 \le q_t < 1$. - δ_{ij} is the Kronecker delta, and $\mathbf{1}[P]$ is 1 if P holds and 0 otherwise. - For a finite set ι, weights $w : \iota \to \mathbb{R}$ and $u, v \in \mathbb{R}^\iota$ we use the abbreviations

$$\|u\|_{1,w} = \sum_b w_b |u_b|, \qquad \|u\|_{2,w}^2 = \sum_b w_b u_b^2, \qquad \langle u, v\rangle_w = \sum_b w_b u_b v_b.$$

These are pure notation; the weights are only assumed nonnegative. - For a function $\varphi : \mathbb{R} \to \mathbb{R}$ and $u \in \mathbb{R}^\iota$, $\Phi(u) \in \mathbb{R}^\iota$ is defined by $\Phi(u)_b = \varphi(u_b)$.

The space ℓ_1. The space ℓ_1 is Mathlib's ℓ^1 space of real sequences indexed by $\mathbb{N}_0 = \{0, 1, 2, \dots\}$. Its norm is $\|f\| = \sum_{k\ge0} |f(k)|$ (Lemma 3.21). Shifting indices by one is a linear isometry onto the source's ℓ_1 indexed by $\{1, 2, \dots\}$, so nothing changes.

Lean: `L1`

```
/-- The real Banach space ℓ₁ of absolutely summable real sequences
(indexed by ℕ; the shift from {1,2,...} to {0,1,...} is an isometry). -/
abbrev L1 : Type := lp (fun _ : ℕ => ℝ) 1
```

Probability vectors. A vector $\pi : \operatorname{Fin} n \to \mathbb{R}$ is a probability vector if $\pi_i \ge 0$ for all i and $\sum_i \pi_i = 1$.

Lean: IsProbVec

```
/-- A probability vector on [n]: nonnegative entries summing to one (zeros allowed). -/
def IsProbVec {n : ℕ} (π : Fin n → ℝ) : Prop :=
  (∀ i, 0 ≤ π i) ∧ ∑ i, π i = 1
```

Stochastic matrices. A real matrix A indexed by $\mathrm{Fin}\, n \times \mathrm{Fin}\, n$ is stochastic if $A_{ij} \geq 0$ for all i, j and $\sum_j A_{ij} = 1$ for every i.

Lean: IsStochastic

```
/-- A stochastic matrix: nonnegative entries and every row sum equal to one. -/
def IsStochastic {n : ℕ} (A : Matrix (Fin n) (Fin n) ℝ) : Prop :=
  (∀ i j, 0 ≤ A i j) ∧ ∀ i, ∑ j, A i j = 1
```

Reversibility. A is reversible with respect to π if $\pi_i A_{ij} = \pi_j A_{ji}$ for all i, j. As a definition, this does not require π to be a probability vector or A to be stochastic.

Lean: IsReversible

```
/-- Reversibility (detailed balance) of `A` with respect to `π`. -/
def IsReversible {n : ℕ} (π : Fin n → ℝ) (A : Matrix (Fin n) (Fin n) ℝ) : Prop :=
  ∀ i j, π i * A i j = π j * A j i
```

The target statement. Lean: target statement openaimath_332_metricmarkovcotypel1, fixed before the proof; solution is checked to have exactly this type

```
theorem openaimath_332_metricmarkovcotypel1
    (n t : ℕ) (ht : 1 ≤ t)
    (π : Fin n → ℝ) (hπ : IsProbVec π)
    (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochastic A)
    (hrev : IsReversible π A)
    (x : Fin n → L1) :
    ∃ y : Fin n → L1,
      ∑ i, π i * ‖x i - y i‖ ^ 2
        + (t : ℝ) * ∑ i, ∑ j, π i * A i j * ‖y i - y j‖ ^ 2
      ≤ 3024 * ∑ i, ∑ j,
          π i * ((1 / (t : ℝ)) * ∑ s ∈ Finset.Icc 1 t, (A ^ s) i j)
            * ‖x i - x j‖ ^ 2
```

It is proved by the theorem `solution` (Section 3.3). The statement reads as follows. Let $n \in \mathbb{N}_0$ and $t \in \mathbb{N}_0$ with $t \geq 1$. Let $\pi : \mathrm{Fin}\, n \to \mathbb{R}$ be a probability vector, A a stochastic matrix reversible with respect to π, and $x : \mathrm{Fin}\, n \to \ell_1$. Then there is $y : \mathrm{Fin}\, n \to \ell_1$ with

$$\sum_i \pi_i \|x_i - y_i\|^2 + t \sum_i \sum_j \pi_i A_{ij} \|y_i - y_j\|^2 \leq 3024 \sum_i \sum_j \pi_i \Big(\frac{1}{t} \sum_{s=1}^{t} A_{ij}^s\Big) \|x_i - x_j\|^2.$$

Here A_{ij}^s denotes the (i, j) entry of the matrix power A^s, and t appears as a real number.

This is exactly the source statement, with the following remarks. - The source's $t \in \mathbb{N} = \{1, 2, \dots\}$ becomes $t \in \mathbb{N}_0$ with the hypothesis $t \geq 1$. - The case $n = 0$ is allowed, but it is vacuous: there is no probability vector on the empty set. - The source's closing remark (stationarity suffices) is not a separate formal theorem. Formally, reversibility enters only through Lemma 3.1. Every subsequent lemma assumes only the stationarity relation $\sum_i \pi_i A_{ij} = \pi_j$.

3 Proof

3.1 Overview

The formal proof builds the same points as the source: $y_i = T(\Phi(h_i))$, where $h_i = \sum_j G_{ij} z_j$ and

$$G = \sum_{s \geq 0} p_t q_t^s A^s$$

is the resolvent. Indeed $h_i = p_t \sum_s q_t^s (A^s z)_i$ is the source's (eq. resolvent). The steps correspond to the source as follows.

1. *Preliminaries on A.* Lemma 3.1 derives stationarity from reversibility. Lemmas 3.2 and 3.3 transfer stochasticity and stationarity to the powers A^s. Lemmas 3.4 to 3.7 show that G is a stationary stochastic matrix satisfying the first-step identities

   $$G = p_t I + q_t AG = p_t I + q_t GA.$$

   Lemma 3.8 is the source's (eq. harmonic).

2. *Finite cuts* (source Lemma **??**). The following lemmas build the cut representation. Lemma 3.29 transfers a cube inequality back to ℓ_1. - Lemmas 3.18 to 3.20 decompose one real coordinate into cuts. - Lemmas 3.21 to 3.26 collect facts about ℓ_1. - Lemmas 3.27 and 3.28 assemble the cut representation.

3. *Flat map* (source Lemma **??**). Lemmas 3.9 and 3.10 give the increment bound (eq. increment) in the squared form $72\|a\|^2\|\delta\|^2 + 18\|\delta\|^4$. Lemma 3.11 is (eq. flat). Lemma 3.14 says that Φ fixes binary vectors.

4. *Martingale and stopped walk* (source Corollary **??** and Section **??**). This part is done without probability. - Lemma 3.12 is the conditional-expectation step: Lemma 3.11 averaged over one transition of the killed chain. The linear term cancels because of the mean-value relation $u = pv + q\sum_k a_k h_k$. - Lemma 3.15 applies it with starting index i_0 (the source's $e = z_{X_0}$) and current index j. The killed transition goes to z_j with weight p_t and to h_k with weight $q_t A_{jk}$. - Lemma 3.13 replaces summation along the martingale. It multiplies these inequalities by $\pi_{i_0} G_{i_0 j}$, which is p_t times the expected occupation time of the alive state j for a chain started at i_0. It then sums over i_0, j and telescopes, using $\pi G = \pi$ and $G = p_t I + q_t GA$. The discarded nonnegative term $p_t \pi_i \|h_i - z_i\|_{2,w}^4$ is the source's initial potential $\mathbb{E}\|M_0 - e\|_H^4$. - Lemmas 3.16 and 3.17 conclude the cube inequality

   $$\sum_i \pi_i \|z_i - \Phi(h_i)\|_{1,w}^2 + t \sum_{i,j} \pi_i A_{ij} \|\Phi(h_i) - \Phi(h_j)\|_{1,w}^2 \leq 108 \sum_{i,j} \pi_i G_{ij} \|z_i - z_j\|_{2,w}^4.$$

   The passage to the source's bound $B + tE_A \leq 108 \sum \pi_i G_{ij} \|x_i - x_j\|_1^2$, in the source's notation B, E_A of Section **??**, is made only in Lemma 3.29. It uses the cut distance identity $\|x_i - x_j\| = \|z_i - z_j\|_{1,w}$ of Lemma 3.28 together with the bit identity $|\alpha - \beta| = (\alpha - \beta)^2$. The exact identity (eq. jump-identity) of the source has no counterpart; only these inequalities are proved.

Geometric versus Cesàro (source Lemma **??**). Put

$$D(s) = \sum_{i,j} \pi_i A_{ij}^s \|x_i - x_j\|^2.$$

The geometric endpoint cost is the series $\sum_s p_t q_t^s D(s)$ (Lemma 3.49). - Subadditivity of $\sqrt{D}$ is Lemma 3.35. It uses Lemmas 3.31 to 3.34. - The bound $D(t) \le 4W$ is Lemma 3.36. - Instead of the random variables $J = \lfloor S/t \rfloor$ and $R = S - tJ$, the series is split termwise by Euclidean division $s = \lfloor s/t \rfloor t + (s \bmod t)$ (Lemmas 3.45 and 3.46). - The bound $\mathbb{E}J^2 \le 3$ becomes $\sum_s p_t q_t^s \lfloor s/t \rfloor^2 \le 3$. It is computed from the closed form of $\sum s^2 q^s$ (Lemmas 3.38 to 3.40). - The bound $\mathbb{P}(R = r) \le 2/t$ becomes the bound $2/t$ on the mass of each residue class, $\sum_{s \equiv r} p_t q_t^s$ (Lemmas 3.41 to 3.43). - The pieces are combined in Lemmas 3.48 and 3.57.

Further deviations from the source. - *Cuts.* The formal cut representation is indexed by all subsets $B \subseteq \operatorname{Fin} n$, with weights $w_B \ge 0$ that may vanish. It is produced by sorting each coordinate (Lemma 3.18), not by the explicit formula $(\min_B - \max_{B^c})_+$. There is no separate case "all points coincide". - *Integral bound.* The integral estimate in the proof of Lemma **??** is replaced by an explicit factorization of $\varphi(r') - \varphi(r)$.

Unused lemmas. The following are proved but not used by the final proof: Lemmas 3.8, 3.9, 3.30, 3.37, 3.44 and 3.53.

3.2 Lemmas

Lemma 3.1 (mc_stationary). *Let $\pi : \operatorname{Fin} n \to \mathbb{R}$ and let A be a stochastic matrix that is reversible with respect to π. Then $\sum_i \pi_i A_{ij} = \pi_j$ for every $j \in \operatorname{Fin} n$.*

Lean: mc_stationary — proved by AIProver, job 1, session 0

```
theorem mc_stationary {n : ℕ} (π : Fin n → ℝ) (A : Matrix (Fin n) (Fin n) ℝ)
    (hA : IsStochastic A) (hrev : IsReversible π A) :
    ∀ j, ∑ i, π i * A i j = π j := by
  intro j
  have hrowsum : ∀ i, ∑ j', A i j' = 1 := hA.2
  calc
    ∑ i, π i * A i j = ∑ i, π j * A j i := by
      refine Finset.sum_congr rfl fun i _ => ?_
      rw [hrev i j]
    _ = π j * ∑ i, A j i := by
      simp [Finset.mul_sum]
    _ = π j * 1 := by rw [hrowsum j]
    _ = π j := by ring
```

Proof. By reversibility, $\sum_i \pi_i A_{ij} = \sum_i \pi_j A_{ji} = \pi_j \sum_i A_{ji}$. The last sum is 1 because A is stochastic. □

Lemma 3.2 (mc_pow_stochastic). *If A is stochastic, then A^s is stochastic for every $s \in \mathbb{N}_0$.*

Lean: mc_pow_stochastic — proved by AIProver, job 3, session 0

```
theorem mc_pow_stochastic {n : ℕ} (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochastic A)
    (s : ℕ) : IsStochastic (A ^ s) := by
  rcases hA with ⟨hA_nonneg, hA_sum⟩
  induction' s with s ih
  ·
    refine ⟨?_, ?_⟩
```

```
    · intro i j
      by_cases h : i = j
      · simp [h]
      · simp [h]
    · intro i
      simp [Matrix.one_apply]
  ·
    rcases ih with ⟨h_nonneg, h_sum⟩
    have h_mul_nonneg : ∀ i j, 0 ≤ (A ^ s * A) i j := by
      intro i j
      rw [Matrix.mul_apply]
      refine Finset.sum_nonneg fun k _ => ?_
      exact mul_nonneg (h_nonneg i k) (hA_nonneg k j)
    have h_mul_sum : ∀ i, ∑ j : Fin n, (A ^ s * A) i j = 1 := by
      intro i
      simp [Matrix.mul_apply]
      calc
        ∑ x : Fin n, ∑ j : Fin n, (A ^ s) i j * A j x = ∑ j : Fin n, ∑ x : Fin n, (A ^ s) i j * A j x :=
          Finset.sum_comm
        _ = ∑ j : Fin n, ((A ^ s) i j * ∑ x : Fin n, A j x) := by
          simp [Finset.mul_sum]
        _ = ∑ j : Fin n, ((A ^ s) i j * 1) := by simp [hA_sum]
        _ = ∑ j : Fin n, (A ^ s) i j := by simp
        _ = 1 := h_sum i
    have : (A : Matrix (Fin n) (Fin n) ℝ) ^ (s + 1) = (A ^ s) * A := pow_succ A s
    rw [this]
    exact ⟨h_mul_nonneg, h_mul_sum⟩
```

Proof. We use induction on s. The identity matrix is stochastic. Write $A^{s+1} = A^s A$. Its entries $\sum_k A^s_{ik} A_{kj}$ are sums of products of nonnegative numbers. Its row sums are

$$\sum_j \sum_k A^s_{ik} A_{kj} = \sum_k A^s_{ik} \sum_j A_{kj} = \sum_k A^s_{ik} = 1. \qquad \square$$

Lemma 3.3 (mc_pow_stationary). *Let $\pi : \mathrm{Fin}\, n \to \mathbb{R}$ and let A be a real matrix with $\sum_i \pi_i A_{ij} = \pi_j$ for all j. Then $\sum_i \pi_i A^s_{ij} = \pi_j$ for all $s \in \mathbb{N}_0$ and all j.*

Lean: mc_pow_stationary — proved by AIProver, job 1, session 0

```
theorem mc_pow_stationary {n : ℕ} (π : Fin n → ℝ) (A : Matrix (Fin n) (Fin n) ℝ)
    (hstat : ∀ j, ∑ i, π i * A i j = π j) (s : ℕ) :
    ∀ j, ∑ i, π i * (A ^ s) i j = π j := by
  induction' s with s ih
  ·
    intro j
    simp [Matrix.one_apply]
  ·
    intro j
    have h_pow_succ : (A ^ (s + 1)) = (A ^ s) * A := by
      simp [pow_succ]
    rw [h_pow_succ]


    simp [Matrix.mul_apply]

    calc
```

```
      ∑ i, π i * (∑ k, (A ^ s) i k * A k j)
          = ∑ i, ∑ k, π i * (A ^ s) i k * A k j := by
        refine Finset.sum_congr rfl fun i _ => ?_
        simp_rw [Finset.mul_sum]
        refine Finset.sum_congr rfl fun k _ => ?_
        ring
      _ = ∑ k, ∑ i, π i * (A ^ s) i k * A k j := Finset.sum_comm
      _ = ∑ k, (∑ i, π i * (A ^ s) i k) * A k j := by
        refine Finset.sum_congr rfl fun k _ => ?_
        simp_rw [Finset.sum_mul]
      _ = ∑ k, π k * A k j := by
        refine Finset.sum_congr rfl fun k _ => ?_
        rw [ih k]
      _ = π j := hstat j
```

Proof. We use induction on s; the case $s = 0$ is trivial. For the step, write $A^{s+1} = A^s A$ and interchange the finite sums:

$$\sum_i \pi_i \sum_k A^s_{ik} A_{kj} = \sum_k \Big(\sum_i \pi_i A^s_{ik} \Big) A_{kj} = \sum_k \pi_k A_{kj} = \pi_j. \qquad \square$$

Lemma 3.4 (mc_resolvent_summable)**.** *Let $t \in \mathbb{N}_0$, let A be stochastic, and let $i, j \in \operatorname{Fin} n$. Then the series $\sum_{s \ge 0} p_t q_t^s A^s_{ij}$ is summable.*

Lean: mc_resolvent_summable — proved by AIProver, job 2, session 0

```
theorem mc_resolvent_summable {n : ℕ} (t : ℕ) (A : Matrix (Fin n) (Fin n) ℝ)
    (hA : IsStochastic A) (i j : Fin n) :
    Summable (fun s : ℕ => (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * (A ^ s) i j) := by
  set r := (t : ℝ) / ((t : ℝ) + 1) with hr
  have hr_nonneg : 0 ≤ r :=
    div_nonneg (Nat.cast_nonneg _) (by positivity)
  have hr_lt_one : r < 1 := by
    refine (div_lt_one ?_).mpr ?_
    · positivity
    · have : (t : ℝ) < (t : ℝ) + 1 := by linarith
      exact_mod_cast this
  have h_geom_summable : Summable fun s : ℕ => r ^ s :=
    summable_geometric_of_lt_one hr_nonneg hr_lt_one
  have h_entry_nonneg : ∀ s : ℕ, ∀ i' j', 0 ≤ (A ^ s) i' j' := by
    intro s
    induction' s with s ih
    · intro i' j'
      by_cases h : i' = j'
      · subst h; simpa [Matrix.one_apply] using (le_refl (1 : ℝ) : (0 : ℝ) ≤ (1 : ℝ))
      · simpa [h, Matrix.one_apply] using (zero_le (0 : ℝ) : (0 : ℝ) ≤ (0 : ℝ))
    · intro i' j'
      rw [pow_succ', Matrix.mul_apply]
      refine Finset.sum_nonneg fun k _ => ?_
      refine mul_nonneg (hA.1 i' k) (ih k j')
  have h_entry_le_one : ∀ s : ℕ, ∀ i' j', (A ^ s) i' j' ≤ 1 := by
    intro s
    induction' s with s ih
    · intro i' j'
      by_cases h : i' = j'
      · subst h; simpa [Matrix.one_apply] using (le_refl (1 : ℝ) : (1 : ℝ) ≤ (1 : ℝ))
```

```
      · simpa [h, Matrix.one_apply] using (zero_le (0 : ℝ) : (0 : ℝ) ≤ (1 : ℝ))
    · intro i' j'
      rw [pow_succ', Matrix.mul_apply]
      calc
        ∑ k, A i' k * (A ^ s) k j' ≤ ∑ k, A i' k * 1 :=
          Finset.sum_le_sum fun k _ => mul_le_mul_of_nonneg_left (ih k j') (hA.1 i' k)
        _ = ∑ k, A i' k := by simp
        _ = 1 := hA.2 i'
  have h_coeff_nonneg : ∀ s : ℕ, 0 ≤ (1 / ((t : ℝ) + 1)) * r ^ s * (A ^ s) i j := by
    intro s
    refine mul_nonneg (mul_nonneg ?_ (pow_nonneg hr_nonneg s)) (h_entry_nonneg s i j)
    refine div_nonneg (by norm_num) (by positivity)
  have h_bound : ∀ s : ℕ, (1 / ((t : ℝ) + 1)) * r ^ s * (A ^ s) i j ≤
      (1 / ((t : ℝ) + 1)) * r ^ s := by
    intro s
    have htemp := h_entry_le_one s i j
    calc
      (1 / ((t : ℝ) + 1)) * r ^ s * (A ^ s) i j ≤
          (1 / ((t : ℝ) + 1)) * r ^ s * 1 :=
        mul_le_mul_of_nonneg_left htemp (by
          refine mul_nonneg (div_nonneg (by norm_num) (by positivity)) (pow_nonneg hr_nonneg s))
      _ = (1 / ((t : ℝ) + 1)) * r ^ s := by simp
  have h_summable_term : Summable fun s : ℕ => (1 / ((t : ℝ) + 1)) * r ^ s :=
    Summable.mul_left (1 / ((t : ℝ) + 1)) h_geom_summable
  exact Summable.of_nonneg_of_le h_coeff_nonneg h_bound h_summable_term
```

Proof. By induction, using $A^{s+1} = AA^s$ together with nonnegativity and the row sums of A, one gets $0 \le A^s_{ij} \le 1$. The terms are therefore nonnegative and bounded by $p_t q_t^s$. The geometric series $\sum q_t^s$ converges because $0 \le q_t < 1$. □

Lemma 3.5 (mc_resolvent_basic). *Let $t \in \mathbb{N}_0$ and $\pi : \mathrm{Fin}\, n \to \mathbb{R}$. Let A be stochastic with $\sum_i \pi_i A_{ij} = \pi_j$ for all j. Let G be the matrix with $G_{ij} = \sum_{s \ge 0} p_t q_t^s A^s_{ij}$. Then: - $G_{ij} \ge 0$ for all i, j; - $\sum_j G_{ij} = 1$ for all i; - $\sum_i \pi_i G_{ij} = \pi_j$ for all j.*

Lean: mc_resolvent_basic — proved by the captain after a hand-back

```
theorem mc_resolvent_basic {n : ℕ} (t : ℕ) (π : Fin n → ℝ)
    (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochastic A)
    (hstat : ∀ j, ∑ i, π i * A i j = π j)
    (G : Matrix (Fin n) (Fin n) ℝ)
    (hG : ∀ i j, G i j = ∑' s : ℕ, (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * (A ^ s) i j) :
    (∀ i j, 0 ≤ G i j) ∧ (∀ i, ∑ j, G i j = 1) ∧ (∀ j, ∑ i, π i * G i j = π j) := by
  have ht1 : (0:ℝ) < (t:ℝ) + 1 := by positivity
  have hr0 : (0:ℝ) ≤ (t:ℝ) / ((t:ℝ) + 1) := div_nonneg (Nat.cast_nonneg t) ht1.le
  have hr1 : (t:ℝ) / ((t:ℝ) + 1) < 1 := by
    rw [div_lt_one ht1]; linarith
  have hsub : (1:ℝ) - (t:ℝ) / ((t:ℝ) + 1) = 1 / ((t:ℝ) + 1) := by
    field_simp <;> ring
  have hc : HasSum (fun s : ℕ => (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s) 1 := by
    have h := (hasSum_geometric_of_lt_one hr0 hr1).mul_left (1 / ((t : ℝ) + 1))
    rw [hsub] at h
    have h1 : (1 / ((t : ℝ) + 1)) * (1 / ((t : ℝ) + 1))⁻¹ = 1 := mul_inv_cancel₀ (by positivity)
    rwa [h1] at h
  have hGs : ∀ i j, HasSum (fun s : ℕ => (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * (A ^ s) i j)
↪   (G i j) := by
    intro i j
```

```
    rw [hG i j]
    exact (mc_resolvent_summable t A hA i j).hasSum
  have hnn : ∀ i j, 0 ≤ G i j := by
    intro i j
    rw [hG i j]
    exact tsum_nonneg (fun s => mul_nonneg (mul_nonneg (by positivity) (pow_nonneg hr0 s))
      ((mc_pow_stochastic A hA s).1 i j))
  have hrow : ∀ i, ∑ j, G i j = 1 := by
    intro i
    have h : HasSum (fun s : ℕ => ∑ j, (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * (A ^ s) i j)
↪  (∑ j, G i j) :=
      hasSum_sum (fun j _ => hGs i j)
    have h2 : HasSum (fun s : ℕ => (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s) (∑ j, G i j) := by
      convert h using 1
      funext s
      rw [← Finset.mul_sum, (mc_pow_stochastic A hA s).2 i, mul_one]
    exact h2.unique hc
  have hstatG : ∀ j, ∑ i, π i * G i j = π j := by
    intro j
    have h : HasSum (fun s : ℕ => ∑ i, π i * ((1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * (A ^
↪  s) i j)) (∑ i, π i * G i j) :=
      hasSum_sum (fun i _ => (hGs i j).mul_left (π i))
    have h2 : HasSum (fun s : ℕ => π j * ((1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s)) (∑ i, π i
↪  * G i j) := by
      convert h using 1
      funext s
      rw [← mc_pow_stationary π A hstat s j, Finset.sum_mul]
      refine Finset.sum_congr rfl (fun i _ => ?_)
      ring
    have h3 := hc.mul_left (π j)
    rw [mul_one] at h3
    exact h2.unique h3
  exact ⟨hnn, hrow, hstatG⟩
```

Proof. Each series converges (Lemma 3.4). Since A^s is stochastic (Lemma 3.2), each term is nonnegative, so $G_{ij} \geq 0$.

The geometric series gives $\sum_s p_t q_t^s = p_t/(1-q_t) = 1$. A finite sum of convergent series converges to the sum of the limits. Hence

$$\sum_j G_{ij} = \sum_s p_t q_t^s \sum_j A_{ij}^s = \sum_s p_t q_t^s = 1.$$

Likewise, by Lemma 3.3,

$$\sum_i \pi_i G_{ij} = \sum_s p_t q_t^s \sum_i \pi_i A_{ij}^s = \pi_j \sum_s p_t q_t^s = \pi_j. \qquad \square$$

Lemma 3.6 (`mc_resolvent_first_step_aux`)**.** *Let $p, q \in \mathbb{R}$ and let A, G be real $\mathrm{Fin}\, n \times \mathrm{Fin}\, n$ matrices. Suppose that for all i, j the series $\sum_s p\, q^s A_{ij}^s$ is summable and $G_{ij} = \sum_{s \geq 0} p\, q^s A_{ij}^s$. Then for all i, j:*

$$G_{ij} = p\, \delta_{ij} + q \sum_k A_{ik} G_{kj} \qquad \text{and} \qquad G_{ij} = p\, \delta_{ij} + q \sum_k G_{ik} A_{kj}.$$

Lean: `mc_resolvent_first_step_aux` — helper of mc_resolvent_first_step, proved by the captain after a hand-back

```
theorem mc_resolvent_first_step_aux {n : ℕ} (p q : ℝ) (A G : Matrix (Fin n) (Fin n) ℝ)
    (hs : ∀ i j, Summable (fun s : ℕ => p * q ^ s * (A ^ s) i j))
    (hG : ∀ i j, G i j = ∑' s : ℕ, p * q ^ s * (A ^ s) i j) :
    (∀ i j, G i j = p * (if i = j then (1:ℝ) else 0) + q * ∑ k, A i k * G k j) ∧
    (∀ i j, G i j = p * (if i = j then (1:ℝ) else 0) + q * ∑ k, G i k * A k j) := by
  constructor
  · intro i j
    have h1 : ∑ k, A i k * G k j = ∑' s : ℕ, ∑ k, A i k * (p * q ^ s * (A ^ s) k j) := by
      rw [Summable.tsum_finsetSum]
      · refine Finset.sum_congr rfl (fun k _ => ?_)
        rw [hG k j]
        exact tsum_mul_left.symm
      · intro k _
        exact (hs k j).mul_left (A i k)
    rw [h1, ← tsum_mul_left, hG i j, (hs i j).tsum_eq_zero_add]
    congr 1
    · first
        | simp only [pow_zero, mul_one, Matrix.one_apply]
        | simp [Matrix.one_apply]
    · refine tsum_congr (fun s => ?_)
      rw [pow_succ' A s, Matrix.mul_apply]
      simp only [Finset.mul_sum]
      refine Finset.sum_congr rfl (fun k _ => ?_)
      ring
  · intro i j
    have h2 : ∑ k, G i k * A k j = ∑' s : ℕ, ∑ k, (p * q ^ s * (A ^ s) i k) * A k j := by
      rw [Summable.tsum_finsetSum]
      · refine Finset.sum_congr rfl (fun k _ => ?_)
        rw [hG i k]
        exact tsum_mul_right.symm
      · intro k _
        exact (hs i k).mul_right (A k j)
    rw [h2, ← tsum_mul_left, hG i j, (hs i j).tsum_eq_zero_add]
    congr 1
    · first
        | simp only [pow_zero, mul_one, Matrix.one_apply]
        | simp [Matrix.one_apply]
    · refine tsum_congr (fun s => ?_)
      rw [pow_succ A s, Matrix.mul_apply]
      simp only [Finset.mul_sum]
      refine Finset.sum_congr rfl (fun k _ => ?_)
      ring
```

Proof. Exchanging the finite sum over k with the convergent series and using $A^{s+1} = AA^s$ gives

$$\sum_k A_{ik} G_{kj} = \sum_s \sum_k A_{ik}\, p\, q^s A^s_{kj} = \sum_s p\, q^s A^{s+1}_{ij}.$$

Splitting off the term $s = 0$ (where $A^0_{ij} = \delta_{ij}$) gives

$$G_{ij} = p\,\delta_{ij} + \sum_s p\, q^{s+1} A^{s+1}_{ij} = p\,\delta_{ij} + q \sum_k A_{ik} G_{kj}.$$

The second identity is proved the same way with $A^{s+1} = A^s A$. □

Lemma 3.7 (mc_resolvent_first_step). *Let $t \in \mathbb{N}_0$, let A be stochastic, and let $G_{ij} = \sum_{s\ge 0} p_t q_t^s A_{ij}^s$. Then for all i, j:*

$$G_{ij} = p_t\delta_{ij} + q_t \sum_k A_{ik} G_{kj} \qquad \text{and} \qquad G_{ij} = p_t\delta_{ij} + q_t \sum_k G_{ik} A_{kj}.$$

Lean: mc_resolvent_first_step — proved by the captain after a hand-back

```
theorem mc_resolvent_first_step {n : ℕ} (t : ℕ) (A : Matrix (Fin n) (Fin n) ℝ)
    (hA : IsStochastic A) (G : Matrix (Fin n) (Fin n) ℝ)
    (hG : ∀ i j, G i j = ∑' s : ℕ, (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * (A ^ s) i j) :
    (∀ i j, G i j = (1 / ((t : ℝ) + 1)) * (if i = j then (1:ℝ) else 0)
        + ((t : ℝ) / ((t : ℝ) + 1)) * ∑ k, A i k * G k j) ∧
    (∀ i j, G i j = (1 / ((t : ℝ) + 1)) * (if i = j then (1:ℝ) else 0)
        + ((t : ℝ) / ((t : ℝ) + 1)) * ∑ k, G i k * A k j) := by
  have hs : ∀ i j, Summable (fun s : ℕ => (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * (A ^ s) i
↪  j) :=
    fun i j => mc_resolvent_summable t A hA i j
  exact mc_resolvent_first_step_aux _ _ A G hs hG
```

Proof. Combine Lemma 3.6, with $p = p_t$ and $q = q_t$, and Lemma 3.4. □

Lemma 3.8 (mc_harmonic_point). *Let $t \in \mathbb{N}_0$, let ι be any set, and let A, G be real matrices. Assume: - $G_{ij} \ge 0$ and $\sum_j G_{ij} = 1$; - $G_{ij} = p_t\delta_{ij} + q_t \sum_k A_{ik} G_{kj}$ for all i, j; - $z : \mathrm{Fin}\, n \to \mathbb{R}^\iota$ satisfies $z_i(b) \in \{0, 1\}$ for all i, b.*

Put $h_i(b) = \sum_j G_{ij} z_j(b)$. Then for all i, b:

$$0 \le h_i(b) \le 1 \qquad \text{and} \qquad h_i(b) = p_t z_i(b) + q_t \sum_k A_{ik} h_k(b).$$

Lean: mc_harmonic_point — proved by AIProver, job 1, session 0

```
theorem mc_harmonic_point {n : ℕ} {ι : Type*} (t : ℕ) (A G : Matrix (Fin n) (Fin n) ℝ)
    (hG0 : ∀ i j, 0 ≤ G i j) (hGrow : ∀ i, ∑ j, G i j = 1)
    (hAG : ∀ i j, G i j = (1 / ((t : ℝ) + 1)) * (if i = j then (1:ℝ) else 0)
        + ((t : ℝ) / ((t : ℝ) + 1)) * ∑ k, A i k * G k j)
    (z : Fin n → ι → ℝ) (hz : ∀ i b, z i b = 0 ∨ z i b = 1) :
    (∀ i b, 0 ≤ ∑ j, G i j * z j b ∧ ∑ j, G i j * z j b ≤ 1) ∧
    (∀ i b, ∑ j, G i j * z j b = (1 / ((t : ℝ) + 1)) * z i b
        + ((t : ℝ) / ((t : ℝ) + 1)) * ∑ k, A i k * ∑ j, G k j * z j b) := by
  have hz_nonneg : ∀ i b, 0 ≤ z i b := by
    intro i b
    rcases hz i b with (h | h)
    · rw [h]
    · rw [h]; norm_num
  have hz_le_one : ∀ i b, z i b ≤ 1 := by
    intro i b
    rcases hz i b with (h | h)
    · rw [h]; norm_num
    · rw [h]
  have part1 : ∀ i b, 0 ≤ ∑ j : Fin n, G i j * z j b ∧ ∑ j : Fin n, G i j * z j b ≤ 1 := by
    intro i b
    have h_nonneg_sum : 0 ≤ ∑ j : Fin n, G i j * z j b := by
      refine Finset.sum_nonneg fun j _ => ?_
      have hGij : 0 ≤ G i j := hG0 i j
```

```
      have hzjb : 0 ≤ z j b := hz_nonneg j b
      nlinarith
    have h_le_one_sum : ∑ j : Fin n, G i j * z j b ≤ 1 := by
      calc
        ∑ j : Fin n, G i j * z j b ≤ ∑ j : Fin n, G i j * 1 := by
          refine Finset.sum_le_sum fun j _ => ?_
          have hzjb : z j b ≤ 1 := hz_le_one j b
          nlinarith [hG0 i j]
        _ = ∑ j : Fin n, G i j := by simp
        _ = 1 := hGrow i
    exact And.intro h_nonneg_sum h_le_one_sum
  have part2 : ∀ i b, ∑ j : Fin n, G i j * z j b = (1 / ((t : ℝ) + 1)) * z i b
      + ((t : ℝ) / ((t : ℝ) + 1)) * ∑ k : Fin n, A i k * ∑ j : Fin n, G k j * z j b := by
    intro i b
    calc
      ∑ j : Fin n, G i j * z j b
          = ∑ j : Fin n, ((1 / ((t : ℝ) + 1)) * (if i = j then (1:ℝ) else 0)
              + ((t : ℝ) / ((t : ℝ) + 1)) * ∑ k : Fin n, A i k * G k j) * z j b := by
        refine Finset.sum_congr rfl fun j _ => ?_
        rw [hAG i j]
      _ = ∑ j : Fin n, ((1 / ((t : ℝ) + 1)) * (if i = j then (1:ℝ) else 0) * z j b
              + ((t : ℝ) / ((t : ℝ) + 1)) * (∑ k : Fin n, A i k * G k j) * z j b) := by
        refine Finset.sum_congr rfl fun j _ => ?_
        ring
      _ = (∑ j : Fin n, (1 / ((t : ℝ) + 1)) * (if i = j then (1:ℝ) else 0) * z j b)
          + (∑ j : Fin n, ((t : ℝ) / ((t : ℝ) + 1)) * (∑ k : Fin n, A i k * G k j) * z j b) :=
        Finset.sum_add_distrib
      _ = (1 / ((t : ℝ) + 1)) * z i b
          + (∑ j : Fin n, ((t : ℝ) / ((t : ℝ) + 1)) * (∑ k : Fin n, A i k * G k j) * z j b) := by
        simp
      _ = (1 / ((t : ℝ) + 1)) * z i b
          + ((t : ℝ) / ((t : ℝ) + 1)) * (∑ k : Fin n, A i k * (∑ j : Fin n, G k j * z j b)) := by
        have hsum : (∑ j : Fin n, (∑ k : Fin n, A i k * G k j) * z j b)
            = (∑ k : Fin n, A i k * (∑ j : Fin n, G k j * z j b)) := by
          calc
            (∑ j : Fin n, (∑ k : Fin n, A i k * G k j) * z j b)
                = (∑ j : Fin n, ∑ k : Fin n, (A i k * G k j) * z j b) := by
              simp_rw [Finset.sum_mul]
            _ = (∑ k : Fin n, ∑ j : Fin n, (A i k * G k j) * z j b) := by
              rw [Finset.sum_comm]
            _ = (∑ k : Fin n, A i k * (∑ j : Fin n, G k j * z j b)) := by
              simp_rw [Finset.mul_sum, mul_assoc]
        calc
          (1 / ((t : ℝ) + 1)) * z i b
              + (∑ j : Fin n, ((t : ℝ) / ((t : ℝ) + 1)) * (∑ k : Fin n, A i k * G k j) * z j b)
          = (1 / ((t : ℝ) + 1)) * z i b
              + ((t : ℝ) / ((t : ℝ) + 1)) * (∑ j : Fin n, (∑ k : Fin n, A i k * G k j) * z j b) := by
            simp [Finset.mul_sum, mul_assoc, mul_comm, mul_left_comm]
          _ = (1 / ((t : ℝ) + 1)) * z i b
              + ((t : ℝ) / ((t : ℝ) + 1)) * (∑ k : Fin n, A i k * (∑ j : Fin n, G k j * z j b)) := by
            rw [hsum]
      _ = (1 / ((t : ℝ) + 1)) * z i b + ((t : ℝ) / ((t : ℝ) + 1)) * ∑ k : Fin n, A i k * ∑ j : Fin n, G k
↪   j * z j b := by
        simp
  exact And.intro part1 part2
```

Proof. The value $h_i(b)$ is a convex combination of numbers in $\{0, 1\}$, since $G \geq 0$ and the rows of G sum to 1. For the identity, substitute the first-step relation for G_{ij}. The δ-term gives $p_t z_i(b)$. In the

other term, exchange the sums over j and k:

$$\sum_j \Big(\sum_k A_{ik} G_{kj}\Big) z_j(b) = \sum_k A_{ik} h_k(b). \qquad \square$$

Lemma 3.9 (mc_phi_scalar). *Let $\varphi : \mathbb{R} \to \mathbb{R}$ satisfy $\varphi(r) = 3r^2 - 2r^3$ for all r. Let $r, r' \in [0,1]$ and $d \in \{0,1\}$. Then*

$$|\varphi(r') - \varphi(r)| \le 6|r - d|\,|r' - r| + 3(r' - r)^2.$$

Lean: mc_phi_scalar — proved by AIProver, job 3, session 1

```
theorem mc_phi_scalar (φ : ℝ → ℝ) (hφ : ∀ r, φ r = 3 * r ^ 2 - 2 * r ^ 3)
    (r r' d : ℝ) (hr : 0 ≤ r ∧ r ≤ 1) (hr' : 0 ≤ r' ∧ r' ≤ 1) (hd : d = 0 ∨ d = 1) :
    |φ r' - φ r| ≤ 6 * |r - d| * |r' - r| + 3 * (r' - r) ^ 2 := by
  rcases hr with ⟨hr_lo, hr_hi⟩
  rcases hr' with ⟨hr'_lo, hr'_hi⟩
  have hφr : φ r = 3 * r ^ 2 - 2 * r ^ 3 := hφ r
  have hφr' : φ r' = 3 * r' ^ 2 - 2 * r' ^ 3 := hφ r'
  rw [hφr, hφr']
  have h_factor : 3 * r' ^ 2 - 2 * r' ^ 3 - (3 * r ^ 2 - 2 * r ^ 3) = (r' - r) * (3 * (r' + r) - 2 * (r' ^ 2
↪  + r' * r + r ^ 2)) := by
    ring
  rw [h_factor, abs_mul]
  by_cases h_eq : r' = r
  · rw [h_eq, sub_self]
    simp
  · set A := 3 * (r' + r) - 2 * (r' ^ 2 + r' * r + r ^ 2) with hA_def
    have h_absA : |A| ≤ 6 * |r - d| + 3 * |r' - r| := by
      rcases hd with (hd | hd)
      ·
        rw [hd, sub_zero, abs_of_nonneg hr_lo]
        by_cases h_le : r' - r ≥ 0
        · have habs_sub : |r' - r| = r' - r := abs_of_nonneg h_le
          rw [habs_sub]
          have hA_le : A ≤ 3 * (r' + r) := by
            dsimp [A]; nlinarith
          have hA_ge : -(6 * r + 3 * (r' - r)) ≤ A := by
            dsimp [A]; ring_nf; nlinarith
          exact abs_le.mpr ⟨by linarith, by linarith⟩
        · have h_le' : r - r' ≥ 0 := by linarith
          have habs_sub : |r' - r| = r - r' := by
            rw [abs_sub_comm, abs_of_nonneg h_le']
          rw [habs_sub]
          have hA_le' : |A| ≤ 3 * (r' + r) :=
            abs_le.mpr ⟨by dsimp [A]; nlinarith, by dsimp [A]; nlinarith⟩
          have h_suff : 3 * (r' + r) ≤ 6 * r + 3 * (r - r') := by
            ring_nf; nlinarith
          nlinarith
      ·
        have hr_abs : |r - 1| = 1 - r := by
          rw [abs_of_nonpos (by linarith : r - 1 ≤ 0)]
          ring
        rw [hd, hr_abs]
        by_cases h_le : r' - r ≥ 0
        · have habs_sub : |r' - r| = r' - r := abs_of_nonneg h_le
          rw [habs_sub]
          dsimp [A]
```

```
        by_cases hA_nonneg : 0 ≤ 3 * (r' + r) - 2 * (r' ^ 2 + r' * r + r ^ 2)
        · rw [abs_of_nonneg hA_nonneg]; nlinarith
        · have hA_neg : 3 * (r' + r) - 2 * (r' ^ 2 + r' * r + r ^ 2) < 0 := by linarith
          rw [abs_of_neg hA_neg]; ring_nf; nlinarith
      · have h_le' : r - r' ≥ 0 := by linarith
        have habs_sub : |r' - r| = r - r' := by
          rw [abs_sub_comm, abs_of_nonneg h_le']
        rw [habs_sub]
        dsimp [A]
        by_cases hA_nonneg : 0 ≤ 3 * (r' + r) - 2 * (r' ^ 2 + r' * r + r ^ 2)
        · rw [abs_of_nonneg hA_nonneg]; nlinarith
        · have hA_neg : 3 * (r' + r) - 2 * (r' ^ 2 + r' * r + r ^ 2) < 0 := by linarith
          rw [abs_of_neg hA_neg]; ring_nf; nlinarith
    have hpos : 0 ≤ |r' - r| := abs_nonneg _
    calc
      |r' - r| * |A| ≤ |r' - r| * (6 * |r - d| + 3 * |r' - r|) := by
        nlinarith
      _ = 6 * |r - d| * |r' - r| + 3 * (|r' - r| ^ 2) := by ring
      _ = 6 * |r - d| * |r' - r| + 3 * (r' - r) ^ 2 := by rw [sq_abs]
```

Proof. We have $\varphi(r') - \varphi(r) = (r' - r)K$ with

$$K = 3(r' + r) - 2(r'^2 + r'r + r^2).$$

It suffices to show $|K| \le 6|r - d| + 3|r' - r|$. With $\delta = r' - r$ one has

$$K = 6r(1 - r) + \delta(3 - 4r - 2r').$$

Here $|3 - 4r - 2r'| \le 3$ on $[0,1]^2$, and $0 \le 6r(1-r) \le 6|r-d|$ for $d \in \{0,1\}$. The formal proof checks the bound on K by a case analysis on d and on the sign of $r' - r$. □

Lemma 3.10 (`mc_flat_increment`). *Let ι be finite, let $w : \iota \to \mathbb{R}$ with $w \ge 0$, and let $\varphi(r) = 3r^2 - 2r^3$ as in Lemma 3.9. Let $u, u' \in [0,1]^\iota$ and $e \in \{0,1\}^\iota$. Then*

$$\|\Phi(u') - \Phi(u)\|_{1,w}^2 \le 72\,\|u - e\|_{2,w}^2\,\|u' - u\|_{2,w}^2 + 18\,\|u' - u\|_{2,w}^4.$$

Lean: `mc_flat_increment` — proved by AIProver, job 3, session 0

```
theorem mc_flat_increment {ι : Type*} [Fintype ι] (w : ι → ℝ) (hw : ∀ b, 0 ≤ w b)
    (φ : ℝ → ℝ) (hφ : ∀ r, φ r = 3 * r ^ 2 - 2 * r ^ 3)
    (u u' e : ι → ℝ) (hu : ∀ b, 0 ≤ u b ∧ u b ≤ 1) (hu' : ∀ b, 0 ≤ u' b ∧ u' b ≤ 1)
    (he : ∀ b, e b = 0 ∨ e b = 1) :
    (∑ b, w b * |φ (u' b) - φ (u b)|) ^ 2
      ≤ 72 * (∑ b, w b * (u b - e b) ^ 2) * (∑ b, w b * (u' b - u b) ^ 2)
        + 18 * (∑ b, w b * (u' b - u b) ^ 2) ^ 2 := by
  have hφ_eq : ∀ r : ℝ, φ r = 3 * r ^ 2 - 2 * r ^ 3 := hφ
  have h_key_scalar : ∀ (r r' e : ℝ), 0 ≤ r → r ≤ 1 → 0 ≤ r' → r' ≤ 1 → (e = 0 ∨ e = 1) →
      |φ r' - φ r| ≤ 6 * |r - e| * |r' - r| + 3 * (r' - r) ^ 2 := by
    intro r r' e hr0 hr1 hr'0 hr'1 he_cases
    have hfactor : φ r' - φ r = (r' - r) * (6 * r * (1 - r) + (r' - r) * (3 - 6 * r - 2 * (r' - r))) := by
      calc
        φ r' - φ r = (3 * r' ^ 2 - 2 * r' ^ 3) - (3 * r ^ 2 - 2 * r ^ 3) := by
          simp [hφ_eq]
        _ = (r' - r) * (6 * r * (1 - r) + (r' - r) * (3 - 6 * r - 2 * (r' - r))) := by
          ring
```

```
  rw [hfactor]
  set δ := r' - r with hδ_def
  have hsq_nonneg : 0 ≤ δ ^ 2 := pow_two_nonneg _
  have habs_nonneg : 0 ≤ |δ| := abs_nonneg _
  have h_inner_bound : |3 - 6 * r - 2 * δ| ≤ 3 := by
    have : 3 - 6 * r - 2 * δ = 3 - 4 * r - 2 * r' := by dsimp [δ]; ring
    rw [this]
    exact abs_le.mpr ⟨by nlinarith, by nlinarith⟩
  have h_expr_eq : δ * (6 * r * (1 - r) + δ * (3 - 6 * r - 2 * δ)) =
      6 * r * (1 - r) * δ + δ ^ 2 * (3 - 6 * r - 2 * δ) := by ring
  rw [h_expr_eq]
  rcases he_cases with (he0 | he1)
  · subst he0
    have h_mul_factor : (1 - r) * |δ| ≤ 1 * |δ| :=
      mul_le_mul_of_nonneg_right (by nlinarith) habs_nonneg
    have h1 : |6 * r * (1 - r) * δ| ≤ 6 * r * |δ| := by
      have hpos : 0 ≤ 6 * r * (1 - r) := by nlinarith
      calc
        |6 * r * (1 - r) * δ| = |6 * r * (1 - r)| * |δ| := abs_mul _ _
        _ = 6 * r * (1 - r) * |δ| := by rw [abs_of_nonneg hpos]
        _ = (6 * r) * ((1 - r) * |δ|) := by ring
        _ ≤ (6 * r) * (1 * |δ|) :=
          mul_le_mul_of_nonneg_left h_mul_factor (by nlinarith)
        _ = 6 * r * |δ| := by ring
    have h2 : |δ ^ 2 * (3 - 6 * r - 2 * δ)| ≤ 3 * δ ^ 2 := by
      calc
        |δ ^ 2 * (3 - 6 * r - 2 * δ)| = |δ ^ 2| * |3 - 6 * r - 2 * δ| := abs_mul _ _
        _ = δ ^ 2 * |3 - 6 * r - 2 * δ| := by rw [abs_of_nonneg hsq_nonneg]
        _ ≤ δ ^ 2 * 3 := mul_le_mul_of_nonneg_left h_inner_bound hsq_nonneg
        _ = 3 * δ ^ 2 := by ring
    calc
      |6 * r * (1 - r) * δ + δ ^ 2 * (3 - 6 * r - 2 * δ)| ≤
          |6 * r * (1 - r) * δ| + |δ ^ 2 * (3 - 6 * r - 2 * δ)| := abs_add _ _
      _ ≤ (6 * r * |δ|) + (3 * δ ^ 2) := by nlinarith
      _ = 6 * |r - 0| * |δ| + 3 * δ ^ 2 := by
        have h_abs : |r - 0| = r := by simpa using abs_of_nonneg hr0
        rw [h_abs]
  · subst he1
    have h_mul_factor : r * |δ| ≤ 1 * |δ| :=
      mul_le_mul_of_nonneg_right (by nlinarith) habs_nonneg
    have h1 : |6 * r * (1 - r) * δ| ≤ 6 * (1 - r) * |δ| := by
      have hpos : 0 ≤ 6 * r * (1 - r) := by nlinarith
      calc
        |6 * r * (1 - r) * δ| = |6 * r * (1 - r)| * |δ| := abs_mul _ _
        _ = 6 * r * (1 - r) * |δ| := by rw [abs_of_nonneg hpos]
        _ = (6 * (1 - r)) * (r * |δ|) := by ring
        _ ≤ (6 * (1 - r)) * (1 * |δ|) :=
          mul_le_mul_of_nonneg_left h_mul_factor (by nlinarith)
        _ = 6 * (1 - r) * |δ| := by ring
    have h2 : |δ ^ 2 * (3 - 6 * r - 2 * δ)| ≤ 3 * δ ^ 2 := by
      calc
        |δ ^ 2 * (3 - 6 * r - 2 * δ)| = |δ ^ 2| * |3 - 6 * r - 2 * δ| := abs_mul _ _
        _ = δ ^ 2 * |3 - 6 * r - 2 * δ| := by rw [abs_of_nonneg hsq_nonneg]
        _ ≤ δ ^ 2 * 3 := mul_le_mul_of_nonneg_left h_inner_bound hsq_nonneg
        _ = 3 * δ ^ 2 := by ring
    calc
      |6 * r * (1 - r) * δ + δ ^ 2 * (3 - 6 * r - 2 * δ)| ≤
          |6 * r * (1 - r) * δ| + |δ ^ 2 * (3 - 6 * r - 2 * δ)| := abs_add _ _
      _ ≤ (6 * (1 - r) * |δ|) + (3 * δ ^ 2) := by nlinarith
```

```
      _ = 6 * |r - 1| * |δ| + 3 * δ ^ 2 := by
        have h_abs : |r - 1| = 1 - r := by
          rw [show |r - 1| = |1 - r| from abs_sub_comm r 1,
            abs_of_nonneg (by nlinarith : 0 ≤ 1 - r)]
        rw [h_abs]
  let S := ∑ b, w b * |u b - e b| * |u' b - u b|
  let B := ∑ b, w b * (u' b - u b) ^ 2
  let A := ∑ b, w b * (u b - e b) ^ 2
  have hA_nonneg : 0 ≤ A := by
    refine Finset.sum_nonneg fun b _ => ?_
    have hw_nonneg := hw b
    have hsq_nonneg := sq_nonneg (u b - e b)
    nlinarith
  have hB_nonneg : 0 ≤ B := by
    refine Finset.sum_nonneg fun b _ => ?_
    have hw_nonneg := hw b
    have hsq_nonneg := sq_nonneg (u' b - u b)
    nlinarith
  have hS_nonneg : 0 ≤ S := by
    refine Finset.sum_nonneg fun b _ => ?_
    have hw_nonneg := hw b
    have h1 : 0 ≤ |u b - e b| := abs_nonneg _
    have h2 : 0 ≤ |u' b - u b| := abs_nonneg _
    have h_nonneg : 0 ≤ w b * |u b - e b| := mul_nonneg hw_nonneg h1
    have h_final : 0 ≤ (w b * |u b - e b|) * |u' b - u b| := mul_nonneg h_nonneg h2
    simpa [mul_assoc] using h_final
  have hX_nonneg : 0 ≤ ∑ b, w b * |φ (u' b) - φ (u b)| := by
    refine Finset.sum_nonneg fun b _ => ?_
    have hw_nonneg := hw b
    have habs_nonneg : 0 ≤ |φ (u' b) - φ (u b)| := abs_nonneg _
    nlinarith
  have h_bound_sum : (∑ b, w b * |φ (u' b) - φ (u b)|) ≤ 6 * S + 3 * B := by
    calc
      (∑ b, w b * |φ (u' b) - φ (u b)|) ≤
          ∑ b, w b * (6 * |u b - e b| * |u' b - u b| + 3 * (u' b - u b) ^ 2) :=
        Finset.sum_le_sum fun b _ => by
          have hb := h_key_scalar (u b) (u' b) (e b) (hu b).1 (hu b).2 (hu' b).1 (hu' b).2 (he b)
          have hw_nonneg := hw b
          have habs_nonneg : 0 ≤ |φ (u' b) - φ (u b)| := abs_nonneg _
          nlinarith
      _ = (∑ b, (w b * (6 * |u b - e b| * |u' b - u b|) + w b * (3 * (u' b - u b) ^ 2))) := by
        refine Finset.sum_congr rfl fun b _ => ?_
        ring
      _ = (∑ b, w b * (6 * |u b - e b| * |u' b - u b|)) +
          (∑ b, w b * (3 * (u' b - u b) ^ 2)) :=
        Finset.sum_add_distrib
      _ = 6 * (∑ b, w b * |u b - e b| * |u' b - u b|) +
          3 * (∑ b, w b * (u' b - u b) ^ 2) := by
        simp [Finset.mul_sum, mul_assoc]
        ring
      _ = 6 * S + 3 * B := rfl
  have h_cauchy_schwarz : S ^ 2 ≤ A * B := by
    set f := fun b : ι => Real.sqrt (w b) * |u b - e b| with hf
```

```
    set g := fun b : ι => Real.sqrt (w b) * |u' b - u b| with hg
    have h_sq_sqrt : ∀ b, (Real.sqrt (w b)) ^ 2 = w b := fun b =>
      Real.sq_sqrt (hw b)
    have hS_eq : S = ∑ b, f b * g b := by
      dsimp [S, f, g]
      refine Finset.sum_congr rfl fun b _ => ?_
      calc
        w b * |u b - e b| * |u' b - u b| =
            ((Real.sqrt (w b)) ^ 2) * |u b - e b| * |u' b - u b| := by rw [h_sq_sqrt b]
        _ = ((Real.sqrt (w b)) * |u b - e b|) * ((Real.sqrt (w b)) * |u' b - u b|) := by ring
    rw [hS_eq]
    calc
      (∑ b, f b * g b) ^ 2 ≤ (∑ b, (f b) ^ 2) * (∑ b, (g b) ^ 2) :=
        Finset.sum_mul_sq_le_sq_mul_sq Finset.univ f g
      _ = (∑ b, (Real.sqrt (w b)) ^ 2 * (|u b - e b|) ^ 2) *
          (∑ b, (Real.sqrt (w b)) ^ 2 * (|u' b - u b|) ^ 2) := by
        simp [hf, hg, mul_pow]
      _ = (∑ b, w b * (|u b - e b|) ^ 2) * (∑ b, w b * (|u' b - u b|) ^ 2) := by
        simp [h_sq_sqrt]
      _ = A * B := by
        simp [A, B, sq_abs]
  have h_ineq : (6 * S + 3 * B) ^ 2 ≤ 72 * (S ^ 2) + 18 * (B ^ 2) := by
    have h_key : 36 * S * B ≤ 36 * (S ^ 2) + 9 * (B ^ 2) := by
      have h_sq_nonneg : 0 ≤ (6 * S - 3 * B) ^ 2 := pow_two_nonneg _
      nlinarith
    nlinarith
  have h_final : (∑ b, w b * |φ (u' b) - φ (u b)|) ^ 2 ≤
      72 * A * B + 18 * B ^ 2 := by
    calc
      (∑ b, w b * |φ (u' b) - φ (u b)|) ^ 2 ≤ (6 * S + 3 * B) ^ 2 := by
        nlinarith
      _ ≤ 72 * (S ^ 2) + 18 * (B ^ 2) := h_ineq
      _ ≤ 72 * (A * B) + 18 * (B ^ 2) := by nlinarith
      _ = 72 * A * B + 18 * B ^ 2 := by ring
  simpa [A, B] using h_final
```

Proof. Coordinatewise bound. The factorization in Lemma 3.9 (reproved here) gives, for each b,

$$|\varphi(u'_b) - \varphi(u_b)| \le 6|u_b - e_b||u'_b - u_b| + 3(u'_b - u_b)^2.$$

Summation. Multiply by $w_b \ge 0$ and sum. Writing $S = \sum_b w_b|u_b - e_b||u'_b - u_b|$, $B = \|u' - u\|_{2,w}^2$ and $A = \|u - e\|_{2,w}^2$, this gives

$$\|\Phi(u') - \Phi(u)\|_{1,w} \le 6S + 3B.$$

The Cauchy–Schwarz inequality, applied to the vectors $(\sqrt{w_b}|u_b - e_b|)_b$ and $(\sqrt{w_b}|u'_b - u_b|)_b$, gives $S^2 \le AB$.

Conclusion. From $(6S - 3B)^2 \ge 0$ we get $(6S + 3B)^2 \le 72S^2 + 18B^2$. Since both sides are nonnegative,

$$\|\Phi(u') - \Phi(u)\|_{1,w}^2 \le (6S + 3B)^2 \le 72S^2 + 18B^2 \le 72AB + 18B^2. \qquad \square$$

Lemma 3.11 (`mc_flat`)**.** *Under the hypotheses of Lemma 3.10,*

$$\|\Phi(u') - \Phi(u)\|_{1,w}^2 \le 108\Big(\|u' - e\|_{2,w}^4 - \|u - e\|_{2,w}^4 - 4\|u - e\|_{2,w}^2\langle u - e, u' - u\rangle_w\Big).$$

Lean: `mc_flat` — proved by the captain after a hand-back

```
theorem mc_flat {ι : Type*} [Fintype ι] (w : ι → ℝ) (hw : ∀ b, 0 ≤ w b)
    (φ : ℝ → ℝ) (hφ : ∀ r, φ r = 3 * r ^ 2 - 2 * r ^ 3)
    (u u' e : ι → ℝ) (hu : ∀ b, 0 ≤ u b ∧ u b ≤ 1) (hu' : ∀ b, 0 ≤ u' b ∧ u' b ≤ 1)
    (he : ∀ b, e b = 0 ∨ e b = 1) :
    (∑ b, w b * |φ (u' b) - φ (u b)|) ^ 2
      ≤ 108 * ((∑ b, w b * (u' b - e b) ^ 2) ^ 2 - (∑ b, w b * (u b - e b) ^ 2) ^ 2
          - 4 * (∑ b, w b * (u b - e b) ^ 2) * ∑ b, w b * ((u b - e b) * (u' b - u b))) := by
  have hinc := mc_flat_increment w hw φ hφ u u' e hu hu' he
  have hexp : ∑ b, w b * (u' b - e b) ^ 2
      = ∑ b, w b * (u b - e b) ^ 2 + 2 * ∑ b, w b * ((u b - e b) * (u' b - u b))
        + ∑ b, w b * (u' b - u b) ^ 2 := by
    rw [Finset.mul_sum, ← Finset.sum_add_distrib, ← Finset.sum_add_distrib]
    exact Finset.sum_congr rfl (fun b _ => by ring)
  have hcs : (∑ b, w b * ((u b - e b) * (u' b - u b))) ^ 2
      ≤ (∑ b, w b * (u b - e b) ^ 2) * ∑ b, w b * (u' b - u b) ^ 2 := by
    have h := Finset.sum_mul_sq_le_sq_mul_sq Finset.univ
      (fun b => Real.sqrt (w b) * (u b - e b)) (fun b => Real.sqrt (w b) * (u' b - u b))
    have h1 : ∀ b, Real.sqrt (w b) * (u b - e b) * (Real.sqrt (w b) * (u' b - u b))
        = w b * ((u b - e b) * (u' b - u b)) := by
      intro b
      have hsq := Real.mul_self_sqrt (hw b)
      calc Real.sqrt (w b) * (u b - e b) * (Real.sqrt (w b) * (u' b - u b))
          = (Real.sqrt (w b) * Real.sqrt (w b)) * ((u b - e b) * (u' b - u b)) := by ring
        _ = w b * ((u b - e b) * (u' b - u b)) := by rw [hsq]
    have h2 : ∀ b (c : ℝ), (Real.sqrt (w b) * c) ^ 2 = w b * c ^ 2 := by
      intro b c
      rw [mul_pow, Real.sq_sqrt (hw b)]
    simp only [h1, h2] at h
    exact h
  rw [hexp]
  set S := ∑ b, w b * (u b - e b) ^ 2 with hS
  set P := ∑ b, w b * (u' b - u b) ^ 2 with hP
  set I := ∑ b, w b * ((u b - e b) * (u' b - u b)) with hI
  nlinarith [hinc, hcs, sq_nonneg (2 * I + P), sq_nonneg (4 * I + P)]
```

Proof. Put $S = \|u-e\|_{2,w}^2$, $P = \|u'-u\|_{2,w}^2$ and $I = \langle u-e, u'-u\rangle_w$.

Exact identity. Expanding the square gives $\|u'-e\|_{2,w}^2 = S + 2I + P$. Hence the bracket equals

$$R = (S+2I+P)^2 - S^2 - 4SI = 2SP + (2I+P)^2.$$

Comparison. By Lemma 3.10 it suffices to show $72SP + 18P^2 \le 108R$. The Cauchy–Schwarz inequality gives $I^2 \le SP$. Therefore

$$P^2 \le 2(2I+P)^2 + 8I^2 \le 2(2I+P)^2 + 8SP.$$

Consequently

$$72SP + 18P^2 \le 216SP + 36(2I+P)^2 \le 108R. \qquad \square$$

Lemma 3.12 (`mc_one_step`)**.** *Let ι be finite, let $w \ge 0$ on ι, let $\varphi(r) = 3r^2 - 2r^3$, and let $e \in \{0,1\}^\iota$. Let $p, q \ge 0$ with $p+q=1$, and let $a : \mathrm{Fin}\, n \to \mathbb{R}$ with $a \ge 0$ and $\sum_k a_k = 1$. Let $u, v \in [0,1]^\iota$ and $h : \mathrm{Fin}\, n \to [0,1]^\iota$ satisfy*

$$u_b = p\, v_b + q \sum_k a_k h_k(b) \qquad \text{for all } b.$$

Then

$$p\,\|\Phi(v)-\Phi(u)\|_{1,w}^2+q\sum_k a_k\|\Phi(h_k)-\Phi(u)\|_{1,w}^2 \le 108\Big(p\,\|v-e\|_{2,w}^4+q\sum_k a_k\|h_k-e\|_{2,w}^4-\|u-e\|_{2,w}^4\Big).$$

Lean: `mc_one_step` — proved by the captain after a hand-back

```
theorem mc_one_step {n : ℕ} {ι : Type*} [Fintype ι] (w : ι → ℝ) (hw : ∀ b, 0 ≤ w b)
    (φ : ℝ → ℝ) (hφ : ∀ r, φ r = 3 * r ^ 2 - 2 * r ^ 3)
    (e : ι → ℝ) (he : ∀ b, e b = 0 ∨ e b = 1)
    (p q : ℝ) (hp : 0 ≤ p) (hq : 0 ≤ q) (hpq : p + q = 1)
    (a : Fin n → ℝ) (ha : ∀ k, 0 ≤ a k) (hsum : ∑ k, a k = 1)
    (u v : ι → ℝ) (h : Fin n → ι → ℝ)
    (hu : ∀ b, 0 ≤ u b ∧ u b ≤ 1) (hv : ∀ b, 0 ≤ v b ∧ v b ≤ 1)
    (hh : ∀ k b, 0 ≤ h k b ∧ h k b ≤ 1)
    (hmean : ∀ b, u b = p * v b + q * ∑ k, a k * h k b) :
    p * (∑ b, w b * |φ (v b) - φ (u b)|) ^ 2
      + q * ∑ k, a k * (∑ b, w b * |φ (h k b) - φ (u b)|) ^ 2
      ≤ 108 * (p * (∑ b, w b * (v b - e b) ^ 2) ^ 2
          + q * ∑ k, a k * (∑ b, w b * (h k b - e b) ^ 2) ^ 2
          - (∑ b, w b * (u b - e b) ^ 2) ^ 2) := by
  have hcross : p * (∑ b, w b * ((u b - e b) * (v b - u b)))
      + q * ∑ k, a k * (∑ b, w b * ((u b - e b) * (h k b - u b))) = 0 := by
    have h1 : ∑ k, a k * (∑ b, w b * ((u b - e b) * (h k b - u b)))
        = ∑ b, w b * ((u b - e b) * (∑ k, a k * h k b - u b)) := by
      simp only [Finset.mul_sum]
      rw [Finset.sum_comm]
      refine Finset.sum_congr rfl fun b _ => ?_
      have hb : ∑ k, a k * h k b - u b = ∑ k, (a k * h k b - a k * u b) := by
        rw [Finset.sum_sub_distrib, ← Finset.sum_mul, hsum, one_mul]
      rw [hb, Finset.mul_sum, Finset.mul_sum]
      exact Finset.sum_congr rfl fun k _ => by ring
    rw [h1, Finset.mul_sum, Finset.mul_sum, ← Finset.sum_add_distrib]
    apply Finset.sum_eq_zero
    intro b _
    linear_combination (-(w b * (u b - e b))) * hmean b + (-(w b * (u b - e b) * u b)) * hpq
  have hsum2 : ∑ k, a k * (∑ b, w b * |φ (h k b) - φ (u b)|) ^ 2
      ≤ ∑ k, a k * (108 * ((∑ b, w b * (h k b - e b) ^ 2) ^ 2 - (∑ b, w b * (u b - e b) ^ 2) ^ 2
          - 4 * (∑ b, w b * (u b - e b) ^ 2) * ∑ b, w b * ((u b - e b) * (h k b - u b)))) :=
    Finset.sum_le_sum fun k _ =>
      mul_le_mul_of_nonneg_left (mc_flat w hw φ hφ u (h k) e hu (hh k) he) (ha k)
  have e2 : ∑ k, 108 * (∑ b, w b * (u b - e b) ^ 2) ^ 2 * a k
      = 108 * (∑ b, w b * (u b - e b) ^ 2) ^ 2 := by
    rw [← Finset.mul_sum, hsum, mul_one]
  have e3 : ∑ k, 432 * (∑ b, w b * (u b - e b) ^ 2) * (a k * ∑ b, w b * ((u b - e b) * (h k b - u b)))
      = 432 * (∑ b, w b * (u b - e b) ^ 2) * ∑ k, a k * (∑ b, w b * ((u b - e b) * (h k b - u b))) :=
    (Finset.mul_sum _ _ _).symm
  have e4 : ∑ k, 108 * (a k * (∑ b, w b * (h k b - e b) ^ 2) ^ 2)
      = 108 * ∑ k, a k * (∑ b, w b * (h k b - e b) ^ 2) ^ 2 :=
    (Finset.mul_sum _ _ _).symm
  have e5 : ∑ k, a k * (108 * ((∑ b, w b * (h k b - e b) ^ 2) ^ 2 - (∑ b, w b * (u b - e b) ^ 2) ^ 2
        - 4 * (∑ b, w b * (u b - e b) ^ 2) * ∑ b, w b * ((u b - e b) * (h k b - u b))))
      = 108 * ∑ k, a k * (∑ b, w b * (h k b - e b) ^ 2) ^ 2
        - 108 * (∑ b, w b * (u b - e b) ^ 2) ^ 2
        - 432 * (∑ b, w b * (u b - e b) ^ 2) * ∑ k, a k * (∑ b, w b * ((u b - e b) * (h k b - u b))) := by
    calc _ = ∑ k, (108 * (a k * (∑ b, w b * (h k b - e b) ^ 2) ^ 2)
          - 108 * (∑ b, w b * (u b - e b) ^ 2) ^ 2 * a k
          - 432 * (∑ b, w b * (u b - e b) ^ 2) * (a k * ∑ b, w b * ((u b - e b) * (h k b - u b)))) :=
        Finset.sum_congr rfl fun k _ => by ring
```

```
      _ = ∑ k, 108 * (a k * (∑ b, w b * (h k b - e b) ^ 2) ^ 2)
            - ∑ k, 108 * (∑ b, w b * (u b - e b) ^ 2) ^ 2 * a k
            - ∑ k, 432 * (∑ b, w b * (u b - e b) ^ 2) * (a k * ∑ b, w b * ((u b - e b) * (h k b - u b)))
↪   := by
          rw [Finset.sum_sub_distrib, Finset.sum_sub_distrib]
      _ = _ := by rw [e4, e2, e3]
  have hP := mul_le_mul_of_nonneg_left (mc_flat w hw φ hφ u v e hu hv he) hp
  have hQ := mul_le_mul_of_nonneg_left (hsum2.trans_eq e5) hq
  have hpS : p * (∑ b, w b * (u b - e b) ^ 2) ^ 2 + q * (∑ b, w b * (u b - e b) ^ 2) ^ 2
      = (∑ b, w b * (u b - e b) ^ 2) ^ 2 := by
    rw [← add_mul, hpq, one_mul]
  have hcS : (∑ b, w b * (u b - e b) ^ 2) * (p * (∑ b, w b * ((u b - e b) * (v b - u b)))
      + q * ∑ k, a k * (∑ b, w b * ((u b - e b) * (h k b - u b)))) = 0 := by
    rw [hcross, mul_zero]
  first
  | linarith
  | nlinarith [hP, hQ, hpS, hcS]
```

Proof. Apply Lemma 3.11 to the pairs (u, v) and (u, h_k). Multiply by p and by $q\,a_k$ respectively, which are nonnegative, and sum.

Potential terms. The terms $-\|u-e\|_{2,w}^4$ add up to $-(p+q\sum_k a_k)\|u-e\|_{2,w}^4 = -\|u-e\|_{2,w}^4$.

Linear terms. They add up to $-432\,\|u-e\|_{2,w}^2$ times

$$p\,\langle u-e, v-u\rangle_w + q\sum_k a_k\langle u-e, h_k-u\rangle_w = \Big\langle u-e,\; p\,v + q\sum_k a_k h_k - (p+q\textstyle\sum_k a_k)u\Big\rangle_w .$$

This vanishes by the mean-value relation together with $p+q=1$ and $\sum_k a_k = 1$. □

Lemma 3.13 (mc_aggregate). *Let $t \in \mathbb{N}_0$, let $\pi : \mathrm{Fin}\,n \to \mathbb{R}$ with $\pi \ge 0$, and let A, G be real matrices. Assume: - $G \ge 0$; - $G_{ij} = p_t\delta_{ij} + q_t\sum_k G_{ik}A_{kj}$ for all i, j; - $\sum_i \pi_i G_{ij} = \pi_j$ for all j.*

Let $B : \mathrm{Fin}\,n \to \mathbb{R}$ and $E, F^z, F^h : \mathrm{Fin}\,n \times \mathrm{Fin}\,n \to \mathbb{R}$ with $F^h \ge 0$. Suppose that for all i_0, j:

$$p_t B_j + q_t\sum_k A_{jk}E_{jk} \le 108\Big(p_t F^z_{i_0 j} + q_t\sum_k A_{jk}F^h_{i_0 k} - F^h_{i_0 j}\Big).$$

Then

$$\sum_j \pi_j B_j + t\sum_j\sum_k \pi_j A_{jk}E_{jk} \le 108\sum_i\sum_j \pi_i G_{ij}F^z_{ij}.$$

Lean: mc_aggregate — proved by the captain after a hand-back

```
theorem mc_aggregate {n : ℕ} (t : ℕ) (π : Fin n → ℝ) (hπ : ∀ i, 0 ≤ π i)
    (A G : Matrix (Fin n) (Fin n) ℝ) (hG0 : ∀ i j, 0 ≤ G i j)
    (hGA : ∀ i j, G i j = (1 / ((t : ℝ) + 1)) * (if i = j then (1:ℝ) else 0)
        + ((t : ℝ) / ((t : ℝ) + 1)) * ∑ k, G i k * A k j)
    (hGπ : ∀ j, ∑ i, π i * G i j = π j)
    (Bv : Fin n → ℝ) (E Fz Fh : Fin n → Fin n → ℝ) (hFh : ∀ i j, 0 ≤ Fh i j)
    (hstep : ∀ i0 j, (1 / ((t : ℝ) + 1)) * Bv j + ((t : ℝ) / ((t : ℝ) + 1)) * ∑ k, A j k * E j k
        ≤ 108 * ((1 / ((t : ℝ) + 1)) * Fz i0 j
            + ((t : ℝ) / ((t : ℝ) + 1)) * ∑ k, A j k * Fh i0 k - Fh i0 j)) :
    ∑ j, π j * Bv j + (t : ℝ) * ∑ j, ∑ k, π j * A j k * E j k
      ≤ 108 * ∑ i, ∑ j, π i * G i j * Fz i j := by
  have ht1 : (t:ℝ) + 1 ≠ 0 := by positivity
  have ht0 : (0:ℝ) ≤ (t:ℝ) + 1 := by positivity
  obtain ⟨p, hp⟩ : ∃ p : ℝ, p = 1 / ((t:ℝ) + 1) := ⟨_, rfl⟩
```

```
  obtain ⟨q, hq⟩ : ∃ q : ℝ, q = (t:ℝ) / ((t:ℝ) + 1) := ⟨_, rfl⟩
  have hp0 : 0 ≤ p := by rw [hp]; positivity
  have hpp : ((t:ℝ) + 1) * p = 1 := by
    rw [hp]
    first
      | exact mul_one_div_cancel ht1
      | field_simp
  have hqq : ((t:ℝ) + 1) * q = t := by
    first
      | (rw [hq, div_eq_mul_one_div, ← hp]; linear_combination (t:ℝ) * hpp)
      | (rw [hq]; field_simp)
  rw [← hp, ← hq] at hstep hGA
  have hcol : ∀ i k, q * ∑ j, G i j * A j k = G i k - p * (if i = k then (1:ℝ) else 0) := by
    intro i k
    have := hGA i k
    linarith
  have hrow : ∀ i, ∑ j, G i j * (q * ∑ k, A j k * Fh i k) = ∑ j, G i j * Fh i j - p * Fh i i := by
    intro i
    calc ∑ j, G i j * (q * ∑ k, A j k * Fh i k)
        = ∑ j, ∑ k, q * (G i j * A j k) * Fh i k := by
          refine Finset.sum_congr rfl (fun j _ => ?_)
          simp only [Finset.mul_sum]
          refine Finset.sum_congr rfl (fun k _ => ?_)
          ring
      _ = ∑ k, (q * ∑ j, G i j * A j k) * Fh i k := by
          rw [Finset.sum_comm]
          refine Finset.sum_congr rfl (fun k _ => ?_)
          rw [Finset.mul_sum, Finset.sum_mul]
      _ = ∑ k, (G i k - p * (if i = k then (1:ℝ) else 0)) * Fh i k := by
          refine Finset.sum_congr rfl (fun k _ => ?_)
          rw [hcol i k]
      _ = ∑ j, G i j * Fh i j - p * Fh i i := by
          have h1 : ∑ k, (G i k - p * (if i = k then (1:ℝ) else 0)) * Fh i k
              = ∑ k, G i k * Fh i k - ∑ k, (if i = k then p * Fh i k else 0) := by
            rw [← Finset.sum_sub_distrib]
            refine Finset.sum_congr rfl (fun k _ => ?_)
            split_ifs <;> ring
          rw [h1]
          simp [Finset.sum_ite_eq]
  have key : ∑ i, ∑ j, π i * G i j * (p * Bv j + q * ∑ k, A j k * E j k)
      ≤ ∑ i, ∑ j, π i * G i j * (108 * (p * Fz i j + q * ∑ k, A j k * Fh i k - Fh i j)) := by
    apply Finset.sum_le_sum
    intro i _
    apply Finset.sum_le_sum
    intro j _
    exact mul_le_mul_of_nonneg_left (hstep i j) (mul_nonneg (hπ i) (hG0 i j))
  have hL : ∑ i, ∑ j, π i * G i j * (p * Bv j + q * ∑ k, A j k * E j k)
      = ∑ j, π j * (p * Bv j + q * ∑ k, A j k * E j k) := by
    rw [Finset.sum_comm]
    refine Finset.sum_congr rfl (fun j _ => ?_)
    rw [← Finset.sum_mul, hGπ j]
  have hR : ∑ i, ∑ j, π i * G i j * (108 * (p * Fz i j + q * ∑ k, A j k * Fh i k - Fh i j))
      = ∑ i, 108 * (π i * (p * ∑ j, G i j * Fz i j - p * Fh i i)) := by
    refine Finset.sum_congr rfl (fun i _ => ?_)
    have hj : ∀ j, π i * G i j * (108 * (p * Fz i j + q * ∑ k, A j k * Fh i k - Fh i j))
        = (108 * π i * p) * (G i j * Fz i j) + (108 * π i) * (G i j * (q * ∑ k, A j k * Fh i k))
          - (108 * π i) * (G i j * Fh i j) := fun j => by ring
    simp only [hj]
    rw [Finset.sum_sub_distrib, Finset.sum_add_distrib]
```

```
    simp only [← Finset.mul_sum]
    rw [hrow i]
    ring
  have hfin : ∑ i, 108 * (π i * (p * ∑ j, G i j * Fz i j - p * Fh i i))
      ≤ 108 * p * ∑ i, ∑ j, π i * G i j * Fz i j := by
    have hi : ∀ i, 108 * (π i * (p * ∑ j, G i j * Fz i j - p * Fh i i))
        ≤ 108 * p * ∑ j, π i * G i j * Fz i j := by
      intro i
      have h1 : ∑ j, π i * G i j * Fz i j = π i * ∑ j, G i j * Fz i j := by
        rw [Finset.mul_sum]
        refine Finset.sum_congr rfl (fun j _ => ?_)
        ring
      rw [h1]
      have hnn : 0 ≤ π i * p * Fh i i := mul_nonneg (mul_nonneg (hπ i) hp0) (hFh i i)
      nlinarith [hnn]
    calc ∑ i, 108 * (π i * (p * ∑ j, G i j * Fz i j - p * Fh i i))
        ≤ ∑ i, 108 * p * ∑ j, π i * G i j * Fz i j := Finset.sum_le_sum (fun i _ => hi i)
      _ = 108 * p * ∑ i, ∑ j, π i * G i j * Fz i j := by rw [← Finset.mul_sum]
  have hsplit : ∑ j, π j * (p * Bv j + q * ∑ k, A j k * E j k)
      = p * ∑ j, π j * Bv j + q * ∑ j, ∑ k, π j * A j k * E j k := by
    have h1 : ∀ j, π j * (p * Bv j + q * ∑ k, A j k * E j k)
        = p * (π j * Bv j) + q * ∑ k, π j * A j k * E j k := by
      intro j
      have h2 : ∑ k, π j * A j k * E j k = π j * ∑ k, A j k * E j k := by
        rw [Finset.mul_sum]
        refine Finset.sum_congr rfl (fun k _ => ?_)
        ring
      rw [h2]
      ring
    simp only [h1, Finset.sum_add_distrib, ← Finset.mul_sum]
  have hmain : p * ∑ j, π j * Bv j + q * ∑ j, ∑ k, π j * A j k * E j k
      ≤ 108 * p * ∑ i, ∑ j, π i * G i j * Fz i j := by
    linarith [key, hL, hR, hfin, hsplit]
  have h2 := mul_le_mul_of_nonneg_left hmain ht0
  have e1 : ((t:ℝ) + 1) * (p * ∑ j, π j * Bv j + q * ∑ j, ∑ k, π j * A j k * E j k)
      = ∑ j, π j * Bv j + (t:ℝ) * ∑ j, ∑ k, π j * A j k * E j k := by
    linear_combination (∑ j, π j * Bv j) * hpp + (∑ j, ∑ k, π j * A j k * E j k) * hqq
  have e2 : ((t:ℝ) + 1) * (108 * p * ∑ i, ∑ j, π i * G i j * Fz i j)
      = 108 * ∑ i, ∑ j, π i * G i j * Fz i j := by
    linear_combination 108 * (∑ i, ∑ j, π i * G i j * Fz i j) * hpp
  linarith [h2, e1, e2]
```

Proof. Multiply the hypothesis for (i, j) by $\pi_i G_{ij} \geq 0$ and sum over i, j.

Left side. Since $\sum_i \pi_i G_{ij} = \pi_j$, it becomes

$$p_t \sum_j \pi_j B_j + q_t \sum_{j,k} \pi_j A_{jk} E_{jk}.$$

Right side. Fix i. By the column first-step identity,

$$\sum_j G_{ij}\, q_t \sum_k A_{jk} F^h_{ik} = \sum_k \Big(q_t \sum_j G_{ij} A_{jk} \Big) F^h_{ik} = \sum_k G_{ik} F^h_{ik} - p_t F^h_{ii}.$$

This telescopes against $-\sum_j G_{ij} F^h_{ij}$, so the right side equals

$$108 \sum_i \pi_i \Big(p_t \sum_j G_{ij} F^z_{ij} - p_t F^h_{ii} \Big).$$

Since $\pi_i p_t F^h_{ii} \geq 0$, this is at most $108\, p_t \sum_{i,j} \pi_i G_{ij} F^z_{ij}$.

Conclusion. Multiply the resulting inequality by $t+1$ and use $(t+1)p_t = 1$ and $(t+1)q_t = t$. □

Lemma 3.14 (mc_phi_vertex). *If $\varphi(r) = 3r^2 - 2r^3$ for all r and $d \in \{0,1\}$, then $\varphi(d) = d$.*

Lean: mc_phi_vertex — proved by AIProver, job 1, session 0

```
theorem mc_phi_vertex (φ : ℝ → ℝ) (hφ : ∀ r, φ r = 3 * r ^ 2 - 2 * r ^ 3)
    (d : ℝ) (hd : d = 0 ∨ d = 1) : φ d = d := by
  rcases hd with (hd | hd)
  · rw [hd, hφ 0]
    ring
  · rw [hd, hφ 1]
    ring
```

Proof. Direct evaluation: $\varphi(0) = 0$ and $\varphi(1) = 1$. □

Lemma 3.15 (mc_cube_step). *Let $t \in \mathbb{N}_0$ and let A be stochastic. Let ι be finite, $w \geq 0$ on ι, and $z : \mathrm{Fin}\, n \to \{0,1\}^\iota$. Let $\varphi(r) = 3r^2 - 2r^3$. Let G satisfy - $G \geq 0$ and $\sum_j G_{ij} = 1$; - $G_{ij} = p_t \delta_{ij} + q_t \sum_k A_{ik} G_{kj}$.*

Put $h_k = \sum_l G_{kl} z_l \in \mathbb{R}^\iota$. Then for all $i_0, j \in \mathrm{Fin}\, n$,

$$p_t \|\Phi(z_j) - \Phi(h_j)\|^2_{1,w} + q_t \sum_k A_{jk} \|\Phi(h_k) - \Phi(h_j)\|^2_{1,w} \leq 108 \Big(p_t \|z_j - z_{i_0}\|^4_{2,w} + q_t \sum_k A_{jk} \|h_k - z_{i_0}\|^4_{2,w} - \|h_j - z_{i_0}\|^4_{2,w} \Big).$$

Lean: mc_cube_step — proved by AIProver, job 2, session 0

```
theorem mc_cube_step {n : ℕ} (t : ℕ) (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochastic A)
    {ι : Type*} [Fintype ι] (w : ι → ℝ) (hw : ∀ b, 0 ≤ w b)
    (z : Fin n → ι → ℝ) (hz : ∀ i b, z i b = 0 ∨ z i b = 1)
    (φ : ℝ → ℝ) (hφ : ∀ r, φ r = 3 * r ^ 2 - 2 * r ^ 3)
    (G : Matrix (Fin n) (Fin n) ℝ) (hG0 : ∀ i j, 0 ≤ G i j) (hGrow : ∀ i, ∑ j, G i j = 1)
    (hAG : ∀ i j, G i j = (1 / ((t : ℝ) + 1)) * (if i = j then (1:ℝ) else 0)
        + ((t : ℝ) / ((t : ℝ) + 1)) * ∑ k, A i k * G k j)
    (i0 j : Fin n) :
    (1 / ((t : ℝ) + 1)) * (∑ b, w b * |φ (z j b) - φ (∑ l, G j l * z l b)|) ^ 2
      + ((t : ℝ) / ((t : ℝ) + 1)) * ∑ k, A j k
          * (∑ b, w b * |φ (∑ l, G k l * z l b) - φ (∑ l, G j l * z l b)|) ^ 2
      ≤ 108 * ((1 / ((t : ℝ) + 1)) * (∑ b, w b * (z j b - z i0 b) ^ 2) ^ 2
          + ((t : ℝ) / ((t : ℝ) + 1)) * ∑ k, A j k * (∑ b, w b * (∑ l, G k l * z l b - z i0 b) ^ 2) ^ 2
          - (∑ b, w b * (∑ l, G j l * z l b - z i0 b) ^ 2) ^ 2) := by
  set p := (1 : ℝ) / ((t : ℝ) + 1) with hpdef
  set q := (t : ℝ) / ((t : ℝ) + 1) with hqdef
  have hp_nonneg : 0 ≤ p := div_nonneg (by norm_num) (by positivity)
  have hq_nonneg : 0 ≤ q := div_nonneg (Nat.cast_nonneg _) (by positivity)
  have hpq : p + q = 1 := by
    dsimp [p, q]
    field_simp [show ((t : ℝ) + 1) ≠ 0 from by positivity]
    ring
  have hu_nonneg : ∀ b, 0 ≤ (∑ l : Fin n, G j l * z l b) := by
    intro b; refine Finset.sum_nonneg (fun l _ => ?_)
    rcases hz l b with (h|h) <;> nlinarith [hG0 j l, h]
  have hv_nonneg : ∀ b, 0 ≤ z j b := by
    intro b; rcases hz j b with (h|h) <;> nlinarith
  have he_nonneg : ∀ b, 0 ≤ z i0 b := by
```

```
    intro b; rcases hz i0 b with (h|h) <;> nlinarith
  have hh_nonneg : ∀ k b, 0 ≤ (∑ l : Fin n, G k l * z l b) := by
    intro k b; refine Finset.sum_nonneg (fun l _ => ?_)
    rcases hz l b with (h|h) <;> nlinarith [hG0 k l, h]
  have hh_le_one : ∀ k b, (∑ l : Fin n, G k l * z l b) ≤ 1 := by
    intro k b
    calc
      (∑ l : Fin n, G k l * z l b) ≤ (∑ l : Fin n, G k l * 1) := by
        refine Finset.sum_le_sum (fun l _ => ?_)
        rcases hz l b with (h|h) <;> nlinarith [hG0 k l, h]
      _ = (∑ l : Fin n, G k l) := by simp
      _ = 1 := hGrow k
  have hsum : ∀ b, (∑ l : Fin n, G j l * z l b) = p * z j b + q * ∑ k, A j k * (∑ l : Fin n, G k l * z l
↪  b) := by
    intro b
    calc
      (∑ l : Fin n, G j l * z l b)
          = (∑ l : Fin n, ((p * (if j = l then (1 : ℝ) else 0) + q * ∑ k, A j k * G k l) * z l b)) := by
        refine Finset.sum_congr rfl (fun l _ => ?_); rw [hAG j l]
      _ = (∑ l : Fin n, ((p * (if j = l then (1 : ℝ) else 0)) * z l b
          + (q * ∑ k, A j k * G k l) * z l b)) := by
        refine Finset.sum_congr rfl (fun x _ => ?_); ring
      _ = (∑ l : Fin n, (p * (if j = l then (1 : ℝ) else 0)) * z l b)
          + (∑ l : Fin n, (q * ∑ k, A j k * G k l) * z l b) :=
        Finset.sum_add_distrib
      _ = (p * z j b) + (∑ l : Fin n, q * (∑ k, A j k * G k l * z l b)) := by
        simp [Finset.sum_ite_eq, Finset.mem_univ, mul_assoc, Finset.mul_sum, mul_comm]
      _ = p * z j b + q * (∑ l : Fin n, ∑ k, A j k * G k l * z l b) := by
        simp [Finset.mul_sum, mul_assoc]
      _ = p * z j b + q * (∑ k, ∑ l : Fin n, A j k * G k l * z l b) := by
        rw [Finset.sum_comm]
      _ = p * z j b + q * (∑ k, A j k * (∑ l : Fin n, G k l * z l b)) := by
        simp [Finset.mul_sum, mul_assoc]
  have hmean (b : ι) : (∑ l : Fin n, G j l * z l b) = p * z j b + q * ∑ k, A j k * (∑ l : Fin n, G k l *
↪  z l b) :=
    hsum b
  have hu_le_one : ∀ b, (∑ l : Fin n, G j l * z l b) ≤ 1 := by
    intro b
    calc
      (∑ l : Fin n, G j l * z l b) ≤ (∑ l : Fin n, G j l * 1) := by
        refine Finset.sum_le_sum (fun l _ => ?_)
        rcases hz l b with (h|h) <;> nlinarith [hG0 j l, h]
      _ = (∑ l : Fin n, G j l) := by simp
      _ = 1 := hGrow j
  have hv_le_one : ∀ b, z j b ≤ 1 := by
    intro b; rcases hz j b with (h|h) <;> nlinarith
  have hkey := mc_one_step w hw φ hφ (fun b => z i0 b) (fun b => hz i0 b) p q hp_nonneg hq_nonneg hpq
    (A j) (hA.1 j) (hA.2 j) (fun b => ∑ l, G j l * z l b) (fun b => z j b)
    (fun k b => ∑ l, G k l * z l b) (fun b => ⟨hu_nonneg b, hu_le_one b⟩) (fun b => ⟨hv_nonneg b, hv_le_one
↪  b⟩)
    (fun k b => ⟨hh_nonneg k b, hh_le_one k b⟩) hmean
  · simpa [hpdef, hqdef] using hkey
```

Proof. Apply Lemma 3.12 with the following data: - $e = z_{i_0}$, $p = p_t$, $q = q_t$ (so $p, q \geq 0$ and $p + q = 1$); - $a = (A_{jk})_k$, a probability vector because A is stochastic; - $u = h_j$, $v = z_j$, and the family $(h_k)_k$.

The values of h_k lie in $[0,1]$ because they are convex combinations of bits. The mean-value relation $h_j = p_t z_j + q_t \sum_k A_{jk} h_k$ follows from the first-step identity, exactly as in Lemma 3.8. □

Lemma 3.16 (mc_cube_energy_raw)**.** *Let $t \in \mathbb{N}_0$, $\pi \geq 0$, and let A be stochastic. Let ι, w, z, φ be as in Lemma 3.15. Let G satisfy - $G \geq 0$ and $\sum_j G_{ij} = 1$; - both first-step identities $G_{ij} = p_t \delta_{ij} + q_t \sum_k A_{ik} G_{kj} = p_t \delta_{ij} + q_t \sum_k G_{ik} A_{kj}$; - $\sum_i \pi_i G_{ij} = \pi_j$.*

With $h_k = \sum_l G_{kl} z_l$,

$$\sum_j \pi_j \|\Phi(z_j) - \Phi(h_j)\|_{1,w}^2 + t \sum_j \sum_k \pi_j A_{jk} \|\Phi(h_k) - \Phi(h_j)\|_{1,w}^2 \leq 108 \sum_i \sum_j \pi_i G_{ij} \|z_j - z_i\|_{2,w}^4.$$

Lean: mc_cube_energy_raw — proved by the captain after a hand-back

```
theorem mc_cube_energy_raw {n : ℕ} (t : ℕ) (π : Fin n → ℝ) (hπ : ∀ i, 0 ≤ π i)
    (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochastic A)
    {ι : Type*} [Fintype ι] (w : ι → ℝ) (hw : ∀ b, 0 ≤ w b)
    (z : Fin n → ι → ℝ) (hz : ∀ i b, z i b = 0 ∨ z i b = 1)
    (φ : ℝ → ℝ) (hφ : ∀ r, φ r = 3 * r ^ 2 - 2 * r ^ 3)
    (G : Matrix (Fin n) (Fin n) ℝ) (hG0 : ∀ i j, 0 ≤ G i j) (hGrow : ∀ i, ∑ j, G i j = 1)
    (hAG : ∀ i j, G i j = (1 / ((t : ℝ) + 1)) * (if i = j then (1:ℝ) else 0)
        + ((t : ℝ) / ((t : ℝ) + 1)) * ∑ k, A i k * G k j)
    (hGA : ∀ i j, G i j = (1 / ((t : ℝ) + 1)) * (if i = j then (1:ℝ) else 0)
        + ((t : ℝ) / ((t : ℝ) + 1)) * ∑ k, G i k * A k j)
    (hGπ : ∀ j, ∑ i, π i * G i j = π j) :
    ∑ j, π j * (∑ b, w b * |φ (z j b) - φ (∑ l, G j l * z l b)|) ^ 2
      + (t : ℝ) * ∑ j, ∑ k, π j * A j k
          * (∑ b, w b * |φ (∑ l, G k l * z l b) - φ (∑ l, G j l * z l b)|) ^ 2
      ≤ 108 * ∑ i, ∑ j, π i * G i j * (∑ b, w b * (z j b - z i b) ^ 2) ^ 2 := by
  have key := mc_aggregate t π hπ A G hG0 hGA hGπ
    (fun j => (∑ b, w b * |φ (z j b) - φ (∑ l, G j l * z l b)|) ^ 2)
    (fun j k => (∑ b, w b * |φ (∑ l, G k l * z l b) - φ (∑ l, G j l * z l b)|) ^ 2)
    (fun i j => (∑ b, w b * (z j b - z i b) ^ 2) ^ 2)
    (fun i j => (∑ b, w b * (∑ l, G j l * z l b - z i b) ^ 2) ^ 2)
    (fun i j => sq_nonneg _)
    (fun i0 j => mc_cube_step t A hA w hw z hz φ hφ G hG0 hGrow hAG i0 j)
  exact key
```

Proof. Apply Lemma 3.13 with

$$B_j = \|\Phi(z_j) - \Phi(h_j)\|_{1,w}^2, \quad E_{jk} = \|\Phi(h_k) - \Phi(h_j)\|_{1,w}^2, \quad F_{ij}^z = \|z_j - z_i\|_{2,w}^4, \quad F_{ij}^h = \|h_j - z_i\|_{2,w}^4 \geq 0.$$

The required step inequality is Lemma 3.15. □

Lemma 3.17 (mc_cube_energy)**.** *Let π be a probability vector, and otherwise keep the hypotheses of Lemma 3.16. Then*

$$\sum_i \pi_i \|z_i - \Phi(h_i)\|_{1,w}^2 + t \sum_i \sum_j \pi_i A_{ij} \|\Phi(h_i) - \Phi(h_j)\|_{1,w}^2 \leq 108 \sum_i \sum_j \pi_i G_{ij} \|z_i - z_j\|_{2,w}^4.$$

Lean: mc_cube_energy — proved by the captain after a hand-back

```
theorem mc_cube_energy {n : ℕ} (t : ℕ) (π : Fin n → ℝ) (hπ : IsProbVec π)
    (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochastic A)
    {ι : Type*} [Fintype ι] (w : ι → ℝ) (hw : ∀ b, 0 ≤ w b)
    (z : Fin n → ι → ℝ) (hz : ∀ i b, z i b = 0 ∨ z i b = 1)
    (φ : ℝ → ℝ) (hφ : ∀ r, φ r = 3 * r ^ 2 - 2 * r ^ 3)
    (G : Matrix (Fin n) (Fin n) ℝ) (hG0 : ∀ i j, 0 ≤ G i j) (hGrow : ∀ i, ∑ j, G i j = 1)
    (hAG : ∀ i j, G i j = (1 / ((t : ℝ) + 1)) * (if i = j then (1:ℝ) else 0)
        + ((t : ℝ) / ((t : ℝ) + 1)) * ∑ k, A i k * G k j)
    (hGA : ∀ i j, G i j = (1 / ((t : ℝ) + 1)) * (if i = j then (1:ℝ) else 0)
        + ((t : ℝ) / ((t : ℝ) + 1)) * ∑ k, G i k * A k j)
    (hGπ : ∀ j, ∑ i, π i * G i j = π j) :
    ∑ i, π i * (∑ b, w b * |z i b - φ (∑ j, G i j * z j b)|) ^ 2
      + (t : ℝ) * ∑ i, ∑ j, π i * A i j
          * (∑ b, w b * |φ (∑ k, G i k * z k b) - φ (∑ k, G j k * z k b)|) ^ 2
      ≤ 108 * ∑ i, ∑ j, π i * G i j * (∑ b, w b * (z i b - z j b) ^ 2) ^ 2 := by
  have h := mc_cube_energy_raw t π hπ.1 A hA w hw z hz φ hφ G hG0 hGrow hAG hGA hGπ
  have key1 : ∑ i, π i * (∑ b, w b * |z i b - φ (∑ j, G i j * z j b)|) ^ 2
      = ∑ j, π j * (∑ b, w b * |φ (z j b) - φ (∑ l, G j l * z l b)|) ^ 2 := by
    refine Finset.sum_congr rfl (fun i _ => ?_)
    have hv : ∀ b, φ (z i b) = z i b := fun b => mc_phi_vertex φ hφ _ (hz i b)
    simp only [hv]
  have key2 : ∑ i, ∑ j, π i * A i j
          * (∑ b, w b * |φ (∑ k, G i k * z k b) - φ (∑ k, G j k * z k b)|) ^ 2
      = ∑ j, ∑ k, π j * A j k
          * (∑ b, w b * |φ (∑ l, G k l * z l b) - φ (∑ l, G j l * z l b)|) ^ 2 := by
    refine Finset.sum_congr rfl (fun i _ => Finset.sum_congr rfl (fun j _ => ?_))
    have hs : ∀ b, |φ (∑ k, G i k * z k b) - φ (∑ k, G j k * z k b)|
        = |φ (∑ l, G j l * z l b) - φ (∑ l, G i l * z l b)| := fun b => abs_sub_comm _ _
    simp only [hs]
  have key3 : ∑ i, ∑ j, π i * G i j * (∑ b, w b * (z i b - z j b) ^ 2) ^ 2
      = ∑ i, ∑ j, π i * G i j * (∑ b, w b * (z j b - z i b) ^ 2) ^ 2 := by
    refine Finset.sum_congr rfl (fun i _ => Finset.sum_congr rfl (fun j _ => ?_))
    have hs : ∀ b, (z i b - z j b) ^ 2 = (z j b - z i b) ^ 2 := fun b => by ring
    simp only [hs]
  rw [key1, key2, key3]
  exact h
```

Proof. This is Lemma 3.16, which only needs $\pi \geq 0$, rewritten in three ways: - $\Phi(z_i) = z_i$ by Lemma 3.14; - $|\alpha - \beta| = |\beta - \alpha|$; - $(\alpha - \beta)^2 = (\beta - \alpha)^2$. □

Lemma 3.18 (mc_cut_real_mono). *Let $a : \mathrm{Fin}\, n \to \mathbb{R}$ be monotone. Then there exist $c : \mathrm{Fin}\, n \to \mathcal{P}(\mathrm{Fin}\, n)$, $\mathrm{gap} : \mathrm{Fin}\, n \to \mathbb{R}$ and $v_0 \in \mathbb{R}$ with:*

1. $\mathrm{gap}_r \geq 0$ *for all r;*
2. $\sum_r \mathrm{gap}_r \leq 2 \sum_i |a_i|$;
3. $|v_0| \leq \sum_i |a_i|$;
4. $a_i = v_0 + \sum_r \mathbf{1}[i \in c_r]\mathrm{gap}_r$ *for all i;*
5. $|a_i - a_j| = \sum_r |\mathbf{1}[i \in c_r] - \mathbf{1}[j \in c_r]|\, \mathrm{gap}_r$ *for all i, j.*

Lean: mc_cut_real_mono — proved by the captain after a hand-back

```
theorem mc_cut_real_mono (n : ℕ) (a : Fin n → ℝ) (ha : Monotone a) :
    ∃ (c : Fin n → Finset (Fin n)) (gap : Fin n → ℝ) (v0 : ℝ),
      (∀ r, 0 ≤ gap r) ∧ ∑ r, gap r ≤ 2 * ∑ i, |a i| ∧ |v0| ≤ ∑ i, |a i| ∧
      (∀ i, a i = v0 + ∑ r, (if i ∈ c r then (1:ℝ) else 0) * gap r) ∧
      (∀ i j, |a i - a j| = ∑ r, |(if i ∈ c r then (1:ℝ) else 0) - (if j ∈ c r then (1:ℝ) else 0)| * gap
↪  r) := by
  rcases Nat.eq_zero_or_pos n with hn | hnpos
  · subst hn
    refine ⟨fun _ => ∅, fun _ => 0, 0, fun _ => le_rfl, by simp, by simp, fun i => i.elim0, fun i => i.elim0⟩
  · obtain ⟨F, hF⟩ : ∃ F : ℕ → ℝ, ∀ m, F m = if h : m < n then a ⟨m, h⟩ else 0 :=
      ⟨fun m => if h : m < n then a ⟨m, h⟩ else 0, fun _ => rfl⟩
    have hFa : ∀ i : Fin n, F i.val = a i := by
      intro i
      rw [hF]
      exact dif_pos i.isLt
    have hFmono : ∀ m, m + 1 < n → F m ≤ F (m + 1) := by
      intro m h
      rw [hF m, hF (m + 1), dif_pos (show m < n by omega), dif_pos h]
      exact ha (show (⟨m, by omega⟩ : Fin n) ≤ ⟨m + 1, h⟩ from
        Fin.le_iff_val_le_val.mpr (by show m ≤ m + 1; omega))
    obtain ⟨g, hg⟩ : ∃ g : ℕ → ℝ, ∀ k, g k = if k + 1 < n then F (k + 1) - F k else 0 :=
      ⟨fun k => if k + 1 < n then F (k + 1) - F k else 0, fun _ => rfl⟩
    obtain ⟨c, hc⟩ : ∃ c : Fin n → Finset (Fin n), ∀ r i, i ∈ c r ↔ r.val < i.val :=
      ⟨fun r => Finset.univ.filter (fun i => r.val < i.val), fun r i => by simp⟩
    obtain ⟨z0, hz0⟩ : ∃ l : Fin n, l.val = 0 := ⟨⟨0, hnpos⟩, rfl⟩
    obtain ⟨last, hlast⟩ : ∃ l : Fin n, l.val = n - 1 := ⟨⟨n - 1, by omega⟩, rfl⟩
    have hF0 : F 0 = a z0 := by rw [← hz0]; exact hFa z0
    have hFl : F (n - 1) = a last := by rw [← hlast]; exact hFa last
    have hle : ∀ j : Fin n, |a j| ≤ ∑ i, |a i| := fun j =>
      Finset.single_le_sum (f := fun i => |a i|) (fun i _ => abs_nonneg (a i)) (Finset.mem_univ j)
    have hgap : ∀ r : Fin n, 0 ≤ g r.val := by
      intro r
      by_cases h : r.val + 1 < n
      · rw [hg, if_pos h]; linarith [hFmono r.val h]
      · simp [hg, h]
    have hsum : ∀ m : ℕ, m < n → ∑ r : Fin n, (if r.val < m then g r.val else 0) = F m - F 0 := by
      intro m
      induction m with
      | zero => intro _; simp
      | succ m ih =>
        intro hm
        have hm' : m < n := by omega
        have key : ∀ r : Fin n, (if r.val < m + 1 then g r.val else 0)
            = (if r.val < m then g r.val else 0) + (if r = (⟨m, hm'⟩ : Fin n) then g m else 0) := by
          intro r
          by_cases h1 : r.val < m
          · have hne : r ≠ ⟨m, hm'⟩ := fun h => by subst h; simp at h1
            rw [if_pos (show r.val < m + 1 by omega), if_pos h1, if_neg hne, add_zero]
          · by_cases h2 : r.val = m
            · have hr : r = ⟨m, hm'⟩ := Fin.ext h2
              subst hr
              simp
            · have hr : r ≠ ⟨m, hm'⟩ := fun h => h2 (by subst h; rfl)
              rw [if_neg (show ¬ (r.val < m + 1) by omega), if_neg h1, if_neg hr, add_zero]
        have hs : ∑ r : Fin n, (if r = (⟨m, hm'⟩ : Fin n) then g m else 0) = g m := by simp
        rw [Finset.sum_congr rfl (fun r _ => key r), Finset.sum_add_distrib, ih hm', hs,
          hg m, if_pos hm]
```

```
    ring
  have hind : ∀ r i : Fin n, (if i ∈ c r then (1:ℝ) else 0) * g r.val
      = if r.val < i.val then g r.val else 0 := by
    intro r i
    by_cases h : r.val < i.val
    · rw [if_pos ((hc r i).mpr h), if_pos h, one_mul]
    · rw [if_neg (fun h' => h ((hc r i).mp h')), if_neg h, zero_mul]
  have hrec : ∀ i : Fin n, a i = a z0 + ∑ r, (if i ∈ c r then (1:ℝ) else 0) * g r.val := by
    intro i
    rw [Finset.sum_congr rfl (fun r _ => hind r i), hsum i.val i.isLt, hFa i, hF0]
    ring
  have hterm : ∀ r i j : Fin n, i.val ≤ j.val →
      |(if i ∈ c r then (1:ℝ) else 0) - (if j ∈ c r then (1:ℝ) else 0)| * g r.val
        = (if r.val < j.val then g r.val else 0) - (if r.val < i.val then g r.val else 0) := by
    intro r i j hij
    by_cases h1 : r.val < i.val
    · have h2 : r.val < j.val := by omega
      rw [if_pos ((hc r i).mpr h1), if_pos ((hc r j).mpr h2), if_pos h1, if_pos h2]
      simp
    · have h1' : i ∉ c r := fun h' => h1 ((hc r i).mp h')
      by_cases h2 : r.val < j.val
      · rw [if_neg h1', if_pos ((hc r j).mpr h2), if_neg h1, if_pos h2]
        simp
      · have h2' : j ∉ c r := fun h' => h2 ((hc r j).mp h')
        rw [if_neg h1', if_neg h2', if_neg h1, if_neg h2]
        simp
  have hdle : ∀ i j : Fin n, i.val ≤ j.val →
     |a i - a j| = ∑ r, |(if i ∈ c r then (1:ℝ) else 0) - (if j ∈ c r then (1:ℝ) else 0)| * g r.val := by
    intro i j hij
    rw [Finset.sum_congr rfl (fun r _ => hterm r i j hij), Finset.sum_sub_distrib,
      hsum j.val j.isLt, hsum i.val i.isLt, hFa i, hFa j]
    have hij' : a i ≤ a j := ha (Fin.le_iff_val_le_val.mpr hij)
    rw [abs_of_nonpos (show a i - a j ≤ 0 by linarith)]
    ring
  refine ⟨c, fun r => g r.val, a z0, hgap, ?_, hle z0, hrec, ?_⟩
  · have e1 : ∑ r : Fin n, g r.val = ∑ r : Fin n, (if r.val < n - 1 then g r.val else 0) := by
      apply Finset.sum_congr rfl
      intro r _
      by_cases h : r.val < n - 1
      · rw [if_pos h]
      · rw [if_neg h, hg, if_neg (show ¬ (r.val + 1 < n) by omega)]
    show ∑ r : Fin n, g r.val ≤ 2 * ∑ i, |a i|
    rw [e1, hsum (n - 1) (by omega), hFl, hF0]
    have h1 := hle last
    have h2 := hle z0
    have h3 := le_abs_self (a last)
    have h4 := neg_abs_le (a z0)
    linarith
  · show ∀ i j : Fin n, |a i - a j|
      = ∑ r, |(if i ∈ c r then (1:ℝ) else 0) - (if j ∈ c r then (1:ℝ) else 0)| * g r.val
    intro i j
    rcases le_total i.val j.val with h | h
    · exact hdle i j h
    · rw [abs_sub_comm (a i) (a j), hdle j i h]
      apply Finset.sum_congr rfl
      intro r _
      rw [abs_sub_comm]
```

Proof. If $n = 0$, everything is empty. Otherwise take: - $c_r = \{i : r < i\}$ (an upper cut); - $\mathrm{gap}_r = a_{r+1} - a_r$ for $r + 1 < n$ and $\mathrm{gap}_{n-1} = 0$; - $v_0 = a_0$.

By monotonicity, $\mathrm{gap}_r \geq 0$. By induction on m, $\sum_{r<m} \mathrm{gap}_r = a_m - a_0$ for $m < n$, and $\sum_r \mathbf{1}[i \in c_r]\mathrm{gap}_r = \sum_{r<i} \mathrm{gap}_r$. This gives (iv).

For (ii), $\sum_r \mathrm{gap}_r = a_{n-1} - a_0 \leq |a_{n-1}| + |a_0| \leq 2\sum |a_i|$. Item (iii) is clear.

For (v), assume $i \leq j$. Then

$$|\mathbf{1}[i \in c_r] - \mathbf{1}[j \in c_r]|\mathrm{gap}_r = \mathbf{1}[r < j]\mathrm{gap}_r - \mathbf{1}[r < i]\mathrm{gap}_r,$$

and the sum is $a_j - a_i = |a_i - a_j|$. The case $j \leq i$ follows by symmetry. □

Lemma 3.19 (mc_cut_real_pointwise)**.** *The conclusion of Lemma 3.18 holds for every $a : \mathrm{Fin}\, n \to \mathbb{R}$, without the monotonicity assumption.*

Lean: mc_cut_real_pointwise — proved by the captain after a hand-back

```
theorem mc_cut_real_pointwise (n : ℕ) (a : Fin n → ℝ) :
    ∃ (c : Fin n → Finset (Fin n)) (gap : Fin n → ℝ) (v0 : ℝ),
      (∀ r, 0 ≤ gap r) ∧ ∑ r, gap r ≤ 2 * ∑ i, |a i| ∧ |v0| ≤ ∑ i, |a i| ∧
      (∀ i, a i = v0 + ∑ r, (if i ∈ c r then (1:ℝ) else 0) * gap r) ∧
      (∀ i j, |a i - a j| = ∑ r, |(if i ∈ c r then (1:ℝ) else 0) - (if j ∈ c r then (1:ℝ) else 0)| * gap
↪  r) := by
  obtain ⟨σ, hmono⟩ : ∃ σ : Equiv.Perm (Fin n), Monotone (a ∘ σ) :=
    ⟨Tuple.sort a, Tuple.monotone_sort a⟩
  obtain ⟨c, gap, v0, hg, hsum, hv, hrep, hdist⟩ := mc_cut_real_mono n (a ∘ σ) hmono
  have hperm : ∑ i, |(a ∘ σ) i| = ∑ i, |a i| := Equiv.sum_comp σ (fun i => |a i|)
  refine ⟨fun r => Finset.univ.filter (fun i => σ.symm i ∈ c r), gap, v0, hg, ?_, ?_, ?_, ?_⟩
  · linarith [hsum, hperm]
  · linarith [hv, hperm]
  · intro i
    have h := hrep (σ.symm i)
    simp only [Function.comp_apply, Equiv.apply_symm_apply] at h
    simp only [Finset.mem_filter, Finset.mem_univ, true_and]
    exact h
  · intro i j
    have h := hdist (σ.symm i) (σ.symm j)
    simp only [Function.comp_apply, Equiv.apply_symm_apply] at h
    simp only [Finset.mem_filter, Finset.mem_univ, true_and]
    exact h
```

Proof. Mathlib's `Tuple.sort` provides a permutation σ of $\mathrm{Fin}\, n$ with $a \circ \sigma$ monotone. Apply Lemma 3.18 to $a \circ \sigma$ and replace c_r by $\{i : \sigma^{-1}(i) \in c_r\}$. Items (iv) and (v) follow by evaluating at $\sigma^{-1}(i)$ and $\sigma^{-1}(j)$. The sum $\sum_i |a_i|$ is invariant under σ. □

Lemma 3.20 (mc_cut_real)**.** *For every n there are functions $g : \mathcal{P}(\mathrm{Fin}\, n) \times \mathbb{R}^{\mathrm{Fin}\, n} \to \mathbb{R}$, written $(B, a) \mapsto g_B(a)$, and $v : \mathbb{R}^{\mathrm{Fin}\, n} \to \mathbb{R}$ such that for every $a \in \mathbb{R}^{\mathrm{Fin}\, n}$:*

1. *$g_B(a) \geq 0$ for every B;*
2. *$g_B(a) \leq 2\sum_i |a_i|$ for every B;*
3. *$|v(a)| \leq \sum_i |a_i|$;*
4. *$a_i = v(a) + \sum_{B \in \mathcal{P}(\mathrm{Fin}\, n)} \mathbf{1}[i \in B] g_B(a)$ for all i;*
5. *$|a_i - a_j| = \sum_{B \in \mathcal{P}(\mathrm{Fin}\, n)} |\mathbf{1}[i \in B] - \mathbf{1}[j \in B]|\, g_B(a)$ for all i, j.*

Lean: `mc_cut_real` — proved by the captain after a hand-back

```
theorem mc_cut_real (n : ℕ) :
    ∃ (g : Finset (Fin n) → (Fin n → ℝ) → ℝ) (v : (Fin n → ℝ) → ℝ),
      ∀ a : Fin n → ℝ,
        (∀ B, 0 ≤ g B a) ∧ (∀ B, g B a ≤ 2 * ∑ i, |a i|) ∧ |v a| ≤ ∑ i, |a i| ∧
        (∀ i, a i = v a + ∑ B, (if i ∈ B then (1:ℝ) else 0) * g B a) ∧
        (∀ i j, |a i - a j| = ∑ B, |(if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0)| * g B a)
↪  := by
  choose c gap v0 hgap0 hgapsum hv0 hrec hdist using fun a : Fin n → ℝ => mc_cut_real_pointwise n a
  have key : ∀ (a : Fin n → ℝ) (f : Finset (Fin n) → ℝ),
      ∑ B, f B * (∑ r, if B = c a r then gap a r else 0) = ∑ r, f (c a r) * gap a r := by
    intro a f
    simp_rw [Finset.mul_sum, mul_ite, mul_zero]
    rw [Finset.sum_comm]
    simp [Finset.sum_ite_eq']
  refine ⟨fun B a => ∑ r, if B = c a r then gap a r else 0, v0, fun a => ?_⟩
  refine ⟨?_, ?_, hv0 a, ?_, ?_⟩
  · intro B
    exact Finset.sum_nonneg (fun r _ => by
      split_ifs
      · exact hgap0 a r
      · exact le_rfl)
  · intro B
    refine le_trans (Finset.sum_le_sum fun r _ => ?_) (hgapsum a)
    split_ifs
    · exact le_rfl
    · exact hgap0 a r
  · intro i
    exact (hrec a i).trans
      (congrArg (v0 a + ·) (key a (fun B => if i ∈ B then (1:ℝ) else 0)).symm)
  · intro i j
    exact (hdist a i j).trans
      (key a (fun B => |(if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0)|)).symm
```

Proof. For each a, choose $c^a, \mathrm{gap}^a, v_0^a$ as in Lemma 3.19. Set

$$v(a) = v_0^a, \qquad g_B(a) = \sum_r \mathbf{1}[B = c_r^a]\,\mathrm{gap}_r^a.$$

Then for every $f : \mathcal{P}(\mathrm{Fin}\, n) \to \mathbb{R}$,

$$\sum_B f(B) g_B(a) = \sum_r f(c_r^a)\mathrm{gap}_r^a.$$

Items (iv) and (v) follow from the corresponding items of Lemma 3.19. Item (i) is clear, and

$$g_B(a) \le \sum_r \mathrm{gap}_r^a \le 2 \sum |a_i|. \qquad \square$$

Lemma 3.21 (`mc_L1_norm`). *For $f \in \ell_1$, the sequence $(|f(k)|)_k$ is summable and $\|f\| = \sum_{k\ge 0} |f(k)|$.*

Lean: mc_L1_norm — proved by AIProver, job 1, session 0

```
theorem mc_L1_norm (f : L1) :
    Summable (fun k : ℕ => |f k|) ∧ ‖f‖ = ∑' k : ℕ, |f k| := by
  have hp : 0 < ENNReal.toReal (1 : ENNReal) := by norm_num
  have hmem : Memℓp (f : ℕ → ℝ) (1 : ENNReal) := lp.memℓp f
  have hsumm : Summable (fun i : ℕ => |f i|) := by
    have := (memℓp_gen_iff hp).mp hmem
    have h1 : ENNReal.toReal (1 : ENNReal) = (1 : ℝ) := by norm_num
    simpa [h1] using this
  have hnorm : ‖f‖ = ∑' k : ℕ, |f k| := by
    rw [lp.norm_eq_tsum_rpow hp f]
    have h1 : ENNReal.toReal (1 : ENNReal) = (1 : ℝ) := by norm_num
    rw [h1]
    simp
  exact And.intro hsumm hnorm
```

Proof. This is Mathlib's characterization of membership in ℓ^p and of the ℓ^p norm, specialised to $p = 1$. □

Lemma 3.22 (mc_L1_of_bound)**.** *Let $f, g : \mathbb{N}_0 \to \mathbb{R}$ with g summable and $|f(k)| \le g(k)$ for all k. Then there is $F \in \ell_1$ with $F(k) = f(k)$ for all k.*

Lean: mc_L1_of_bound — proved by AIProver, job 2, session 0

```
theorem mc_L1_of_bound (f g : ℕ → ℝ) (hg : Summable g) (hfg : ∀ k, |f k| ≤ g k) :
    ∃ F : L1, ∀ k, F k = f k := by
  have h_abs_summable : Summable fun k : ℕ => |f k| :=
    Summable.of_nonneg_of_le (fun _k => abs_nonneg _) hfg hg
  have h_mem : Memℓp f (1 : ENNReal) := by
    have h_norm_eq : ∀ k, ‖f k‖ = |f k| := fun _k => by simp
    have : Summable (fun k : ℕ => ‖f k‖ ^ (1 : ENNReal).toReal) := by
      simpa [ENNReal.toReal_one, h_norm_eq] using h_abs_summable
    exact memℓp_gen this
  refine ⟨⟨f, h_mem⟩, fun _k => rfl⟩
```

Proof. By comparison, $\sum |f(k)|$ converges, so f belongs to ℓ^1. □

Lemma 3.23 (mc_L1_affine_apply)**.** *Let ι be finite, $V \in \ell_1$, $b : \iota \to \ell_1$, $u \in \mathbb{R}^\iota$ and $k \in \mathbb{N}_0$. Then*

$$\Big(V + \sum_B u_B b_B\Big)(k) = V(k) + \sum_B u_B b_B(k).$$

Lean: mc_L1_affine_apply — proved by AIProver, job 1, session 0

```
theorem mc_L1_affine_apply {ι : Type*} [Fintype ι] (V : L1) (b : ι → L1) (u : ι → ℝ) (k : ℕ) :
    (V + ∑ B, u B • b B) k = V k + ∑ B, u B * b B k := by
  simp [lp.coeFn_smul, Finset.sum_apply, smul_eq_mul]
```

Proof. Evaluation at k is linear. □

Lemma 3.24 (mc_L1_affine_contr)**.** *With ι, V, b as in Lemma 3.23 and $u, u' \in \mathbb{R}^\iota$,*

$$\Big\|\Big(V + \sum_B u_B b_B\Big) - \Big(V + \sum_B u'_B b_B\Big)\Big\| \le \sum_B \|b_B\|\,|u_B - u'_B|.$$

Lean: mc_L1_affine_contr — proved by AIProver, job 1, session 0

```
theorem mc_L1_affine_contr {ι : Type*} [Fintype ι] (V : L1) (b : ι → L1) (u u' : ι → ℝ) :
    ‖(V + ∑ B, u B • b B) - (V + ∑ B, u' B • b B)‖ ≤ ∑ B, ‖b B‖ * |u B - u' B| := by
  have hsub : (V + ∑ B, u B • b B) - (V + ∑ B, u' B • b B) = ∑ B, (u B - u' B) • b B := by
    simp [sub_smul, Finset.sum_sub_distrib]
  rw [hsub]
  calc
    ‖∑ B, (u B - u' B) • b B‖ ≤ ∑ B, ‖(u B - u' B) • b B‖ := norm_sum_le _ _
    _ ≤ ∑ B, (‖(u B - u' B : ℝ)‖ * ‖b B‖) := by
      refine Finset.sum_le_sum fun i _ => ?_
      exact norm_smul_le _ _
    _ = ∑ B, |u B - u' B| * ‖b B‖ := by
      refine Finset.sum_congr rfl fun i _ => ?_
      rw [Real.norm_eq_abs]
    _ = ∑ B, ‖b B‖ * |u B - u' B| := by
      simp [mul_comm]
```

Proof. The difference is $\sum_B (u_B - u'_B) b_B$. Apply the triangle inequality and homogeneity of the norm. □

Lemma 3.25 (mc_L1_dist_eq_aiprover_s0)**.** *Let ι be finite, $x : \operatorname{Fin} n \to \ell_1$, and $b : \iota \to \ell_1$ with $b_B(k) \geq 0$ for all B, k. Let $z : \operatorname{Fin} n \to \mathbb{R}^\iota$ satisfy*

$$|x_i(k) - x_j(k)| = \sum_B |z_i(B) - z_j(B)|\, b_B(k) \qquad \text{for all } i, j, k.$$

Then $\|x_i - x_j\| = \sum_B \|b_B\|\, |z_i(B) - z_j(B)|$ for all i, j.

Lean: mc_L1_dist_eq_aiprover_s0 — helper of mc_L1_dist_eq, proved by AIProver, job 2, session 0

```
open ENNReal MarkovCotype in
theorem mc_L1_dist_eq_aiprover_s0 {n : ℕ} {ι : Type*} [Fintype ι] (x : Fin n → L1) (b : ι → L1)
    (hb : ∀ B k, 0 ≤ b B k) (z : Fin n → ι → ℝ)
    (hrep : ∀ i j k, |x i k - x j k| = ∑ B, |z i B - z j B| * b B k) (i j : Fin n) :
    ‖x i - x j‖ = ∑ B, ‖b B‖ * |z i B - z j B| := by
  have hp : 0 < (1 : ℝ≥0∞).toReal := by simp
  have hnorm := fun (f : L1) => (mc_L1_norm f).2
  have hsum_b (B : ι) : Summable (fun k : ℕ => b B k) := by
    have := (mc_L1_norm (b B)).1



    have hpos := hb B



    have hpos' : ∀ k, 0 ≤ b B k := hpos
    refine Summable.of_nonneg_of_le hpos' (fun k => ?_) this
    rw [abs_of_nonneg (hpos k)]
  have hsum_inner (B : ι) : Summable (fun k : ℕ => |z i B - z j B| * b B k) :=
    Summable.mul_left (|z i B - z j B|) (hsum_b B)
  calc
    ‖x i - x j‖ = (∑' k : ℕ, ‖(x i - x j) k‖) := by
      rw [lp.norm_eq_tsum_rpow hp (x i - x j)]
      simp [toReal_one]
```

```
    _ = (∑' k : ℕ, |x i k - x j k|) := by
      simp [Real.norm_eq_abs]
    _ = (∑' k : ℕ, ∑ B, |z i B - z j B| * b B k) := by
      simp_rw [hrep i j]
    _ = ∑ B, (∑' k : ℕ, |z i B - z j B| * b B k) := by
      have h := Summable.tsum_finsetSum (s := Finset.univ) (fun B _ => hsum_inner B)
      simpa using h
    _ = ∑ B, |z i B - z j B| * (∑' k : ℕ, b B k) := by
      refine Finset.sum_congr rfl fun B _ => ?_
      simp [tsum_mul_left]
    _ = ∑ B, |z i B - z j B| * ‖b B‖ := by
      refine Finset.sum_congr rfl fun B _ => ?_
      have h := hnorm (b B)
      have hsum_eq : (∑' k : ℕ, |(b B : ℕ → ℝ) k|) = (∑' k : ℕ, (b B : ℕ → ℝ) k) := by
        refine tsum_congr (fun k => ?_)
        rw [abs_of_nonneg (hb B k)]
      rw [h, hsum_eq]
    _ = ∑ B, ‖b B‖ * |z i B - z j B| := by
      refine Finset.sum_congr rfl fun B _ => ?_
      rw [mul_comm]
```

Proof. By Lemma 3.21 and the hypothesis,

$$\|x_i - x_j\| = \sum_k |x_i(k) - x_j(k)| = \sum_k \sum_B |z_i(B) - z_j(B)|\, b_B(k).$$

Each series $\sum_k b_B(k)$ converges, since $b_B \in \ell_1$. So the finite sum over B and the series over k may be interchanged. Finally, $\sum_k b_B(k) = \sum_k |b_B(k)| = \|b_B\|$ by nonnegativity. □

Lemma 3.26 (mc_L1_dist_eq)**.** *Same statement as Lemma 3.25.*

Lean: mc_L1_dist_eq — proved by AIProver, job 2, session 0

```
theorem mc_L1_dist_eq {n : ℕ} {ι : Type*} [Fintype ι] (x : Fin n → L1) (b : ι → L1)
    (hb : ∀ B k, 0 ≤ b B k) (z : Fin n → ι → ℝ)
    (hrep : ∀ i j k, |x i k - x j k| = ∑ B, |z i B - z j B| * b B k) (i j : Fin n) :
    ‖x i - x j‖ = ∑ B, ‖b B‖ * |z i B - z j B| := by
  first
    | exact mc_L1_dist_eq_aiprover_s0
    | (intros; apply mc_L1_dist_eq_aiprover_s0 <;> assumption)
```

Proof. This is Lemma 3.25. □

Lemma 3.27 (mc_cut_coords)**.** *Let $x : \mathrm{Fin}\, n \to \ell_1$. Then there exist $b : \mathcal{P}(\mathrm{Fin}\, n) \to \ell_1$ and $V \in \ell_1$ such that for all B, i, j, k: - $b_B(k) \geq 0$; - $x_i(k) = V(k) + \sum_B \mathbf{1}[i \in B] b_B(k)$; - $|x_i(k) - x_j(k)| = \sum_B |\mathbf{1}[i \in B] - \mathbf{1}[j \in B]|\, b_B(k)$.*

Lean: mc_cut_coords — proved by the captain after a hand-back

```
theorem mc_cut_coords {n : ℕ} (x : Fin n → L1) :
    ∃ (b : Finset (Fin n) → L1) (V : L1),
      (∀ B k, 0 ≤ b B k) ∧
      (∀ i k, x i k = V k + ∑ B, (if i ∈ B then (1:ℝ) else 0) * b B k) ∧
      (∀ i j k, |x i k - x j k|
        = ∑ B, |(if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0)| * b B k) := by
  obtain ⟨g, v, hgv⟩ := mc_cut_real n
  have hS : Summable (fun k : ℕ => ∑ i, |x i k|) := by
    first
    | exact summable_sum (fun i _ => (mc_L1_norm (x i)).1)
    | exact Summable.sum (fun i _ => (mc_L1_norm (x i)).1)
    | (have key : ∀ s : Finset (Fin n), Summable (fun k : ℕ => ∑ i ∈ s, |x i k|) := by
         intro s
         refine Finset.induction_on s ?_ ?_
         · simp
         · intro a s ha ih
           simpa [Finset.sum_insert ha] using ((mc_L1_norm (x a)).1).add ih
       exact key Finset.univ)
  choose b hb using fun B : Finset (Fin n) =>
    mc_L1_of_bound (fun k => g B (fun i => x i k)) (fun k => 2 * ∑ i, |x i k|) (hS.mul_left 2)
      (fun k => (abs_of_nonneg ((hgv (fun i => x i k)).1 B)).le.trans ((hgv (fun i => x i k)).2.1 B))
  obtain ⟨V, hV⟩ := mc_L1_of_bound (fun k => v (fun i => x i k)) (fun k => ∑ i, |x i k|) hS
    (fun k => (hgv (fun i => x i k)).2.2.1)
  refine ⟨b, V, ?_, ?_, ?_⟩
  · intro B k
    rw [hb B k]
    exact (hgv (fun i => x i k)).1 B
  · intro i k
    rw [hV k]
    simp only [hb]
    exact (hgv (fun i => x i k)).2.2.2.1 i
  · intro i j k
    simp only [hb]
    exact (hgv (fun i => x i k)).2.2.2.2 i j
```

Proof. Let g, v be as in Lemma 3.20 and write $a^{(k)} = (x_i(k))_i$. Define

$$b_B(k) = g_B(a^{(k)}), \qquad V(k) = v(a^{(k)}).$$

The sequence $k \mapsto \sum_i |x_i(k)|$ is summable, as a finite sum of summable sequences (Lemma 3.21). By Lemma 3.20 (i)–(iii),

$$|b_B(k)| \le 2 \sum_i |x_i(k)| \qquad \text{and} \qquad |V(k)| \le \sum_i |x_i(k)|.$$

Hence $b_B, V \in \ell_1$ by Lemma 3.22. The three properties are items (i), (iv) and (v) of Lemma 3.20, applied to $a^{(k)}$. □

Lemma 3.28 (mc_cut_L1)**.** *Let $x : \mathrm{Fin}\, n \to \ell_1$, and for $i \in \mathrm{Fin}\, n$ let $z_i \in \{0,1\}^{\mathcal{P}(\mathrm{Fin}\, n)}$ be given by $z_i(B) = \mathbf{1}[i \in B]$. Then there exist $w : \mathcal{P}(\mathrm{Fin}\, n) \to \mathbb{R}$ and $T : \mathbb{R}^{\mathcal{P}(\mathrm{Fin}\, n)} \to \ell_1$ such that: - $w_B \ge 0$ for all B; - $T(z_i) = x_i$ for all i; - $\|x_i - x_j\| = \|z_i - z_j\|_{1,w}$ for all i, j; - $\|T(u) - T(u')\| \le \|u - u'\|_{1,w}$ for all $u, u' \in \mathbb{R}^{\mathcal{P}(\mathrm{Fin}\, n)}$.*

Lean: mc_cut_L1 — proved by the captain after a hand-back

```
theorem mc_cut_L1 {n : ℕ} (x : Fin n → L1) :
    ∃ (w : Finset (Fin n) → ℝ) (T : (Finset (Fin n) → ℝ) → L1),
      (∀ B, 0 ≤ w B) ∧
      (∀ i, T (fun B => if i ∈ B then (1:ℝ) else 0) = x i) ∧
      (∀ i j, ‖x i - x j‖ = ∑ B, w B * |(if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0)|) ∧
      (∀ u u', ‖T u - T u'‖ ≤ ∑ B, w B * |u B - u' B|) := by
  obtain ⟨b, V, hb, hx, hrep⟩ := mc_cut_coords x
  refine ⟨fun B => ‖b B‖, fun u => V + ∑ B, u B • b B, fun B => norm_nonneg _, ?_, ?_, ?_⟩
  · intro i
    have h : ∀ k, (V + ∑ B, (fun B : Finset (Fin n) => if i ∈ B then (1:ℝ) else 0) B • b B) k = x i k := by
      intro k
      first
        | (rw [mc_L1_affine_apply, hx i k])
        | (simp only [mc_L1_affine_apply, hx i k])
        | (rw [mc_L1_affine_apply]; exact (hx i k).symm)
    first
      | exact lp.ext (funext h)
      | exact Subtype.ext (funext h)
  · intro i j
    exact mc_L1_dist_eq x b hb (fun i B => if i ∈ B then (1:ℝ) else 0) hrep i j
  · intro u u'
    exact mc_L1_affine_contr V b u u'
```

Proof. Take b, V from Lemma 3.27 and set

$$w_B = \|b_B\|, \qquad T(u) = V + \sum_B u_B b_B.$$

- $T(z_i) = x_i$ coordinatewise, by Lemma 3.23. - The distance identity is Lemma 3.26. - The contraction property is Lemma 3.24. □

Lemma 3.29 (mc_transfer). *Let $t \in \mathbb{N}_0$, $\pi \geq 0$, and let A have nonnegative entries. Let $x : \mathrm{Fin}\, n \to \ell_1$, ι finite, $w \geq 0$ on ι, and $z : \mathrm{Fin}\, n \to \{0,1\}^\iota$. Let $T : \mathbb{R}^\iota \to \ell_1$ satisfy - $T(z_i) = x_i$; - $\|x_i - x_j\| = \|z_i - z_j\|_{1,w}$ for all i, j; - $\|T(u) - T(u')\| \leq \|u - u'\|_{1,w}$ for all u, u'.*

Let G be any real matrix and $c : \mathrm{Fin}\, n \to \mathbb{R}^\iota$ such that

$$\sum_i \pi_i \|z_i - c_i\|_{1,w}^2 + t \sum_{i,j} \pi_i A_{ij} \|c_i - c_j\|_{1,w}^2 \leq 108 \sum_{i,j} \pi_i G_{ij} \|z_i - z_j\|_{2,w}^4.$$

Then there is $y : \mathrm{Fin}\, n \to \ell_1$ with

$$\sum_i \pi_i \|x_i - y_i\|^2 + t \sum_{i,j} \pi_i A_{ij} \|y_i - y_j\|^2 \leq 108 \sum_{i,j} \pi_i G_{ij} \|x_i - x_j\|^2.$$

Lean: mc_transfer — proved by AIProver, job 1, session 3

```
theorem mc_transfer {n : ℕ} (t : ℕ) (π : Fin n → ℝ) (hπ : ∀ i, 0 ≤ π i)
    (A : Matrix (Fin n) (Fin n) ℝ) (hA0 : ∀ i j, 0 ≤ A i j) (x : Fin n → L1)
    {ι : Type*} [Fintype ι] (w : ι → ℝ) (hw : ∀ b, 0 ≤ w b)
    (z : Fin n → ι → ℝ) (hz : ∀ i b, z i b = 0 ∨ z i b = 1)
    (T : (ι → ℝ) → L1) (hT : ∀ i, T (z i) = x i)
    (hdist : ∀ i j, ‖x i - x j‖ = ∑ b, w b * |z i b - z j b|)
    (hcontr : ∀ u u', ‖T u - T u'‖ ≤ ∑ b, w b * |u b - u' b|)
```

```
    (G : Matrix (Fin n) (Fin n) ℝ) (c : Fin n → ι → ℝ)
    (hcube : ∑ i, π i * (∑ b, w b * |z i b - c i b|) ^ 2
      + (t : ℝ) * ∑ i, ∑ j, π i * A i j * (∑ b, w b * |c i b - c j b|) ^ 2
      ≤ 108 * ∑ i, ∑ j, π i * G i j * (∑ b, w b * (z i b - z j b) ^ 2) ^ 2) :
    ∃ y : Fin n → L1,
      ∑ i, π i * ‖x i - y i‖ ^ 2 + (t : ℝ) * ∑ i, ∑ j, π i * A i j * ‖y i - y j‖ ^ 2
        ≤ 108 * ∑ i, ∑ j, π i * G i j * ‖x i - x j‖ ^ 2 := by
  have h_binary_eq (a b : ℝ) (ha : a = 0 ∨ a = 1) (hb : b = 0 ∨ b = 1) : |a - b| = (a - b) ^ 2 := by
    rcases ha with (rfl | rfl) <;> rcases hb with (rfl | rfl) <;> norm_num
  have h_sum_eq (i j : Fin n) : ∑ b : ι, w b * |z i b - z j b| = ∑ b : ι, w b * (z i b - z j b) ^ 2 := by
    refine Finset.sum_congr rfl fun b _ => ?_
    rw [h_binary_eq (z i b) (z j b) (hz i b) (hz j b)]
  refine ⟨fun i => T (c i), ?_⟩
  have h_nonneg_sum (u v : ι → ℝ) : 0 ≤ ∑ b : ι, w b * |u b - v b| :=
    Finset.sum_nonneg fun b _ => mul_nonneg (hw b) (abs_nonneg _)
  have h1 (i : Fin n) : ‖x i - T (c i)‖ ≤ ∑ b : ι, w b * |z i b - c i b| := by
    rw [← hT i]
    exact hcontr (z i) (c i)
  have h2 (i j : Fin n) : ‖T (c i) - T (c j)‖ ≤ ∑ b : ι, w b * |c i b - c j b| :=
    hcontr (c i) (c j)
  have h1_sq (i : Fin n) : ‖x i - T (c i)‖ ^ 2 ≤ (∑ b : ι, w b * |z i b - c i b|) ^ 2 := by
    have h := mul_self_le_mul_self (norm_nonneg _) (h1 i)
    rw [sq, sq]
    exact h
  have h2_sq (i j : Fin n) : ‖T (c i) - T (c j)‖ ^ 2 ≤ (∑ b : ι, w b * |c i b - c j b|) ^ 2 := by
    have h := mul_self_le_mul_self (norm_nonneg _) (h2 i j)
    rw [sq, sq]
    exact h
  calc
    ∑ i : Fin n, π i * ‖x i - T (c i)‖ ^ 2 + (t : ℝ) * ∑ i : Fin n, ∑ j : Fin n, π i * A i j * ‖T (c i)
↪   - T (c j)‖ ^ 2
        ≤ ∑ i : Fin n, π i * (∑ b : ι, w b * |z i b - c i b|) ^ 2
          + (t : ℝ) * ∑ i : Fin n, ∑ j : Fin n, π i * A i j * (∑ b : ι, w b * |c i b - c j b|) ^ 2 := by
      refine add_le_add ?_ ?_
      · refine Finset.sum_le_sum fun i _ => ?_
        have h_nonneg_pi : 0 ≤ π i := hπ i
        have h_sq := h1_sq i
        nlinarith
      · have h_nonneg_t : 0 ≤ (t : ℝ) := Nat.cast_nonneg _
        refine mul_le_mul_of_nonneg_left ?_ h_nonneg_t
        refine Finset.sum_le_sum fun i _ => ?_
        refine Finset.sum_le_sum fun j _ => ?_
        have h_nonneg_piA : 0 ≤ π i * A i j := mul_nonneg (hπ i) (hA0 i j)
        have h_sq := h2_sq i j
        nlinarith
    _ ≤ 108 * ∑ i : Fin n, ∑ j : Fin n, π i * G i j * (∑ b : ι, w b * (z i b - z j b) ^ 2) ^ 2 := hcube
    _ = 108 * ∑ i : Fin n, ∑ j : Fin n, π i * G i j * (‖x i - x j‖) ^ 2 := by
      simp only [h_sum_eq, hdist]
```

Proof. Take $y_i = T(c_i)$. By the contraction property,

$$\|x_i - y_i\| = \|T(z_i) - T(c_i)\| \le \|z_i - c_i\|_{1,w} \qquad \text{and} \qquad \|y_i - y_j\| \le \|c_i - c_j\|_{1,w}.$$

Square these, which is allowed since both sides are nonnegative. Multiply by $\pi_i \ge 0$, resp. by $t\,\pi_i A_{ij} \ge 0$, and sum; the left side is then bounded by the left side of the hypothesis.

For bits α, β we have $|\alpha - \beta| = (\alpha - \beta)^2$. Hence $\|z_i - z_j\|_{2,w}^2 = \|z_i - z_j\|_{1,w} = \|x_i - x_j\|$, and the right side of the hypothesis equals $108 \sum \pi_i G_{ij} \|x_i - x_j\|^2$. □

Lemma 3.30 (mc_minkowski). *Let α be finite, $\mu \ge 0$ on α, and $a, b \in \mathbb{R}^\alpha$. Then*

$$\Big(\sum_k \mu_k (a_k + b_k)^2\Big)^{1/2} \le \Big(\sum_k \mu_k a_k^2\Big)^{1/2} + \Big(\sum_k \mu_k b_k^2\Big)^{1/2}.$$

Lean: mc_minkowski — proved by AIProver, job 1, session 0

```
theorem mc_minkowski {α : Type*} [Fintype α] (μ a b : α → ℝ) (hμ : ∀ k, 0 ≤ μ k) :
    Real.sqrt (∑ k, μ k * (a k + b k) ^ 2)
      ≤ Real.sqrt (∑ k, μ k * a k ^ 2) + Real.sqrt (∑ k, μ k * b k ^ 2) := by
  have hp : (1 : ℝ) ≤ 2 := by norm_num
  set f : α → ℝ := fun k => Real.sqrt (μ k) * a k
  set g : α → ℝ := fun k => Real.sqrt (μ k) * b k
  have hsum_add : (∑ k, μ k * (a k + b k) ^ 2) = ∑ k, (f k + g k) ^ 2 := by
    refine Finset.sum_congr rfl fun k _ => ?_
    dsimp [f, g]
    calc
      μ k * (a k + b k) ^ 2 = (Real.sqrt (μ k)) ^ 2 * (a k + b k) ^ 2 := by
        rw [Real.sq_sqrt (hμ k)]
      _ = ((Real.sqrt (μ k)) * (a k + b k)) ^ 2 := by ring
      _ = ((Real.sqrt (μ k)) * a k + (Real.sqrt (μ k)) * b k) ^ 2 := by ring
  have hsum_a : (∑ k, μ k * a k ^ 2) = ∑ k, (f k) ^ 2 := by
    refine Finset.sum_congr rfl fun k _ => ?_
    dsimp [f]
    calc
      μ k * a k ^ 2 = (Real.sqrt (μ k)) ^ 2 * a k ^ 2 := by rw [Real.sq_sqrt (hμ k)]
      _ = ((Real.sqrt (μ k)) * a k) ^ 2 := by ring
  have hsum_b : (∑ k, μ k * b k ^ 2) = ∑ k, (g k) ^ 2 := by
    refine Finset.sum_congr rfl fun k _ => ?_
    dsimp [g]
    calc
      μ k * b k ^ 2 = (Real.sqrt (μ k)) ^ 2 * b k ^ 2 := by rw [Real.sq_sqrt (hμ k)]
      _ = ((Real.sqrt (μ k)) * b k) ^ 2 := by ring
      _ = (g k) ^ 2 := rfl
  rw [hsum_add, hsum_a, hsum_b]
  rw [Real.sqrt_eq_rpow, Real.sqrt_eq_rpow, Real.sqrt_eq_rpow]
  have hLp_raw := Real.Lp_add_le (Finset.univ : Finset α) f g hp
  have hLp : (∑ k, |f k + g k| ^ 2) ^ (1 / (2 : ℝ)) ≤
            (∑ k, |f k| ^ 2) ^ (1 / (2 : ℝ)) + (∑ k, |g k| ^ 2) ^ (1 / (2 : ℝ)) := by
    simpa [Real.rpow_two] using hLp_raw
  have hsum_eq : ∀ (h : α → ℝ), (∑ k, h k ^ 2) = (∑ k, |h k| ^ 2) := by
    intro h; refine Finset.sum_congr rfl fun k _ => ?_
    rw [sq_abs]
  calc
    (∑ k, (f k + g k) ^ 2) ^ (1 / (2 : ℝ)) = (∑ k, |f k + g k| ^ 2) ^ (1 / (2 : ℝ)) := by
      simpa [Pi.add_apply] using congrArg (· ^ (1 / (2 : ℝ))) (hsum_eq (f + g))
    _ ≤ (∑ k, |f k| ^ 2) ^ (1 / (2 : ℝ)) + (∑ k, |g k| ^ 2) ^ (1 / (2 : ℝ)) := hLp
    _ = (∑ k, f k ^ 2) ^ (1 / (2 : ℝ)) + (∑ k, g k ^ 2) ^ (1 / (2 : ℝ)) := by
      rw [hsum_eq f, hsum_eq g]
```

Proof. Apply Mathlib's Minkowski inequality `Real.Lp_add_le`, with exponent 2, to the vectors $(\sqrt{\mu_k} a_k)_k$ and $(\sqrt{\mu_k} b_k)_k$. □

Lemma 3.31 (mc_weighted_tri). *Let α be finite, $\mu \ge 0$ on α, and $a, b, c \in \mathbb{R}^\alpha$ with $0 \le c_k \le a_k + b_k$ for all k. Then*

$$\Big(\sum_k \mu_k c_k^2\Big)^{1/2} \le \Big(\sum_k \mu_k a_k^2\Big)^{1/2} + \Big(\sum_k \mu_k b_k^2\Big)^{1/2}.$$

Lean: `mc_weighted_tri` — proved by AIProver, job 2, session 0

```
theorem mc_weighted_tri {α : Type*} [Fintype α] (μ a b c : α → ℝ) (hμ : ∀ k, 0 ≤ μ k)
    (hc : ∀ k, 0 ≤ c k) (hcab : ∀ k, c k ≤ a k + b k) :
    Real.sqrt (∑ k, μ k * c k ^ 2)
      ≤ Real.sqrt (∑ k, μ k * a k ^ 2) + Real.sqrt (∑ k, μ k * b k ^ 2) := by

  set A := Real.sqrt (∑ k, μ k * a k ^ 2) with hA_def
  set B := Real.sqrt (∑ k, μ k * b k ^ 2) with hB_def
  have hA_nonneg : 0 ≤ A := Real.sqrt_nonneg _
  have hB_nonneg : 0 ≤ B := Real.sqrt_nonneg _
  have hA_sq : A ^ 2 = ∑ k, μ k * a k ^ 2 :=
    Real.sq_sqrt (Finset.sum_nonneg (fun i _ => mul_nonneg (hμ i) (sq_nonneg _)))
  have hB_sq : B ^ 2 = ∑ k, μ k * b k ^ 2 :=
    Real.sq_sqrt (Finset.sum_nonneg (fun i _ => mul_nonneg (hμ i) (sq_nonneg _)))

  have h_nonneg_sum_c : 0 ≤ ∑ k, μ k * c k ^ 2 :=
    Finset.sum_nonneg (fun i _ => mul_nonneg (hμ i) (sq_nonneg _))
  have h_sq_goal : (Real.sqrt (∑ k, μ k * c k ^ 2)) ^ 2 ≤ (A + B) ^ 2 := by
    calc
      (Real.sqrt (∑ k, μ k * c k ^ 2)) ^ 2 = ∑ k, μ k * c k ^ 2 :=
        Real.sq_sqrt h_nonneg_sum_c
      _ ≤ (A + B) ^ 2 := by


        have hc_sq_bound : ∀ k, c k ^ 2 ≤ (|a k| + |b k|) ^ 2 := by
          intro k
          have hck : 0 ≤ c k := hc k
          have hle : c k ≤ |a k| + |b k| := by
            have hle' := hcab k
            have h_tri : a k + b k ≤ |a k| + |b k| := by
              have h1 : a k ≤ |a k| := le_abs_self (a k)
              have h2 : b k ≤ |b k| := le_abs_self (b k)
              linarith
            linarith
          have hsum_nonneg : 0 ≤ |a k| + |b k| := add_nonneg (abs_nonneg _) (abs_nonneg _)
          nlinarith

        have h_sum_c_sq_le : ∑ k, μ k * c k ^ 2 ≤ ∑ k, μ k * (|a k| + |b k|) ^ 2 :=
          Finset.sum_le_sum (fun k _ => mul_le_mul_of_nonneg_left (hc_sq_bound k) (hμ k))

        have h_expand : ∑ k, μ k * (|a k| + |b k|) ^ 2 =
            (∑ k, μ k * a k ^ 2) + 2 * (∑ k, μ k * |a k| * |b k|) + (∑ k, μ k * b k ^ 2) := by
          calc
            ∑ k, μ k * (|a k| + |b k|) ^ 2
                = ∑ k, μ k * (|a k| ^ 2 + 2 * |a k| * |b k| + |b k| ^ 2) := by
              refine Finset.sum_congr rfl (fun k _ => ?_)
              ring
            _ = (∑ k, μ k * |a k| ^ 2) + (∑ k, μ k * (2 * |a k| * |b k|)) + (∑ k, μ k * |b k| ^ 2) := by
              simp [Finset.sum_add_distrib, mul_add]
            _ = (∑ k, μ k * |a k| ^ 2) + 2 * (∑ k, μ k * |a k| * |b k|) + (∑ k, μ k * |b k| ^ 2) := by
              simp [Finset.mul_sum, mul_comm, mul_left_comm, mul_assoc]
            _ = (∑ k, μ k * a k ^ 2) + 2 * (∑ k, μ k * |a k| * |b k|) + (∑ k, μ k * b k ^ 2) := by
              simp [sq_abs]


        have h_cauchy : (∑ k, μ k * |a k| * |b k|) ^ 2 ≤ (∑ k, μ k * a k ^ 2) * (∑ k, μ k * b k ^ 2) := by
          have h_cs := Finset.sum_mul_sq_le_sq_mul_sq (Finset.univ : Finset α)
            (fun k => Real.sqrt (μ k) * |a k|) (fun k => Real.sqrt (μ k) * |b k|)
```

```
        have h_left : (∑ k : α, (Real.sqrt (μ k) * |a k|) * (Real.sqrt (μ k) * |b k|)) = ∑ k, μ k * |a
↪  k| * |b k| := by
          calc
            (∑ k : α, (Real.sqrt (μ k) * |a k|) * (Real.sqrt (μ k) * |b k|))
                = ∑ k, (Real.sqrt (μ k)) ^ 2 * |a k| * |b k| := by
              refine Finset.sum_congr rfl (fun k _ => ?_)
              ring
            _ = ∑ k, μ k * |a k| * |b k| := by
              simp [Real.sq_sqrt (hμ _)]
        have h_r1 : (∑ k : α, ((Real.sqrt (μ k) * |a k|)) ^ 2) = ∑ k, μ k * a k ^ 2 := by
          simp [mul_pow, Real.sq_sqrt (hμ _), sq_abs]
        have h_r2 : (∑ k : α, ((Real.sqrt (μ k) * |b k|)) ^ 2) = ∑ k, μ k * b k ^ 2 := by
          simp [mul_pow, Real.sq_sqrt (hμ _), sq_abs]
        simpa [h_left, h_r1, h_r2] using h_cs
      have h_main : ∑ k, μ k * |a k| * |b k| ≤ A * B := by
        have h_sq_le : (∑ k, μ k * |a k| * |b k|) ^ 2 ≤ (A * B) ^ 2 := by
          calc
            (∑ k, μ k * |a k| * |b k|) ^ 2 ≤ (∑ k, μ k * a k ^ 2) * (∑ k, μ k * b k ^ 2) := h_cauchy
            _ = A ^ 2 * B ^ 2 := by rw [hA_sq, hB_sq]
            _ = (A * B) ^ 2 := by ring
        have h_AB_nonneg : 0 ≤ A * B := mul_nonneg hA_nonneg hB_nonneg
        nlinarith

      calc
        ∑ k, μ k * c k ^ 2 ≤ ∑ k, μ k * (|a k| + |b k|) ^ 2 := h_sum_c_sq_le
        _ = (∑ k, μ k * a k ^ 2) + 2 * (∑ k, μ k * |a k| * |b k|) + (∑ k, μ k * b k ^ 2) := h_expand
        _ ≤ (∑ k, μ k * a k ^ 2) + 2 * (A * B) + (∑ k, μ k * b k ^ 2) := by
          nlinarith
        _ = A * B * 2 + A ^ 2 + B ^ 2 := by rw [hA_sq, hB_sq]; ring
        _ = (A + B) ^ 2 := by ring

  have h_sq_mono : (Real.sqrt (∑ k, μ k * c k ^ 2)) ^ 2 ≤ (A + B) ^ 2 →
      Real.sqrt (∑ k, μ k * c k ^ 2) ≤ A + B := by
    have h_left_nonneg : 0 ≤ Real.sqrt (∑ k, μ k * c k ^ 2) := Real.sqrt_nonneg _
    have h_right_nonneg : 0 ≤ A + B := add_nonneg hA_nonneg hB_nonneg
    intro h_sq
    nlinarith
  exact h_sq_mono h_sq_goal
```

Proof. Let A, B denote the two roots on the right. Since $0 \le c_k \le |a_k| + |b_k|$,

$$\sum_k \mu_k c_k^2 \le \sum_k \mu_k (|a_k| + |b_k|)^2 = A^2 + 2\sum_k \mu_k |a_k||b_k| + B^2.$$

By Cauchy–Schwarz with the weights $\sqrt{\mu_k}$, $\sum_k \mu_k |a_k||b_k| \le AB$. Hence $\sum_k \mu_k c_k^2 \le (A+B)^2$, and we take square roots, both sides being nonnegative. □

Lemma 3.32 (`mc_triple_mul`)**.** *For $\pi : \mathrm{Fin}\, n \to \mathbb{R}$, real matrices P, Q and $d : \mathrm{Fin}\, n \times \mathrm{Fin}\, n \to \mathbb{R}$,*

$$\sum_i \sum_j \pi_i (PQ)_{ij} d_{ij} = \sum_{(i,k,j) \in \mathrm{Fin}\, n^3} \pi_i P_{ik} Q_{kj} d_{ij}.$$

Lean: mc_triple_mul — proved by AIProver, job 2, session 0

```
theorem mc_triple_mul {n : ℕ} (π : Fin n → ℝ) (P Q : Matrix (Fin n) (Fin n) ℝ)
    (d : Fin n → Fin n → ℝ) :
    ∑ i, ∑ j, π i * (P * Q) i j * d i j
      = ∑ p : Fin n × Fin n × Fin n, π p.1 * P p.1 p.2.1 * Q p.2.1 p.2.2 * d p.1 p.2.2 := by
  classical
  calc
    ∑ i, ∑ j, π i * (P * Q) i j * d i j
        = ∑ i, ∑ j, π i * (∑ k, P i k * Q k j) * d i j := by
      simp_rw [Matrix.mul_apply]
    _ = ∑ i, ∑ j, ∑ k, π i * P i k * Q k j * d i j := by
      simp [Finset.mul_sum, Finset.sum_mul, mul_assoc]
    _ = ∑ i, ∑ k, ∑ j, (π i * P i k * Q k j) * d i j := by
      refine Finset.sum_congr rfl fun i _ => ?_
      rw [Finset.sum_comm]
    _ = ∑ p : Fin n × Fin n × Fin n, π p.1 * P p.1 p.2.1 * Q p.2.1 p.2.2 * d p.1 p.2.2 := by
      simp_rw [Fintype.sum_prod_type]
```

Proof. Expand $(PQ)_{ij} = \sum_k P_{ik}Q_{kj}$, distribute, and reorder the finite sums. □

Lemma 3.33 (mc_triple_first). *If $\sum_j Q_{kj} = 1$ for all k, then*

$$\sum_{(i,k,j)} \pi_i P_{ik} Q_{kj} d_{ik} = \sum_i \sum_k \pi_i P_{ik} d_{ik}.$$

Lean: mc_triple_first — proved by AIProver, job 2, session 2

```
theorem mc_triple_first {n : ℕ} (π : Fin n → ℝ) (P Q : Matrix (Fin n) (Fin n) ℝ)
    (d : Fin n → Fin n → ℝ) (hQ : ∀ k, ∑ j, Q k j = 1) :
    ∑ p : Fin n × Fin n × Fin n, π p.1 * P p.1 p.2.1 * Q p.2.1 p.2.2 * d p.1 p.2.1
      = ∑ i, ∑ k, π i * P i k * d i k := by


  have h_reindex : (∑ p : Fin n × Fin n × Fin n, π p.1 * P p.1 p.2.1 * Q p.2.1 p.2.2 * d p.1 p.2.1) =
      (∑ i : Fin n, ∑ j : Fin n, ∑ k : Fin n, π i * P i j * Q j k * d i j) := by
    calc
      (∑ p : Fin n × Fin n × Fin n, π p.1 * P p.1 p.2.1 * Q p.2.1 p.2.2 * d p.1 p.2.1)
          = (∑ p ∈ ((Finset.univ : Finset (Fin n)) ×ˢ (Finset.univ : Finset (Fin n × Fin n))),
            π p.1 * P p.1 p.2.1 * Q p.2.1 p.2.2 * d p.1 p.2.1) := by
        simp [Finset.univ_product_univ]
      _ = (∑ i ∈ (Finset.univ : Finset (Fin n)),
            ∑ jk ∈ (Finset.univ : Finset (Fin n × Fin n)),
            π i * P i jk.1 * Q jk.1 jk.2 * d i jk.1) := by
        rw [Finset.sum_product]
      _ = (∑ i : Fin n, ∑ jk : Fin n × Fin n, π i * P i jk.1 * Q jk.1 jk.2 * d i jk.1) := by
        simp
      _ = (∑ i : Fin n, (∑ j : Fin n, ∑ k : Fin n, π i * P i j * Q j k * d i j)) := by
        refine Finset.sum_congr rfl fun i hi => ?_
        calc
          ∑ jk : Fin n × Fin n, π i * P i jk.1 * Q jk.1 jk.2 * d i jk.1
              = (∑ jk ∈ ((Finset.univ : Finset (Fin n)) ×ˢ (Finset.univ : Finset (Fin n))),
                π i * P i jk.1 * Q jk.1 jk.2 * d i jk.1) := by
            simp [Finset.univ_product_univ]
          _ = ∑ j ∈ (Finset.univ : Finset (Fin n)), ∑ k ∈ (Finset.univ : Finset (Fin n)),
                π i * P i j * Q j k * d i j := by
```

```
          rw [Finset.sum_product]
        _ = ∑ j : Fin n, ∑ k : Fin n, π i * P i j * Q j k * d i j := by
          simp
      _ = (∑ i : Fin n, ∑ j : Fin n, ∑ k : Fin n, π i * P i j * Q j k * d i j) := by
        simp

  have h_collapse : (∑ i : Fin n, ∑ j : Fin n, ∑ k : Fin n, π i * P i j * Q j k * d i j) =
      (∑ i : Fin n, ∑ j : Fin n, π i * P i j * d i j * (∑ k : Fin n, Q j k)) := by
    simp [Finset.mul_sum, mul_comm, mul_left_comm]

  have h_sum_one : (∑ i : Fin n, ∑ j : Fin n, π i * P i j * d i j * (∑ k : Fin n, Q j k)) =
      (∑ i : Fin n, ∑ j : Fin n, π i * P i j * d i j) := by
    simp [hQ, mul_comm, mul_left_comm]

  calc
    ∑ p : Fin n × Fin n × Fin n, π p.1 * P p.1 p.2.1 * Q p.2.1 p.2.2 * d p.1 p.2.1
        = (∑ i : Fin n, ∑ j : Fin n, ∑ k : Fin n, π i * P i j * Q j k * d i j) := h_reindex
    _ = (∑ i : Fin n, ∑ j : Fin n, π i * P i j * d i j * (∑ k : Fin n, Q j k)) := h_collapse
    _ = (∑ i : Fin n, ∑ j : Fin n, π i * P i j * d i j) := h_sum_one
    _ = ∑ i, ∑ k, π i * P i k * d i k := by
      simp
```

Proof. Sum over j first and use $\sum_j Q_{kj} = 1$. □

Lemma 3.34 (mc_triple_second). *If $\sum_i \pi_i P_{ik} = \pi_k$ for all k, then*

$$\sum_{(i,k,j)} \pi_i P_{ik} Q_{kj} d_{kj} = \sum_k \sum_j \pi_k Q_{kj} d_{kj}.$$

Lean: mc_triple_second — proved by AIProver, job 1, session 0

```
theorem mc_triple_second {n : ℕ} (π : Fin n → ℝ) (P Q : Matrix (Fin n) (Fin n) ℝ)
    (d : Fin n → Fin n → ℝ) (hP : ∀ k, ∑ i, π i * P i k = π k) :
    ∑ p : Fin n × Fin n × Fin n, π p.1 * P p.1 p.2.1 * Q p.2.1 p.2.2 * d p.2.1 p.2.2
      = ∑ k, ∑ j, π k * Q k j * d k j := by

  have h1 : (∑ p : Fin n × Fin n × Fin n, π p.1 * P p.1 p.2.1 * Q p.2.1 p.2.2 * d p.2.1 p.2.2) =
      (∑ i : Fin n, ∑ j : Fin n, ∑ k : Fin n, π i * P i j * Q j k * d j k) := by
    calc
      (∑ p : Fin n × Fin n × Fin n, π p.1 * P p.1 p.2.1 * Q p.2.1 p.2.2 * d p.2.1 p.2.2)
        = (∑ p ∈ (Finset.univ : Finset (Fin n × Fin n × Fin n)), π p.1 * P p.1 p.2.1 * Q p.2.1 p.2.2 *
↪  d p.2.1 p.2.2) := rfl
      _ = (∑ p ∈ (Finset.univ : Finset (Fin n)) ×[U+02E2] (Finset.univ : Finset (Fin n × Fin n)), π p.1 * P
↪  p.1 p.2.1 * Q p.2.1 p.2.2 * d p.2.1 p.2.2) := by
        rw [Finset.univ_product_univ]
      _ = (∑ i ∈ (Finset.univ : Finset (Fin n)), ∑ pair ∈ (Finset.univ : Finset (Fin n × Fin n)), π i *
↪  P i pair.1 * Q pair.1 pair.2 * d pair.1 pair.2) := by
        rw [Finset.sum_product]
      _ = (∑ i : Fin n, ∑ pair : Fin n × Fin n, π i * P i pair.1 * Q pair.1 pair.2 * d pair.1 pair.2) :=
↪  by simp
      _ = (∑ i : Fin n, ∑ j : Fin n, ∑ k : Fin n, π i * P i j * Q j k * d j k) := by
        refine Finset.sum_congr rfl (λ i hi => ?_)
        calc
          ∑ pair : Fin n × Fin n, π i * P i pair.1 * Q pair.1 pair.2 * d pair.1 pair.2
            = ∑ pair ∈ (Finset.univ : Finset (Fin n × Fin n)), π i * P i pair.1 * Q pair.1 pair.2 * d
↪  pair.1 pair.2 := rfl
```

```
          _ = ∑ pair ∈ (Finset.univ : Finset (Fin n)) ×[U+02E2] (Finset.univ : Finset (Fin n)), π i * P i
↪  pair.1 * Q pair.1 pair.2 * d pair.1 pair.2 := by
            rw [Finset.univ_product_univ]
          _ = ∑ j ∈ (Finset.univ : Finset (Fin n)), ∑ k ∈ (Finset.univ : Finset (Fin n)), π i * P i j * Q
↪  j k * d j k := by
            rw [Finset.sum_product]
          _ = ∑ j : Fin n, ∑ k : Fin n, π i * P i j * Q j k * d j k := by simp
      _ = (∑ i : Fin n, ∑ j : Fin n, ∑ k : Fin n, π i * P i j * Q j k * d j k) := rfl

  rw [h1]




  rw [Finset.sum_comm]



  have h_factor : (∑ j : Fin n, ∑ i : Fin n, ∑ k : Fin n, π i * P i j * Q j k * d j k) =
      (∑ j : Fin n, ∑ k : Fin n, (∑ i : Fin n, π i * P i j) * Q j k * d j k) := by
    calc
      (∑ j : Fin n, ∑ i : Fin n, ∑ k : Fin n, π i * P i j * Q j k * d j k)
          = (∑ j : Fin n, ∑ k : Fin n, ∑ i : Fin n, π i * P i j * Q j k * d j k) := by
        refine Finset.sum_congr rfl (λ j hj => ?_)
        rw [Finset.sum_comm]
      _ = (∑ j : Fin n, ∑ k : Fin n, (∑ i : Fin n, π i * P i j * Q j k * d j k)) := by
        simp
      _ = (∑ j : Fin n, ∑ k : Fin n, (∑ i : Fin n, π i * P i j) * Q j k * d j k) := by
        refine Finset.sum_congr rfl (λ j hj => ?_)
        refine Finset.sum_congr rfl (λ k hk => ?_)
        simp [Finset.mul_sum, mul_comm, mul_left_comm]

  rw [h_factor]




  simp [hP]
```

Proof. Sum over i first and use $\sum_i \pi_i P_{ik} = \pi_k$. □

Lemma 3.35 (mc_D_subadd)**.** *Let π be a probability vector and let A be stochastic with $\sum_i \pi_i A_{ij} = \pi_j$ for all j. Let $x : \mathrm{Fin}\, n \to \ell_1$ and*

$$D(s) = \sum_i \sum_j \pi_i A^s_{ij} \|x_i - x_j\|^2.$$

Then $\sqrt{D(r+s)} \le \sqrt{D(r)} + \sqrt{D(s)}$ for all $r, s \in \mathbb{N}_0$.

Lean: mc_D_subadd — proved by the captain after a hand-back

```
theorem mc_D_subadd {n : ℕ} (π : Fin n → ℝ) (hπ : IsProbVec π)
    (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochastic A)
    (hstat : ∀ j, ∑ i, π i * A i j = π j) (x : Fin n → L1) (D : ℕ → ℝ)
    (hD : ∀ s, D s = ∑ i, ∑ j, π i * (A ^ s) i j * ‖x i - x j‖ ^ 2) (r s : ℕ) :
```

```
    Real.sqrt (D (r + s)) ≤ Real.sqrt (D r) + Real.sqrt (D s) := by
  have hμ : ∀ p : Fin n × Fin n × Fin n,
      0 ≤ π p.1 * (A ^ r) p.1 p.2.1 * (A ^ s) p.2.1 p.2.2 := by
    intro p
    exact mul_nonneg (mul_nonneg (hπ.1 p.1) ((mc_pow_stochastic A hA r).1 p.1 p.2.1))
      ((mc_pow_stochastic A hA s).1 p.2.1 p.2.2)
  have htri : ∀ p : Fin n × Fin n × Fin n,
      ‖x p.1 - x p.2.2‖ ≤ ‖x p.1 - x p.2.1‖ + ‖x p.2.1 - x p.2.2‖ := by
    intro p
    calc ‖x p.1 - x p.2.2‖ = ‖(x p.1 - x p.2.1) + (x p.2.1 - x p.2.2)‖ := by
          rw [sub_add_sub_cancel]
      _ ≤ ‖x p.1 - x p.2.1‖ + ‖x p.2.1 - x p.2.2‖ := norm_add_le _ _
  have hDrs : D (r + s) = ∑ p : Fin n × Fin n × Fin n,
      π p.1 * (A ^ r) p.1 p.2.1 * (A ^ s) p.2.1 p.2.2 * ‖x p.1 - x p.2.2‖ ^ 2 := by
    rw [hD, pow_add]
    exact mc_triple_mul π (A ^ r) (A ^ s) (fun i j => ‖x i - x j‖ ^ 2)
  have hDr : D r = ∑ p : Fin n × Fin n × Fin n,
      π p.1 * (A ^ r) p.1 p.2.1 * (A ^ s) p.2.1 p.2.2 * ‖x p.1 - x p.2.1‖ ^ 2 := by
    rw [hD]
    exact (mc_triple_first π (A ^ r) (A ^ s) (fun i j => ‖x i - x j‖ ^ 2)
      (mc_pow_stochastic A hA s).2).symm
  have hDs : D s = ∑ p : Fin n × Fin n × Fin n,
      π p.1 * (A ^ r) p.1 p.2.1 * (A ^ s) p.2.1 p.2.2 * ‖x p.2.1 - x p.2.2‖ ^ 2 := by
    rw [hD]
    exact (mc_triple_second π (A ^ r) (A ^ s) (fun i j => ‖x i - x j‖ ^ 2)
      (mc_pow_stationary π A hstat r)).symm
  have key := mc_weighted_tri
    (fun p : Fin n × Fin n × Fin n => π p.1 * (A ^ r) p.1 p.2.1 * (A ^ s) p.2.1 p.2.2)
    (fun p => ‖x p.1 - x p.2.1‖) (fun p => ‖x p.2.1 - x p.2.2‖) (fun p => ‖x p.1 - x p.2.2‖)
    hμ (fun p => norm_nonneg _) htri
  rw [hDrs, hDr, hDs]
  exact key
```

Proof. On $\mathrm{Fin}\, n^3$ put $\mu(i,k,j) = \pi_i A^r_{ik} A^s_{kj}$; then $\mu \geq 0$ by Lemma 3.2. We express the three quantities through μ: - Since $A^{r+s} = A^r A^s$, Lemma 3.32 gives $D(r+s) = \sum \mu \, \|x_i - x_j\|^2$. - The rows of A^s sum to 1 (Lemma 3.2), so Lemma 3.33 gives $D(r) = \sum \mu \, \|x_i - x_k\|^2$. - $\pi A^r = \pi$ (Lemma 3.3), so Lemma 3.34 gives $D(s) = \sum \mu \, \|x_k - x_j\|^2$.

Finally, $\|x_i - x_j\| \leq \|x_i - x_k\| + \|x_k - x_j\|$. Apply Lemma 3.31. □

Lemma 3.36 (`mc_cesaro_Dt`). *Let $t \geq 1$ and let $D : \mathbb{N}_0 \to \mathbb{R}$ satisfy - $D(0) = 0$ and $D \geq 0$; - $\sqrt{D(r+s)} \leq \sqrt{D(r)} + \sqrt{D(s)}$ for all r, s.*

Then

$$D(t) \leq 4 \cdot \frac{1}{t} \sum_{s=1}^{t} D(s).$$

Lean: `mc_cesaro_Dt` — proved by AIProver, job 1, session 0

```
theorem mc_cesaro_Dt (t : ℕ) (ht : 1 ≤ t) (D : ℕ → ℝ) (hD0 : D 0 = 0)
    (hnn : ∀ s, 0 ≤ D s)
    (hsub : ∀ r s, Real.sqrt (D (r + s)) ≤ Real.sqrt (D r) + Real.sqrt (D s)) :
    D t ≤ 4 * ((1 / (t : ℝ)) * ∑ s ∈ Finset.Icc 1 t, D s) := by

  have hblock : ∀ r, 1 ≤ r → r ≤ t → D t ≤ 2 * D r + 2 * D (t - r) := by
    intro r hr1 hr2
    have h := hsub r (t - r)
```

```
  have hsum : r + (t - r) = t := Nat.add_sub_cancel' hr2
  rw [hsum] at h
  have hsq : D t ≤ (Real.sqrt (D r) + Real.sqrt (D (t - r))) ^ 2 := by
    have h_sq_ineq : (Real.sqrt (D t)) ^ 2 ≤ (Real.sqrt (D r) + Real.sqrt (D (t - r))) ^ 2 := by
      have h_sqrt_nonneg : 0 ≤ Real.sqrt (D t) := Real.sqrt_nonneg _
      have h_sum_nonneg : 0 ≤ Real.sqrt (D r) + Real.sqrt (D (t - r)) := by positivity
      nlinarith
    have h_sq_Dt : (Real.sqrt (D t)) ^ 2 = D t := Real.sq_sqrt (hnn t)
    rw [h_sq_Dt] at h_sq_ineq
    exact h_sq_ineq
  have hsq_bound : (Real.sqrt (D r) + Real.sqrt (D (t - r))) ^ 2 ≤ 2 * D r + 2 * D (t - r) := by
    have h_amgm : 2 * Real.sqrt (D r) * Real.sqrt (D (t - r))
        ≤ (Real.sqrt (D r)) ^ 2 + (Real.sqrt (D (t - r))) ^ 2 := by
      nlinarith [sq_nonneg (Real.sqrt (D r) - Real.sqrt (D (t - r)))]
    have hsqD_r : (Real.sqrt (D r)) ^ 2 = D r := Real.sq_sqrt (hnn r)
    have hsqD_tr : (Real.sqrt (D (t - r))) ^ 2 = D (t - r) := Real.sq_sqrt (hnn (t - r))
    nlinarith
  nlinarith


have hcard : (Finset.Icc 1 t : Finset ℕ).card = t := by
  simp

have hsum : ∑ r ∈ Finset.Icc 1 t, D t ≤ ∑ r ∈ Finset.Icc 1 t, (2 * D r + 2 * D (t - r)) := by
  apply Finset.sum_le_sum
  intro r hr
  rcases Finset.mem_Icc.mp hr with ⟨hr1, hr2⟩
  exact hblock r hr1 hr2


have hright_split : ∑ r ∈ Finset.Icc 1 t, (2 * D r + 2 * D (t - r))
    = 2 * (∑ r ∈ Finset.Icc 1 t, D r) + 2 * (∑ r ∈ Finset.Icc 1 t, D (t - r)) := by
  simp [Finset.sum_add_distrib, Finset.mul_sum]


have hreindex : ∑ r ∈ Finset.Icc 1 t, D (t - r) = ∑ s ∈ Finset.Icc 1 (t-1), D s := by
  by_cases ht1' : t = 1
  · subst ht1'; simp [hD0]
  · have ht_ge_2 : 2 ≤ t := by omega

    have h_disjoint : Finset.Icc 1 t = {t} ∪ Finset.Icc 1 (t-1) := by
      ext x; constructor
      · intro hx
        rcases Finset.mem_Icc.mp hx with ⟨hx1, hx2⟩
        by_cases hxt : x = t
        · apply Finset.mem_union_left; exact Finset.mem_singleton.mpr hxt
        · apply Finset.mem_union_right; apply Finset.mem_Icc.mpr; omega
      · intro hx
        rcases Finset.mem_union.mp hx with (hx' | hx')
        · rw [Finset.mem_singleton.mp hx']; apply Finset.mem_Icc.mpr; omega
        · rcases Finset.mem_Icc.mp hx' with ⟨hx1, hx2⟩; apply Finset.mem_Icc.mpr; omega
    have h_disjoint' : Disjoint ({t} : Finset ℕ) (Finset.Icc 1 (t-1)) := by
      apply Finset.disjoint_singleton_left.mpr
      intro hx; rcases Finset.mem_Icc.mp hx with ⟨hx1, hx2⟩; omega
    rw [h_disjoint, Finset.sum_union h_disjoint', Finset.sum_singleton]

    have hsub_zero : t - t = 0 := Nat.sub_self _
    rw [hsub_zero, hD0, zero_add]
```

```
    apply Finset.sum_nbij (fun r => t - r) ?_ ?_ ?_ ?_
    · intro r hr
      rcases Finset.mem_Icc.mp hr with ⟨hr1, hr2⟩
      apply Finset.mem_Icc.mpr
      have hpos : 1 ≤ t - r := by omega
      have hle : t - r ≤ t - 1 := by omega
      exact ⟨hpos, hle⟩
    · intro r₁ hr₁ r₂ hr₂ h_eq
      rcases Finset.mem_Icc.mp hr₁ with ⟨hr₁1, hr₁2⟩
      rcases Finset.mem_Icc.mp hr₂ with ⟨hr₂1, hr₂2⟩
      simp at h_eq
      omega
    ·

      intro s hs
      rcases Finset.mem_Icc.mp hs with ⟨hs1, hs2⟩
      have hs_le_t : s ≤ t := by omega
      have hpos : 1 ≤ t - s := by omega
      have hle : t - s ≤ t - 1 := by omega
      refine ⟨t - s, Finset.mem_Icc.mpr ⟨hpos, hle⟩, ?_⟩

      simp
      omega
    ·









      intro r hr
      rfl


  have h_ineq : (t : ℝ) * D t ≤ 2 * (∑ r ∈ Finset.Icc 1 t, D r) + 2 * (∑ s ∈ Finset.Icc 1 (t-1), D s) :=
↪   by
    calc
      (t : ℝ) * D t = ∑ r ∈ Finset.Icc 1 t, D t := by symm; simp [hcard]
      _ ≤ ∑ r ∈ Finset.Icc 1 t, (2 * D r + 2 * D (t - r)) := hsum
      _ = 2 * (∑ r ∈ Finset.Icc 1 t, D r) + 2 * (∑ r ∈ Finset.Icc 1 t, D (t - r)) := by
        simp [Finset.sum_add_distrib, Finset.mul_sum]
      _ = 2 * (∑ r ∈ Finset.Icc 1 t, D r) + 2 * (∑ s ∈ Finset.Icc 1 (t-1), D s) := by
        rw [hreindex]


  by_cases ht1 : t = 1
  · subst ht1
    have hsum1 : (∑ s ∈ Finset.Icc 1 1, D s) = D 1 := by simp
    rw [hsum1]
    have hpos1 : 0 ≤ D 1 := hnn 1
    nlinarith
  · have ht_ge_2 : 2 ≤ t := by omega
    have hsplit : (∑ r ∈ Finset.Icc 1 t, D r) = D t + (∑ r ∈ Finset.Icc 1 (t-1), D r) := by
```

```
  have h_disjoint : Finset.Icc 1 t = {t} ∪ Finset.Icc 1 (t-1) := by
    ext x; constructor
    · intro hx
      rcases Finset.mem_Icc.mp hx with ⟨hx1, hx2⟩
      by_cases hxt : x = t
      · apply Finset.mem_union_left; exact Finset.mem_singleton.mpr hxt
      · apply Finset.mem_union_right; apply Finset.mem_Icc.mpr; omega
    · intro hx
      rcases Finset.mem_union.mp hx with (hx' | hx')
      · rw [Finset.mem_singleton.mp hx']; apply Finset.mem_Icc.mpr; omega
      · rcases Finset.mem_Icc.mp hx' with ⟨hx1, hx2⟩; apply Finset.mem_Icc.mpr; omega
  have h_disjoint' : Disjoint ({t} : Finset ℕ) (Finset.Icc 1 (t-1)) := by
    apply Finset.disjoint_singleton_left.mpr
    intro hx; rcases Finset.mem_Icc.mp hx with ⟨hx1, hx2⟩; omega
  rw [h_disjoint, Finset.sum_union h_disjoint', Finset.sum_singleton, add_comm]
rw [hsplit] at h_ineq

set S := ∑ s ∈ Finset.Icc 1 (t-1), D s with hS

have hS_nonneg : 0 ≤ S := by
  rw [hS]
  apply Finset.sum_nonneg
  intro s _
  exact hnn s



have h_derived : ((t : ℝ) - 2) * D t ≤ 4 * S := by
  nlinarith







have hgoal : D t ≤ 4 * ((1 / (t : ℝ)) * ∑ s ∈ Finset.Icc 1 t, D s) := by
  rw [hsplit, hS]
  have htpos : 0 < (t : ℝ) := by
    have : 0 < t := Nat.lt_of_lt_of_le (by norm_num) ht
    exact_mod_cast this


  have htarget : (t : ℝ) * D t ≤ 4 * (D t + (∑ s ∈ Finset.Icc 1 (t-1), D s)) := by
    by_cases ht_le_4 : (t : ℝ) ≤ 4
    ·
      have h_target : ((t : ℝ) - 4) * D t ≤ 4 * (∑ s ∈ Finset.Icc 1 (t-1), D s) := by
        have h_nonpos : ((t : ℝ) - 4) * D t ≤ 0 := by
          have : (t : ℝ) - 4 ≤ 0 := by linarith
          nlinarith [hnn t]
        nlinarith
      nlinarith
    ·
      have h_target : ((t : ℝ) - 4) * D t ≤ 4 * (∑ s ∈ Finset.Icc 1 (t-1), D s) := by
        have hcoeff : (t : ℝ) - 4 ≤ (t : ℝ) - 2 := by nlinarith
        have hDt_nonneg : 0 ≤ D t := hnn t
        nlinarith
      nlinarith

  calc
```

```
        D t = ((t : ℝ) * D t) / (t : ℝ) := by field_simp [ne_of_gt htpos]
        _ ≤ (4 * (D t + (∑ s ∈ Finset.Icc 1 (t-1), D s))) / (t : ℝ) := by
          apply div_le_div_of_nonneg_right htarget
          positivity
        _ = 4 * ((1 / (t : ℝ)) * (D t + (∑ s ∈ Finset.Icc 1 (t-1), D s))) := by ring

    exact hgoal
```

Proof. For $1 \le r \le t$, subadditivity and $(\alpha+\beta)^2 \le 2\alpha^2 + 2\beta^2$ give

$$D(t) \le 2D(r) + 2D(t-r).$$

Summing over r and reindexing $s = t - r$ (the term $r = t$ contributes $D(0) = 0$) gives

$$tD(t) \le 2\sum_{r=1}^{t} D(r) + 2\sum_{s=1}^{t-1} D(s) \le 4\sum_{s=1}^{t} D(s),$$

using $D \ge 0$ in the last step. Divide by t. □

Lemma 3.37 (mc_cesaro_blocks). *Let $t \in \mathbb{N}_0$, and let $D : \mathbb{N}_0 \to \mathbb{R}$ with $D \ge 0$ and $\sqrt{D}$ subadditive as above. Then for all $j, r \in \mathbb{N}_0$,*

$$D(jt + r) \le 2j^2 D(t) + 2D(r).$$

Lean: mc_cesaro_blocks — proved by AIProver, job 3, session 1

```
theorem mc_cesaro_blocks (t : ℕ) (D : ℕ → ℝ) (hnn : ∀ s, 0 ≤ D s)
    (hsub : ∀ r s, Real.sqrt (D (r + s)) ≤ Real.sqrt (D r) + Real.sqrt (D s)) (j r : ℕ) :
    D (j * t + r) ≤ 2 * (j : ℝ) ^ 2 * D t + 2 * D r := by

  have h_sqrt_bound : Real.sqrt (D (j * t + r)) ≤ (j : ℝ) * Real.sqrt (D t) + Real.sqrt (D r) := by
    induction' j with k ih
    · simp
    · have h_sub := hsub (k * t + r) t
      have h_eq : (k.succ * t + r) = ((k * t + r) + t) := by
        calc
          k.succ * t + r = (k * t + t) + r := by rw [Nat.succ_mul]
          _ = (k * t + r) + t := by omega
      have h_target : Real.sqrt (D (k.succ * t + r)) ≤ Real.sqrt (D (k * t + r)) + Real.sqrt (D t) := by
        simpa [h_eq] using h_sub
      have h_comb : Real.sqrt (D (k.succ * t + r)) ≤ ((k : ℝ) * Real.sqrt (D t) + Real.sqrt (D r)) +
↪  Real.sqrt (D t) := by
        linarith

      simpa [Nat.cast_succ, add_comm, add_left_comm, add_assoc, mul_add, add_mul] using h_comb

  have h_sq : D (j * t + r) ≤ ((j : ℝ) * Real.sqrt (D t) + Real.sqrt (D r)) ^ 2 := by
    have h_nonneg_sqrt : 0 ≤ Real.sqrt (D (j * t + r)) := Real.sqrt_nonneg _
    have h := mul_self_le_mul_self h_nonneg_sqrt h_sqrt_bound

    rw [Real.mul_self_sqrt (hnn (j * t + r))] at h

    simpa [sq] using h

  have h_cross : 2 * ((j : ℝ) * Real.sqrt (D t)) * Real.sqrt (D r) ≤
```

```
      ((j : ℝ) * Real.sqrt (D t)) ^ 2 + (Real.sqrt (D r)) ^ 2 := by
    have h_sq_nonneg : 0 ≤ ((j : ℝ) * Real.sqrt (D t) - Real.sqrt (D r)) ^ 2 := by
      apply pow_two_nonneg
    nlinarith
  calc
    D (j * t + r) ≤ ((j : ℝ) * Real.sqrt (D t) + Real.sqrt (D r)) ^ 2 := h_sq
    _ = ((j : ℝ) * Real.sqrt (D t)) ^ 2 + (Real.sqrt (D r)) ^ 2 + 2 * ((j : ℝ) * Real.sqrt (D t)) *
↪  Real.sqrt (D r) := by ring
    _ ≤ ((j : ℝ) * Real.sqrt (D t)) ^ 2 + (Real.sqrt (D r)) ^ 2 + (((j : ℝ) * Real.sqrt (D t)) ^ 2 +
↪  (Real.sqrt (D r)) ^ 2) := by
      nlinarith
    _ = 2 * (((j : ℝ) * Real.sqrt (D t)) ^ 2 + (Real.sqrt (D r)) ^ 2) := by ring
    _ = 2 * (((j : ℝ) ^ 2 * (Real.sqrt (D t)) ^ 2) + (Real.sqrt (D r)) ^ 2) := by ring
    _ = 2 * (((j : ℝ) ^ 2 * D t) + D r) := by simp [Real.sq_sqrt (hnn t), Real.sq_sqrt (hnn r)]
    _ = 2 * (j : ℝ) ^ 2 * D t + 2 * D r := by ring
```

Proof. By induction on j, $\sqrt{D(jt+r)} \le j\sqrt{D(t)} + \sqrt{D(r)}$. Square both sides and use $2\alpha\beta \le \alpha^2 + \beta^2$. □

Lemma 3.38 (mc_hasSum_sq_geom). *For $0 \le r < 1$, the series $\sum_{s\ge 0} s^2 r^s$ converges to $\dfrac{r(1+r)}{(1-r)^3}$.*

Lean: mc_hasSum_sq_geom — proved by AIProver, job 2, session 0

```
theorem mc_hasSum_sq_geom (r : ℝ) (h0 : 0 ≤ r) (h1 : r < 1) :
    HasSum (fun s : ℕ => (s : ℝ) ^ 2 * r ^ s) (r * (1 + r) / (1 - r) ^ 3) := by
  have hr_norm : ‖r‖ < 1 := by
    rw [Real.norm_eq_abs]
    exact abs_lt.mpr ⟨by linarith, h1⟩
  have h_choose2 : HasSum (fun n : ℕ => ((n + 2 : ℕ).choose 2 : ℝ) * r ^ n) ((1 : ℝ) / (1 - r) ^ 3) := by
    have h := hasSum_choose_mul_geometric_of_norm_lt_one (𝓡 := ℝ) 2 hr_norm
    simpa [show (2:ℕ)+1 = 3 by decide] using h
  have h_n_mul : HasSum (fun n : ℕ => (n : ℝ) * r ^ n) (r / (1 - r) ^ 2) :=
    hasSum_coe_mul_geometric_of_norm_lt_one hr_norm
  have h_geom : HasSum (fun n : ℕ => r ^ n) ((1 - r)⁻¹) :=
    hasSum_geometric_of_lt_one h0 h1

  have h_sq_eq : ∀ n : ℕ, (n : ℝ) ^ 2 = (2 : ℝ) * (((n : ℕ) + 2).choose 2 : ℝ) - (3 : ℝ) * (n : ℝ) - 2 := by
    intro n
    have h_choose_real : (((n : ℕ) + 2).choose 2 : ℝ) = ((n : ℝ) + 2) * ((n : ℝ) + 1) / 2 := by
      have h := Nat.cast_choose_two (a := n+2) (K := ℝ)
      push_cast at h
      have h' : ((n : ℝ) + 2) - 1 = (n : ℝ) + 1 := by ring
      simpa [h'] using h
    rw [h_choose_real]
    ring

  have h_sum : HasSum (fun s : ℕ => (s : ℝ) ^ 2 * r ^ s) (r * (1 + r) / (1 - r) ^ 3) := by
    have h1 : HasSum (fun s : ℕ => (2 : ℝ) * (((s : ℕ) + 2).choose 2 : ℝ) * r ^ s)
        ((2 : ℝ) * ((1 : ℝ) / (1 - r) ^ 3)) := by
      simpa [mul_assoc] using h_choose2.mul_left (2 : ℝ)
    have h2 : HasSum (fun s : ℕ => (3 : ℝ) * ((s : ℝ) * r ^ s))
        ((3 : ℝ) * (r / (1 - r) ^ 2)) := by
      simpa [mul_assoc] using h_n_mul.mul_left (3 : ℝ)
    have h3 : HasSum (fun s : ℕ => (2 : ℝ) * (r ^ s))
        ((2 : ℝ) * ((1 - r)⁻¹)) :=
      h_geom.mul_left (2 : ℝ)
```

```
    have h_total : HasSum (fun s : ℕ =>
        ((2 : ℝ) * (((s : ℕ) + 2).choose 2 : ℝ) * r ^ s)
        - ((3 : ℝ) * ((s : ℝ) * r ^ s))
        - ((2 : ℝ) * (r ^ s)))
        (((2 : ℝ) * ((1 : ℝ) / (1 - r) ^ 3))
        - ((3 : ℝ) * (r / (1 - r) ^ 2))
        - ((2 : ℝ) * ((1 - r)⁻¹))) := by
      simpa using ((h1.sub h2).sub h3)

    have h_fun_eq : (fun s : ℕ => ((2 : ℝ) * (((s : ℕ) + 2).choose 2 : ℝ) * r ^ s)
        - ((3 : ℝ) * ((s : ℝ) * r ^ s))
        - ((2 : ℝ) * (r ^ s))) = (fun s : ℕ => (s : ℝ) ^ 2 * r ^ s) := by
      ext s
      calc
        ((2 : ℝ) * (((s : ℕ) + 2).choose 2 : ℝ) * r ^ s)
            - ((3 : ℝ) * ((s : ℝ) * r ^ s))
            - ((2 : ℝ) * (r ^ s))
          = ((2 : ℝ) * (((s : ℕ) + 2).choose 2 : ℝ) - (3 : ℝ) * (s : ℝ) - 2) * r ^ s := by ring
        _ = (s : ℝ) ^ 2 * r ^ s := by rw [h_sq_eq s]
    have h_total' : HasSum (fun s : ℕ => (s : ℝ) ^ 2 * r ^ s)
        (((2 : ℝ) * ((1 : ℝ) / (1 - r) ^ 3))
        - ((3 : ℝ) * (r / (1 - r) ^ 2))
        - ((2 : ℝ) * ((1 - r)⁻¹))) := by
      simpa [h_fun_eq] using h_total

    have h_sum_eq : ((2 : ℝ) * ((1 : ℝ) / (1 - r) ^ 3))
        - ((3 : ℝ) * (r / (1 - r) ^ 2))
        - ((2 : ℝ) * ((1 - r)⁻¹)) = r * (1 + r) / (1 - r) ^ 3 := by
      field_simp [show (1 : ℝ) - r ≠ 0 by linarith]
      ring
    rw [h_sum_eq] at h_total'
    exact h_total'
  exact h_sum
```

Proof. Write $s^2 = 2\binom{s+2}{2} - 3s - 2$. Mathlib provides the sums

$$\sum \binom{s+2}{2} r^s = (1-r)^{-3}, \qquad \sum s r^s = r(1-r)^{-2}, \qquad \sum r^s = (1-r)^{-1}.$$

Hence the series converges to

$$\frac{2}{(1-r)^3} - \frac{3r}{(1-r)^2} - \frac{2}{1-r} = \frac{r + r^2}{(1-r)^3}.$$

□

Lemma 3.39 (mc_floor_div_sq_le). *For $t \geq 1$ and $s \in \mathbb{N}_0$, $\lfloor s/t \rfloor^2 \leq s^2/t^2$.*

Lean: mc_floor_div_sq_le — proved by AIProver, job 1, session 0

```
theorem mc_floor_div_sq_le (t s : ℕ) (ht : 1 ≤ t) :
    (((s / t : ℕ) : ℝ)) ^ 2 ≤ (s : ℝ) ^ 2 / (t : ℝ) ^ 2 := by
  have ht_pos : 0 < (t : ℝ) := by
    have : 0 < t := Nat.lt_of_lt_of_le (by decide : 0 < 1) ht
    exact Nat.cast_pos.mpr this
  have hdiv : ((s / t : ℕ) : ℝ) ≤ (s : ℝ) / (t : ℝ) := by
    have hle : ((s / t : ℕ) : ℝ) * (t : ℝ) ≤ (s : ℝ) := by
      have := Nat.div_mul_le_self s t
      exact mod_cast this
```

```
    calc
      ((s / t : ℕ) : ℝ) = (((s / t : ℕ) : ℝ) * (t : ℝ)) / (t : ℝ) := by
        field_simp [ne_of_gt ht_pos]
      _ ≤ (s : ℝ) / (t : ℝ) := by
        gcongr
  have hsq : (((s / t : ℕ) : ℝ)) ^ 2 ≤ ((s : ℝ) / (t : ℝ)) ^ 2 := by
    have hnonneg_left : 0 ≤ ((s / t : ℕ) : ℝ) := by exact mod_cast Nat.zero_le _
    have hnonneg_right : 0 ≤ (s : ℝ) / (t : ℝ) :=
      div_nonneg (by exact mod_cast Nat.zero_le s) (by exact mod_cast Nat.zero_le t)
    nlinarith
  calc
    (((s / t : ℕ) : ℝ)) ^ 2 ≤ ((s : ℝ) / (t : ℝ)) ^ 2 := hsq
    _ = (s : ℝ) ^ 2 / (t : ℝ) ^ 2 := by ring
```

Proof. We have $0 \le \lfloor s/t \rfloor \le s/t$, because $\lfloor s/t \rfloor\, t \le s$. Square. □

Lemma 3.40 (mc_geom_sq_div). *For $t \ge 1$, the series $\sum_s p_t q_t^s \lfloor s/t \rfloor^2$ is summable and its sum is at most 3.*

Lean: mc_geom_sq_div — proved by the captain after a hand-back

```
theorem mc_geom_sq_div (t : ℕ) (ht : 1 ≤ t) :
    Summable (fun s : ℕ => (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * (((s / t : ℕ) : ℝ)) ^ 2) ∧
    ∑' s : ℕ, (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * (((s / t : ℕ) : ℝ)) ^ 2 ≤ 3 := by
  have htpos : (0:ℝ) < t := by exact_mod_cast ht
  have ht0 : (t:ℝ) ≠ 0 := ne_of_gt htpos
  have ht1 : (t:ℝ) + 1 ≠ 0 := by positivity
  set r : ℝ := (t : ℝ) / ((t : ℝ) + 1) with hr
  have hr0 : 0 ≤ r := by rw [hr]; positivity
  have hr1 : r < 1 := by
    rw [hr, div_lt_one (by positivity)]; linarith
  have hS := mc_hasSum_sq_geom r hr0 hr1
  have hg : HasSum (fun s : ℕ => (1 / ((t : ℝ) + 1)) / (t:ℝ)^2 * ((s : ℝ) ^ 2 * r ^ s))
      ((1 / ((t : ℝ) + 1)) / (t:ℝ)^2 * (r * (1 + r) / (1 - r) ^ 3)) := hS.mul_left _
  have hp0 : (0:ℝ) ≤ 1 / ((t : ℝ) + 1) := by positivity
  have hle : ∀ s : ℕ, (1 / ((t : ℝ) + 1)) * r ^ s * (((s / t : ℕ) : ℝ)) ^ 2
      ≤ (1 / ((t : ℝ) + 1)) / (t:ℝ)^2 * ((s : ℝ) ^ 2 * r ^ s) := by
    intro s
    have h1 := mc_floor_div_sq_le t s ht
    have hp : 0 ≤ (1 / ((t : ℝ) + 1)) * r ^ s := mul_nonneg hp0 (pow_nonneg hr0 s)
    calc (1 / ((t : ℝ) + 1)) * r ^ s * (((s / t : ℕ) : ℝ)) ^ 2
        ≤ (1 / ((t : ℝ) + 1)) * r ^ s * ((s : ℝ) ^ 2 / (t : ℝ) ^ 2) :=
          mul_le_mul_of_nonneg_left h1 hp
      _ = (1 / ((t : ℝ) + 1)) / (t:ℝ)^2 * ((s : ℝ) ^ 2 * r ^ s) := by ring
  have hnn : ∀ s : ℕ, 0 ≤ (1 / ((t : ℝ) + 1)) * r ^ s * (((s / t : ℕ) : ℝ)) ^ 2 :=
    fun s => mul_nonneg (mul_nonneg hp0 (pow_nonneg hr0 s)) (sq_nonneg _)
  have hsum : Summable (fun s : ℕ => (1 / ((t : ℝ) + 1)) * r ^ s * (((s / t : ℕ) : ℝ)) ^ 2) :=
    Summable.of_nonneg_of_le hnn hle hg.summable
  have h1r : 1 - r = 1 / ((t:ℝ) + 1) := by
    rw [hr, eq_div_iff ht1, sub_mul, div_mul_cancel₀ _ ht1]; ring
  have key : (1 / ((t : ℝ) + 1)) / (t:ℝ)^2 * (r * (1 + r) / (1 - r) ^ 3)
      = (2 * (t:ℝ) + 1) / t := by
    rw [h1r, hr]
    field_simp
    ring
  refine ⟨hsum, ?_⟩
  calc ∑' s : ℕ, (1 / ((t : ℝ) + 1)) * r ^ s * (((s / t : ℕ) : ℝ)) ^ 2
```

```
    ≤ ∑' s : ℕ, (1 / ((t : ℝ) + 1)) / (t:ℝ)^2 * ((s : ℝ) ^ 2 * r ^ s) :=
      Summable.tsum_le_tsum hle hsum hg.summable
  _ = (1 / ((t : ℝ) + 1)) / (t:ℝ)^2 * (r * (1 + r) / (1 - r) ^ 3) := hg.tsum_eq
  _ = (2 * (t:ℝ) + 1) / t := key
  _ ≤ 3 := by
    rw [div_le_iff₀ htpos]
    have : (1:ℝ) ≤ t := by exact_mod_cast ht
    linarith
```

Proof. By Lemma 3.39, each term is between 0 and $(p_t/t^2)s^2q_t^s$. By Lemma 3.38, together with $1-q_t=p_t$, the dominating series has sum

$$\frac{p_t}{t^2}\cdot\frac{q_t(1+q_t)}{p_t^3}=\frac{2t+1}{t}\le 3.$$

□

Lemma 3.41 (mc_geom_mod_coeff_geom)**.** *For every $M\in\mathbb{N}_0$, $\sum_{k<M}2^{-k}\le 2-2\cdot 2^{-M}$.*

Lean: mc_geom_mod_coeff_geom — helper of mc_geom_mod_coeff, proved by the captain after a hand-back

```
theorem mc_geom_mod_coeff_geom (M : ℕ) :
    ∑ k ∈ Finset.range M, (1/2:ℝ)^k ≤ 2 - 2 * (1/2:ℝ)^M := by
  induction M with
  | zero => norm_num
  | succ M ih =>
    rw [Finset.sum_range_succ, pow_succ]
    linarith
```

Proof. Induction on M. □

Lemma 3.42 (mc_geom_mod_coeff)**.** *For $t\ge 1$ and $r,N\in\mathbb{N}_0$,*

$$\sum_{\substack{s<N\\ s\bmod t=r}} p_tq_t^s\le\frac{2}{t}.$$

Lean: mc_geom_mod_coeff — proved by the captain after a hand-back

```
theorem mc_geom_mod_coeff (t : ℕ) (ht : 1 ≤ t) (r N : ℕ) :
    ∑ s ∈ (Finset.range N).filter (fun s => s % t = r),
      (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s ≤ 2 / (t : ℝ) := by
  have ht_pos : (0:ℝ) < (t:ℝ) := Nat.cast_pos.mpr (by omega)
  have ht0 : (t:ℝ) ≠ 0 := ht_pos.ne'
  have ht1 : (t:ℝ) + 1 ≠ 0 := (by linarith : (0:ℝ) < (t:ℝ) + 1).ne'
  obtain ⟨q, hq⟩ : ∃ q : ℝ, q = (t:ℝ) / ((t:ℝ) + 1) := ⟨_, rfl⟩
  rw [← hq]
  have hq0 : 0 ≤ q := by rw [hq]; positivity
  have hq1 : q ≤ 1 := by
    rw [hq, div_le_one (by positivity : (0:ℝ) < (t:ℝ) + 1)]
    linarith
  have hqmul : q * (1 + 1 / (t:ℝ)) = 1 := by
    rw [hq]
    field_simp
    try ring
  have hbern : (2:ℝ) ≤ (1 + 1 / (t:ℝ)) ^ t := by
```

```
  have ha : (-2:ℝ) ≤ 1 / (t:ℝ) := by
    have : (0:ℝ) ≤ 1 / (t:ℝ) := by positivity
    linarith
  have h := one_add_mul_le_pow ha t
  rw [mul_one_div_cancel ht0] at h
  linarith
have hqt : q ^ t ≤ 1 / 2 := by
  have h1 : q ^ t * (1 + 1 / (t:ℝ)) ^ t = 1 := by
    rw [← mul_pow, hqmul, one_pow]
  have h2 : q ^ t * 2 ≤ q ^ t * (1 + 1 / (t:ℝ)) ^ t :=
    mul_le_mul_of_nonneg_left hbern (pow_nonneg hq0 t)
  linarith
have hterm : ∀ s ∈ (Finset.range N).filter (fun s => s % t = r),
    1 / ((t:ℝ) + 1) * q ^ s ≤ 1 / ((t:ℝ) + 1) * (1/2:ℝ) ^ (s / t) := by
  intro s _
  refine mul_le_mul_of_nonneg_left ?_ (by positivity)
  have hsplit : q ^ s = q ^ (s % t) * (q ^ t) ^ (s / t) := by
    rw [← pow_mul, ← pow_add, Nat.mod_add_div]
  rw [hsplit]
  have h1 : q ^ (s % t) ≤ 1 := pow_le_one₀ hq0 hq1
  have h2 : (q ^ t) ^ (s / t) ≤ (1/2:ℝ) ^ (s / t) := by
    first
      | exact pow_le_pow_left₀ (pow_nonneg hq0 t) hqt (s / t)
      | exact pow_le_pow_left (pow_nonneg hq0 t) hqt (s / t)
  have h3 : 0 ≤ (q ^ t) ^ (s / t) := pow_nonneg (pow_nonneg hq0 t) _
  calc q ^ (s % t) * (q ^ t) ^ (s / t) ≤ 1 * (q ^ t) ^ (s / t) :=
        mul_le_mul_of_nonneg_right h1 h3
    _ = (q ^ t) ^ (s / t) := one_mul _
    _ ≤ (1/2:ℝ) ^ (s / t) := h2
have hinj : Set.InjOn (fun s : ℕ => s / t)
    ↑((Finset.range N).filter (fun s => s % t = r)) := by
  intro x hx y hy hxy
  have hxm : x % t = r := (Finset.mem_filter.mp (Finset.mem_coe.mp hx)).2
  have hym : y % t = r := (Finset.mem_filter.mp (Finset.mem_coe.mp hy)).2
  have hxy' : x / t = y / t := hxy
  calc x = x % t + t * (x / t) := (Nat.mod_add_div x t).symm
    _ = y % t + t * (y / t) := by rw [hxm, hym, hxy']
    _ = y := Nat.mod_add_div y t
have hsub : ((Finset.range N).filter (fun s => s % t = r)).image (fun s => s / t)
    ⊆ Finset.range N := by
  intro k hk
  obtain ⟨s, hs, rfl⟩ := Finset.mem_image.mp hk
  exact Finset.mem_range.mpr (lt_of_le_of_lt (Nat.div_le_self s t)
    (Finset.mem_range.mp (Finset.mem_filter.mp hs).1))
have hgeom : ∑ k ∈ Finset.range N, (1/2:ℝ) ^ k ≤ 2 := by
  have h := mc_geom_mod_coeff_geom N
  have : (0:ℝ) ≤ (1/2:ℝ) ^ N := by positivity
  linarith
calc ∑ s ∈ (Finset.range N).filter (fun s => s % t = r), 1 / ((t:ℝ) + 1) * q ^ s
    ≤ ∑ s ∈ (Finset.range N).filter (fun s => s % t = r),
        1 / ((t:ℝ) + 1) * (1/2:ℝ) ^ (s / t) := Finset.sum_le_sum hterm
  _ = 1 / ((t:ℝ) + 1) * ∑ s ∈ (Finset.range N).filter (fun s => s % t = r),
        (1/2:ℝ) ^ (s / t) := by rw [Finset.mul_sum]
  _ = 1 / ((t:ℝ) + 1) * ∑ k ∈ ((Finset.range N).filter (fun s => s % t = r)).image
        (fun s => s / t), (1/2:ℝ) ^ k := by rw [Finset.sum_image hinj]
  _ ≤ 1 / ((t:ℝ) + 1) * ∑ k ∈ Finset.range N, (1/2:ℝ) ^ k :=
      mul_le_mul_of_nonneg_left
        (Finset.sum_le_sum_of_subset_of_nonneg hsub (fun k _ _ => by positivity))
        (by positivity)
```

```
    _ ≤ 1 / ((t:ℝ) + 1) * 2 := mul_le_mul_of_nonneg_left hgeom (by positivity)
    _ ≤ 1 / (t:ℝ) * 2 :=
        mul_le_mul_of_nonneg_right (one_div_le_one_div_of_le ht_pos (by linarith)) (by norm_num)
    _ = 2 / (t:ℝ) := by ring
```

Proof. *A bound on q_t^t.* Bernoulli's inequality gives $(1+1/t)^t \geq 2$. Since $q_t(1+1/t) = 1$, it follows that $q_t^t \leq 1/2$.

Termwise bound. Hence

$$q_t^s = q_t^{\,s \bmod t}\,(q_t^t)^{\lfloor s/t \rfloor} \leq 2^{-\lfloor s/t \rfloor}.$$

Reindexing. On the residue class $\{s : s \bmod t = r\}$, the map $s \mapsto \lfloor s/t \rfloor$ is injective, and it maps $\{s < N\}$ into $\{k < N\}$. Therefore the sum is at most $p_t \sum_{k<N} 2^{-k} \leq 2p_t \leq 2/t$, by Lemma 3.41. □

Lemma 3.43 (mc_geom_mod)**.** *Let $t \geq 1$ and $f : \mathbb{N}_0 \to \mathbb{R}$ with $f \geq 0$. Then the series $\sum_s p_t q_t^s f(s \bmod t)$ is summable, and*

$$\sum_{s \geq 0} p_t q_t^s f(s \bmod t) \leq \frac{2}{t} \sum_{r=0}^{t-1} f(r).$$

Lean: mc_geom_mod — proved by the captain after a hand-back

```
theorem mc_geom_mod (t : ℕ) (ht : 1 ≤ t) (f : ℕ → ℝ) (hf : ∀ r, 0 ≤ f r) :
    Summable (fun s : ℕ => (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * f (s % t)) ∧
    ∑' s : ℕ, (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * f (s % t)
      ≤ (2 / (t : ℝ)) * ∑ r ∈ Finset.range t, f r := by
  have ht0 : 0 < t := ht
  have htR : (0:ℝ) < t := by exact_mod_cast ht0
  have hq0 : (0:ℝ) ≤ (t : ℝ) / ((t : ℝ) + 1) := by positivity
  have hq1 : (t : ℝ) / ((t : ℝ) + 1) < 1 := by
    rw [div_lt_one (by positivity)]; linarith
  have hnn : ∀ s : ℕ, 0 ≤ (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * f (s % t) := by
    intro s
    exact mul_nonneg (by positivity) (hf _)
  have hfle : ∀ s : ℕ, f (s % t) ≤ ∑ r ∈ Finset.range t, f r := fun s =>
    Finset.single_le_sum (fun i _ => hf i) (Finset.mem_range.mpr (Nat.mod_lt s ht0))
  have hpart : ∀ N : ℕ, ∑ s ∈ Finset.range N,
      (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * f (s % t)
      ≤ (2 / (t : ℝ)) * ∑ r ∈ Finset.range t, f r := by
    intro N
    rw [← Finset.sum_fiberwise_of_maps_to (g := fun s => s % t)
      (fun s _ => Finset.mem_range.mpr (Nat.mod_lt s ht0))]
    rw [Finset.mul_sum]
    refine Finset.sum_le_sum (fun r _ => ?_)
    have hc := mc_geom_mod_coeff t ht r N
    calc ∑ s ∈ (Finset.range N).filter (fun s => s % t = r),
          (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * f (s % t)
        = (∑ s ∈ (Finset.range N).filter (fun s => s % t = r),
            (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s) * f r := by
          rw [Finset.sum_mul]
          refine Finset.sum_congr rfl (fun s hs => ?_)
          rw [(Finset.mem_filter.mp hs).2]
      _ ≤ 2 / (t : ℝ) * f r := mul_le_mul_of_nonneg_right hc (hf r)
  have hsum : Summable (fun s : ℕ =>
      (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * f (s % t)) := by
    refine Summable.of_nonneg_of_le hnn (fun s => ?_)
    ((summable_geometric_of_lt_one hq0 hq1).mul_left
```

```
      ((1 / ((t : ℝ) + 1)) * ∑ r ∈ Finset.range t, f r))
  show _ ≤ (1 / ((t : ℝ) + 1)) * (∑ r ∈ Finset.range t, f r) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s
  have h1 := hfle s
  have h2 : 0 ≤ (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s := by positivity
  calc (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * f (s % t)
      ≤ (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * ∑ r ∈ Finset.range t, f r :=
        mul_le_mul_of_nonneg_left h1 h2
    _ = (1 / ((t : ℝ) + 1)) * (∑ r ∈ Finset.range t, f r) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s := by
        ring
exact ⟨hsum, Real.tsum_le_of_sum_range_le hnn hpart⟩
```

Proof. Summability. We have $f(s \bmod t) \le \sum_{r<t} f(r)$, so the series is dominated by a multiple of the geometric series.

Partial sums. Split each partial sum over $s < N$ according to the residue $r = s \bmod t \in \{0, \dots, t-1\}$. On each class the factor $f(r)$ is constant, so by Lemma 3.42 that class contributes at most $(2/t) f(r)$.

Conclusion. A series of nonnegative terms whose partial sums are uniformly bounded has sum at most that bound. □

Lemma 3.44 (mc_geom_summable_bdd). *Let $t \in \mathbb{N}_0$ and let $f : \mathbb{N}_0 \to \mathbb{R}$ be nonnegative and bounded above. Then $\sum_s p_t q_t^s f(s)$ is summable.*

Lean: mc_geom_summable_bdd — proved by AIProver, job 1, session 0

```
theorem mc_geom_summable_bdd (t : ℕ) (f : ℕ → ℝ) (hnn : ∀ s, 0 ≤ f s)
    (hbdd : ∃ C, ∀ s, f s ≤ C) :
    Summable (fun s : ℕ => (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * f s) := by
  rcases hbdd with ⟨C, hC⟩
  have h_nonneg : ∀ s, 0 ≤ (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * f s := by
    intro s
    have hpos1 : 0 ≤ 1 / ((t : ℝ) + 1) := div_nonneg (by norm_num) (by positivity)
    have hpos2 : 0 ≤ ((t : ℝ) / ((t : ℝ) + 1)) ^ s := pow_nonneg (div_nonneg (Nat.cast_nonneg _) (by
↪  positivity)) s
    have hpos3 : 0 ≤ f s := hnn s
    positivity
  have h_bound : ∀ s, (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * f s ≤ (1 / ((t : ℝ) + 1)) *
↪  ((t : ℝ) / ((t : ℝ) + 1)) ^ s * C := by
    intro s
    gcongr
    exact hC s
  have h_geometric_summable : Summable (fun s : ℕ => ((t : ℝ) / ((t : ℝ) + 1)) ^ s) := by
    have h_ratio_nonneg : 0 ≤ (t : ℝ) / ((t : ℝ) + 1) := div_nonneg (Nat.cast_nonneg _) (by positivity)
    by_cases ht0 : (t : ℝ) = 0
    · have h : Summable (fun s : ℕ => (0 : ℝ) ^ s) := by
        simp
      simpa [ht0] using h
    · have hpos : 0 < (t : ℝ) := by
        apply Nat.cast_pos.mpr
        exact Nat.pos_of_ne_zero (by
          intro hzero
          apply ht0
          exact_mod_cast hzero)
      have h_ratio_lt_one : (t : ℝ) / ((t : ℝ) + 1) < 1 := by
        refine (div_lt_one ?_).mpr ?_
        · positivity
```

```
      · linarith
    exact summable_geometric_of_lt_one h_ratio_nonneg h_ratio_lt_one
  have hC_nonneg : 0 ≤ C := by
    have h0 := hC 0
    have h0nn := hnn 0
    linarith
  have h_geometric_summable_mul : Summable (fun s : ℕ => (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^
↪  s * C) := by
    have htemp := h_geometric_summable.mul_left C
    have htemp2 := htemp.mul_right (1 / ((t : ℝ) + 1))
    simpa [mul_assoc, mul_comm, mul_left_comm] using htemp2
  exact Summable.of_nonneg_of_le h_nonneg h_bound h_geometric_summable_mul
```

Proof. Compare with $C\,p_t q_t^s$, where C is an upper bound for f. For $t = 0$ we have $q_0 = 0$, and the dominating series is trivially summable. □

Lemma 3.45 (mc_D_divmod)**.** *Let $t \in \mathbb{N}_0$ and $D \geq 0$ with $\sqrt{D}$ subadditive. Then for all s,*

$$D(s) \leq 2\lfloor s/t \rfloor^2 D(t) + 2D(s \bmod t),$$

with Lean's conventions $\lfloor s/0 \rfloor = 0$ and $s \bmod 0 = s$.

Lean: mc_D_divmod — proved by AIProver, job 2, session 0

```
theorem mc_D_divmod (t : ℕ) (D : ℕ → ℝ) (hnn : ∀ s, 0 ≤ D s)
    (hsub : ∀ r s, Real.sqrt (D (r + s)) ≤ Real.sqrt (D r) + Real.sqrt (D s)) (s : ℕ) :
    D s ≤ 2 * (((s / t : ℕ) : ℝ)) ^ 2 * D t + 2 * D (s % t) := by
  rcases Nat.eq_zero_or_pos t with (htz | htp)
  · subst htz
    simp
    have hDs : 0 ≤ D s := hnn s
    nlinarith
  · have h_sq_subadd : ∀ r s : ℕ, D (r + s) ≤ 2 * D r + 2 * D s := by
      intro r s
      have h_sqrt := hsub r s
      have h_nonneg_r : 0 ≤ Real.sqrt (D r) := Real.sqrt_nonneg _
      have h_nonneg_s : 0 ≤ Real.sqrt (D s) := Real.sqrt_nonneg _
      have h_nonneg_Drs : 0 ≤ D (r + s) := hnn (r + s)
      have h_sq_sqrt : (Real.sqrt (D (r + s))) ^ 2 = D (r + s) := Real.sq_sqrt (hnn (r + s))
      have h_nonneg_sqrt_sum : 0 ≤ Real.sqrt (D r) + Real.sqrt (D s) :=
        add_nonneg h_nonneg_r h_nonneg_s
      have hsq : (Real.sqrt (D (r + s))) ^ 2 ≤ (Real.sqrt (D r) + Real.sqrt (D s)) ^ 2 := by
        have : Real.sqrt (D (r + s)) ≤ Real.sqrt (D r) + Real.sqrt (D s) := h_sqrt
        have hpos1 : 0 ≤ Real.sqrt (D (r + s)) := Real.sqrt_nonneg _
        have hpos2 : 0 ≤ Real.sqrt (D r) + Real.sqrt (D s) := h_nonneg_sqrt_sum
        nlinarith
      rw [h_sq_sqrt] at hsq
      have h_expand : (Real.sqrt (D r) + Real.sqrt (D s)) ^ 2 =
          D r + D s + 2 * Real.sqrt (D r) * Real.sqrt (D s) := by
        ring
        simp [Real.sq_sqrt (hnn r), Real.sq_sqrt (hnn s)]
      have h_amgm : 2 * Real.sqrt (D r) * Real.sqrt (D s) ≤ D r + D s := by
        have h_sq_nonneg : (Real.sqrt (D r) - Real.sqrt (D s)) ^ 2 ≥ 0 := by positivity
        have h_expand' : (Real.sqrt (D r) - Real.sqrt (D s)) ^ 2 =
            D r + D s - 2 * Real.sqrt (D r) * Real.sqrt (D s) := by
          ring
```

```
      simp [Real.sq_sqrt (hnn r), Real.sq_sqrt (hnn s)]
    nlinarith
  nlinarith
set q := s / t with hq_def
set r' := s % t with hr'_def
have h_divmod : s = q * t + r' := by
  rw [hq_def, hr'_def]
  rw [mul_comm]
  exact (Nat.div_add_mod s t).symm
by_cases hqz : q = 0
·
  have hs_eq_r' : s = r' := by
    rw [hqz] at h_divmod
    simpa using h_divmod
  rw [hs_eq_r']

  have hq0 : (q : ℝ) = 0 := by exact_mod_cast hqz
  simp [hq0]
  have hDr' : 0 ≤ D r' := hnn r'
  nlinarith
·
  have hq_pos : 1 ≤ q := Nat.one_le_of_lt (Nat.pos_of_ne_zero hqz)
  have h_D_qt : D (q * t) ≤ ((q : ℕ) : ℝ) ^ 2 * D t := by
    have h_sqrt_qt : Real.sqrt (D (q * t)) ≤ (q : ℝ) * Real.sqrt (D t) := by
      refine Nat.le_induction ?_ (fun k hk ih => ?_) q hq_pos
      ·
        simp
      ·
        have h_eq : (k.succ : ℕ) * t = k * t + t := by
          rw [Nat.succ_mul]
        rw [h_eq]
        calc
          Real.sqrt (D (k * t + t)) ≤ Real.sqrt (D (k * t)) + Real.sqrt (D t) := hsub (k * t) t
          _ ≤ (k : ℝ) * Real.sqrt (D t) + Real.sqrt (D t) := by nlinarith
          _ = ((k.succ : ℕ) : ℝ) * Real.sqrt (D t) := by push_cast; ring
    have h_sq : D (q * t) ≤ ((q : ℝ) * Real.sqrt (D t)) ^ 2 := by
      have h_sq_sqrt : (Real.sqrt (D (q * t))) ^ 2 = D (q * t) := Real.sq_sqrt (hnn (q * t))
      have h_nonneg_sqrt : 0 ≤ Real.sqrt (D (q * t)) := Real.sqrt_nonneg _
      have hsq' : (Real.sqrt (D (q * t))) ^ 2 ≤ ((q : ℝ) * Real.sqrt (D t)) ^ 2 := by
        have : Real.sqrt (D (q * t)) ≤ (q : ℝ) * Real.sqrt (D t) := h_sqrt_qt
        have : 0 ≤ Real.sqrt (D (q * t)) := h_nonneg_sqrt
        have : 0 ≤ (q : ℝ) * Real.sqrt (D t) :=
          mul_nonneg (by exact_mod_cast Nat.zero_le _) (Real.sqrt_nonneg _)
        nlinarith
      rw [h_sq_sqrt] at hsq'
      exact hsq'
    calc
      D (q * t) ≤ ((q : ℝ) * Real.sqrt (D t)) ^ 2 := h_sq
      _ = ((q : ℝ) ^ 2) * ((Real.sqrt (D t)) ^ 2) := by ring
      _ = ((q : ℝ) ^ 2) * D t := by simp [Real.sq_sqrt (hnn t)]
      _ = ((q : ℕ) : ℝ) ^ 2 * D t := by simp
  rw [h_divmod]
  calc
    D (q * t + r') ≤ 2 * D (q * t) + 2 * D r' := h_sq_subadd (q * t) r'
    _ ≤ 2 * (((q : ℕ) : ℝ) ^ 2 * D t) + 2 * D r' := by nlinarith
    _ = 2 * (((q : ℕ) : ℝ) ^ 2) * D t + 2 * D r' := by ring
    _ = 2 * (((s / t : ℕ) : ℝ)) ^ 2 * D t + 2 * D (s % t) := by simp [hq_def, hr'_def]
```

Proof. For $t = 0$ the claim reads $D(s) \le 2D(s)$. For $t \ge 1$, write $s = jt + r$ with $j = \lfloor s/t \rfloor$ and $r = s \bmod t$. - If $j = 0$, then $D(s) = D(r) \le 2D(r)$. - If $j \ge 1$, induction gives $\sqrt{D(jt)} \le j\sqrt{D(t)}$, so $D(jt) \le j^2 D(t)$. Subadditivity gives $D(a+b) \le 2D(a)+2D(b)$. Hence $D(s) \le 2j^2 D(t) + 2D(r)$. □

Lemma 3.46 (mc_geom_cesaro_pre)**.** *Let $t \ge 1$, and let $D \ge 0$ be bounded above with $\sqrt{D}$ subadditive. Then*

$$\sum_{s\ge 0} p_t q_t^s D(s) \le 2D(t) \sum_{s\ge 0} p_t q_t^s \lfloor s/t \rfloor^2 + 2 \sum_{s\ge 0} p_t q_t^s D(s \bmod t).$$

Lean: mc_geom_cesaro_pre — proved by AIProver, job 3, session 2

```
theorem mc_geom_cesaro_pre (t : ℕ) (ht : 1 ≤ t) (D : ℕ → ℝ)
    (hnn : ∀ s, 0 ≤ D s) (hbdd : ∃ C, ∀ s, D s ≤ C)
    (hsub : ∀ r s, Real.sqrt (D (r + s)) ≤ Real.sqrt (D r) + Real.sqrt (D s)) :
    ∑' s : ℕ, (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * D s
      ≤ 2 * D t * ∑' s : ℕ, (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * (((s / t : ℕ) : ℝ)) ^ 2
        + 2 * ∑' s : ℕ, (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * D (s % t) := by
  set p := 1 / ((t : ℝ) + 1) with hp_def
  set q := (t : ℝ) / ((t : ℝ) + 1) with hq_def
  have ht_nonneg' : 0 ≤ (t : ℝ) := Nat.cast_nonneg _
  have hp_nonneg : 0 ≤ p := by
    rw [hp_def]
    exact div_nonneg (by norm_num) (by nlinarith)
  have hq_nonneg : 0 ≤ q := by
    rw [hq_def]
    exact div_nonneg ht_nonneg' (by nlinarith)
  have hq_lt_one : q < 1 := by
    rw [hq_def]
    apply (div_lt_one (by nlinarith)).mpr
    linarith
  have hq_norm_lt_one : ‖q‖ < 1 := by
    rwa [Real.norm_of_nonneg hq_nonneg]
  have hq_summable : Summable (fun s : ℕ => q ^ s) :=
    summable_geometric_of_lt_one hq_nonneg hq_lt_one
  have hq_pow_summable (k : ℕ) : Summable (fun s : ℕ => (s : ℝ) ^ k * q ^ s) := by
    apply summable_pow_mul_geometric_of_norm_lt_one k hq_norm_lt_one
  have h_divmod (s : ℕ) : D s ≤ 2 * (((s / t : ℕ) : ℝ)) ^ 2 * D t + 2 * D (s % t) :=
    mc_D_divmod t D hnn hsub s

  have h_div_sq_le_sq (s : ℕ) : ((s / t : ℕ) : ℝ) ^ 2 ≤ (s : ℝ) ^ 2 := by
    have hdiv : (s / t : ℕ) ≤ s := Nat.div_le_self s t
    have hcast : ((s / t : ℕ) : ℝ) ≤ (s : ℝ) := Nat.cast_le.mpr hdiv
    nlinarith

  have h_summable_div_sq_mul_q : Summable (fun s : ℕ => ((s / t : ℕ) : ℝ) ^ 2 * q ^ s) := by
    have h_summable_sq_mul_q : Summable (fun s : ℕ => (s : ℝ) ^ 2 * q ^ s) := hq_pow_summable 2
    refine h_summable_sq_mul_q.of_nonneg_of_le
      (fun s => by
        have hsq : 0 ≤ ((s / t : ℕ) : ℝ) ^ 2 := by positivity
        have hqpow : 0 ≤ q ^ s := pow_nonneg hq_nonneg s
        positivity)
      (fun s => mul_le_mul_of_nonneg_right (h_div_sq_le_sq s) (pow_nonneg hq_nonneg s))

  have h_summable_sq_term : Summable (fun s : ℕ => p * q ^ s * (((s / t : ℕ) : ℝ)) ^ 2) := by
    simpa [mul_comm, mul_left_comm, mul_assoc] using h_summable_div_sq_mul_q.mul_left p
  have h_summable_mod_term : Summable (fun s : ℕ => p * q ^ s * D (s % t)) := by
    rcases hbdd with ⟨C, hC⟩
    have h_nonneg (s : ℕ) : 0 ≤ p * q ^ s * D (s % t) := by
```

```
      have hq_pow : 0 ≤ q ^ s := pow_nonneg hq_nonneg s
      have hDmod_nonneg : 0 ≤ D (s % t) := hnn _
      apply mul_nonneg
      · apply mul_nonneg hp_nonneg hq_pow
      · exact hDmod_nonneg
    have h_bound (s : ℕ) : p * q ^ s * D (s % t) ≤ p * q ^ s * C := by
      have hD : D (s % t) ≤ C := hC (s % t)
      have hq_pow : 0 ≤ q ^ s := pow_nonneg hq_nonneg s
      have h_nonneg_prod : 0 ≤ p * q ^ s := mul_nonneg hp_nonneg hq_pow
      nlinarith
    have h_summable_C : Summable (fun s : ℕ => p * q ^ s * C) := by
      have := hq_summable.mul_left p |>.mul_right C
      simpa [mul_comm, mul_left_comm, mul_assoc] using this
    exact h_summable_C.of_nonneg_of_le h_nonneg h_bound
  have h_summable_rhs_first : Summable (fun s : ℕ => 2 * D t * (p * q ^ s * (((s / t : ℕ) : ℝ)) ^ 2)) := by
    simpa [mul_comm, mul_left_comm, mul_assoc] using h_summable_sq_term.mul_left (2 * D t)
  have h_summable_rhs_second : Summable (fun s : ℕ => 2 * (p * q ^ s * D (s % t))) := by
    simpa [mul_comm, mul_left_comm, mul_assoc] using h_summable_mod_term.mul_left 2
  have h_summable_rhs : Summable (fun s : ℕ =>
      2 * D t * (p * q ^ s * (((s / t : ℕ) : ℝ)) ^ 2) + 2 * (p * q ^ s * D (s % t))) :=
    h_summable_rhs_first.add h_summable_rhs_second

  have h_summable_lhs : Summable (fun s : ℕ => p * q ^ s * D s) := by
    rcases hbdd with ⟨C, hC⟩
    have h_nonneg (s : ℕ) : 0 ≤ p * q ^ s * D s := by
      have hq_pow : 0 ≤ q ^ s := pow_nonneg hq_nonneg s
      have hD_nonneg : 0 ≤ D s := hnn s
      apply mul_nonneg
      · apply mul_nonneg hp_nonneg hq_pow
      · exact hD_nonneg
    have h_bound (s : ℕ) : p * q ^ s * D s ≤ p * q ^ s * C := by
      have hD : D s ≤ C := hC s
      have h_nonneg_prod : 0 ≤ p * q ^ s := mul_nonneg hp_nonneg (pow_nonneg hq_nonneg s)
      nlinarith
    have h_summable_C : Summable (fun s : ℕ => p * q ^ s * C) := by
      have := hq_summable.mul_left p |>.mul_right C
      simpa [mul_comm, mul_left_comm, mul_assoc] using this
    exact h_summable_C.of_nonneg_of_le h_nonneg h_bound

  have h_term_bound (s : ℕ) : p * q ^ s * D s ≤
      p * q ^ s * (2 * (((s / t : ℕ) : ℝ)) ^ 2 * D t + 2 * D (s % t)) := by
    have hpos : 0 ≤ p * q ^ s := mul_nonneg hp_nonneg (pow_nonneg hq_nonneg s)
    have hDmod_nonneg : 0 ≤ 2 * (((s / t : ℕ) : ℝ)) ^ 2 * D t + 2 * D (s % t) := by
      have hsq : 0 ≤ ((s / t : ℕ) : ℝ) ^ 2 := by positivity
      have hD_nonneg : 0 ≤ D t := hnn t
      have hDmod_nonneg' : 0 ≤ D (s % t) := hnn _
      nlinarith
    have hdivmod := h_divmod s
    nlinarith

  have h_summable_rhs' : Summable (fun s : ℕ => p * q ^ s * (2 * (((s / t : ℕ) : ℝ)) ^ 2 * D t + 2 * D (s %
↪  t))) := by
    simpa [mul_add, add_mul, mul_comm, mul_left_comm, mul_assoc] using h_summable_rhs
  calc
    ∑' s : ℕ, p * q ^ s * D s
    _ ≤ ∑' s : ℕ, p * q ^ s * (2 * (((s / t : ℕ) : ℝ)) ^ 2 * D t + 2 * D (s % t)) :=
      h_summable_lhs.tsum_le_tsum h_term_bound h_summable_rhs'
    _ = ∑' s : ℕ, (2 * D t * (p * q ^ s * (((s / t : ℕ) : ℝ)) ^ 2) + 2 * (p * q ^ s * D (s % t))) := by
      refine tsum_congr (fun s => ?_)
```

```
    ring
  _ = ∑' s : ℕ, 2 * D t * (p * q ^ s * (((s / t : ℕ) : ℝ)) ^ 2)
      + ∑' s : ℕ, 2 * (p * q ^ s * D (s % t)) :=
    h_summable_rhs_first.tsum_add h_summable_rhs_second
  _ = 2 * D t * (∑' s : ℕ, p * q ^ s * (((s / t : ℕ) : ℝ)) ^ 2)
      + 2 * (∑' s : ℕ, p * q ^ s * D (s % t)) := by
    simp [tsum_mul_left]
  _ = 2 * D t * ∑' s : ℕ, (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * (((s / t : ℕ) : ℝ)) ^ 2
      + 2 * ∑' s : ℕ, (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * D (s % t) := by
    simp [hp_def, hq_def]
```

Proof. All three series are summable: - the first and the third by boundedness of D and comparison with the geometric series; - the second because $\lfloor s/t \rfloor^2 \le s^2$ and $\sum s^2 q_t^s$ converges (Mathlib).

Multiply Lemma 3.45 by $p_t q_t^s \ge 0$, sum termwise, and split the right-hand series by linearity. □

Lemma 3.47 (mc_range_le_Icc). *Let $t \in \mathbb{N}_0$, $D(0) = 0$ and $D \ge 0$. Then $\sum_{r=0}^{t-1} D(r) \le \sum_{s=1}^{t} D(s)$.*

Lean: mc_range_le_Icc — proved by AIProver, job 1, session 0

```
theorem mc_range_le_Icc (t : ℕ) (D : ℕ → ℝ) (hD0 : D 0 = 0) (hnn : ∀ s, 0 ≤ D s) :
    ∑ r ∈ Finset.range t, D r ≤ ∑ s ∈ Finset.Icc 1 t, D s := by
  by_cases ht : t = 0
  · subst ht; simp
  · have htpos : 1 ≤ t := by omega
    have h0_mem_range : (0 : ℕ) ∈ Finset.range t := by
      simpa [htpos] using Finset.mem_range.2 (by omega)
    have ht_mem_Icc : t ∈ Finset.Icc (1 : ℕ) t := by
      simp [htpos]
    have h_erase_eq : (Finset.range t).erase 0 = (Finset.Icc (1 : ℕ) t).erase t := by
      ext x
      simp [Finset.mem_range]
      omega
    have hsum_range_erase : ∑ r ∈ Finset.range t, D r = D 0 + ∑ r ∈ (Finset.range t).erase 0, D r := by
      have := Finset.sum_erase_add (Finset.range t) D h0_mem_range

      linarith [this]
    have hsum_Icc_erase : D t + ∑ r ∈ (Finset.Icc (1 : ℕ) t).erase t, D r = ∑ s ∈ Finset.Icc (1 : ℕ) t,
↪  D s := by
      have := Finset.sum_erase_add (Finset.Icc (1 : ℕ) t) D ht_mem_Icc
      linarith [this]
    calc
      ∑ r ∈ Finset.range t, D r = D 0 + ∑ r ∈ (Finset.range t).erase 0, D r := hsum_range_erase
      _ = (0 : ℝ) + ∑ r ∈ (Finset.range t).erase 0, D r := by rw [hD0]
      _ = ∑ r ∈ (Finset.range t).erase 0, D r := by simp
      _ = ∑ r ∈ (Finset.Icc (1 : ℕ) t).erase t, D r := by rw [h_erase_eq]
      _ ≤ D t + ∑ r ∈ (Finset.Icc (1 : ℕ) t).erase t, D r := by
        nlinarith [hnn t]
      _ = ∑ s ∈ Finset.Icc (1 : ℕ) t, D s := by rw [hsum_Icc_erase]
```

Proof. The two sums share the terms $1 \le r \le t-1$. The left side has the extra term $D(0) = 0$, and the right side has the extra term $D(t) \ge 0$. □

Lemma 3.48 (mc_geom_cesaro). *Let $t \geq 1$ and let $D : \mathbb{N}_0 \to \mathbb{R}$ satisfy - $D(0) = 0$, $D \geq 0$ and D bounded above; - $\sqrt{D}$ subadditive.*

Then

$$\sum_{s \geq 0} p_t q_t^s D(s) \leq 28 \cdot \frac{1}{t} \sum_{s=1}^{t} D(s).$$

Lean: mc_geom_cesaro — proved by the captain after a hand-back

```
theorem mc_geom_cesaro (t : ℕ) (ht : 1 ≤ t) (D : ℕ → ℝ) (hD0 : D 0 = 0)
    (hnn : ∀ s, 0 ≤ D s) (hbdd : ∃ C, ∀ s, D s ≤ C)
    (hsub : ∀ r s, Real.sqrt (D (r + s)) ≤ Real.sqrt (D r) + Real.sqrt (D s)) :
    ∑' s : ℕ, (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * D s
      ≤ 28 * ((1 / (t : ℝ)) * ∑ s ∈ Finset.Icc 1 t, D s) := by
  have hpre := mc_geom_cesaro_pre t ht D hnn hbdd hsub
  obtain ⟨_, hS2⟩ := mc_geom_sq_div t ht
  obtain ⟨_, hmod⟩ := mc_geom_mod t ht D hnn
  have hDt := mc_cesaro_Dt t ht D hD0 hnn hsub
  have hrange := mc_range_le_Icc t D hD0 hnn
  have htpos : (0:ℝ) < t := by exact_mod_cast ht
  have hDt0 := hnn t
  have h1 : (2 / (t:ℝ)) * ∑ r ∈ Finset.range t, D r
      ≤ 2 * ((1 / (t:ℝ)) * ∑ s ∈ Finset.Icc 1 t, D s) := by
    rw [show (2 / (t:ℝ)) * ∑ r ∈ Finset.range t, D r
        = 2 * ((1 / (t:ℝ)) * ∑ r ∈ Finset.range t, D r) by ring]
    exact mul_le_mul_of_nonneg_left
      (mul_le_mul_of_nonneg_left hrange (by positivity)) (by norm_num)
  have h2 : 2 * D t * ∑' s : ℕ, (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * (((s / t : ℕ) : ℝ))
↪   ^ 2
      ≤ 2 * D t * 3 :=
    mul_le_mul_of_nonneg_left hS2 (by linarith)
  linarith [hpre, h2, hmod, h1, hDt]
```

Proof. Let $W = \frac{1}{t}\sum_{s=1}^{t} D(s)$. Bound the two terms of Lemma 3.46: - By Lemma 3.40 and then Lemma 3.36, the first term is at most $2D(t) \cdot 3 \leq 24W$. - By Lemma 3.43 and then Lemma 3.47, the second term is at most $2 \cdot \frac{2}{t}\sum_{r<t} D(r) \leq 4W$.

The total is at most $28W$. □

Lemma 3.49 (mc_resolvent_D_eq). *Let $t \in \mathbb{N}_0$, $\pi : \mathrm{Fin}\, n \to \mathbb{R}$, A stochastic, and $x : \mathrm{Fin}\, n \to \ell_1$. Let $G_{ij} = \sum_s p_t q_t^s A_{ij}^s$ and $D(s) = \sum_{i,j} \pi_i A_{ij}^s \|x_i - x_j\|^2$. Then*

$$\sum_i \sum_j \pi_i G_{ij} \|x_i - x_j\|^2 = \sum_{s \geq 0} p_t q_t^s D(s).$$

Lean: mc_resolvent_D_eq — proved by the captain after a hand-back

```
theorem mc_resolvent_D_eq {n : ℕ} (t : ℕ) (π : Fin n → ℝ)
    (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochastic A) (x : Fin n → L1)
    (G : Matrix (Fin n) (Fin n) ℝ)
    (hG : ∀ i j, G i j = ∑' s : ℕ, (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * (A ^ s) i j)
    (D : ℕ → ℝ) (hD : ∀ s, D s = ∑ i, ∑ j, π i * (A ^ s) i j * ‖x i - x j‖ ^ 2) :
    ∑ i, ∑ j, π i * G i j * ‖x i - x j‖ ^ 2
      = ∑' s : ℕ, (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * D s := by
  classical
  have hsum := mc_resolvent_summable t A hA
```

```
  have h1 : ∀ i j, HasSum (fun s : ℕ => π i * ((1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * (A ^
↪  s) i j) * ‖x i - x j‖ ^ 2)
      (π i * G i j * ‖x i - x j‖ ^ 2) := by
    intro i j
    rw [hG i j]
    exact ((hsum i j).hasSum.mul_left (π i)).mul_right _
  have h2 := hasSum_sum (s := Finset.univ) (fun i _ => hasSum_sum (s := Finset.univ) (fun j _ => h1 i j))
  have key : HasSum (fun s : ℕ => (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * D s)
      (∑ i, ∑ j, π i * G i j * ‖x i - x j‖ ^ 2) := by
    convert h2 using 1
    funext s
    rw [hD s, Finset.mul_sum]
    refine Finset.sum_congr rfl (fun i _ => ?_)
    rw [Finset.mul_sum]
    refine Finset.sum_congr rfl (fun j _ => ?_)
    ring
  exact key.tsum_eq.symm
```

Proof. Each series defining G_{ij} converges (Lemma 3.4). Multiply it by the constant $\pi_i\|x_i - x_j\|^2$. A finite sum over (i,j) of convergent series converges to the sum of the limits. Its terms are $p_t q_t^s D(s)$. In particular, the proof shows that the series on the right converges; this is not part of the formal statement. □

Lemma 3.50 (pow_nonneg_entry). *If $A_{ij} \geq 0$ for all i, j, then $A^s_{ij} \geq 0$ for all s, i, j.*

Lean: pow_nonneg_entry — helper of mc_D_props, proved by AIProver, job 2, session 0

```
open MarkovCotype in
lemma pow_nonneg_entry {n : ℕ} (A : Matrix (Fin n) (Fin n) ℝ) (hA : ∀ i j, 0 ≤ A i j) :
    ∀ s i j, 0 ≤ (A ^ s) i j := by
  intro s i j
  induction s generalizing i j with
  | zero =>
    simp [Matrix.one_apply]
    split <;> positivity
  | succ k ih =>
    rw [pow_succ', Matrix.mul_apply]
    refine Finset.sum_nonneg (fun k' _ => ?_)
    have h1 : 0 ≤ A i k' := hA i k'
    have h2 : 0 ≤ (A ^ k) k' j := ih k' j
    exact mul_nonneg h1 h2
```

Proof. Induction on s, using $A^{s+1} = AA^s$. □

Lemma 3.51 (pow_stochastic_row_sum). *If A is stochastic, then $\sum_j A^s_{ij} = 1$ for all s and i.*

Lean: pow_stochastic_row_sum — helper of mc_D_props, proved by AIProver, job 2, session 0

```
open MarkovCotype in
lemma pow_stochastic_row_sum {n : ℕ} (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochastic A) (s : ℕ) (i : Fin
↪  n) :
    ∑ j, (A ^ s) i j = 1 := by
  have hA_row_sum : ∀ i, ∑ j, A i j = 1 := hA.2
  induction s with
```

```
  | zero => simp [Matrix.one_apply, Finset.sum_ite_eq, Finset.mem_univ]
  | succ k ih =>
    rw [pow_succ]
    calc
      ∑ j, (A ^ k * A) i j = ∑ j, ∑ l, (A ^ k) i l * A l j := by
        simp [Matrix.mul_apply]
      _ = ∑ l, ∑ j, (A ^ k) i l * A l j := by rw [Finset.sum_comm]
      _ = ∑ l, (A ^ k) i l * (∑ j, A l j) := by simp [Finset.mul_sum]
      _ = ∑ l, (A ^ k) i l * 1 := by simp [hA_row_sum]
      _ = ∑ l, (A ^ k) i l := by simp
      _ = 1 := ih
```

Proof. Induction on s, using $A^{s+1} = A^s A$ and exchanging the order of summation. □

Lemma 3.52 (pow_stochastic_entry_le_one). *If A is stochastic, then $A^s_{ij} \le 1$ for all s, i, j.*

Lean: pow_stochastic_entry_le_one — helper of mc_D_props, proved by AIProver, job 2, session 0

```
open MarkovCotype in
lemma pow_stochastic_entry_le_one {n : ℕ} (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochastic A)
    (s : ℕ) (i j : Fin n) : (A ^ s) i j ≤ 1 := by
  have h_entry_nonneg : ∀ i j, 0 ≤ (A ^ s) i j := fun i j =>
    pow_nonneg_entry A hA.1 s i j
  have h_row_sum := pow_stochastic_row_sum A hA s i
  have h_single_le_sum : (A ^ s) i j ≤ ∑ j', (A ^ s) i j' :=
    Finset.single_le_sum (fun k _ => h_entry_nonneg i k) (Finset.mem_univ j)
  rw [h_row_sum] at h_single_le_sum
  exact h_single_le_sum
```

Proof. A single nonnegative term (Lemma 3.50) is at most the row sum, which equals 1 (Lemma 3.51). □

Lemma 3.53 (pow_stochastic). *If A is stochastic, then A^s is stochastic for every s.*

Lean: pow_stochastic — helper of mc_D_props, proved by AIProver, job 2, session 0

```
open MarkovCotype in
lemma pow_stochastic {n : ℕ} (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochastic A) (s : ℕ) :
    IsStochastic (A ^ s) := by
  have hA_nonneg : ∀ i j, 0 ≤ A i j := hA.1
  have h_entry_nonneg : ∀ i j, 0 ≤ (A ^ s) i j := fun i j =>
    pow_nonneg_entry A hA_nonneg s i j
  have h_row_sum : ∀ i, ∑ j, (A ^ s) i j = 1 :=
    pow_stochastic_row_sum A hA s
  exact ⟨h_entry_nonneg, h_row_sum⟩
```

Proof. Combine Lemmas 3.50 and 3.51. □

Lemma 3.54 (mc_D_props). *Let π be a probability vector, A stochastic, $x : \mathrm{Fin}\, n \to \ell_1$, and $D(s) = \sum_{i,j} \pi_i A^s_{ij} \|x_i - x_j\|^2$. Then $D(0) = 0$, $D(s) \ge 0$ for all s, and D is bounded above.*

Lean: mc_D_props — proved by AIProver, job 2, session 0

```
theorem mc_D_props {n : ℕ} (π : Fin n → ℝ) (hπ : IsProbVec π)
    (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochastic A) (x : Fin n → L1) (D : ℕ → ℝ)
    (hD : ∀ s, D s = ∑ i, ∑ j, π i * (A ^ s) i j * ‖x i - x j‖ ^ 2) :
    D 0 = 0 ∧ (∀ s, 0 ≤ D s) ∧ ∃ C, ∀ s, D s ≤ C := by
  have hπ_nonneg : ∀ i, 0 ≤ π i := hπ.1
  have hπ_sum : ∑ i, π i = 1 := hπ.2
  have hA_nonneg : ∀ i j, 0 ≤ A i j := hA.1

  have hpow0 : (A ^ 0 : Matrix (Fin n) (Fin n) ℝ) = 1 := by
    simpa using pow_zero A
  have hD0 : D 0 = 0 := by
    rw [hD 0, hpow0]
    simp [Matrix.one_apply]
  have hnonneg : ∀ s, 0 ≤ D s := by
    intro s
    rw [hD s]
    refine Finset.sum_nonneg (fun i _ => Finset.sum_nonneg (fun j _ => ?_))
    have hA_nonneg_s : 0 ≤ (A ^ s) i j := pow_nonneg_entry A hA_nonneg s i j
    have h_norm_sq_nonneg : 0 ≤ ‖x i - x j‖ ^ 2 := pow_two_nonneg _
    apply mul_nonneg
    · apply mul_nonneg (hπ_nonneg i) hA_nonneg_s
    · exact h_norm_sq_nonneg
  have hbdd : ∃ C, ∀ s, D s ≤ C := by
    by_cases h_ne : Nonempty (Fin n)
    · obtain ⟨i₀⟩ := h_ne
      let B : ℝ := ∑ i, ∑ j, ‖x i - x j‖ ^ 2
      have hB : 0 ≤ B := Finset.sum_nonneg (fun i _ => Finset.sum_nonneg (fun j _ => by positivity))
      use B
      intro s
      rw [hD s]
      calc
        ∑ i, ∑ j, π i * (A ^ s) i j * ‖x i - x j‖ ^ 2
            ≤ ∑ i, ∑ j, π i * 1 * ‖x i - x j‖ ^ 2 := by
          refine Finset.sum_le_sum (fun i _ => ?_)
          refine Finset.sum_le_sum (fun j _ => ?_)
          have h_entry_le_one : (A ^ s) i j ≤ 1 :=
            pow_stochastic_entry_le_one A hA s i j
          have h_entry_nonneg : 0 ≤ (A ^ s) i j := pow_nonneg_entry A hA_nonneg s i j
          have hπ_nonneg_i : 0 ≤ π i := hπ_nonneg i
          have h_norm_sq_nonneg : 0 ≤ ‖x i - x j‖ ^ 2 := by positivity
          gcongr
        _ = ∑ i, ∑ j, π i * ‖x i - x j‖ ^ 2 := by simp
        _ ≤ ∑ i, ∑ j, 1 * ‖x i - x j‖ ^ 2 := by
          refine Finset.sum_le_sum (fun i _ => Finset.sum_le_sum (fun j _ => ?_))
          have hπi : π i ≤ 1 := by
            have : π i ≤ ∑ k, π k := Finset.single_le_sum (fun k _ => hπ_nonneg k) (Finset.mem_univ i)
            rw [hπ_sum] at this
            exact this
          gcongr
        _ = ∑ i, ∑ j, ‖x i - x j‖ ^ 2 := by simp
        _ = B := rfl
    ·
      have h_empty : IsEmpty (Fin n) := not_nonempty_iff.mp h_ne
      use 0
      intro s
      rw [hD s]
      simp
```

```
  exact ⟨hD0, hnonneg, hbdd⟩
```

Proof. - $A^0 = I$, so $D(0) = \sum_i \pi_i \|x_i - x_i\|^2 = 0$. - Nonnegativity follows from Lemma 3.50 and $\pi \geq 0$. - Since $0 \leq A^s_{ij} \leq 1$ (Lemma 3.52) and $0 \leq \pi_i \leq 1$, we get $D(s) \leq \sum_{i,j} \|x_i - x_j\|^2$. For $n = 0$ the bound 0 works. □

Lemma 3.55 (mc_cesaro_rhs_eq). *For $t \in \mathbb{N}_0$, π, any real matrix A, $x : \mathrm{Fin}\, n \to \ell_1$ and D as above,*

$$\sum_i \sum_j \pi_i \Big(\frac{1}{t} \sum_{s=1}^{t} A^s_{ij}\Big) \|x_i - x_j\|^2 = \frac{1}{t} \sum_{s=1}^{t} D(s).$$

Lean: mc_cesaro_rhs_eq — proved by AIProver, job 3, session 0

```
theorem mc_cesaro_rhs_eq {n : ℕ} (t : ℕ) (π : Fin n → ℝ)
    (A : Matrix (Fin n) (Fin n) ℝ) (x : Fin n → L1) (D : ℕ → ℝ)
    (hD : ∀ s, D s = ∑ i, ∑ j, π i * (A ^ s) i j * ‖x i - x j‖ ^ 2) :
    ∑ i, ∑ j, π i * ((1 / (t : ℝ)) * ∑ s ∈ Finset.Icc 1 t, (A ^ s) i j) * ‖x i - x j‖ ^ 2
      = (1 / (t : ℝ)) * ∑ s ∈ Finset.Icc 1 t, D s := by
  have hsum : (∑ i, ∑ j, ∑ s ∈ Finset.Icc 1 t, π i * (A ^ s) i j * ‖x i - x j‖ ^ 2)
      = (∑ s ∈ Finset.Icc 1 t, ∑ i, ∑ j, π i * (A ^ s) i j * ‖x i - x j‖ ^ 2) := by
    calc
      (∑ i, ∑ j, ∑ s ∈ Finset.Icc 1 t, π i * (A ^ s) i j * ‖x i - x j‖ ^ 2)
          = (∑ i, ∑ s ∈ Finset.Icc 1 t, ∑ j, π i * (A ^ s) i j * ‖x i - x j‖ ^ 2) := by
        refine Finset.sum_congr rfl fun i _ => ?_
        rw [Finset.sum_comm]
      _ = (∑ s ∈ Finset.Icc 1 t, ∑ i, ∑ j, π i * (A ^ s) i j * ‖x i - x j‖ ^ 2) := by
        rw [Finset.sum_comm]
  calc
    ∑ i, ∑ j, π i * ((1 / (t : ℝ)) * ∑ s ∈ Finset.Icc 1 t, (A ^ s) i j) * ‖x i - x j‖ ^ 2
        = (1 / (t : ℝ)) * ∑ i, ∑ j, ∑ s ∈ Finset.Icc 1 t, π i * (A ^ s) i j * ‖x i - x j‖ ^ 2 := by
      simp only [Finset.mul_sum, mul_assoc, mul_comm, mul_left_comm]
    _ = (1 / (t : ℝ)) * (∑ i, ∑ j, ∑ s ∈ Finset.Icc 1 t, π i * (A ^ s) i j * ‖x i - x j‖ ^ 2) := rfl
    _ = (1 / (t : ℝ)) * (∑ s ∈ Finset.Icc 1 t, ∑ i, ∑ j, π i * (A ^ s) i j * ‖x i - x j‖ ^ 2) := by
      rw [hsum]
    _ = (1 / (t : ℝ)) * ∑ s ∈ Finset.Icc 1 t, D s := by
      simp only [hD]
```

Proof. Distribute, and interchange the finite sums over i, j and s. □

Lemma 3.56 (mc_comparison_aux). *Let $t \geq 1$, let π be a probability vector, and let A be stochastic with $\sum_i \pi_i A_{ij} = \pi_j$. Let $x : \mathrm{Fin}\, n \to \ell_1$, $G_{ij} = \sum_s p_t q^s_t A^s_{ij}$ and $D(s) = \sum_{i,j} \pi_i A^s_{ij} \|x_i - x_j\|^2$. Then*

$$\sum_i \sum_j \pi_i G_{ij} \|x_i - x_j\|^2 \leq 28 \sum_i \sum_j \pi_i \Big(\frac{1}{t} \sum_{s=1}^{t} A^s_{ij}\Big) \|x_i - x_j\|^2.$$

Lean: mc_comparison_aux — helper of mc_comparison, proved by the captain after a hand-back

```
theorem mc_comparison_aux {n : ℕ} (t : ℕ) (ht : 1 ≤ t) (π : Fin n → ℝ) (hπ : IsProbVec π)
    (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochastic A)
    (hstat : ∀ j, ∑ i, π i * A i j = π j) (x : Fin n → L1)
    (G : Matrix (Fin n) (Fin n) ℝ)
    (hG : ∀ i j, G i j = ∑' s : ℕ, (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * (A ^ s) i j)
```

```
    (D : ℕ → ℝ) (hD : ∀ s : ℕ, D s = ∑ i, ∑ j, π i * (A ^ s) i j * ‖x i - x j‖ ^ 2) :
    ∑ i, ∑ j, π i * G i j * ‖x i - x j‖ ^ 2
      ≤ 28 * ∑ i, ∑ j,
          π i * ((1 / (t : ℝ)) * ∑ s ∈ Finset.Icc 1 t, (A ^ s) i j) * ‖x i - x j‖ ^ 2 := by
  have h1 := mc_resolvent_D_eq t π A hA x G hG D hD
  have h2 := mc_cesaro_rhs_eq t π A x D hD
  have hprops := mc_D_props π hπ A hA x D hD
  have hsub : ∀ r s : ℕ, Real.sqrt (D (r + s)) ≤ Real.sqrt (D r) + Real.sqrt (D s) :=
    fun r s => mc_D_subadd π hπ A hA hstat x D hD r s
  have hgc := mc_geom_cesaro t ht D hprops.1 hprops.2.1 hprops.2.2 hsub
  first
    | (rw [h1, h2]; exact hgc)
    | linarith [h1, h2, hgc]
```

Proof. Rewrite the left side by Lemma 3.49 and the right side by Lemma 3.55. The hypotheses of Lemma 3.48 hold: - $D(0) = 0$, $D \geq 0$ and D is bounded, by Lemma 3.54; - $\sqrt{D}$ is subadditive, by Lemma 3.35.

Apply Lemma 3.48. □

Lemma 3.57 (mc_comparison). *Under the hypotheses of Lemma 3.56, without the auxiliary function D,*

$$\sum_i \sum_j \pi_i G_{ij} \|x_i - x_j\|^2 \leq 28 \sum_i \sum_j \pi_i \Big(\frac{1}{t} \sum_{s=1}^{t} A_{ij}^s\Big) \|x_i - x_j\|^2.$$

Lean: mc_comparison — proved by the captain after a hand-back

```
theorem mc_comparison {n : ℕ} (t : ℕ) (ht : 1 ≤ t) (π : Fin n → ℝ) (hπ : IsProbVec π)
    (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochastic A)
    (hstat : ∀ j, ∑ i, π i * A i j = π j) (x : Fin n → L1)
    (G : Matrix (Fin n) (Fin n) ℝ)
    (hG : ∀ i j, G i j = ∑' s : ℕ, (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * (A ^ s) i j) :
    ∑ i, ∑ j, π i * G i j * ‖x i - x j‖ ^ 2
      ≤ 28 * ∑ i, ∑ j,
          π i * ((1 / (t : ℝ)) * ∑ s ∈ Finset.Icc 1 t, (A ^ s) i j) * ‖x i - x j‖ ^ 2 := by
  set_option maxHeartbeats 1000000 in
  exact mc_comparison_aux t ht π hπ A hA hstat x G hG _ (fun s => rfl)
```

Proof. Apply Lemma 3.56 with D defined by its formula. □

3.3 Proof of the theorem

Lean: solution — main proof from the captain's sketch

```
theorem solution
    (n t : ℕ) (ht : 1 ≤ t)
    (π : Fin n → ℝ) (hπ : IsProbVec π)
    (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochastic A)
    (hrev : IsReversible π A)
    (x : Fin n → L1) :
    ∃ y : Fin n → L1,
      ∑ i, π i * ‖x i - y i‖ ^ 2
        + (t : ℝ) * ∑ i, ∑ j, π i * A i j * ‖y i - y j‖ ^ 2
```

```
      ≤ 3024 * ∑ i, ∑ j,
          π i * ((1 / (t : ℝ)) * ∑ s ∈ Finset.Icc 1 t, (A ^ s) i j)
            * ‖x i - x j‖ ^ 2 := by
  have hstat := mc_stationary π A hA hrev
  obtain ⟨w, T, hw, hT, hdist, hcontr⟩ := mc_cut_L1 x
  obtain ⟨G, hG⟩ : ∃ G : Matrix (Fin n) (Fin n) ℝ, ∀ i j, G i j = ∑' s : ℕ, (1 / ((t : ℝ) + 1)) * ((t :
↪  ℝ) / ((t : ℝ) + 1)) ^ s * (A ^ s) i j :=
    ⟨fun i j => ∑' s : ℕ, (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * (A ^ s) i j, fun _ _ => rfl⟩
  obtain ⟨hG0, hGrow, hGπ⟩ := mc_resolvent_basic t π A hA hstat G hG
  obtain ⟨hAG, hGA⟩ := mc_resolvent_first_step t A hA G hG
  obtain ⟨φ, hφ⟩ : ∃ φ : ℝ → ℝ, ∀ r, φ r = 3 * r ^ 2 - 2 * r ^ 3 :=
    ⟨fun r => 3 * r ^ 2 - 2 * r ^ 3, fun _ => rfl⟩
  have hz : ∀ (i : Fin n) (B : Finset (Fin n)),
      (if i ∈ B then (1:ℝ) else 0) = 0 ∨ (if i ∈ B then (1:ℝ) else 0) = 1 := by
    intro i B
    by_cases h : i ∈ B <;> simp [h]
  have hcube := mc_cube_energy t π hπ A hA w hw (fun i B => if i ∈ B then (1:ℝ) else 0) hz φ hφ G hG0
↪  hGrow hAG hGA hGπ
  obtain ⟨y, hy⟩ := mc_transfer t π hπ.1 A hA.1 x w hw (fun i B => if i ∈ B then (1:ℝ) else 0) hz T hT
↪  hdist hcontr G
    (fun i b => φ (∑ j, G i j * (if j ∈ b then (1:ℝ) else 0))) hcube
  have hcomp := mc_comparison t ht π hπ A hA hstat x G hG
  exact ⟨y, by linarith⟩
```

Proof. Let n, t, π, A, x be as in the target statement, with $t \geq 1$.

Step 1: stationarity. By Lemma 3.1, $\sum_i \pi_i A_{ij} = \pi_j$ for all j. This is the only use of reversibility.

Step 2: cuts. Put $z_i(B) = \mathbf{1}[i \in B]$ for $B \in \mathcal{P}(\mathrm{Fin}\, n)$; these are bits. Lemma 3.28 gives weights $w \geq 0$ on $\mathcal{P}(\mathrm{Fin}\, n)$ and a map $T : \mathbb{R}^{\mathcal{P}(\mathrm{Fin}\, n)} \to \ell_1$ such that - $T(z_i) = x_i$; - $\|x_i - x_j\| = \|z_i - z_j\|_{1,w}$; - $\|T(u) - T(u')\| \leq \|u - u'\|_{1,w}$.

Step 3: resolvent. Define $G_{ij} = \sum_{s \geq 0} p_t q_t^s A_{ij}^s$. By Lemma 3.5, $G \geq 0$, G has row sums 1, and $\sum_i \pi_i G_{ij} = \pi_j$. By Lemma 3.7, $G = p_t I + q_t A G = p_t I + q_t G A$.

Step 4: cube inequality. Let $\varphi(r) = 3r^2 - 2r^3$ and $h_i = \sum_j G_{ij} z_j$. Lemma 3.17, with $\iota = \mathcal{P}(\mathrm{Fin}\, n)$, gives

$$\sum_i \pi_i \|z_i - \Phi(h_i)\|_{1,w}^2 + t \sum_{i,j} \pi_i A_{ij} \|\Phi(h_i) - \Phi(h_j)\|_{1,w}^2 \leq 108 \sum_{i,j} \pi_i G_{ij} \|z_i - z_j\|_{2,w}^4.$$

Step 5: transfer to ℓ_1. Lemma 3.29, with $c_i = \Phi(h_i)$ and using $\pi \geq 0$ and $A \geq 0$, produces $y_i = T(\Phi(h_i)) \in \ell_1$ with

$$\sum_i \pi_i \|x_i - y_i\|^2 + t \sum_{i,j} \pi_i A_{ij} \|y_i - y_j\|^2 \leq 108 \sum_{i,j} \pi_i G_{ij} \|x_i - x_j\|^2.$$

Step 6: comparison. By Lemma 3.57 (which uses $t \geq 1$ and stationarity), the right side is at most

$$108 \cdot 28 \sum_{i,j} \pi_i \Big(\frac{1}{t} \sum_{s=1}^{t} A_{ij}^s \Big) \|x_i - x_j\|^2.$$

Since $108 \cdot 28 = 3024$, this is the asserted inequality. □

4 Independent review

A referee model (claude-opus-5-5), which took no part in the formalization or the proof, compared the verified Lean file with the source. Verdict: **FAITHFUL**.

Definitions. `L1` is `lp` with constant fibre $\mathbb{R}$ over $\mathbb{N} = \{0, 1, \ldots\}$ and exponent 1. Its norm is $\sum_k |x(k)|$, which the proof confirms in `mc_L1_norm`. Re-indexing from $\{1, 2, \ldots\}$ to $\{0, 1, \ldots\}$ is an isometric isomorphism, so this is the source's ℓ_1.

`IsProbVec` says the entries are nonnegative and sum to 1; zero entries are allowed, as in the source.

`IsStochastic` says the entries are nonnegative and every row sums to 1.

`IsReversible` is $\pi_i a_{ij} = \pi_j a_{ji}$ for all i, j, which is detailed balance as stated.

A^s is the Mathlib monoid power on `Matrix (Fin n) (Fin n) Real`, i.e. the ordinary matrix power.

No discrepancies.

Statement. Quantification is over all $n, t \in \mathbb{N}$ with $1 \le t$, every π satisfying `IsProbVec`, every A satisfying `IsStochastic` and `IsReversible`, and every family $x : \mathrm{Fin}\, n \to \ell_1$. This matches the source.

The case $n = 0$ is admitted, but there the hypothesis $\sum_i \pi_i = 1$ fails. That case is therefore vacuous, which is harmless because the source only concerns $n \ge 1$.

The conclusion asserts the existence of y with

$$\sum_i \pi_i \|x_i - y_i\|^2 + t \sum_{i,j} \pi_i a_{ij} \|y_i - y_j\|^2 \le 3024 \sum_{i,j} \pi_i \Big(\tfrac{1}{t} \sum_{s \in [1,t]} (A^s)_{ij} \Big) \|x_i - x_j\|^2.$$

The range `Finset.Icc 1 t` is $\{1, \ldots, t\}$, and $1/t$ is well defined since $t \ge 1$. This is exactly the source inequality with constant 3024. Nothing extra needs to be supplied.

Proof. Cut representation (Lemma 1). The cuts are built per coordinate.

- `mc_cut_real_mono` handles a sorted vector using upper cuts with gaps. `mc_cut_real_pointwise` reduces the general case to it via `Tuple.sort`.
- `mc_cut_real` re-indexes the cuts by all subsets $B \subseteq [n]$, using choice. Zero weights and the empty and full sets are allowed; they are harmless.
- `mc_cut_coords` and `mc_cut_L1` establish summability from the bounds $|v| \le \sum_i |x_i(k)|$ and $b_B \le 2 \sum_i |x_i(k)|$. They define $T(u) = V + \sum_B u_B b_B$ and prove $T(z_i) = x_i$, the distance identity, and the contraction property.

Resolvent. `mc_resolvent_basic` and `mc_resolvent_first_step` define $G = p \sum_s q^s A^s$ and show it is stochastic and π-stationary. They also prove both first-step identities, $G = pI + qAG$ and $G = pI + qGA$. Stationarity of π is derived from reversibility in `mc_stationary`, exactly as the source remarks.

Flat map (Lemma 2).

- `mc_flat_increment` proves the increment estimate (eq:increment) in squared form.
- `mc_flat` proves (eq:flat) with constant 108.

Stopped walk (Corollary and Section 4). This is the one place the formalization departs from the source. The martingale and stopping-time argument is replaced by a deterministic algebraic one.

- `mc_one_step` averages (eq:flat) over a single transition of the killed chain, with center $e = z_{i_0}$. The linear term cancels exactly by the harmonic identity.

- `mc_aggregate` weights these one-step inequalities by $\pi_i G_{ij}$ and telescopes using $G = pI + qGA$. It then discards the nonnegative term $p\,F_h(i,i)$ and uses $\pi G = \pi$ on the left.

This is precisely the expectation over the stopped chain that the source computes, written as a Green's-function sum. It yields $B + tE_A \le 108 \sum_{i,j} \pi_i G_{ij} \|x_i - x_j\|_1^2$, i.e. the combination of (eq:stopped-bound) and (eq:jump-identity). `mc_transfer` then pushes this through T using contractivity.

Comparison (Lemma 3). The structure follows the source closely; constants match ($EJ^2 \le 3$, residue weight $\le 2/t$, total 28).

- `mc_resolvent_D_eq` rewrites the resolvent sum as $\sum_s pq^s D(s)$.
- `mc_D_subadd` proves L_2 subadditivity, via Minkowski and stationarity.
- `mc_cesaro_Dt` proves $D(t) \le 4W$.
- `mc_D_divmod` gives the block bound.
- `mc_geom_sq_div` gives the bound 3.
- `mc_geom_mod` gives the $2/t$ residue weight.
- `mc_range_le_Icc` uses $D(0) = 0$.

Conclusion. The final step combines 108 and 28 by `linarith`, giving 3024. No intermediate lemma is weaker than the step it represents.

Concerns. None

A Complete Lean file

The verified file, as checked by Lean.

```
import Mathlib
set_option autoImplicit false

namespace MarkovCotype

/-- The real Banach space ℓ₁ of absolutely summable real sequences
(indexed by ℕ; the shift from {1,2,...} to {0,1,...} is an isometry). -/
abbrev L1 : Type := lp (fun _ : ℕ => ℝ) 1

/-- A probability vector on [n]: nonnegative entries summing to one (zeros allowed). -/
def IsProbVec {n : ℕ} (π : Fin n → ℝ) : Prop :=
  (∀ i, 0 ≤ π i) ∧ ∑ i, π i = 1

/-- A stochastic matrix: nonnegative entries and every row sum equal to one. -/
def IsStochastic {n : ℕ} (A : Matrix (Fin n) (Fin n) ℝ) : Prop :=
  (∀ i j, 0 ≤ A i j) ∧ ∀ i, ∑ j, A i j = 1

/-- Reversibility (detailed balance) of `A` with respect to `π`. -/
def IsReversible {n : ℕ} (π : Fin n → ℝ) (A : Matrix (Fin n) (Fin n) ℝ) : Prop :=
  ∀ i j, π i * A i j = π j * A j i

end MarkovCotype
```

```
open MarkovCotype

theorem mc_stationary {n : ℕ} (π : Fin n → ℝ) (A : Matrix (Fin n) (Fin n) ℝ)
    (hA : IsStochastic A) (hrev : IsReversible π A) :
    ∀ j, ∑ i, π i * A i j = π j := by
  intro j
  have hrowsum : ∀ i, ∑ j', A i j' = 1 := hA.2
  calc
    ∑ i, π i * A i j = ∑ i, π j * A j i := by
      refine Finset.sum_congr rfl fun i _ => ?_
      rw [hrev i j]
    _ = π j * ∑ i, A j i := by
      simp [Finset.mul_sum]
    _ = π j * 1 := by rw [hrowsum j]
    _ = π j := by ring

theorem mc_pow_stochastic {n : ℕ} (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochastic A)
    (s : ℕ) : IsStochastic (A ^ s) := by
  rcases hA with ⟨hA_nonneg, hA_sum⟩
  induction' s with s ih
  ·
    refine ⟨?_, ?_⟩
    · intro i j
      by_cases h : i = j
      · simp [h]
      · simp [h]
    · intro i
      simp [Matrix.one_apply]
  ·
    rcases ih with ⟨h_nonneg, h_sum⟩
    have h_mul_nonneg : ∀ i j, 0 ≤ (A ^ s * A) i j := by
      intro i j
      rw [Matrix.mul_apply]
      refine Finset.sum_nonneg fun k _ => ?_
      exact mul_nonneg (h_nonneg i k) (hA_nonneg k j)
    have h_mul_sum : ∀ i, ∑ j : Fin n, (A ^ s * A) i j = 1 := by
      intro i
      simp [Matrix.mul_apply]
      calc
        ∑ x : Fin n, ∑ j : Fin n, (A ^ s) i j * A j x = ∑ j : Fin n, ∑ x : Fin n, (A ^ s) i j * A j x :=
          Finset.sum_comm
        _ = ∑ j : Fin n, ((A ^ s) i j * ∑ x : Fin n, A j x) := by
          simp [Finset.mul_sum]
        _ = ∑ j : Fin n, ((A ^ s) i j * 1) := by simp [hA_sum]
        _ = ∑ j : Fin n, (A ^ s) i j := by simp
        _ = 1 := h_sum i
    have : (A : Matrix (Fin n) (Fin n) ℝ) ^ (s + 1) = (A ^ s) * A := pow_succ A s
    rw [this]
    exact ⟨h_mul_nonneg, h_mul_sum⟩

theorem mc_pow_stationary {n : ℕ} (π : Fin n → ℝ) (A : Matrix (Fin n) (Fin n) ℝ)
    (hstat : ∀ j, ∑ i, π i * A i j = π j) (s : ℕ) :
    ∀ j, ∑ i, π i * (A ^ s) i j = π j := by
  induction' s with s ih
  ·
    intro j
    simp [Matrix.one_apply]
  ·
```

```
    intro j
    have h_pow_succ : (A ^ (s + 1)) = (A ^ s) * A := by
      simp [pow_succ]
    rw [h_pow_succ]


    simp [Matrix.mul_apply]

    calc
      ∑ i, π i * (∑ k, (A ^ s) i k * A k j)
          = ∑ i, ∑ k, π i * (A ^ s) i k * A k j := by
        refine Finset.sum_congr rfl fun i _ => ?_
        simp_rw [Finset.mul_sum]
        refine Finset.sum_congr rfl fun k _ => ?_
        ring
      _ = ∑ k, ∑ i, π i * (A ^ s) i k * A k j := Finset.sum_comm
      _ = ∑ k, (∑ i, π i * (A ^ s) i k) * A k j := by
        refine Finset.sum_congr rfl fun k _ => ?_
        simp_rw [Finset.sum_mul]
      _ = ∑ k, π k * A k j := by
        refine Finset.sum_congr rfl fun k _ => ?_
        rw [ih k]
      _ = π j := hstat j

theorem mc_resolvent_summable {n : ℕ} (t : ℕ) (A : Matrix (Fin n) (Fin n) ℝ)
    (hA : IsStochastic A) (i j : Fin n) :
    Summable (fun s : ℕ => (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * (A ^ s) i j) := by
  set r := (t : ℝ) / ((t : ℝ) + 1) with hr
  have hr_nonneg : 0 ≤ r :=
    div_nonneg (Nat.cast_nonneg _) (by positivity)
  have hr_lt_one : r < 1 := by
    refine (div_lt_one ?_).mpr ?_
    · positivity
    · have : (t : ℝ) < (t : ℝ) + 1 := by linarith
      exact_mod_cast this
  have h_geom_summable : Summable fun s : ℕ => r ^ s :=
    summable_geometric_of_lt_one hr_nonneg hr_lt_one
  have h_entry_nonneg : ∀ s : ℕ, ∀ i' j', 0 ≤ (A ^ s) i' j' := by
    intro s
    induction' s with s ih
    · intro i' j'
      by_cases h : i' = j'
      · subst h; simpa [Matrix.one_apply] using (le_refl (1 : ℝ) : (0 : ℝ) ≤ (1 : ℝ))
      · simpa [h, Matrix.one_apply] using (zero_le (0 : ℝ) : (0 : ℝ) ≤ (0 : ℝ))
    · intro i' j'
      rw [pow_succ', Matrix.mul_apply]
      refine Finset.sum_nonneg fun k _ => ?_
      refine mul_nonneg (hA.1 i' k) (ih k j')
  have h_entry_le_one : ∀ s : ℕ, ∀ i' j', (A ^ s) i' j' ≤ 1 := by
    intro s
    induction' s with s ih
    · intro i' j'
      by_cases h : i' = j'
      · subst h; simpa [Matrix.one_apply] using (le_refl (1 : ℝ) : (1 : ℝ) ≤ (1 : ℝ))
      · simpa [h, Matrix.one_apply] using (zero_le (0 : ℝ) : (0 : ℝ) ≤ (1 : ℝ))
    · intro i' j'
      rw [pow_succ', Matrix.mul_apply]
      calc
        ∑ k, A i' k * (A ^ s) k j' ≤ ∑ k, A i' k * 1 :=
```

```
          Finset.sum_le_sum fun k _ => mul_le_mul_of_nonneg_left (ih k j') (hA.1 i' k)
        _ = ∑ k, A i' k := by simp
        _ = 1 := hA.2 i'
  have h_coeff_nonneg : ∀ s : ℕ, 0 ≤ (1 / ((t : ℝ) + 1)) * r ^ s * (A ^ s) i j := by
    intro s
    refine mul_nonneg (mul_nonneg ?_ (pow_nonneg hr_nonneg s)) (h_entry_nonneg s i j)
    refine div_nonneg (by norm_num) (by positivity)
  have h_bound : ∀ s : ℕ, (1 / ((t : ℝ) + 1)) * r ^ s * (A ^ s) i j ≤
      (1 / ((t : ℝ) + 1)) * r ^ s := by
    intro s
    have htemp := h_entry_le_one s i j
    calc
      (1 / ((t : ℝ) + 1)) * r ^ s * (A ^ s) i j ≤
          (1 / ((t : ℝ) + 1)) * r ^ s * 1 :=
        mul_le_mul_of_nonneg_left htemp (by
          refine mul_nonneg (div_nonneg (by norm_num) (by positivity)) (pow_nonneg hr_nonneg s))
      _ = (1 / ((t : ℝ) + 1)) * r ^ s := by simp
  have h_summable_term : Summable fun s : ℕ => (1 / ((t : ℝ) + 1)) * r ^ s :=
    Summable.mul_left (1 / ((t : ℝ) + 1)) h_geom_summable
  exact Summable.of_nonneg_of_le h_coeff_nonneg h_bound h_summable_term

theorem mc_resolvent_basic {n : ℕ} (t : ℕ) (π : Fin n → ℝ)
    (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochastic A)
    (hstat : ∀ j, ∑ i, π i * A i j = π j)
    (G : Matrix (Fin n) (Fin n) ℝ)
    (hG : ∀ i j, G i j = ∑' s : ℕ, (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * (A ^ s) i j) :
    (∀ i j, 0 ≤ G i j) ∧ (∀ i, ∑ j, G i j = 1) ∧ (∀ j, ∑ i, π i * G i j = π j) := by
  have ht1 : (0:ℝ) < (t:ℝ) + 1 := by positivity
  have hr0 : (0:ℝ) ≤ (t:ℝ) / ((t:ℝ) + 1) := div_nonneg (Nat.cast_nonneg t) ht1.le
  have hr1 : (t:ℝ) / ((t:ℝ) + 1) < 1 := by
    rw [div_lt_one ht1]; linarith
  have hsub : (1:ℝ) - (t:ℝ) / ((t:ℝ) + 1) = 1 / ((t:ℝ) + 1) := by
    field_simp <;> ring
  have hc : HasSum (fun s : ℕ => (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s) 1 := by
    have h := (hasSum_geometric_of_lt_one hr0 hr1).mul_left (1 / ((t : ℝ) + 1))
    rw [hsub] at h
    have h1 : (1 / ((t : ℝ) + 1)) * (1 / ((t : ℝ) + 1))⁻¹ = 1 := mul_inv_cancel₀ (by positivity)
    rwa [h1] at h
  have hGs : ∀ i j, HasSum (fun s : ℕ => (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * (A ^ s) i j)
↪  (G i j) := by
    intro i j
    rw [hG i j]
    exact (mc_resolvent_summable t A hA i j).hasSum
  have hnn : ∀ i j, 0 ≤ G i j := by
    intro i j
    rw [hG i j]
    exact tsum_nonneg (fun s => mul_nonneg (mul_nonneg (by positivity) (pow_nonneg hr0 s))
      ((mc_pow_stochastic A hA s).1 i j))
  have hrow : ∀ i, ∑ j, G i j = 1 := by
    intro i
    have h : HasSum (fun s : ℕ => ∑ j, (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * (A ^ s) i j)
↪  (∑ j, G i j) :=
      hasSum_sum (fun j _ => hGs i j)
    have h2 : HasSum (fun s : ℕ => (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s) (∑ j, G i j) := by
      convert h using 1
      funext s
      rw [← Finset.mul_sum, (mc_pow_stochastic A hA s).2 i, mul_one]
    exact h2.unique hc
  have hstatG : ∀ j, ∑ i, π i * G i j = π j := by
```

```
    intro j
    have h : HasSum (fun s : ℕ => ∑ i, π i * ((1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * (A ^
↪  s) i j)) (∑ i, π i * G i j) :=
      hasSum_sum (fun i _ => (hGs i j).mul_left (π i))
    have h2 : HasSum (fun s : ℕ => π j * ((1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s)) (∑ i, π i
↪  * G i j) := by
      convert h using 1
      funext s
      rw [← mc_pow_stationary π A hstat s j, Finset.sum_mul]
      refine Finset.sum_congr rfl (fun i _ => ?_)
      ring
    have h3 := hc.mul_left (π j)
    rw [mul_one] at h3
    exact h2.unique h3
  exact ⟨hnn, hrow, hstatG⟩

theorem mc_resolvent_first_step_aux {n : ℕ} (p q : ℝ) (A G : Matrix (Fin n) (Fin n) ℝ)
    (hs : ∀ i j, Summable (fun s : ℕ => p * q ^ s * (A ^ s) i j))
    (hG : ∀ i j, G i j = ∑' s : ℕ, p * q ^ s * (A ^ s) i j) :
    (∀ i j, G i j = p * (if i = j then (1:ℝ) else 0) + q * ∑ k, A i k * G k j) ∧
    (∀ i j, G i j = p * (if i = j then (1:ℝ) else 0) + q * ∑ k, G i k * A k j) := by
  constructor
  · intro i j
    have h1 : ∑ k, A i k * G k j = ∑' s : ℕ, ∑ k, A i k * (p * q ^ s * (A ^ s) k j) := by
      rw [Summable.tsum_finsetSum]
      · refine Finset.sum_congr rfl (fun k _ => ?_)
        rw [hG k j]
        exact tsum_mul_left.symm
      · intro k _
        exact (hs k j).mul_left (A i k)
    rw [h1, ← tsum_mul_left, hG i j, (hs i j).tsum_eq_zero_add]
    congr 1
    · first
        | simp only [pow_zero, mul_one, Matrix.one_apply]
        | simp [Matrix.one_apply]
    · refine tsum_congr (fun s => ?_)
      rw [pow_succ' A s, Matrix.mul_apply]
      simp only [Finset.mul_sum]
      refine Finset.sum_congr rfl (fun k _ => ?_)
      ring
  · intro i j
    have h2 : ∑ k, G i k * A k j = ∑' s : ℕ, ∑ k, (p * q ^ s * (A ^ s) i k) * A k j := by
      rw [Summable.tsum_finsetSum]
      · refine Finset.sum_congr rfl (fun k _ => ?_)
        rw [hG i k]
        exact tsum_mul_right.symm
      · intro k _
        exact (hs i k).mul_right (A k j)
    rw [h2, ← tsum_mul_left, hG i j, (hs i j).tsum_eq_zero_add]
    congr 1
    · first
        | simp only [pow_zero, mul_one, Matrix.one_apply]
        | simp [Matrix.one_apply]
    · refine tsum_congr (fun s => ?_)
      rw [pow_succ A s, Matrix.mul_apply]
      simp only [Finset.mul_sum]
      refine Finset.sum_congr rfl (fun k _ => ?_)
      ring
```

```
theorem mc_resolvent_first_step {n : ℕ} (t : ℕ) (A : Matrix (Fin n) (Fin n) ℝ)
    (hA : IsStochastic A) (G : Matrix (Fin n) (Fin n) ℝ)
    (hG : ∀ i j, G i j = ∑' s : ℕ, (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * (A ^ s) i j) :
    (∀ i j, G i j = (1 / ((t : ℝ) + 1)) * (if i = j then (1:ℝ) else 0)
        + ((t : ℝ) / ((t : ℝ) + 1)) * ∑ k, A i k * G k j) ∧
    (∀ i j, G i j = (1 / ((t : ℝ) + 1)) * (if i = j then (1:ℝ) else 0)
        + ((t : ℝ) / ((t : ℝ) + 1)) * ∑ k, G i k * A k j) := by
  have hs : ∀ i j, Summable (fun s : ℕ => (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * (A ^ s) i
↪  j) :=
    fun i j => mc_resolvent_summable t A hA i j
  exact mc_resolvent_first_step_aux _ _ A G hs hG

theorem mc_harmonic_point {n : ℕ} {ι : Type*} (t : ℕ) (A G : Matrix (Fin n) (Fin n) ℝ)
    (hG0 : ∀ i j, 0 ≤ G i j) (hGrow : ∀ i, ∑ j, G i j = 1)
    (hAG : ∀ i j, G i j = (1 / ((t : ℝ) + 1)) * (if i = j then (1:ℝ) else 0)
        + ((t : ℝ) / ((t : ℝ) + 1)) * ∑ k, A i k * G k j)
    (z : Fin n → ι → ℝ) (hz : ∀ i b, z i b = 0 ∨ z i b = 1) :
    (∀ i b, 0 ≤ ∑ j, G i j * z j b ∧ ∑ j, G i j * z j b ≤ 1) ∧
    (∀ i b, ∑ j, G i j * z j b = (1 / ((t : ℝ) + 1)) * z i b
        + ((t : ℝ) / ((t : ℝ) + 1)) * ∑ k, A i k * ∑ j, G k j * z j b) := by
  have hz_nonneg : ∀ i b, 0 ≤ z i b := by
    intro i b
    rcases hz i b with (h | h)
    · rw [h]
    · rw [h]; norm_num
  have hz_le_one : ∀ i b, z i b ≤ 1 := by
    intro i b
    rcases hz i b with (h | h)
    · rw [h]; norm_num
    · rw [h]
  have part1 : ∀ i b, 0 ≤ ∑ j : Fin n, G i j * z j b ∧ ∑ j : Fin n, G i j * z j b ≤ 1 := by
    intro i b
    have h_nonneg_sum : 0 ≤ ∑ j : Fin n, G i j * z j b := by
      refine Finset.sum_nonneg fun j _ => ?_
      have hGij : 0 ≤ G i j := hG0 i j
      have hzjb : 0 ≤ z j b := hz_nonneg j b
      nlinarith
    have h_le_one_sum : ∑ j : Fin n, G i j * z j b ≤ 1 := by
      calc
        ∑ j : Fin n, G i j * z j b ≤ ∑ j : Fin n, G i j * 1 := by
          refine Finset.sum_le_sum fun j _ => ?_
          have hzjb : z j b ≤ 1 := hz_le_one j b
          nlinarith [hG0 i j]
        _ = ∑ j : Fin n, G i j := by simp
        _ = 1 := hGrow i
    exact And.intro h_nonneg_sum h_le_one_sum
  have part2 : ∀ i b, ∑ j : Fin n, G i j * z j b = (1 / ((t : ℝ) + 1)) * z i b
      + ((t : ℝ) / ((t : ℝ) + 1)) * ∑ k : Fin n, A i k * ∑ j : Fin n, G k j * z j b := by
    intro i b
    calc
      ∑ j : Fin n, G i j * z j b
          = ∑ j : Fin n, ((1 / ((t : ℝ) + 1)) * (if i = j then (1:ℝ) else 0)
              + ((t : ℝ) / ((t : ℝ) + 1)) * ∑ k : Fin n, A i k * G k j) * z j b := by
        refine Finset.sum_congr rfl fun j _ => ?_
        rw [hAG i j]
      _ = ∑ j : Fin n, ((1 / ((t : ℝ) + 1)) * (if i = j then (1:ℝ) else 0) * z j b
              + ((t : ℝ) / ((t : ℝ) + 1)) * (∑ k : Fin n, A i k * G k j) * z j b) := by
        refine Finset.sum_congr rfl fun j _ => ?_
        ring
```

```
      _ = (∑ j : Fin n, (1 / ((t : ℝ) + 1)) * (if i = j then (1:ℝ) else 0) * z j b)
        + (∑ j : Fin n, ((t : ℝ) / ((t : ℝ) + 1)) * (∑ k : Fin n, A i k * G k j) * z j b) :=
        Finset.sum_add_distrib
      _ = (1 / ((t : ℝ) + 1)) * z i b
        + (∑ j : Fin n, ((t : ℝ) / ((t : ℝ) + 1)) * (∑ k : Fin n, A i k * G k j) * z j b) := by
        simp
      _ = (1 / ((t : ℝ) + 1)) * z i b
        + ((t : ℝ) / ((t : ℝ) + 1)) * (∑ k : Fin n, A i k * (∑ j : Fin n, G k j * z j b)) := by
        have hsum : (∑ j : Fin n, (∑ k : Fin n, A i k * G k j) * z j b)
            = (∑ k : Fin n, A i k * (∑ j : Fin n, G k j * z j b)) := by
          calc
            (∑ j : Fin n, (∑ k : Fin n, A i k * G k j) * z j b)
                = (∑ j : Fin n, ∑ k : Fin n, (A i k * G k j) * z j b) := by
              simp_rw [Finset.sum_mul]
            _ = (∑ k : Fin n, ∑ j : Fin n, (A i k * G k j) * z j b) := by
              rw [Finset.sum_comm]
            _ = (∑ k : Fin n, A i k * (∑ j : Fin n, G k j * z j b)) := by
              simp_rw [Finset.mul_sum, mul_assoc]
        calc
          (1 / ((t : ℝ) + 1)) * z i b
              + (∑ j : Fin n, ((t : ℝ) / ((t : ℝ) + 1)) * (∑ k : Fin n, A i k * G k j) * z j b)
          = (1 / ((t : ℝ) + 1)) * z i b
              + ((t : ℝ) / ((t : ℝ) + 1)) * (∑ j : Fin n, (∑ k : Fin n, A i k * G k j) * z j b) := by
            simp [Finset.mul_sum, mul_assoc, mul_comm, mul_left_comm]
          _ = (1 / ((t : ℝ) + 1)) * z i b
              + ((t : ℝ) / ((t : ℝ) + 1)) * (∑ k : Fin n, A i k * (∑ j : Fin n, G k j * z j b)) := by
            rw [hsum]
      _ = (1 / ((t : ℝ) + 1)) * z i b + ((t : ℝ) / ((t : ℝ) + 1)) * ∑ k : Fin n, A i k * ∑ j : Fin n, G k
↪  j * z j b := by
        simp
  exact And.intro part1 part2

theorem mc_phi_scalar (φ : ℝ → ℝ) (hφ : ∀ r, φ r = 3 * r ^ 2 - 2 * r ^ 3)
    (r r' d : ℝ) (hr : 0 ≤ r ∧ r ≤ 1) (hr' : 0 ≤ r' ∧ r' ≤ 1) (hd : d = 0 ∨ d = 1) :
    |φ r' - φ r| ≤ 6 * |r - d| * |r' - r| + 3 * (r' - r) ^ 2 := by
  rcases hr with ⟨hr_lo, hr_hi⟩
  rcases hr' with ⟨hr'_lo, hr'_hi⟩
  have hφr : φ r = 3 * r ^ 2 - 2 * r ^ 3 := hφ r
  have hφr' : φ r' = 3 * r' ^ 2 - 2 * r' ^ 3 := hφ r'
  rw [hφr, hφr']
  have h_factor : 3 * r' ^ 2 - 2 * r' ^ 3 - (3 * r ^ 2 - 2 * r ^ 3) = (r' - r) * (3 * (r' + r) - 2 * (r' ^ 2
↪  + r' * r + r ^ 2)) := by
    ring
  rw [h_factor, abs_mul]
  by_cases h_eq : r' = r
  · rw [h_eq, sub_self]
    simp
  · set A := 3 * (r' + r) - 2 * (r' ^ 2 + r' * r + r ^ 2) with hA_def
    have h_absA : |A| ≤ 6 * |r - d| + 3 * |r' - r| := by
      rcases hd with (hd | hd)
      ·
        rw [hd, sub_zero, abs_of_nonneg hr_lo]
        by_cases h_le : r' - r ≥ 0
        · have habs_sub : |r' - r| = r' - r := abs_of_nonneg h_le
          rw [habs_sub]
          have hA_le : A ≤ 3 * (r' + r) := by
            dsimp [A]; nlinarith
          have hA_ge : -(6 * r + 3 * (r' - r)) ≤ A := by
            dsimp [A]; ring_nf; nlinarith
```

```
          exact abs_le.mpr ⟨by linarith, by linarith⟩
        · have h_le' : r - r' ≥ 0 := by linarith
          have habs_sub : |r' - r| = r - r' := by
            rw [abs_sub_comm, abs_of_nonneg h_le']
          rw [habs_sub]
          have hA_le' : |A| ≤ 3 * (r' + r) :=
            abs_le.mpr ⟨by dsimp [A]; nlinarith, by dsimp [A]; nlinarith⟩
          have h_suff : 3 * (r' + r) ≤ 6 * r + 3 * (r - r') := by
            ring_nf; nlinarith
          nlinarith
      ·
        have hr_abs : |r - 1| = 1 - r := by
          rw [abs_of_nonpos (by linarith : r - 1 ≤ 0)]
          ring
        rw [hd, hr_abs]
        by_cases h_le : r' - r ≥ 0
        · have habs_sub : |r' - r| = r' - r := abs_of_nonneg h_le
          rw [habs_sub]
          dsimp [A]
          by_cases hA_nonneg : 0 ≤ 3 * (r' + r) - 2 * (r' ^ 2 + r' * r + r ^ 2)
          · rw [abs_of_nonneg hA_nonneg]; nlinarith
          · have hA_neg : 3 * (r' + r) - 2 * (r' ^ 2 + r' * r + r ^ 2) < 0 := by linarith
            rw [abs_of_neg hA_neg]; ring_nf; nlinarith
        · have h_le' : r - r' ≥ 0 := by linarith
          have habs_sub : |r' - r| = r - r' := by
            rw [abs_sub_comm, abs_of_nonneg h_le']
          rw [habs_sub]
          dsimp [A]
          by_cases hA_nonneg : 0 ≤ 3 * (r' + r) - 2 * (r' ^ 2 + r' * r + r ^ 2)
          · rw [abs_of_nonneg hA_nonneg]; nlinarith
          · have hA_neg : 3 * (r' + r) - 2 * (r' ^ 2 + r' * r + r ^ 2) < 0 := by linarith
            rw [abs_of_neg hA_neg]; ring_nf; nlinarith
    have hpos : 0 ≤ |r' - r| := abs_nonneg _
    calc
      |r' - r| * |A| ≤ |r' - r| * (6 * |r - d| + 3 * |r' - r|) := by
        nlinarith
      _ = 6 * |r - d| * |r' - r| + 3 * (|r' - r| ^ 2) := by ring
      _ = 6 * |r - d| * |r' - r| + 3 * (r' - r) ^ 2 := by rw [sq_abs]

theorem mc_flat_increment {ι : Type*} [Fintype ι] (w : ι → ℝ) (hw : ∀ b, 0 ≤ w b)
    (φ : ℝ → ℝ) (hφ : ∀ r, φ r = 3 * r ^ 2 - 2 * r ^ 3)
    (u u' e : ι → ℝ) (hu : ∀ b, 0 ≤ u b ∧ u b ≤ 1) (hu' : ∀ b, 0 ≤ u' b ∧ u' b ≤ 1)
    (he : ∀ b, e b = 0 ∨ e b = 1) :
    (∑ b, w b * |φ (u' b) - φ (u b)|) ^ 2
      ≤ 72 * (∑ b, w b * (u b - e b) ^ 2) * (∑ b, w b * (u' b - u b) ^ 2)
        + 18 * (∑ b, w b * (u' b - u b) ^ 2) ^ 2 := by
  have hφ_eq : ∀ r : ℝ, φ r = 3 * r ^ 2 - 2 * r ^ 3 := hφ
  have h_key_scalar : ∀ (r r' e : ℝ), 0 ≤ r → r ≤ 1 → 0 ≤ r' → r' ≤ 1 → (e = 0 ∨ e = 1) →
      |φ r' - φ r| ≤ 6 * |r - e| * |r' - r| + 3 * (r' - r) ^ 2 := by
    intro r r' e hr0 hr1 hr'0 hr'1 he_cases
    have hfactor : φ r' - φ r = (r' - r) * (6 * r * (1 - r) + (r' - r) * (3 - 6 * r - 2 * (r' - r))) := by
      calc
        φ r' - φ r = (3 * r' ^ 2 - 2 * r' ^ 3) - (3 * r ^ 2 - 2 * r ^ 3) := by
          simp [hφ_eq]
        _ = (r' - r) * (6 * r * (1 - r) + (r' - r) * (3 - 6 * r - 2 * (r' - r))) := by
          ring
    rw [hfactor]
    set δ := r' - r with hδ_def
    have hsq_nonneg : 0 ≤ δ ^ 2 := pow_two_nonneg _
```

```
  have habs_nonneg : 0 ≤ |δ| := abs_nonneg _
  have h_inner_bound : |3 - 6 * r - 2 * δ| ≤ 3 := by
    have : 3 - 6 * r - 2 * δ = 3 - 4 * r - 2 * r' := by dsimp [δ]; ring
    rw [this]
    exact abs_le.mpr ⟨by nlinarith, by nlinarith⟩
  have h_expr_eq : δ * (6 * r * (1 - r) + δ * (3 - 6 * r - 2 * δ)) =
      6 * r * (1 - r) * δ + δ ^ 2 * (3 - 6 * r - 2 * δ) := by ring
  rw [h_expr_eq]
  rcases he_cases with (he0 | he1)
  · subst he0
    have h_mul_factor : (1 - r) * |δ| ≤ 1 * |δ| :=
      mul_le_mul_of_nonneg_right (by nlinarith) habs_nonneg
    have h1 : |6 * r * (1 - r) * δ| ≤ 6 * r * |δ| := by
      have hpos : 0 ≤ 6 * r * (1 - r) := by nlinarith
      calc
        |6 * r * (1 - r) * δ| = |6 * r * (1 - r)| * |δ| := abs_mul _ _
        _ = 6 * r * (1 - r) * |δ| := by rw [abs_of_nonneg hpos]
        _ = (6 * r) * ((1 - r) * |δ|) := by ring
        _ ≤ (6 * r) * (1 * |δ|) :=
          mul_le_mul_of_nonneg_left h_mul_factor (by nlinarith)
        _ = 6 * r * |δ| := by ring
    have h2 : |δ ^ 2 * (3 - 6 * r - 2 * δ)| ≤ 3 * δ ^ 2 := by
      calc
        |δ ^ 2 * (3 - 6 * r - 2 * δ)| = |δ ^ 2| * |3 - 6 * r - 2 * δ| := abs_mul _ _
        _ = δ ^ 2 * |3 - 6 * r - 2 * δ| := by rw [abs_of_nonneg hsq_nonneg]
        _ ≤ δ ^ 2 * 3 := mul_le_mul_of_nonneg_left h_inner_bound hsq_nonneg
        _ = 3 * δ ^ 2 := by ring
    calc
      |6 * r * (1 - r) * δ + δ ^ 2 * (3 - 6 * r - 2 * δ)| ≤
          |6 * r * (1 - r) * δ| + |δ ^ 2 * (3 - 6 * r - 2 * δ)| := abs_add _ _
      _ ≤ (6 * r * |δ|) + (3 * δ ^ 2) := by nlinarith
      _ = 6 * |r - 0| * |δ| + 3 * δ ^ 2 := by
        have h_abs : |r - 0| = r := by simpa using abs_of_nonneg hr0
        rw [h_abs]
  · subst he1
    have h_mul_factor : r * |δ| ≤ 1 * |δ| :=
      mul_le_mul_of_nonneg_right (by nlinarith) habs_nonneg
    have h1 : |6 * r * (1 - r) * δ| ≤ 6 * (1 - r) * |δ| := by
      have hpos : 0 ≤ 6 * r * (1 - r) := by nlinarith
      calc
        |6 * r * (1 - r) * δ| = |6 * r * (1 - r)| * |δ| := abs_mul _ _
        _ = 6 * r * (1 - r) * |δ| := by rw [abs_of_nonneg hpos]
        _ = (6 * (1 - r)) * (r * |δ|) := by ring
        _ ≤ (6 * (1 - r)) * (1 * |δ|) :=
          mul_le_mul_of_nonneg_left h_mul_factor (by nlinarith)
        _ = 6 * (1 - r) * |δ| := by ring
    have h2 : |δ ^ 2 * (3 - 6 * r - 2 * δ)| ≤ 3 * δ ^ 2 := by
      calc
        |δ ^ 2 * (3 - 6 * r - 2 * δ)| = |δ ^ 2| * |3 - 6 * r - 2 * δ| := abs_mul _ _
        _ = δ ^ 2 * |3 - 6 * r - 2 * δ| := by rw [abs_of_nonneg hsq_nonneg]
        _ ≤ δ ^ 2 * 3 := mul_le_mul_of_nonneg_left h_inner_bound hsq_nonneg
        _ = 3 * δ ^ 2 := by ring
    calc
      |6 * r * (1 - r) * δ + δ ^ 2 * (3 - 6 * r - 2 * δ)| ≤
          |6 * r * (1 - r) * δ| + |δ ^ 2 * (3 - 6 * r - 2 * δ)| := abs_add _ _
      _ ≤ (6 * (1 - r) * |δ|) + (3 * δ ^ 2) := by nlinarith
      _ = 6 * |r - 1| * |δ| + 3 * δ ^ 2 := by
        have h_abs : |r - 1| = 1 - r := by
          rw [show |r - 1| = |1 - r| from abs_sub_comm r 1,
```

```
            abs_of_nonneg (by nlinarith : 0 ≤ 1 - r)]
        rw [h_abs]
  let S := ∑ b, w b * |u b - e b| * |u' b - u b|
  let B := ∑ b, w b * (u' b - u b) ^ 2
  let A := ∑ b, w b * (u b - e b) ^ 2
  have hA_nonneg : 0 ≤ A := by
    refine Finset.sum_nonneg fun b _ => ?_
    have hw_nonneg := hw b
    have hsq_nonneg := sq_nonneg (u b - e b)
    nlinarith
  have hB_nonneg : 0 ≤ B := by
    refine Finset.sum_nonneg fun b _ => ?_
    have hw_nonneg := hw b
    have hsq_nonneg := sq_nonneg (u' b - u b)
    nlinarith
  have hS_nonneg : 0 ≤ S := by
    refine Finset.sum_nonneg fun b _ => ?_
    have hw_nonneg := hw b
    have h1 : 0 ≤ |u b - e b| := abs_nonneg _
    have h2 : 0 ≤ |u' b - u b| := abs_nonneg _
    have h_nonneg : 0 ≤ w b * |u b - e b| := mul_nonneg hw_nonneg h1
    have h_final : 0 ≤ (w b * |u b - e b|) * |u' b - u b| := mul_nonneg h_nonneg h2
    simpa [mul_assoc] using h_final
  have hX_nonneg : 0 ≤ ∑ b, w b * |φ (u' b) - φ (u b)| := by
    refine Finset.sum_nonneg fun b _ => ?_
    have hw_nonneg := hw b
    have habs_nonneg : 0 ≤ |φ (u' b) - φ (u b)| := abs_nonneg _
    nlinarith
  have h_bound_sum : (∑ b, w b * |φ (u' b) - φ (u b)|) ≤ 6 * S + 3 * B := by
    calc
      (∑ b, w b * |φ (u' b) - φ (u b)|) ≤
          ∑ b, w b * (6 * |u b - e b| * |u' b - u b| + 3 * (u' b - u b) ^ 2) :=
        Finset.sum_le_sum fun b _ => by
          have hb := h_key_scalar (u b) (u' b) (e b) (hu b).1 (hu b).2 (hu' b).1 (hu' b).2 (he b)
          have hw_nonneg := hw b
          have habs_nonneg : 0 ≤ |φ (u' b) - φ (u b)| := abs_nonneg _
          nlinarith
      _ = (∑ b, (w b * (6 * |u b - e b| * |u' b - u b|) + w b * (3 * (u' b - u b) ^ 2))) := by
        refine Finset.sum_congr rfl fun b _ => ?_
        ring
      _ = (∑ b, w b * (6 * |u b - e b| * |u' b - u b|)) +
          (∑ b, w b * (3 * (u' b - u b) ^ 2)) :=
        Finset.sum_add_distrib
      _ = 6 * (∑ b, w b * |u b - e b| * |u' b - u b|) +
          3 * (∑ b, w b * (u' b - u b) ^ 2) := by
        simp [Finset.mul_sum, mul_assoc]
        ring
      _ = 6 * S + 3 * B := rfl
  have h_cauchy_schwarz : S ^ 2 ≤ A * B := by
    set f := fun b : ι => Real.sqrt (w b) * |u b - e b| with hf
    set g := fun b : ι => Real.sqrt (w b) * |u' b - u b| with hg
    have h_sq_sqrt : ∀ b, (Real.sqrt (w b)) ^ 2 = w b := fun b =>
      Real.sq_sqrt (hw b)
```

```
    have hS_eq : S = ∑ b, f b * g b := by
      dsimp [S, f, g]
      refine Finset.sum_congr rfl fun b _ => ?_
      calc
        w b * |u b - e b| * |u' b - u b| =
            ((Real.sqrt (w b)) ^ 2) * |u b - e b| * |u' b - u b| := by rw [h_sq_sqrt b]
        _ = ((Real.sqrt (w b)) * |u b - e b|) * ((Real.sqrt (w b)) * |u' b - u b|) := by ring
    rw [hS_eq]
    calc
      (∑ b, f b * g b) ^ 2 ≤ (∑ b, (f b) ^ 2) * (∑ b, (g b) ^ 2) :=
        Finset.sum_mul_sq_le_sq_mul_sq Finset.univ f g
      _ = (∑ b, (Real.sqrt (w b)) ^ 2 * (|u b - e b|) ^ 2) *
          (∑ b, (Real.sqrt (w b)) ^ 2 * (|u' b - u b|) ^ 2) := by
        simp [hf, hg, mul_pow]
      _ = (∑ b, w b * (|u b - e b|) ^ 2) * (∑ b, w b * (|u' b - u b|) ^ 2) := by
        simp [h_sq_sqrt]
      _ = A * B := by
        simp [A, B, sq_abs]
  have h_ineq : (6 * S + 3 * B) ^ 2 ≤ 72 * (S ^ 2) + 18 * (B ^ 2) := by
    have h_key : 36 * S * B ≤ 36 * (S ^ 2) + 9 * (B ^ 2) := by
      have h_sq_nonneg : 0 ≤ (6 * S - 3 * B) ^ 2 := pow_two_nonneg _
      nlinarith
    nlinarith
  have h_final : (∑ b, w b * |φ (u' b) - φ (u b)|) ^ 2 ≤
      72 * A * B + 18 * B ^ 2 := by
    calc
      (∑ b, w b * |φ (u' b) - φ (u b)|) ^ 2 ≤ (6 * S + 3 * B) ^ 2 := by
        nlinarith
      _ ≤ 72 * (S ^ 2) + 18 * (B ^ 2) := h_ineq
      _ ≤ 72 * (A * B) + 18 * (B ^ 2) := by nlinarith
      _ = 72 * A * B + 18 * B ^ 2 := by ring
  simpa [A, B] using h_final

theorem mc_flat {ι : Type*} [Fintype ι] (w : ι → ℝ) (hw : ∀ b, 0 ≤ w b)
    (φ : ℝ → ℝ) (hφ : ∀ r, φ r = 3 * r ^ 2 - 2 * r ^ 3)
    (u u' e : ι → ℝ) (hu : ∀ b, 0 ≤ u b ∧ u b ≤ 1) (hu' : ∀ b, 0 ≤ u' b ∧ u' b ≤ 1)
    (he : ∀ b, e b = 0 ∨ e b = 1) :
    (∑ b, w b * |φ (u' b) - φ (u b)|) ^ 2
      ≤ 108 * ((∑ b, w b * (u' b - e b) ^ 2) ^ 2 - (∑ b, w b * (u b - e b) ^ 2) ^ 2
          - 4 * (∑ b, w b * (u b - e b) ^ 2) * ∑ b, w b * ((u b - e b) * (u' b - u b))) := by
  have hinc := mc_flat_increment w hw φ hφ u u' e hu hu' he
  have hexp : ∑ b, w b * (u' b - e b) ^ 2
      = ∑ b, w b * (u b - e b) ^ 2 + 2 * ∑ b, w b * ((u b - e b) * (u' b - u b))
        + ∑ b, w b * (u' b - u b) ^ 2 := by
    rw [Finset.mul_sum, ← Finset.sum_add_distrib, ← Finset.sum_add_distrib]
    exact Finset.sum_congr rfl (fun b _ => by ring)
  have hcs : (∑ b, w b * ((u b - e b) * (u' b - u b))) ^ 2
      ≤ (∑ b, w b * (u b - e b) ^ 2) * ∑ b, w b * (u' b - u b) ^ 2 := by
    have h := Finset.sum_mul_sq_le_sq_mul_sq Finset.univ
      (fun b => Real.sqrt (w b) * (u b - e b)) (fun b => Real.sqrt (w b) * (u' b - u b))
    have h1 : ∀ b, Real.sqrt (w b) * (u b - e b) * (Real.sqrt (w b) * (u' b - u b))
        = w b * ((u b - e b) * (u' b - u b)) := by
      intro b
      have hsq := Real.mul_self_sqrt (hw b)
      calc Real.sqrt (w b) * (u b - e b) * (Real.sqrt (w b) * (u' b - u b))
          = (Real.sqrt (w b) * Real.sqrt (w b)) * ((u b - e b) * (u' b - u b)) := by ring
        _ = w b * ((u b - e b) * (u' b - u b)) := by rw [hsq]
    have h2 : ∀ b (c : ℝ), (Real.sqrt (w b) * c) ^ 2 = w b * c ^ 2 := by
      intro b c
```

```
      rw [mul_pow, Real.sq_sqrt (hw b)]
    simp only [h1, h2] at h
    exact h
  rw [hexp]
  set S := ∑ b, w b * (u b - e b) ^ 2 with hS
  set P := ∑ b, w b * (u' b - u b) ^ 2 with hP
  set I := ∑ b, w b * ((u b - e b) * (u' b - u b)) with hI
  nlinarith [hinc, hcs, sq_nonneg (2 * I + P), sq_nonneg (4 * I + P)]

theorem mc_one_step {n : ℕ} {ι : Type*} [Fintype ι] (w : ι → ℝ) (hw : ∀ b, 0 ≤ w b)
    (φ : ℝ → ℝ) (hφ : ∀ r, φ r = 3 * r ^ 2 - 2 * r ^ 3)
    (e : ι → ℝ) (he : ∀ b, e b = 0 ∨ e b = 1)
    (p q : ℝ) (hp : 0 ≤ p) (hq : 0 ≤ q) (hpq : p + q = 1)
    (a : Fin n → ℝ) (ha : ∀ k, 0 ≤ a k) (hsum : ∑ k, a k = 1)
    (u v : ι → ℝ) (h : Fin n → ι → ℝ)
    (hu : ∀ b, 0 ≤ u b ∧ u b ≤ 1) (hv : ∀ b, 0 ≤ v b ∧ v b ≤ 1)
    (hh : ∀ k b, 0 ≤ h k b ∧ h k b ≤ 1)
    (hmean : ∀ b, u b = p * v b + q * ∑ k, a k * h k b) :
    p * (∑ b, w b * |φ (v b) - φ (u b)|) ^ 2
      + q * ∑ k, a k * (∑ b, w b * |φ (h k b) - φ (u b)|) ^ 2
      ≤ 108 * (p * (∑ b, w b * (v b - e b) ^ 2) ^ 2
          + q * ∑ k, a k * (∑ b, w b * (h k b - e b) ^ 2) ^ 2
          - (∑ b, w b * (u b - e b) ^ 2) ^ 2) := by
  have hcross : p * (∑ b, w b * ((u b - e b) * (v b - u b)))
      + q * ∑ k, a k * (∑ b, w b * ((u b - e b) * (h k b - u b))) = 0 := by
    have h1 : ∑ k, a k * (∑ b, w b * ((u b - e b) * (h k b - u b)))
        = ∑ b, w b * ((u b - e b) * (∑ k, a k * h k b - u b)) := by
      simp only [Finset.mul_sum]
      rw [Finset.sum_comm]
      refine Finset.sum_congr rfl fun b _ => ?_
      have hb : ∑ k, a k * h k b - u b = ∑ k, (a k * h k b - a k * u b) := by
        rw [Finset.sum_sub_distrib, ← Finset.sum_mul, hsum, one_mul]
      rw [hb, Finset.mul_sum, Finset.mul_sum]
      exact Finset.sum_congr rfl fun k _ => by ring
    rw [h1, Finset.mul_sum, Finset.mul_sum, ← Finset.sum_add_distrib]
    apply Finset.sum_eq_zero
    intro b _
    linear_combination (-(w b * (u b - e b))) * hmean b + (-(w b * (u b - e b) * u b)) * hpq
  have hsum2 : ∑ k, a k * (∑ b, w b * |φ (h k b) - φ (u b)|) ^ 2
      ≤ ∑ k, a k * (108 * ((∑ b, w b * (h k b - e b) ^ 2) ^ 2 - (∑ b, w b * (u b - e b) ^ 2) ^ 2
          - 4 * (∑ b, w b * (u b - e b) ^ 2) * ∑ b, w b * ((u b - e b) * (h k b - u b)))) :=
    Finset.sum_le_sum fun k _ =>
      mul_le_mul_of_nonneg_left (mc_flat w hw φ hφ u (h k) e hu (hh k) he) (ha k)
  have e2 : ∑ k, 108 * (∑ b, w b * (u b - e b) ^ 2) ^ 2 * a k
      = 108 * (∑ b, w b * (u b - e b) ^ 2) ^ 2 := by
    rw [← Finset.mul_sum, hsum, mul_one]
  have e3 : ∑ k, 432 * (∑ b, w b * (u b - e b) ^ 2) * (a k * ∑ b, w b * ((u b - e b) * (h k b - u b)))
      = 432 * (∑ b, w b * (u b - e b) ^ 2) * ∑ k, a k * (∑ b, w b * ((u b - e b) * (h k b - u b))) :=
    (Finset.mul_sum _ _ _).symm
  have e4 : ∑ k, 108 * (a k * (∑ b, w b * (h k b - e b) ^ 2) ^ 2)
      = 108 * ∑ k, a k * (∑ b, w b * (h k b - e b) ^ 2) ^ 2 :=
    (Finset.mul_sum _ _ _).symm
  have e5 : ∑ k, a k * (108 * ((∑ b, w b * (h k b - e b) ^ 2) ^ 2 - (∑ b, w b * (u b - e b) ^ 2) ^ 2
        - 4 * (∑ b, w b * (u b - e b) ^ 2) * ∑ b, w b * ((u b - e b) * (h k b - u b))))
      = 108 * ∑ k, a k * (∑ b, w b * (h k b - e b) ^ 2) ^ 2
        - 108 * (∑ b, w b * (u b - e b) ^ 2) ^ 2
        - 432 * (∑ b, w b * (u b - e b) ^ 2) * ∑ k, a k * (∑ b, w b * ((u b - e b) * (h k b - u b))) := by
    calc _ = ∑ k, (108 * (a k * (∑ b, w b * (h k b - e b) ^ 2) ^ 2)
          - 108 * (∑ b, w b * (u b - e b) ^ 2) ^ 2 * a k
```

```
          - 432 * (∑ b, w b * (u b - e b) ^ 2) * (a k * ∑ b, w b * ((u b - e b) * (h k b - u b)))) :=
        Finset.sum_congr rfl fun k _ => by ring
    _ = ∑ k, 108 * (a k * (∑ b, w b * (h k b - e b) ^ 2) ^ 2)
          - ∑ k, 108 * (∑ b, w b * (u b - e b) ^ 2) ^ 2 * a k
          - ∑ k, 432 * (∑ b, w b * (u b - e b) ^ 2) * (a k * ∑ b, w b * ((u b - e b) * (h k b - u b)))
↪  := by
        rw [Finset.sum_sub_distrib, Finset.sum_sub_distrib]
    _ = _ := by rw [e4, e2, e3]
  have hP := mul_le_mul_of_nonneg_left (mc_flat w hw φ hφ u v e hu hv he) hp
  have hQ := mul_le_mul_of_nonneg_left (hsum2.trans_eq e5) hq
  have hpS : p * (∑ b, w b * (u b - e b) ^ 2) ^ 2 + q * (∑ b, w b * (u b - e b) ^ 2) ^ 2
      = (∑ b, w b * (u b - e b) ^ 2) ^ 2 := by
    rw [← add_mul, hpq, one_mul]
  have hcS : (∑ b, w b * (u b - e b) ^ 2) * (p * (∑ b, w b * ((u b - e b) * (v b - u b)))
      + q * ∑ k, a k * (∑ b, w b * ((u b - e b) * (h k b - u b)))) = 0 := by
    rw [hcross, mul_zero]
  first
  | linarith
  | nlinarith [hP, hQ, hpS, hcS]

theorem mc_aggregate {n : ℕ} (t : ℕ) (π : Fin n → ℝ) (hπ : ∀ i, 0 ≤ π i)
    (A G : Matrix (Fin n) (Fin n) ℝ) (hG0 : ∀ i j, 0 ≤ G i j)
    (hGA : ∀ i j, G i j = (1 / ((t : ℝ) + 1)) * (if i = j then (1:ℝ) else 0)
        + ((t : ℝ) / ((t : ℝ) + 1)) * ∑ k, G i k * A k j)
    (hGπ : ∀ j, ∑ i, π i * G i j = π j)
    (Bv : Fin n → ℝ) (E Fz Fh : Fin n → Fin n → ℝ) (hFh : ∀ i j, 0 ≤ Fh i j)
    (hstep : ∀ i0 j, (1 / ((t : ℝ) + 1)) * Bv j + ((t : ℝ) / ((t : ℝ) + 1)) * ∑ k, A j k * E j k
        ≤ 108 * ((1 / ((t : ℝ) + 1)) * Fz i0 j
            + ((t : ℝ) / ((t : ℝ) + 1)) * ∑ k, A j k * Fh i0 k - Fh i0 j)) :
    ∑ j, π j * Bv j + (t : ℝ) * ∑ j, ∑ k, π j * A j k * E j k
      ≤ 108 * ∑ i, ∑ j, π i * G i j * Fz i j := by
  have ht1 : (t:ℝ) + 1 ≠ 0 := by positivity
  have ht0 : (0:ℝ) ≤ (t:ℝ) + 1 := by positivity
  obtain ⟨p, hp⟩ : ∃ p : ℝ, p = 1 / ((t:ℝ) + 1) := ⟨_, rfl⟩
  obtain ⟨q, hq⟩ : ∃ q : ℝ, q = (t:ℝ) / ((t:ℝ) + 1) := ⟨_, rfl⟩
  have hp0 : 0 ≤ p := by rw [hp]; positivity
  have hpp : ((t:ℝ) + 1) * p = 1 := by
    rw [hp]
    first
      | exact mul_one_div_cancel ht1
      | field_simp
  have hqq : ((t:ℝ) + 1) * q = t := by
    first
      | (rw [hq, div_eq_mul_one_div, ← hp]; linear_combination (t:ℝ) * hpp)
      | (rw [hq]; field_simp)
  rw [← hp, ← hq] at hstep hGA
  have hcol : ∀ i k, q * ∑ j, G i j * A j k = G i k - p * (if i = k then (1:ℝ) else 0) := by
    intro i k
    have := hGA i k
    linarith
  have hrow : ∀ i, ∑ j, G i j * (q * ∑ k, A j k * Fh i k) = ∑ j, G i j * Fh i j - p * Fh i i := by
    intro i
    calc ∑ j, G i j * (q * ∑ k, A j k * Fh i k)
        = ∑ j, ∑ k, q * (G i j * A j k) * Fh i k := by
          refine Finset.sum_congr rfl (fun j _ => ?_)
          simp only [Finset.mul_sum]
          refine Finset.sum_congr rfl (fun k _ => ?_)
          ring
      _ = ∑ k, (q * ∑ j, G i j * A j k) * Fh i k := by
```

```
        rw [Finset.sum_comm]
        refine Finset.sum_congr rfl (fun k _ => ?_)
        rw [Finset.mul_sum, Finset.sum_mul]
    _ = ∑ k, (G i k - p * (if i = k then (1:ℝ) else 0)) * Fh i k := by
        refine Finset.sum_congr rfl (fun k _ => ?_)
        rw [hcol i k]
    _ = ∑ j, G i j * Fh i j - p * Fh i i := by
        have h1 : ∑ k, (G i k - p * (if i = k then (1:ℝ) else 0)) * Fh i k
            = ∑ k, G i k * Fh i k - ∑ k, (if i = k then p * Fh i k else 0) := by
          rw [← Finset.sum_sub_distrib]
          refine Finset.sum_congr rfl (fun k _ => ?_)
          split_ifs <;> ring
        rw [h1]
        simp [Finset.sum_ite_eq]
have key : ∑ i, ∑ j, π i * G i j * (p * Bv j + q * ∑ k, A j k * E j k)
    ≤ ∑ i, ∑ j, π i * G i j * (108 * (p * Fz i j + q * ∑ k, A j k * Fh i k - Fh i j)) := by
  apply Finset.sum_le_sum
  intro i _
  apply Finset.sum_le_sum
  intro j _
  exact mul_le_mul_of_nonneg_left (hstep i j) (mul_nonneg (hπ i) (hG0 i j))
have hL : ∑ i, ∑ j, π i * G i j * (p * Bv j + q * ∑ k, A j k * E j k)
    = ∑ j, π j * (p * Bv j + q * ∑ k, A j k * E j k) := by
  rw [Finset.sum_comm]
  refine Finset.sum_congr rfl (fun j _ => ?_)
  rw [← Finset.sum_mul, hGπ j]
have hR : ∑ i, ∑ j, π i * G i j * (108 * (p * Fz i j + q * ∑ k, A j k * Fh i k - Fh i j))
    = ∑ i, 108 * (π i * (p * ∑ j, G i j * Fz i j - p * Fh i i)) := by
  refine Finset.sum_congr rfl (fun i _ => ?_)
  have hj : ∀ j, π i * G i j * (108 * (p * Fz i j + q * ∑ k, A j k * Fh i k - Fh i j))
      = (108 * π i * p) * (G i j * Fz i j) + (108 * π i) * (G i j * (q * ∑ k, A j k * Fh i k))
        - (108 * π i) * (G i j * Fh i j) := fun j => by ring
  simp only [hj]
  rw [Finset.sum_sub_distrib, Finset.sum_add_distrib]
  simp only [← Finset.mul_sum]
  rw [hrow i]
  ring
have hfin : ∑ i, 108 * (π i * (p * ∑ j, G i j * Fz i j - p * Fh i i))
    ≤ 108 * p * ∑ i, ∑ j, π i * G i j * Fz i j := by
  have hi : ∀ i, 108 * (π i * (p * ∑ j, G i j * Fz i j - p * Fh i i))
      ≤ 108 * p * ∑ j, π i * G i j * Fz i j := by
    intro i
    have h1 : ∑ j, π i * G i j * Fz i j = π i * ∑ j, G i j * Fz i j := by
      rw [Finset.mul_sum]
      refine Finset.sum_congr rfl (fun j _ => ?_)
      ring
    rw [h1]
    have hnn : 0 ≤ π i * p * Fh i i := mul_nonneg (mul_nonneg (hπ i) hp0) (hFh i i)
    nlinarith [hnn]
  calc ∑ i, 108 * (π i * (p * ∑ j, G i j * Fz i j - p * Fh i i))
      ≤ ∑ i, 108 * p * ∑ j, π i * G i j * Fz i j := Finset.sum_le_sum (fun i _ => hi i)
    _ = 108 * p * ∑ i, ∑ j, π i * G i j * Fz i j := by rw [← Finset.mul_sum]
have hsplit : ∑ j, π j * (p * Bv j + q * ∑ k, A j k * E j k)
    = p * ∑ j, π j * Bv j + q * ∑ j, ∑ k, π j * A j k * E j k := by
  have h1 : ∀ j, π j * (p * Bv j + q * ∑ k, A j k * E j k)
      = p * (π j * Bv j) + q * ∑ k, π j * A j k * E j k := by
    intro j
    have h2 : ∑ k, π j * A j k * E j k = π j * ∑ k, A j k * E j k := by
      rw [Finset.mul_sum]
```

```
        refine Finset.sum_congr rfl (fun k _ => ?_)
        ring
      rw [h2]
      ring
    simp only [h1, Finset.sum_add_distrib, ← Finset.mul_sum]
  have hmain : p * ∑ j, π j * Bv j + q * ∑ j, ∑ k, π j * A j k * E j k
      ≤ 108 * p * ∑ i, ∑ j, π i * G i j * Fz i j := by
    linarith [key, hL, hR, hfin, hsplit]
  have h2 := mul_le_mul_of_nonneg_left hmain ht0
  have e1 : ((t:ℝ) + 1) * (p * ∑ j, π j * Bv j + q * ∑ j, ∑ k, π j * A j k * E j k)
      = ∑ j, π j * Bv j + (t:ℝ) * ∑ j, ∑ k, π j * A j k * E j k := by
    linear_combination (∑ j, π j * Bv j) * hpp + (∑ j, ∑ k, π j * A j k * E j k) * hqq
  have e2 : ((t:ℝ) + 1) * (108 * p * ∑ i, ∑ j, π i * G i j * Fz i j)
      = 108 * ∑ i, ∑ j, π i * G i j * Fz i j := by
    linear_combination 108 * (∑ i, ∑ j, π i * G i j * Fz i j) * hpp
  linarith [h2, e1, e2]

theorem mc_phi_vertex (φ : ℝ → ℝ) (hφ : ∀ r, φ r = 3 * r ^ 2 - 2 * r ^ 3)
    (d : ℝ) (hd : d = 0 ∨ d = 1) : φ d = d := by
  rcases hd with (hd | hd)
  · rw [hd, hφ 0]
    ring
  · rw [hd, hφ 1]
    ring

theorem mc_cube_step {n : ℕ} (t : ℕ) (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochastic A)
    {ι : Type*} [Fintype ι] (w : ι → ℝ) (hw : ∀ b, 0 ≤ w b)
    (z : Fin n → ι → ℝ) (hz : ∀ i b, z i b = 0 ∨ z i b = 1)
    (φ : ℝ → ℝ) (hφ : ∀ r, φ r = 3 * r ^ 2 - 2 * r ^ 3)
    (G : Matrix (Fin n) (Fin n) ℝ) (hG0 : ∀ i j, 0 ≤ G i j) (hGrow : ∀ i, ∑ j, G i j = 1)
    (hAG : ∀ i j, G i j = (1 / ((t : ℝ) + 1)) * (if i = j then (1:ℝ) else 0)
        + ((t : ℝ) / ((t : ℝ) + 1)) * ∑ k, A i k * G k j)
    (i0 j : Fin n) :
    (1 / ((t : ℝ) + 1)) * (∑ b, w b * |φ (z j b) - φ (∑ l, G j l * z l b)|) ^ 2
      + ((t : ℝ) / ((t : ℝ) + 1)) * ∑ k, A j k
          * (∑ b, w b * |φ (∑ l, G k l * z l b) - φ (∑ l, G j l * z l b)|) ^ 2
      ≤ 108 * ((1 / ((t : ℝ) + 1)) * (∑ b, w b * (z j b - z i0 b) ^ 2) ^ 2
          + ((t : ℝ) / ((t : ℝ) + 1)) * ∑ k, A j k * (∑ b, w b * (∑ l, G k l * z l b - z i0 b) ^ 2) ^ 2
          - (∑ b, w b * (∑ l, G j l * z l b - z i0 b) ^ 2) ^ 2) := by
  set p := (1 : ℝ) / ((t : ℝ) + 1) with hpdef
  set q := (t : ℝ) / ((t : ℝ) + 1) with hqdef
  have hp_nonneg : 0 ≤ p := div_nonneg (by norm_num) (by positivity)
  have hq_nonneg : 0 ≤ q := div_nonneg (Nat.cast_nonneg _) (by positivity)
  have hpq : p + q = 1 := by
    dsimp [p, q]
    field_simp [show ((t : ℝ) + 1) ≠ 0 from by positivity]
    ring
  have hu_nonneg : ∀ b, 0 ≤ (∑ l : Fin n, G j l * z l b) := by
    intro b; refine Finset.sum_nonneg (fun l _ => ?_)
    rcases hz l b with (h|h) <;> nlinarith [hG0 j l, h]
  have hv_nonneg : ∀ b, 0 ≤ z j b := by
    intro b; rcases hz j b with (h|h) <;> nlinarith
  have he_nonneg : ∀ b, 0 ≤ z i0 b := by
    intro b; rcases hz i0 b with (h|h) <;> nlinarith
  have hh_nonneg : ∀ k b, 0 ≤ (∑ l : Fin n, G k l * z l b) := by
    intro k b; refine Finset.sum_nonneg (fun l _ => ?_)
    rcases hz l b with (h|h) <;> nlinarith [hG0 k l, h]
  have hh_le_one : ∀ k b, (∑ l : Fin n, G k l * z l b) ≤ 1 := by
    intro k b
```

```
    calc
      (∑ l : Fin n, G k l * z l b) ≤ (∑ l : Fin n, G k l * 1) := by
        refine Finset.sum_le_sum (fun l _ => ?_)
        rcases hz l b with (h|h) <;> nlinarith [hG0 k l, h]
      _ = (∑ l : Fin n, G k l) := by simp
      _ = 1 := hGrow k
  have hsum : ∀ b, (∑ l : Fin n, G j l * z l b) = p * z j b + q * ∑ k, A j k * (∑ l : Fin n, G k l * z l
↪  b) := by
    intro b
    calc
      (∑ l : Fin n, G j l * z l b)
          = (∑ l : Fin n, ((p * (if j = l then (1 : ℝ) else 0) + q * ∑ k, A j k * G k l) * z l b)) := by
        refine Finset.sum_congr rfl (fun l _ => ?_); rw [hAG j l]
      _ = (∑ l : Fin n, ((p * (if j = l then (1 : ℝ) else 0)) * z l b
          + (q * ∑ k, A j k * G k l) * z l b)) := by
        refine Finset.sum_congr rfl (fun x _ => ?_); ring
      _ = (∑ l : Fin n, (p * (if j = l then (1 : ℝ) else 0)) * z l b)
          + (∑ l : Fin n, (q * ∑ k, A j k * G k l) * z l b) :=
        Finset.sum_add_distrib
      _ = (p * z j b) + (∑ l : Fin n, q * (∑ k, A j k * G k l * z l b)) := by
        simp [Finset.sum_ite_eq, Finset.mem_univ, mul_assoc, Finset.mul_sum, mul_comm]
      _ = p * z j b + q * (∑ l : Fin n, ∑ k, A j k * G k l * z l b) := by
        simp [Finset.mul_sum, mul_assoc]
      _ = p * z j b + q * (∑ k, ∑ l : Fin n, A j k * G k l * z l b) := by
        rw [Finset.sum_comm]
      _ = p * z j b + q * (∑ k, A j k * (∑ l : Fin n, G k l * z l b)) := by
        simp [Finset.mul_sum, mul_assoc]
  have hmean (b : ι) : (∑ l : Fin n, G j l * z l b) = p * z j b + q * ∑ k, A j k * (∑ l : Fin n, G k l *
↪  z l b) :=
    hsum b
  have hu_le_one : ∀ b, (∑ l : Fin n, G j l * z l b) ≤ 1 := by
    intro b
    calc
      (∑ l : Fin n, G j l * z l b) ≤ (∑ l : Fin n, G j l * 1) := by
        refine Finset.sum_le_sum (fun l _ => ?_)
        rcases hz l b with (h|h) <;> nlinarith [hG0 j l, h]
      _ = (∑ l : Fin n, G j l) := by simp
      _ = 1 := hGrow j
  have hv_le_one : ∀ b, z j b ≤ 1 := by
    intro b; rcases hz j b with (h|h) <;> nlinarith
  have hkey := mc_one_step w hw φ hφ (fun b => z i0 b) (fun b => hz i0 b) p q hp_nonneg hq_nonneg hpq
    (A j) (hA.1 j) (hA.2 j) (fun b => ∑ l, G j l * z l b) (fun b => z j b)
    (fun k b => ∑ l, G k l * z l b) (fun b => ⟨hu_nonneg b, hu_le_one b⟩) (fun b => ⟨hv_nonneg b, hv_le_one
↪  b⟩)
    (fun k b => ⟨hh_nonneg k b, hh_le_one k b⟩) hmean
  · simpa [hpdef, hqdef] using hkey

theorem mc_cube_energy_raw {n : ℕ} (t : ℕ) (π : Fin n → ℝ) (hπ : ∀ i, 0 ≤ π i)
    (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochastic A)
    {ι : Type*} [Fintype ι] (w : ι → ℝ) (hw : ∀ b, 0 ≤ w b)
    (z : Fin n → ι → ℝ) (hz : ∀ i b, z i b = 0 ∨ z i b = 1)
    (φ : ℝ → ℝ) (hφ : ∀ r, φ r = 3 * r ^ 2 - 2 * r ^ 3)
    (G : Matrix (Fin n) (Fin n) ℝ) (hG0 : ∀ i j, 0 ≤ G i j) (hGrow : ∀ i, ∑ j, G i j = 1)
    (hAG : ∀ i j, G i j = (1 / ((t : ℝ) + 1)) * (if i = j then (1:ℝ) else 0)
        + ((t : ℝ) / ((t : ℝ) + 1)) * ∑ k, A i k * G k j)
    (hGA : ∀ i j, G i j = (1 / ((t : ℝ) + 1)) * (if i = j then (1:ℝ) else 0)
        + ((t : ℝ) / ((t : ℝ) + 1)) * ∑ k, G i k * A k j)
    (hGπ : ∀ j, ∑ i, π i * G i j = π j) :
    ∑ j, π j * (∑ b, w b * |φ (z j b) - φ (∑ l, G j l * z l b)|) ^ 2
```

```
      + (t : ℝ) * ∑ j, ∑ k, π j * A j k
          * (∑ b, w b * |φ (∑ l, G k l * z l b) - φ (∑ l, G j l * z l b)|) ^ 2
      ≤ 108 * ∑ i, ∑ j, π i * G i j * (∑ b, w b * (z j b - z i b) ^ 2) ^ 2 := by
  have key := mc_aggregate t π hπ A G hG0 hGA hGπ
    (fun j => (∑ b, w b * |φ (z j b) - φ (∑ l, G j l * z l b)|) ^ 2)
    (fun j k => (∑ b, w b * |φ (∑ l, G k l * z l b) - φ (∑ l, G j l * z l b)|) ^ 2)
    (fun i j => (∑ b, w b * (z j b - z i b) ^ 2) ^ 2)
    (fun i j => (∑ b, w b * (∑ l, G j l * z l b - z i b) ^ 2) ^ 2)
    (fun i j => sq_nonneg _)
    (fun i0 j => mc_cube_step t A hA w hw z hz φ hφ G hG0 hGrow hAG i0 j)
  exact key

theorem mc_cube_energy {n : ℕ} (t : ℕ) (π : Fin n → ℝ) (hπ : IsProbVec π)
    (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochastic A)
    {ι : Type*} [Fintype ι] (w : ι → ℝ) (hw : ∀ b, 0 ≤ w b)
    (z : Fin n → ι → ℝ) (hz : ∀ i b, z i b = 0 ∨ z i b = 1)
    (φ : ℝ → ℝ) (hφ : ∀ r, φ r = 3 * r ^ 2 - 2 * r ^ 3)
    (G : Matrix (Fin n) (Fin n) ℝ) (hG0 : ∀ i j, 0 ≤ G i j) (hGrow : ∀ i, ∑ j, G i j = 1)
    (hAG : ∀ i j, G i j = (1 / ((t : ℝ) + 1)) * (if i = j then (1:ℝ) else 0)
        + ((t : ℝ) / ((t : ℝ) + 1)) * ∑ k, A i k * G k j)
    (hGA : ∀ i j, G i j = (1 / ((t : ℝ) + 1)) * (if i = j then (1:ℝ) else 0)
        + ((t : ℝ) / ((t : ℝ) + 1)) * ∑ k, G i k * A k j)
    (hGπ : ∀ j, ∑ i, π i * G i j = π j) :
    ∑ i, π i * (∑ b, w b * |z i b - φ (∑ j, G i j * z j b)|) ^ 2
      + (t : ℝ) * ∑ i, ∑ j, π i * A i j
          * (∑ b, w b * |φ (∑ k, G i k * z k b) - φ (∑ k, G j k * z k b)|) ^ 2
      ≤ 108 * ∑ i, ∑ j, π i * G i j * (∑ b, w b * (z i b - z j b) ^ 2) ^ 2 := by
  have h := mc_cube_energy_raw t π hπ.1 A hA w hw z hz φ hφ G hG0 hGrow hAG hGA hGπ
  have key1 : ∑ i, π i * (∑ b, w b * |z i b - φ (∑ j, G i j * z j b)|) ^ 2
      = ∑ j, π j * (∑ b, w b * |φ (z j b) - φ (∑ l, G j l * z l b)|) ^ 2 := by
    refine Finset.sum_congr rfl (fun i _ => ?_)
    have hv : ∀ b, φ (z i b) = z i b := fun b => mc_phi_vertex φ hφ _ (hz i b)
    simp only [hv]
  have key2 : ∑ i, ∑ j, π i * A i j
        * (∑ b, w b * |φ (∑ k, G i k * z k b) - φ (∑ k, G j k * z k b)|) ^ 2
      = ∑ j, ∑ k, π j * A j k
        * (∑ b, w b * |φ (∑ l, G k l * z l b) - φ (∑ l, G j l * z l b)|) ^ 2 := by
    refine Finset.sum_congr rfl (fun i _ => Finset.sum_congr rfl (fun j _ => ?_))
    have hs : ∀ b, |φ (∑ k, G i k * z k b) - φ (∑ k, G j k * z k b)|
        = |φ (∑ l, G j l * z l b) - φ (∑ l, G i l * z l b)| := fun b => abs_sub_comm _ _
    simp only [hs]
  have key3 : ∑ i, ∑ j, π i * G i j * (∑ b, w b * (z i b - z j b) ^ 2) ^ 2
      = ∑ i, ∑ j, π i * G i j * (∑ b, w b * (z j b - z i b) ^ 2) ^ 2 := by
    refine Finset.sum_congr rfl (fun i _ => Finset.sum_congr rfl (fun j _ => ?_))
    have hs : ∀ b, (z i b - z j b) ^ 2 = (z j b - z i b) ^ 2 := fun b => by ring
    simp only [hs]
  rw [key1, key2, key3]
  exact h

theorem mc_cut_real_mono (n : ℕ) (a : Fin n → ℝ) (ha : Monotone a) :
    ∃ (c : Fin n → Finset (Fin n)) (gap : Fin n → ℝ) (v0 : ℝ),
      (∀ r, 0 ≤ gap r) ∧ ∑ r, gap r ≤ 2 * ∑ i, |a i| ∧ |v0| ≤ ∑ i, |a i| ∧
      (∀ i, a i = v0 + ∑ r, (if i ∈ c r then (1:ℝ) else 0) * gap r) ∧
      (∀ i j, |a i - a j| = ∑ r, |(if i ∈ c r then (1:ℝ) else 0) - (if j ∈ c r then (1:ℝ) else 0)| * gap
↪  r) := by
  rcases Nat.eq_zero_or_pos n with hn | hnpos
  · subst hn
    refine ⟨fun _ => ∅, fun _ => 0, 0, fun _ => le_rfl, by simp, by simp, fun i => i.elim0, fun i => i.elim0⟩
  · obtain ⟨F, hF⟩ : ∃ F : ℕ → ℝ, ∀ m, F m = if h : m < n then a ⟨m, h⟩ else 0 :=
```

```
    ⟨fun m => if h : m < n then a ⟨m, h⟩ else 0, fun _ => rfl⟩
  have hFa : ∀ i : Fin n, F i.val = a i := by
    intro i
    rw [hF]
    exact dif_pos i.isLt
  have hFmono : ∀ m, m + 1 < n → F m ≤ F (m + 1) := by
    intro m h
    rw [hF m, hF (m + 1), dif_pos (show m < n by omega), dif_pos h]
    exact ha (show (⟨m, by omega⟩ : Fin n) ≤ ⟨m + 1, h⟩ from
      Fin.le_iff_val_le_val.mpr (by show m ≤ m + 1; omega))
  obtain ⟨g, hg⟩ : ∃ g : ℕ → ℝ, ∀ k, g k = if k + 1 < n then F (k + 1) - F k else 0 :=
    ⟨fun k => if k + 1 < n then F (k + 1) - F k else 0, fun _ => rfl⟩
  obtain ⟨c, hc⟩ : ∃ c : Fin n → Finset (Fin n), ∀ r i, i ∈ c r ↔ r.val < i.val :=
    ⟨fun r => Finset.univ.filter (fun i => r.val < i.val), fun r i => by simp⟩
  obtain ⟨z0, hz0⟩ : ∃ l : Fin n, l.val = 0 := ⟨⟨0, hnpos⟩, rfl⟩
  obtain ⟨last, hlast⟩ : ∃ l : Fin n, l.val = n - 1 := ⟨⟨n - 1, by omega⟩, rfl⟩
  have hF0 : F 0 = a z0 := by rw [← hz0]; exact hFa z0
  have hFl : F (n - 1) = a last := by rw [← hlast]; exact hFa last
  have hle : ∀ j : Fin n, |a j| ≤ ∑ i, |a i| := fun j =>
    Finset.single_le_sum (f := fun i => |a i|) (fun i _ => abs_nonneg (a i)) (Finset.mem_univ j)
  have hgap : ∀ r : Fin n, 0 ≤ g r.val := by
    intro r
    by_cases h : r.val + 1 < n
    · rw [hg, if_pos h]; linarith [hFmono r.val h]
    · simp [hg, h]
  have hsum : ∀ m : ℕ, m < n → ∑ r : Fin n, (if r.val < m then g r.val else 0) = F m - F 0 := by
    intro m
    induction m with
    | zero => intro _; simp
    | succ m ih =>
      intro hm
      have hm' : m < n := by omega
      have key : ∀ r : Fin n, (if r.val < m + 1 then g r.val else 0)
          = (if r.val < m then g r.val else 0) + (if r = (⟨m, hm'⟩ : Fin n) then g m else 0) := by
        intro r
        by_cases h1 : r.val < m
        · have hne : r ≠ ⟨m, hm'⟩ := fun h => by subst h; simp at h1
          rw [if_pos (show r.val < m + 1 by omega), if_pos h1, if_neg hne, add_zero]
        · by_cases h2 : r.val = m
          · have hr : r = ⟨m, hm'⟩ := Fin.ext h2
            subst hr
            simp
          · have hr : r ≠ ⟨m, hm'⟩ := fun h => h2 (by subst h; rfl)
            rw [if_neg (show ¬ (r.val < m + 1) by omega), if_neg h1, if_neg hr, add_zero]
      have hs : ∑ r : Fin n, (if r = (⟨m, hm'⟩ : Fin n) then g m else 0) = g m := by simp
      rw [Finset.sum_congr rfl (fun r _ => key r), Finset.sum_add_distrib, ih hm', hs,
        hg m, if_pos hm]
      ring
  have hind : ∀ r i : Fin n, (if i ∈ c r then (1:ℝ) else 0) * g r.val
      = if r.val < i.val then g r.val else 0 := by
    intro r i
    by_cases h : r.val < i.val
    · rw [if_pos ((hc r i).mpr h), if_pos h, one_mul]
    · rw [if_neg (fun h' => h ((hc r i).mp h')), if_neg h, zero_mul]
  have hrec : ∀ i : Fin n, a i = a z0 + ∑ r, (if i ∈ c r then (1:ℝ) else 0) * g r.val := by
    intro i
    rw [Finset.sum_congr rfl (fun r _ => hind r i), hsum i.val i.isLt, hFa i, hF0]
    ring
  have hterm : ∀ r i j : Fin n, i.val ≤ j.val →
```

```
        |(if i ∈ c r then (1:ℝ) else 0) - (if j ∈ c r then (1:ℝ) else 0)| * g r.val
          = (if r.val < j.val then g r.val else 0) - (if r.val < i.val then g r.val else 0) := by
      intro r i j hij
      by_cases h1 : r.val < i.val
      · have h2 : r.val < j.val := by omega
        rw [if_pos ((hc r i).mpr h1), if_pos ((hc r j).mpr h2), if_pos h1, if_pos h2]
        simp
      · have h1' : i ∉ c r := fun h' => h1 ((hc r i).mp h')
        by_cases h2 : r.val < j.val
        · rw [if_neg h1', if_pos ((hc r j).mpr h2), if_neg h1, if_pos h2]
          simp
        · have h2' : j ∉ c r := fun h' => h2 ((hc r j).mp h')
          rw [if_neg h1', if_neg h2', if_neg h1, if_neg h2]
          simp
    have hdle : ∀ i j : Fin n, i.val ≤ j.val →
       |a i - a j| = ∑ r, |(if i ∈ c r then (1:ℝ) else 0) - (if j ∈ c r then (1:ℝ) else 0)| * g r.val := by
      intro i j hij
      rw [Finset.sum_congr rfl (fun r _ => hterm r i j hij), Finset.sum_sub_distrib,
        hsum j.val j.isLt, hsum i.val i.isLt, hFa i, hFa j]
      have hij' : a i ≤ a j := ha (Fin.le_iff_val_le_val.mpr hij)
      rw [abs_of_nonpos (show a i - a j ≤ 0 by linarith)]
      ring
    refine ⟨c, fun r => g r.val, a z0, hgap, ?_, hle z0, hrec, ?_⟩
    · have e1 : ∑ r : Fin n, g r.val = ∑ r : Fin n, (if r.val < n - 1 then g r.val else 0) := by
        apply Finset.sum_congr rfl
        intro r _
        by_cases h : r.val < n - 1
        · rw [if_pos h]
        · rw [if_neg h, hg, if_neg (show ¬ (r.val + 1 < n) by omega)]
      show ∑ r : Fin n, g r.val ≤ 2 * ∑ i, |a i|
      rw [e1, hsum (n - 1) (by omega), hFl, hF0]
      have h1 := hle last
      have h2 := hle z0
      have h3 := le_abs_self (a last)
      have h4 := neg_abs_le (a z0)
      linarith
    · show ∀ i j : Fin n, |a i - a j|
        = ∑ r, |(if i ∈ c r then (1:ℝ) else 0) - (if j ∈ c r then (1:ℝ) else 0)| * g r.val
      intro i j
      rcases le_total i.val j.val with h | h
      · exact hdle i j h
      · rw [abs_sub_comm (a i) (a j), hdle j i h]
        apply Finset.sum_congr rfl
        intro r _
        rw [abs_sub_comm]

theorem mc_cut_real_pointwise (n : ℕ) (a : Fin n → ℝ) :
    ∃ (c : Fin n → Finset (Fin n)) (gap : Fin n → ℝ) (v0 : ℝ),
      (∀ r, 0 ≤ gap r) ∧ ∑ r, gap r ≤ 2 * ∑ i, |a i| ∧ |v0| ≤ ∑ i, |a i| ∧
      (∀ i, a i = v0 + ∑ r, (if i ∈ c r then (1:ℝ) else 0) * gap r) ∧
      (∀ i j, |a i - a j| = ∑ r, |(if i ∈ c r then (1:ℝ) else 0) - (if j ∈ c r then (1:ℝ) else 0)| * gap
↪  r) := by
  obtain ⟨σ, hmono⟩ : ∃ σ : Equiv.Perm (Fin n), Monotone (a ∘ σ) :=
    ⟨Tuple.sort a, Tuple.monotone_sort a⟩
  obtain ⟨c, gap, v0, hg, hsum, hv, hrep, hdist⟩ := mc_cut_real_mono n (a ∘ σ) hmono
  have hperm : ∑ i, |(a ∘ σ) i| = ∑ i, |a i| := Equiv.sum_comp σ (fun i => |a i|)
  refine ⟨fun r => Finset.univ.filter (fun i => σ.symm i ∈ c r), gap, v0, hg, ?_, ?_, ?_, ?_⟩
  · linarith [hsum, hperm]
  · linarith [hv, hperm]
```

```
  · intro i
    have h := hrep (σ.symm i)
    simp only [Function.comp_apply, Equiv.apply_symm_apply] at h
    simp only [Finset.mem_filter, Finset.mem_univ, true_and]
    exact h
  · intro i j
    have h := hdist (σ.symm i) (σ.symm j)
    simp only [Function.comp_apply, Equiv.apply_symm_apply] at h
    simp only [Finset.mem_filter, Finset.mem_univ, true_and]
    exact h

theorem mc_cut_real (n : ℕ) :
    ∃ (g : Finset (Fin n) → (Fin n → ℝ) → ℝ) (v : (Fin n → ℝ) → ℝ),
      ∀ a : Fin n → ℝ,
        (∀ B, 0 ≤ g B a) ∧ (∀ B, g B a ≤ 2 * ∑ i, |a i|) ∧ |v a| ≤ ∑ i, |a i| ∧
        (∀ i, a i = v a + ∑ B, (if i ∈ B then (1:ℝ) else 0) * g B a) ∧
        (∀ i j, |a i - a j| = ∑ B, |(if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0)| * g B a)
↪  := by
  choose c gap v0 hgap0 hgapsum hv0 hrec hdist using fun a : Fin n → ℝ => mc_cut_real_pointwise n a
  have key : ∀ (a : Fin n → ℝ) (f : Finset (Fin n) → ℝ),
      ∑ B, f B * (∑ r, if B = c a r then gap a r else 0) = ∑ r, f (c a r) * gap a r := by
    intro a f
    simp_rw [Finset.mul_sum, mul_ite, mul_zero]
    rw [Finset.sum_comm]
    simp [Finset.sum_ite_eq']
  refine ⟨fun B a => ∑ r, if B = c a r then gap a r else 0, v0, fun a => ?_⟩
  refine ⟨?_, ?_, hv0 a, ?_, ?_⟩
  · intro B
    exact Finset.sum_nonneg (fun r _ => by
      split_ifs
      · exact hgap0 a r
      · exact le_rfl)
  · intro B
    refine le_trans (Finset.sum_le_sum fun r _ => ?_) (hgapsum a)
    split_ifs
    · exact le_rfl
    · exact hgap0 a r
  · intro i
    exact (hrec a i).trans
      (congrArg (v0 a + ·) (key a (fun B => if i ∈ B then (1:ℝ) else 0)).symm)
  · intro i j
    exact (hdist a i j).trans
      (key a (fun B => |(if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0)|)).symm

theorem mc_L1_norm (f : L1) :
    Summable (fun k : ℕ => |f k|) ∧ ‖f‖ = ∑' k : ℕ, |f k| := by
  have hp : 0 < ENNReal.toReal (1 : ENNReal) := by norm_num
  have hmem : Memℓp (f : ℕ → ℝ) (1 : ENNReal) := lp.memℓp f
  have hsumm : Summable (fun i : ℕ => |f i|) := by
    have := (memℓp_gen_iff hp).mp hmem
    have h1 : ENNReal.toReal (1 : ENNReal) = (1 : ℝ) := by norm_num
    simpa [h1] using this
  have hnorm : ‖f‖ = ∑' k : ℕ, |f k| := by
    rw [lp.norm_eq_tsum_rpow hp f]
    have h1 : ENNReal.toReal (1 : ENNReal) = (1 : ℝ) := by norm_num
    rw [h1]
    simp
  exact And.intro hsumm hnorm
```

```
theorem mc_L1_of_bound (f g : ℕ → ℝ) (hg : Summable g) (hfg : ∀ k, |f k| ≤ g k) :
    ∃ F : L1, ∀ k, F k = f k := by
  have h_abs_summable : Summable fun k : ℕ => |f k| :=
    Summable.of_nonneg_of_le (fun _k => abs_nonneg _) hfg hg
  have h_mem : Memℓp f (1 : ENNReal) := by
    have h_norm_eq : ∀ k, ‖f k‖ = |f k| := fun _k => by simp
    have : Summable (fun k : ℕ => ‖f k‖ ^ (1 : ENNReal).toReal) := by
      simpa [ENNReal.toReal_one, h_norm_eq] using h_abs_summable
    exact memℓp_gen this
  refine ⟨⟨f, h_mem⟩, fun _k => rfl⟩

theorem mc_L1_affine_apply {ι : Type*} [Fintype ι] (V : L1) (b : ι → L1) (u : ι → ℝ) (k : ℕ) :
    (V + ∑ B, u B • b B) k = V k + ∑ B, u B * b B k := by
  simp [lp.coeFn_smul, Finset.sum_apply, smul_eq_mul]

theorem mc_L1_affine_contr {ι : Type*} [Fintype ι] (V : L1) (b : ι → L1) (u u' : ι → ℝ) :
    ‖(V + ∑ B, u B • b B) - (V + ∑ B, u' B • b B)‖ ≤ ∑ B, ‖b B‖ * |u B - u' B| := by
  have hsub : (V + ∑ B, u B • b B) - (V + ∑ B, u' B • b B) = ∑ B, (u B - u' B) • b B := by
    simp [sub_smul, Finset.sum_sub_distrib]
  rw [hsub]
  calc
    ‖∑ B, (u B - u' B) • b B‖ ≤ ∑ B, ‖(u B - u' B) • b B‖ := norm_sum_le _ _
    _ ≤ ∑ B, (‖(u B - u' B : ℝ)‖ * ‖b B‖) := by
      refine Finset.sum_le_sum fun i _ => ?_
      exact norm_smul_le _ _
    _ = ∑ B, |u B - u' B| * ‖b B‖ := by
      refine Finset.sum_congr rfl fun i _ => ?_
      rw [Real.norm_eq_abs]
    _ = ∑ B, ‖b B‖ * |u B - u' B| := by
      simp [mul_comm]

open ENNReal MarkovCotype in
theorem mc_L1_dist_eq_aiprover_s0 {n : ℕ} {ι : Type*} [Fintype ι] (x : Fin n → L1) (b : ι → L1)
    (hb : ∀ B k, 0 ≤ b B k) (z : Fin n → ι → ℝ)
    (hrep : ∀ i j k, |x i k - x j k| = ∑ B, |z i B - z j B| * b B k) (i j : Fin n) :
    ‖x i - x j‖ = ∑ B, ‖b B‖ * |z i B - z j B| := by
  have hp : 0 < (1 : ℝ≥0∞).toReal := by simp
  have hnorm := fun (f : L1) => (mc_L1_norm f).2
  have hsum_b (B : ι) : Summable (fun k : ℕ => b B k) := by
    have := (mc_L1_norm (b B)).1



    have hpos := hb B



    have hpos' : ∀ k, 0 ≤ b B k := hpos
    refine Summable.of_nonneg_of_le hpos' (fun k => ?_) this
    rw [abs_of_nonneg (hpos k)]
  have hsum_inner (B : ι) : Summable (fun k : ℕ => |z i B - z j B| * b B k) :=
    Summable.mul_left (|z i B - z j B|) (hsum_b B)
  calc
    ‖x i - x j‖ = (∑' k : ℕ, ‖(x i - x j) k‖) := by
      rw [lp.norm_eq_tsum_rpow hp (x i - x j)]
      simp [toReal_one]
    _ = (∑' k : ℕ, |x i k - x j k|) := by
      simp [Real.norm_eq_abs]
    _ = (∑' k : ℕ, ∑ B, |z i B - z j B| * b B k) := by
```

```
      simp_rw [hrep i j]
    _ = ∑ B, (∑' k : ℕ, |z i B - z j B| * b B k) := by
      have h := Summable.tsum_finsetSum (s := Finset.univ) (fun B _ => hsum_inner B)
      simpa using h
    _ = ∑ B, |z i B - z j B| * (∑' k : ℕ, b B k) := by
      refine Finset.sum_congr rfl fun B _ => ?_
      simp [tsum_mul_left]
    _ = ∑ B, |z i B - z j B| * ‖b B‖ := by
      refine Finset.sum_congr rfl fun B _ => ?_
      have h := hnorm (b B)









      have hsum_eq : (∑' k : ℕ, |(b B : ℕ → ℝ) k|) = (∑' k : ℕ, (b B : ℕ → ℝ) k) := by
        refine tsum_congr (fun k => ?_)
        rw [abs_of_nonneg (hb B k)]
      rw [h, hsum_eq]
    _ = ∑ B, ‖b B‖ * |z i B - z j B| := by
      refine Finset.sum_congr rfl fun B _ => ?_
      rw [mul_comm]

theorem mc_L1_dist_eq {n : ℕ} {ι : Type*} [Fintype ι] (x : Fin n → L1) (b : ι → L1)
    (hb : ∀ B k, 0 ≤ b B k) (z : Fin n → ι → ℝ)
    (hrep : ∀ i j k, |x i k - x j k| = ∑ B, |z i B - z j B| * b B k) (i j : Fin n) :
    ‖x i - x j‖ = ∑ B, ‖b B‖ * |z i B - z j B| := by
  first
    | exact mc_L1_dist_eq_aiprover_s0
    | (intros; apply mc_L1_dist_eq_aiprover_s0 <;> assumption)

theorem mc_cut_coords {n : ℕ} (x : Fin n → L1) :
    ∃ (b : Finset (Fin n) → L1) (V : L1),
      (∀ B k, 0 ≤ b B k) ∧
      (∀ i k, x i k = V k + ∑ B, (if i ∈ B then (1:ℝ) else 0) * b B k) ∧
      (∀ i j k, |x i k - x j k|
        = ∑ B, |(if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0)| * b B k) := by
  obtain ⟨g, v, hgv⟩ := mc_cut_real n
  have hS : Summable (fun k : ℕ => ∑ i, |x i k|) := by
    first
    | exact summable_sum (fun i _ => (mc_L1_norm (x i)).1)
    | exact Summable.sum (fun i _ => (mc_L1_norm (x i)).1)
    | (have key : ∀ s : Finset (Fin n), Summable (fun k : ℕ => ∑ i ∈ s, |x i k|) := by
         intro s
         refine Finset.induction_on s ?_ ?_
         · simp
         · intro a s ha ih
           simpa [Finset.sum_insert ha] using ((mc_L1_norm (x a)).1).add ih
       exact key Finset.univ)
  choose b hb using fun B : Finset (Fin n) =>
    mc_L1_of_bound (fun k => g B (fun i => x i k)) (fun k => 2 * ∑ i, |x i k|) (hS.mul_left 2)
      (fun k => (abs_of_nonneg ((hgv (fun i => x i k)).1 B)).le.trans ((hgv (fun i => x i k)).2.1 B))
  obtain ⟨V, hV⟩ := mc_L1_of_bound (fun k => v (fun i => x i k)) (fun k => ∑ i, |x i k|) hS
    (fun k => (hgv (fun i => x i k)).2.2.1)
  refine ⟨b, V, ?_, ?_, ?_⟩
  · intro B k
    rw [hb B k]
```

```
    exact (hgv (fun i => x i k)).1 B
  · intro i k
    rw [hV k]
    simp only [hb]
    exact (hgv (fun i => x i k)).2.2.2.1 i
  · intro i j k
    simp only [hb]
    exact (hgv (fun i => x i k)).2.2.2.2 i j

theorem mc_cut_L1 {n : ℕ} (x : Fin n → L1) :
    ∃ (w : Finset (Fin n) → ℝ) (T : (Finset (Fin n) → ℝ) → L1),
      (∀ B, 0 ≤ w B) ∧
      (∀ i, T (fun B => if i ∈ B then (1:ℝ) else 0) = x i) ∧
      (∀ i j, ‖x i - x j‖ = ∑ B, w B * |(if i ∈ B then (1:ℝ) else 0) - (if j ∈ B then (1:ℝ) else 0)|) ∧
      (∀ u u', ‖T u - T u'‖ ≤ ∑ B, w B * |u B - u' B|) := by
  obtain ⟨b, V, hb, hx, hrep⟩ := mc_cut_coords x
  refine ⟨fun B => ‖b B‖, fun u => V + ∑ B, u B • b B, fun B => norm_nonneg _, ?_, ?_, ?_⟩
  · intro i
    have h : ∀ k, (V + ∑ B, (fun B : Finset (Fin n) => if i ∈ B then (1:ℝ) else 0) B • b B) k = x i k := by
      intro k
      first
        | (rw [mc_L1_affine_apply, hx i k])
        | (simp only [mc_L1_affine_apply, hx i k])
        | (rw [mc_L1_affine_apply]; exact (hx i k).symm)
    first
      | exact lp.ext (funext h)
      | exact Subtype.ext (funext h)
  · intro i j
    exact mc_L1_dist_eq x b hb (fun i B => if i ∈ B then (1:ℝ) else 0) hrep i j
  · intro u u'
    exact mc_L1_affine_contr V b u u'

theorem mc_transfer {n : ℕ} (t : ℕ) (π : Fin n → ℝ) (hπ : ∀ i, 0 ≤ π i)
    (A : Matrix (Fin n) (Fin n) ℝ) (hA0 : ∀ i j, 0 ≤ A i j) (x : Fin n → L1)
    {ι : Type*} [Fintype ι] (w : ι → ℝ) (hw : ∀ b, 0 ≤ w b)
    (z : Fin n → ι → ℝ) (hz : ∀ i b, z i b = 0 ∨ z i b = 1)
    (T : (ι → ℝ) → L1) (hT : ∀ i, T (z i) = x i)
    (hdist : ∀ i j, ‖x i - x j‖ = ∑ b, w b * |z i b - z j b|)
    (hcontr : ∀ u u', ‖T u - T u'‖ ≤ ∑ b, w b * |u b - u' b|)
    (G : Matrix (Fin n) (Fin n) ℝ) (c : Fin n → ι → ℝ)
    (hcube : ∑ i, π i * (∑ b, w b * |z i b - c i b|) ^ 2
      + (t : ℝ) * ∑ i, ∑ j, π i * A i j * (∑ b, w b * |c i b - c j b|) ^ 2
      ≤ 108 * ∑ i, ∑ j, π i * G i j * (∑ b, w b * (z i b - z j b) ^ 2) ^ 2) :
    ∃ y : Fin n → L1,
      ∑ i, π i * ‖x i - y i‖ ^ 2 + (t : ℝ) * ∑ i, ∑ j, π i * A i j * ‖y i - y j‖ ^ 2
        ≤ 108 * ∑ i, ∑ j, π i * G i j * ‖x i - x j‖ ^ 2 := by
  have h_binary_eq (a b : ℝ) (ha : a = 0 ∨ a = 1) (hb : b = 0 ∨ b = 1) : |a - b| = (a - b) ^ 2 := by
    rcases ha with (rfl | rfl) <;> rcases hb with (rfl | rfl) <;> norm_num
  have h_sum_eq (i j : Fin n) : ∑ b : ι, w b * |z i b - z j b| = ∑ b : ι, w b * (z i b - z j b) ^ 2 := by
    refine Finset.sum_congr rfl fun b _ => ?_
    rw [h_binary_eq (z i b) (z j b) (hz i b) (hz j b)]
  refine ⟨fun i => T (c i), ?_⟩
  have h_nonneg_sum (u v : ι → ℝ) : 0 ≤ ∑ b : ι, w b * |u b - v b| :=
    Finset.sum_nonneg fun b _ => mul_nonneg (hw b) (abs_nonneg _)
  have h1 (i : Fin n) : ‖x i - T (c i)‖ ≤ ∑ b : ι, w b * |z i b - c i b| := by
    rw [← hT i]
    exact hcontr (z i) (c i)
  have h2 (i j : Fin n) : ‖T (c i) - T (c j)‖ ≤ ∑ b : ι, w b * |c i b - c j b| :=
    hcontr (c i) (c j)
```

```
  have h1_sq (i : Fin n) : ‖x i - T (c i)‖ ^ 2 ≤ (∑ b : ι, w b * |z i b - c i b|) ^ 2 := by
    have h := mul_self_le_mul_self (norm_nonneg _) (h1 i)
    rw [sq, sq]
    exact h
  have h2_sq (i j : Fin n) : ‖T (c i) - T (c j)‖ ^ 2 ≤ (∑ b : ι, w b * |c i b - c j b|) ^ 2 := by
    have h := mul_self_le_mul_self (norm_nonneg _) (h2 i j)
    rw [sq, sq]
    exact h
  calc
    ∑ i : Fin n, π i * ‖x i - T (c i)‖ ^ 2 + (t : ℝ) * ∑ i : Fin n, ∑ j : Fin n, π i * A i j * ‖T (c i)
↪   - T (c j)‖ ^ 2
      ≤ ∑ i : Fin n, π i * (∑ b : ι, w b * |z i b - c i b|) ^ 2
        + (t : ℝ) * ∑ i : Fin n, ∑ j : Fin n, π i * A i j * (∑ b : ι, w b * |c i b - c j b|) ^ 2 := by
      refine add_le_add ?_ ?_
      · refine Finset.sum_le_sum fun i _ => ?_
        have h_nonneg_pi : 0 ≤ π i := hπ i
        have h_sq := h1_sq i
        nlinarith
      · have h_nonneg_t : 0 ≤ (t : ℝ) := Nat.cast_nonneg _
        refine mul_le_mul_of_nonneg_left ?_ h_nonneg_t
        refine Finset.sum_le_sum fun i _ => ?_
        refine Finset.sum_le_sum fun j _ => ?_
        have h_nonneg_piA : 0 ≤ π i * A i j := mul_nonneg (hπ i) (hA0 i j)
        have h_sq := h2_sq i j
        nlinarith
    _ ≤ 108 * ∑ i : Fin n, ∑ j : Fin n, π i * G i j * (∑ b : ι, w b * (z i b - z j b) ^ 2) ^ 2 := hcube
    _ = 108 * ∑ i : Fin n, ∑ j : Fin n, π i * G i j * (‖x i - x j‖) ^ 2 := by
      simp only [h_sum_eq, hdist]

theorem mc_minkowski {α : Type*} [Fintype α] (μ a b : α → ℝ) (hμ : ∀ k, 0 ≤ μ k) :
    Real.sqrt (∑ k, μ k * (a k + b k) ^ 2)
      ≤ Real.sqrt (∑ k, μ k * a k ^ 2) + Real.sqrt (∑ k, μ k * b k ^ 2) := by
  have hp : (1 : ℝ) ≤ 2 := by norm_num
  set f : α → ℝ := fun k => Real.sqrt (μ k) * a k
  set g : α → ℝ := fun k => Real.sqrt (μ k) * b k
  have hsum_add : (∑ k, μ k * (a k + b k) ^ 2) = ∑ k, (f k + g k) ^ 2 := by
    refine Finset.sum_congr rfl fun k _ => ?_
    dsimp [f, g]
    calc
      μ k * (a k + b k) ^ 2 = (Real.sqrt (μ k)) ^ 2 * (a k + b k) ^ 2 := by
        rw [Real.sq_sqrt (hμ k)]
      _ = ((Real.sqrt (μ k)) * (a k + b k)) ^ 2 := by ring
      _ = ((Real.sqrt (μ k)) * a k + (Real.sqrt (μ k)) * b k) ^ 2 := by ring
  have hsum_a : (∑ k, μ k * a k ^ 2) = ∑ k, (f k) ^ 2 := by
    refine Finset.sum_congr rfl fun k _ => ?_
    dsimp [f]
    calc
      μ k * a k ^ 2 = (Real.sqrt (μ k)) ^ 2 * a k ^ 2 := by rw [Real.sq_sqrt (hμ k)]
      _ = ((Real.sqrt (μ k)) * a k) ^ 2 := by ring
  have hsum_b : (∑ k, μ k * b k ^ 2) = ∑ k, (g k) ^ 2 := by
    refine Finset.sum_congr rfl fun k _ => ?_
    dsimp [g]
    calc
      μ k * b k ^ 2 = (Real.sqrt (μ k)) ^ 2 * b k ^ 2 := by rw [Real.sq_sqrt (hμ k)]
      _ = ((Real.sqrt (μ k)) * b k) ^ 2 := by ring
      _ = (g k) ^ 2 := rfl
  rw [hsum_add, hsum_a, hsum_b]
  rw [Real.sqrt_eq_rpow, Real.sqrt_eq_rpow, Real.sqrt_eq_rpow]
  have hLp_raw := Real.Lp_add_le (Finset.univ : Finset α) f g hp
```

```
  have hLp : (∑ k, |f k + g k| ^ 2) ^ (1 / (2 : ℝ)) ≤
           (∑ k, |f k| ^ 2) ^ (1 / (2 : ℝ)) + (∑ k, |g k| ^ 2) ^ (1 / (2 : ℝ)) := by
    simpa [Real.rpow_two] using hLp_raw
  have hsum_eq : ∀ (h : α → ℝ), (∑ k, h k ^ 2) = (∑ k, |h k| ^ 2) := by
    intro h; refine Finset.sum_congr rfl fun k _ => ?_
    rw [sq_abs]
  calc
    (∑ k, (f k + g k) ^ 2) ^ (1 / (2 : ℝ)) = (∑ k, |f k + g k| ^ 2) ^ (1 / (2 : ℝ)) := by
      simpa [Pi.add_apply] using congrArg (· ^ (1 / (2 : ℝ))) (hsum_eq (f + g))
    _ ≤ (∑ k, |f k| ^ 2) ^ (1 / (2 : ℝ)) + (∑ k, |g k| ^ 2) ^ (1 / (2 : ℝ)) := hLp
    _ = (∑ k, f k ^ 2) ^ (1 / (2 : ℝ)) + (∑ k, g k ^ 2) ^ (1 / (2 : ℝ)) := by
      rw [hsum_eq f, hsum_eq g]

theorem mc_weighted_tri {α : Type*} [Fintype α] (μ a b c : α → ℝ) (hμ : ∀ k, 0 ≤ μ k)
    (hc : ∀ k, 0 ≤ c k) (hcab : ∀ k, c k ≤ a k + b k) :
    Real.sqrt (∑ k, μ k * c k ^ 2)
      ≤ Real.sqrt (∑ k, μ k * a k ^ 2) + Real.sqrt (∑ k, μ k * b k ^ 2) := by

  set A := Real.sqrt (∑ k, μ k * a k ^ 2) with hA_def
  set B := Real.sqrt (∑ k, μ k * b k ^ 2) with hB_def
  have hA_nonneg : 0 ≤ A := Real.sqrt_nonneg _
  have hB_nonneg : 0 ≤ B := Real.sqrt_nonneg _
  have hA_sq : A ^ 2 = ∑ k, μ k * a k ^ 2 :=
    Real.sq_sqrt (Finset.sum_nonneg (fun i _ => mul_nonneg (hμ i) (sq_nonneg _)))
  have hB_sq : B ^ 2 = ∑ k, μ k * b k ^ 2 :=
    Real.sq_sqrt (Finset.sum_nonneg (fun i _ => mul_nonneg (hμ i) (sq_nonneg _)))

  have h_nonneg_sum_c : 0 ≤ ∑ k, μ k * c k ^ 2 :=
    Finset.sum_nonneg (fun i _ => mul_nonneg (hμ i) (sq_nonneg _))
  have h_sq_goal : (Real.sqrt (∑ k, μ k * c k ^ 2)) ^ 2 ≤ (A + B) ^ 2 := by
    calc
      (Real.sqrt (∑ k, μ k * c k ^ 2)) ^ 2 = ∑ k, μ k * c k ^ 2 :=
        Real.sq_sqrt h_nonneg_sum_c
      _ ≤ (A + B) ^ 2 := by


        have hc_sq_bound : ∀ k, c k ^ 2 ≤ (|a k| + |b k|) ^ 2 := by
          intro k
          have hck : 0 ≤ c k := hc k
          have hle : c k ≤ |a k| + |b k| := by
            have hle' := hcab k
            have h_tri : a k + b k ≤ |a k| + |b k| := by
              have h1 : a k ≤ |a k| := le_abs_self (a k)
              have h2 : b k ≤ |b k| := le_abs_self (b k)
              linarith
            linarith
          have hsum_nonneg : 0 ≤ |a k| + |b k| := add_nonneg (abs_nonneg _) (abs_nonneg _)
          nlinarith

        have h_sum_c_sq_le : ∑ k, μ k * c k ^ 2 ≤ ∑ k, μ k * (|a k| + |b k|) ^ 2 :=
          Finset.sum_le_sum (fun k _ => mul_le_mul_of_nonneg_left (hc_sq_bound k) (hμ k))

        have h_expand : ∑ k, μ k * (|a k| + |b k|) ^ 2 =
            (∑ k, μ k * a k ^ 2) + 2 * (∑ k, μ k * |a k| * |b k|) + (∑ k, μ k * b k ^ 2) := by
          calc
            ∑ k, μ k * (|a k| + |b k|) ^ 2
                = ∑ k, μ k * (|a k| ^ 2 + 2 * |a k| * |b k| + |b k| ^ 2) := by
              refine Finset.sum_congr rfl (fun k _ => ?_)
              ring
```

```
          _ = (∑ k, μ k * |a k| ^ 2) + (∑ k, μ k * (2 * |a k| * |b k|)) + (∑ k, μ k * |b k| ^ 2) := by
            simp [Finset.sum_add_distrib, mul_add]
          _ = (∑ k, μ k * |a k| ^ 2) + 2 * (∑ k, μ k * |a k| * |b k|) + (∑ k, μ k * |b k| ^ 2) := by
            simp [Finset.mul_sum, mul_comm, mul_left_comm, mul_assoc]
          _ = (∑ k, μ k * a k ^ 2) + 2 * (∑ k, μ k * |a k| * |b k|) + (∑ k, μ k * b k ^ 2) := by
            simp [sq_abs]


      have h_cauchy : (∑ k, μ k * |a k| * |b k|) ^ 2 ≤ (∑ k, μ k * a k ^ 2) * (∑ k, μ k * b k ^ 2) := by
        have h_cs := Finset.sum_mul_sq_le_sq_mul_sq (Finset.univ : Finset α)
          (fun k => Real.sqrt (μ k) * |a k|) (fun k => Real.sqrt (μ k) * |b k|)
        have h_left : (∑ k : α, (Real.sqrt (μ k) * |a k|) * (Real.sqrt (μ k) * |b k|)) = ∑ k, μ k * |a
↪  k| * |b k| := by
          calc
            (∑ k : α, (Real.sqrt (μ k) * |a k|) * (Real.sqrt (μ k) * |b k|))
                = ∑ k, (Real.sqrt (μ k)) ^ 2 * |a k| * |b k| := by
              refine Finset.sum_congr rfl (fun k _ => ?_)
              ring
            _ = ∑ k, μ k * |a k| * |b k| := by
              simp [Real.sq_sqrt (hμ _)]
        have h_r1 : (∑ k : α, ((Real.sqrt (μ k) * |a k|)) ^ 2) = ∑ k, μ k * a k ^ 2 := by
          simp [mul_pow, Real.sq_sqrt (hμ _), sq_abs]
        have h_r2 : (∑ k : α, ((Real.sqrt (μ k) * |b k|)) ^ 2) = ∑ k, μ k * b k ^ 2 := by
          simp [mul_pow, Real.sq_sqrt (hμ _), sq_abs]
        simpa [h_left, h_r1, h_r2] using h_cs
      have h_main : ∑ k, μ k * |a k| * |b k| ≤ A * B := by
        have h_sq_le : (∑ k, μ k * |a k| * |b k|) ^ 2 ≤ (A * B) ^ 2 := by
          calc
            (∑ k, μ k * |a k| * |b k|) ^ 2 ≤ (∑ k, μ k * a k ^ 2) * (∑ k, μ k * b k ^ 2) := h_cauchy
            _ = A ^ 2 * B ^ 2 := by rw [hA_sq, hB_sq]
            _ = (A * B) ^ 2 := by ring
        have h_AB_nonneg : 0 ≤ A * B := mul_nonneg hA_nonneg hB_nonneg
        nlinarith

      calc
        ∑ k, μ k * c k ^ 2 ≤ ∑ k, μ k * (|a k| + |b k|) ^ 2 := h_sum_c_sq_le
        _ = (∑ k, μ k * a k ^ 2) + 2 * (∑ k, μ k * |a k| * |b k|) + (∑ k, μ k * b k ^ 2) := h_expand
        _ ≤ (∑ k, μ k * a k ^ 2) + 2 * (A * B) + (∑ k, μ k * b k ^ 2) := by
          nlinarith
        _ = A * B * 2 + A ^ 2 + B ^ 2 := by rw [hA_sq, hB_sq]; ring
        _ = (A + B) ^ 2 := by ring

  have h_sq_mono : (Real.sqrt (∑ k, μ k * c k ^ 2)) ^ 2 ≤ (A + B) ^ 2 →
      Real.sqrt (∑ k, μ k * c k ^ 2) ≤ A + B := by
    have h_left_nonneg : 0 ≤ Real.sqrt (∑ k, μ k * c k ^ 2) := Real.sqrt_nonneg _
    have h_right_nonneg : 0 ≤ A + B := add_nonneg hA_nonneg hB_nonneg
    intro h_sq
    nlinarith
  exact h_sq_mono h_sq_goal

theorem mc_triple_mul {n : ℕ} (π : Fin n → ℝ) (P Q : Matrix (Fin n) (Fin n) ℝ)
    (d : Fin n → Fin n → ℝ) :
    ∑ i, ∑ j, π i * (P * Q) i j * d i j
      = ∑ p : Fin n × Fin n × Fin n, π p.1 * P p.1 p.2.1 * Q p.2.1 p.2.2 * d p.1 p.2.2 := by
  classical
  calc
    ∑ i, ∑ j, π i * (P * Q) i j * d i j
        = ∑ i, ∑ j, π i * (∑ k, P i k * Q k j) * d i j := by
      simp_rw [Matrix.mul_apply]
```

```
    _ = ∑ i, ∑ j, ∑ k, π i * P i k * Q k j * d i j := by
      simp [Finset.mul_sum, Finset.sum_mul, mul_assoc]
    _ = ∑ i, ∑ k, ∑ j, (π i * P i k * Q k j) * d i j := by
      refine Finset.sum_congr rfl fun i _ => ?_
      rw [Finset.sum_comm]
    _ = ∑ p : Fin n × Fin n × Fin n, π p.1 * P p.1 p.2.1 * Q p.2.1 p.2.2 * d p.1 p.2.2 := by
      simp_rw [Fintype.sum_prod_type]

theorem mc_triple_first {n : ℕ} (π : Fin n → ℝ) (P Q : Matrix (Fin n) (Fin n) ℝ)
    (d : Fin n → Fin n → ℝ) (hQ : ∀ k, ∑ j, Q k j = 1) :
    ∑ p : Fin n × Fin n × Fin n, π p.1 * P p.1 p.2.1 * Q p.2.1 p.2.2 * d p.1 p.2.1
      = ∑ i, ∑ k, π i * P i k * d i k := by


  have h_reindex : (∑ p : Fin n × Fin n × Fin n, π p.1 * P p.1 p.2.1 * Q p.2.1 p.2.2 * d p.1 p.2.1) =
      (∑ i : Fin n, ∑ j : Fin n, ∑ k : Fin n, π i * P i j * Q j k * d i j) := by
    calc
      (∑ p : Fin n × Fin n × Fin n, π p.1 * P p.1 p.2.1 * Q p.2.1 p.2.2 * d p.1 p.2.1)
          = (∑ p ∈ ((Finset.univ : Finset (Fin n)) ×□ (Finset.univ : Finset (Fin n × Fin n))),
            π p.1 * P p.1 p.2.1 * Q p.2.1 p.2.2 * d p.1 p.2.1) := by
        simp [Finset.univ_product_univ]
      _ = (∑ i ∈ (Finset.univ : Finset (Fin n)),
            ∑ jk ∈ (Finset.univ : Finset (Fin n × Fin n)),
            π i * P i jk.1 * Q jk.1 jk.2 * d i jk.1) := by
        rw [Finset.sum_product]
      _ = (∑ i : Fin n, ∑ jk : Fin n × Fin n, π i * P i jk.1 * Q jk.1 jk.2 * d i jk.1) := by
        simp
      _ = (∑ i : Fin n, (∑ j : Fin n, ∑ k : Fin n, π i * P i j * Q j k * d i j)) := by
        refine Finset.sum_congr rfl fun i hi => ?_
        calc
          ∑ jk : Fin n × Fin n, π i * P i jk.1 * Q jk.1 jk.2 * d i jk.1
              = (∑ jk ∈ ((Finset.univ : Finset (Fin n)) ×□ (Finset.univ : Finset (Fin n))),
                π i * P i jk.1 * Q jk.1 jk.2 * d i jk.1) := by
            simp [Finset.univ_product_univ]
          _ = ∑ j ∈ (Finset.univ : Finset (Fin n)), ∑ k ∈ (Finset.univ : Finset (Fin n)),
                π i * P i j * Q j k * d i j := by
            rw [Finset.sum_product]
          _ = ∑ j : Fin n, ∑ k : Fin n, π i * P i j * Q j k * d i j := by
            simp
      _ = (∑ i : Fin n, ∑ j : Fin n, ∑ k : Fin n, π i * P i j * Q j k * d i j) := by
        simp

  have h_collapse : (∑ i : Fin n, ∑ j : Fin n, ∑ k : Fin n, π i * P i j * Q j k * d i j) =
      (∑ i : Fin n, ∑ j : Fin n, π i * P i j * d i j * (∑ k : Fin n, Q j k)) := by
    simp [Finset.mul_sum, mul_comm, mul_left_comm]

  have h_sum_one : (∑ i : Fin n, ∑ j : Fin n, π i * P i j * d i j * (∑ k : Fin n, Q j k)) =
      (∑ i : Fin n, ∑ j : Fin n, π i * P i j * d i j) := by
    simp [hQ, mul_comm, mul_left_comm]

  calc
    ∑ p : Fin n × Fin n × Fin n, π p.1 * P p.1 p.2.1 * Q p.2.1 p.2.2 * d p.1 p.2.1
        = (∑ i : Fin n, ∑ j : Fin n, ∑ k : Fin n, π i * P i j * Q j k * d i j) := h_reindex
    _ = (∑ i : Fin n, ∑ j : Fin n, π i * P i j * d i j * (∑ k : Fin n, Q j k)) := h_collapse
    _ = (∑ i : Fin n, ∑ j : Fin n, π i * P i j * d i j) := h_sum_one
    _ = ∑ i, ∑ k, π i * P i k * d i k := by
      simp

theorem mc_triple_second {n : ℕ} (π : Fin n → ℝ) (P Q : Matrix (Fin n) (Fin n) ℝ)
```

```
    (d : Fin n → Fin n → ℝ) (hP : ∀ k, ∑ i, π i * P i k = π k) :
    ∑ p : Fin n × Fin n × Fin n, π p.1 * P p.1 p.2.1 * Q p.2.1 p.2.2 * d p.2.1 p.2.2
      = ∑ k, ∑ j, π k * Q k j * d k j := by

  have h1 : (∑ p : Fin n × Fin n × Fin n, π p.1 * P p.1 p.2.1 * Q p.2.1 p.2.2 * d p.2.1 p.2.2) =
      (∑ i : Fin n, ∑ j : Fin n, ∑ k : Fin n, π i * P i j * Q j k * d j k) := by
    calc
      (∑ p : Fin n × Fin n × Fin n, π p.1 * P p.1 p.2.1 * Q p.2.1 p.2.2 * d p.2.1 p.2.2)
        = (∑ p ∈ (Finset.univ : Finset (Fin n × Fin n × Fin n)), π p.1 * P p.1 p.2.1 * Q p.2.1 p.2.2 *
↪  d p.2.1 p.2.2) := rfl
      _ = (∑ p ∈ (Finset.univ : Finset (Fin n)) ×U+02E2 (Finset.univ : Finset (Fin n × Fin n)), π p.1 * P
↪  p.1 p.2.1 * Q p.2.1 p.2.2 * d p.2.1 p.2.2) := by
        rw [Finset.univ_product_univ]
      _ = (∑ i ∈ (Finset.univ : Finset (Fin n)), ∑ pair ∈ (Finset.univ : Finset (Fin n × Fin n)), π i *
↪  P i pair.1 * Q pair.1 pair.2 * d pair.1 pair.2) := by
        rw [Finset.sum_product]
      _ = (∑ i : Fin n, ∑ pair : Fin n × Fin n, π i * P i pair.1 * Q pair.1 pair.2 * d pair.1 pair.2) :=
↪  by simp
      _ = (∑ i : Fin n, ∑ j : Fin n, ∑ k : Fin n, π i * P i j * Q j k * d j k) := by
        refine Finset.sum_congr rfl (λ i hi => ?_)
        calc
          ∑ pair : Fin n × Fin n, π i * P i pair.1 * Q pair.1 pair.2 * d pair.1 pair.2
            = ∑ pair ∈ (Finset.univ : Finset (Fin n × Fin n)), π i * P i pair.1 * Q pair.1 pair.2 * d
↪  pair.1 pair.2 := rfl
          _ = ∑ pair ∈ (Finset.univ : Finset (Fin n)) ×U+02E2 (Finset.univ : Finset (Fin n)), π i * P i
↪  pair.1 * Q pair.1 pair.2 * d pair.1 pair.2 := by
            rw [Finset.univ_product_univ]
          _ = ∑ j ∈ (Finset.univ : Finset (Fin n)), ∑ k ∈ (Finset.univ : Finset (Fin n)), π i * P i j * Q
↪  j k * d j k := by
            rw [Finset.sum_product]
          _ = ∑ j : Fin n, ∑ k : Fin n, π i * P i j * Q j k * d j k := by simp
      _ = (∑ i : Fin n, ∑ j : Fin n, ∑ k : Fin n, π i * P i j * Q j k * d j k) := rfl

  rw [h1]




  rw [Finset.sum_comm]



  have h_factor : (∑ j : Fin n, ∑ i : Fin n, ∑ k : Fin n, π i * P i j * Q j k * d j k) =
      (∑ j : Fin n, ∑ k : Fin n, (∑ i : Fin n, π i * P i j) * Q j k * d j k) := by
    calc
      (∑ j : Fin n, ∑ i : Fin n, ∑ k : Fin n, π i * P i j * Q j k * d j k)
        = (∑ j : Fin n, ∑ k : Fin n, ∑ i : Fin n, π i * P i j * Q j k * d j k) := by
          refine Finset.sum_congr rfl (λ j hj => ?_)
          rw [Finset.sum_comm]
      _ = (∑ j : Fin n, ∑ k : Fin n, (∑ i : Fin n, π i * P i j * Q j k * d j k)) := by
          simp
      _ = (∑ j : Fin n, ∑ k : Fin n, (∑ i : Fin n, π i * P i j) * Q j k * d j k) := by
          refine Finset.sum_congr rfl (λ j hj => ?_)
          refine Finset.sum_congr rfl (λ k hk => ?_)
          simp [Finset.mul_sum, mul_comm, mul_left_comm]

  rw [h_factor]
```

```
  simp [hP]

theorem mc_D_subadd {n : ℕ} (π : Fin n → ℝ) (hπ : IsProbVec π)
    (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochastic A)
    (hstat : ∀ j, ∑ i, π i * A i j = π j) (x : Fin n → L1) (D : ℕ → ℝ)
    (hD : ∀ s, D s = ∑ i, ∑ j, π i * (A ^ s) i j * ‖x i - x j‖ ^ 2) (r s : ℕ) :
    Real.sqrt (D (r + s)) ≤ Real.sqrt (D r) + Real.sqrt (D s) := by
  have hμ : ∀ p : Fin n × Fin n × Fin n,
      0 ≤ π p.1 * (A ^ r) p.1 p.2.1 * (A ^ s) p.2.1 p.2.2 := by
    intro p
    exact mul_nonneg (mul_nonneg (hπ.1 p.1) ((mc_pow_stochastic A hA r).1 p.1 p.2.1))
      ((mc_pow_stochastic A hA s).1 p.2.1 p.2.2)
  have htri : ∀ p : Fin n × Fin n × Fin n,
      ‖x p.1 - x p.2.2‖ ≤ ‖x p.1 - x p.2.1‖ + ‖x p.2.1 - x p.2.2‖ := by
    intro p
    calc ‖x p.1 - x p.2.2‖ = ‖(x p.1 - x p.2.1) + (x p.2.1 - x p.2.2)‖ := by
          rw [sub_add_sub_cancel]
      _ ≤ ‖x p.1 - x p.2.1‖ + ‖x p.2.1 - x p.2.2‖ := norm_add_le _ _
  have hDrs : D (r + s) = ∑ p : Fin n × Fin n × Fin n,
      π p.1 * (A ^ r) p.1 p.2.1 * (A ^ s) p.2.1 p.2.2 * ‖x p.1 - x p.2.2‖ ^ 2 := by
    rw [hD, pow_add]
    exact mc_triple_mul π (A ^ r) (A ^ s) (fun i j => ‖x i - x j‖ ^ 2)
  have hDr : D r = ∑ p : Fin n × Fin n × Fin n,
      π p.1 * (A ^ r) p.1 p.2.1 * (A ^ s) p.2.1 p.2.2 * ‖x p.1 - x p.2.1‖ ^ 2 := by
    rw [hD]
    exact (mc_triple_first π (A ^ r) (A ^ s) (fun i j => ‖x i - x j‖ ^ 2)
      (mc_pow_stochastic A hA s).2).symm
  have hDs : D s = ∑ p : Fin n × Fin n × Fin n,
      π p.1 * (A ^ r) p.1 p.2.1 * (A ^ s) p.2.1 p.2.2 * ‖x p.2.1 - x p.2.2‖ ^ 2 := by
    rw [hD]
    exact (mc_triple_second π (A ^ r) (A ^ s) (fun i j => ‖x i - x j‖ ^ 2)
      (mc_pow_stationary π A hstat r)).symm
  have key := mc_weighted_tri
    (fun p : Fin n × Fin n × Fin n => π p.1 * (A ^ r) p.1 p.2.1 * (A ^ s) p.2.1 p.2.2)
    (fun p => ‖x p.1 - x p.2.1‖) (fun p => ‖x p.2.1 - x p.2.2‖) (fun p => ‖x p.1 - x p.2.2‖)
    hμ (fun p => norm_nonneg _) htri
  rw [hDrs, hDr, hDs]
  exact key

theorem mc_cesaro_Dt (t : ℕ) (ht : 1 ≤ t) (D : ℕ → ℝ) (hD0 : D 0 = 0)
    (hnn : ∀ s, 0 ≤ D s)
    (hsub : ∀ r s, Real.sqrt (D (r + s)) ≤ Real.sqrt (D r) + Real.sqrt (D s)) :
    D t ≤ 4 * ((1 / (t : ℝ)) * ∑ s ∈ Finset.Icc 1 t, D s) := by

  have hblock : ∀ r, 1 ≤ r → r ≤ t → D t ≤ 2 * D r + 2 * D (t - r) := by
    intro r hr1 hr2
    have h := hsub r (t - r)
    have hsum : r + (t - r) = t := Nat.add_sub_cancel' hr2
    rw [hsum] at h
    have hsq : D t ≤ (Real.sqrt (D r) + Real.sqrt (D (t - r))) ^ 2 := by
      have h_sq_ineq : (Real.sqrt (D t)) ^ 2 ≤ (Real.sqrt (D r) + Real.sqrt (D (t - r))) ^ 2 := by
        have h_sqrt_nonneg : 0 ≤ Real.sqrt (D t) := Real.sqrt_nonneg _
        have h_sum_nonneg : 0 ≤ Real.sqrt (D r) + Real.sqrt (D (t - r)) := by positivity
        nlinarith
      have h_sq_Dt : (Real.sqrt (D t)) ^ 2 = D t := Real.sq_sqrt (hnn t)
```

```
    rw [h_sq_Dt] at h_sq_ineq
    exact h_sq_ineq
  have hsq_bound : (Real.sqrt (D r) + Real.sqrt (D (t - r))) ^ 2 ≤ 2 * D r + 2 * D (t - r) := by
    have h_amgm : 2 * Real.sqrt (D r) * Real.sqrt (D (t - r))
        ≤ (Real.sqrt (D r)) ^ 2 + (Real.sqrt (D (t - r))) ^ 2 := by
      nlinarith [sq_nonneg (Real.sqrt (D r) - Real.sqrt (D (t - r)))]
    have hsqD_r : (Real.sqrt (D r)) ^ 2 = D r := Real.sq_sqrt (hnn r)
    have hsqD_tr : (Real.sqrt (D (t - r))) ^ 2 = D (t - r) := Real.sq_sqrt (hnn (t - r))
    nlinarith
  nlinarith


have hcard : (Finset.Icc 1 t : Finset ℕ).card = t := by
  simp

have hsum : ∑ r ∈ Finset.Icc 1 t, D t ≤ ∑ r ∈ Finset.Icc 1 t, (2 * D r + 2 * D (t - r)) := by
  apply Finset.sum_le_sum
  intro r hr
  rcases Finset.mem_Icc.mp hr with ⟨hr1, hr2⟩
  exact hblock r hr1 hr2


have hright_split : ∑ r ∈ Finset.Icc 1 t, (2 * D r + 2 * D (t - r))
    = 2 * (∑ r ∈ Finset.Icc 1 t, D r) + 2 * (∑ r ∈ Finset.Icc 1 t, D (t - r)) := by
  simp [Finset.sum_add_distrib, Finset.mul_sum]


have hreindex : ∑ r ∈ Finset.Icc 1 t, D (t - r) = ∑ s ∈ Finset.Icc 1 (t-1), D s := by
  by_cases ht1' : t = 1
  · subst ht1'; simp [hD0]
  · have ht_ge_2 : 2 ≤ t := by omega

    have h_disjoint : Finset.Icc 1 t = {t} ∪ Finset.Icc 1 (t-1) := by
      ext x; constructor
      · intro hx
        rcases Finset.mem_Icc.mp hx with ⟨hx1, hx2⟩
        by_cases hxt : x = t
        · apply Finset.mem_union_left; exact Finset.mem_singleton.mpr hxt
        · apply Finset.mem_union_right; apply Finset.mem_Icc.mpr; omega
      · intro hx
        rcases Finset.mem_union.mp hx with (hx' | hx')
        · rw [Finset.mem_singleton.mp hx']; apply Finset.mem_Icc.mpr; omega
        · rcases Finset.mem_Icc.mp hx' with ⟨hx1, hx2⟩; apply Finset.mem_Icc.mpr; omega
    have h_disjoint' : Disjoint ({t} : Finset ℕ) (Finset.Icc 1 (t-1)) := by
      apply Finset.disjoint_singleton_left.mpr
      intro hx; rcases Finset.mem_Icc.mp hx with ⟨hx1, hx2⟩; omega
    rw [h_disjoint, Finset.sum_union h_disjoint', Finset.sum_singleton]

    have hsub_zero : t - t = 0 := Nat.sub_self _
    rw [hsub_zero, hD0, zero_add]



    apply Finset.sum_nbij (fun r => t - r) ?_ ?_ ?_ ?_
    · intro r hr
      rcases Finset.mem_Icc.mp hr with ⟨hr1, hr2⟩
      apply Finset.mem_Icc.mpr
      have hpos : 1 ≤ t - r := by omega
      have hle : t - r ≤ t - 1 := by omega
```

```
      exact ⟨hpos, hle⟩
    · intro r₁ hr₁ r₂ hr₂ h_eq
      rcases Finset.mem_Icc.mp hr₁ with ⟨hr₁1, hr₁2⟩
      rcases Finset.mem_Icc.mp hr₂ with ⟨hr₂1, hr₂2⟩
      simp at h_eq
      omega
    ·

      intro s hs
      rcases Finset.mem_Icc.mp hs with ⟨hs1, hs2⟩
      have hs_le_t : s ≤ t := by omega
      have hpos : 1 ≤ t - s := by omega
      have hle : t - s ≤ t - 1 := by omega
      refine ⟨t - s, Finset.mem_Icc.mpr ⟨hpos, hle⟩, ?_⟩

      simp
      omega
    ·









      intro r hr
      rfl


  have h_ineq : (t : ℝ) * D t ≤ 2 * (∑ r ∈ Finset.Icc 1 t, D r) + 2 * (∑ s ∈ Finset.Icc 1 (t-1), D s) :=
↪   by
    calc
      (t : ℝ) * D t = ∑ r ∈ Finset.Icc 1 t, D t := by symm; simp [hcard]
      _ ≤ ∑ r ∈ Finset.Icc 1 t, (2 * D r + 2 * D (t - r)) := hsum
      _ = 2 * (∑ r ∈ Finset.Icc 1 t, D r) + 2 * (∑ r ∈ Finset.Icc 1 t, D (t - r)) := by
        simp [Finset.sum_add_distrib, Finset.mul_sum]
      _ = 2 * (∑ r ∈ Finset.Icc 1 t, D r) + 2 * (∑ s ∈ Finset.Icc 1 (t-1), D s) := by
        rw [hreindex]


  by_cases ht1 : t = 1
  · subst ht1
    have hsum1 : (∑ s ∈ Finset.Icc 1 1, D s) = D 1 := by simp
    rw [hsum1]
    have hpos1 : 0 ≤ D 1 := hnn 1
    nlinarith
  · have ht_ge_2 : 2 ≤ t := by omega
    have hsplit : (∑ r ∈ Finset.Icc 1 t, D r) = D t + (∑ r ∈ Finset.Icc 1 (t-1), D r) := by

      have h_disjoint : Finset.Icc 1 t = {t} ∪ Finset.Icc 1 (t-1) := by
        ext x; constructor
        · intro hx
          rcases Finset.mem_Icc.mp hx with ⟨hx1, hx2⟩
          by_cases hxt : x = t
          · apply Finset.mem_union_left; exact Finset.mem_singleton.mpr hxt
          · apply Finset.mem_union_right; apply Finset.mem_Icc.mpr; omega
        · intro hx
```

```
      rcases Finset.mem_union.mp hx with (hx' | hx')
      · rw [Finset.mem_singleton.mp hx']; apply Finset.mem_Icc.mpr; omega
      · rcases Finset.mem_Icc.mp hx' with ⟨hx1, hx2⟩; apply Finset.mem_Icc.mpr; omega
  have h_disjoint' : Disjoint ({t} : Finset ℕ) (Finset.Icc 1 (t-1)) := by
    apply Finset.disjoint_singleton_left.mpr
    intro hx; rcases Finset.mem_Icc.mp hx with ⟨hx1, hx2⟩; omega
  rw [h_disjoint, Finset.sum_union h_disjoint', Finset.sum_singleton, add_comm]
rw [hsplit] at h_ineq

set S := ∑ s ∈ Finset.Icc 1 (t-1), D s with hS

have hS_nonneg : 0 ≤ S := by
  rw [hS]
  apply Finset.sum_nonneg
  intro s _
  exact hnn s



have h_derived : ((t : ℝ) - 2) * D t ≤ 4 * S := by
  nlinarith







have hgoal : D t ≤ 4 * ((1 / (t : ℝ)) * ∑ s ∈ Finset.Icc 1 t, D s) := by
  rw [hsplit, hS]
  have htpos : 0 < (t : ℝ) := by
    have : 0 < t := Nat.lt_of_lt_of_le (by norm_num) ht
    exact_mod_cast this


  have htarget : (t : ℝ) * D t ≤ 4 * (D t + (∑ s ∈ Finset.Icc 1 (t-1), D s)) := by
    by_cases ht_le_4 : (t : ℝ) ≤ 4
    ·
      have h_target : ((t : ℝ) - 4) * D t ≤ 4 * (∑ s ∈ Finset.Icc 1 (t-1), D s) := by
        have h_nonpos : ((t : ℝ) - 4) * D t ≤ 0 := by
          have : (t : ℝ) - 4 ≤ 0 := by linarith
          nlinarith [hnn t]
        nlinarith
      nlinarith
    ·
      have h_target : ((t : ℝ) - 4) * D t ≤ 4 * (∑ s ∈ Finset.Icc 1 (t-1), D s) := by
        have hcoeff : (t : ℝ) - 4 ≤ (t : ℝ) - 2 := by nlinarith
        have hDt_nonneg : 0 ≤ D t := hnn t
        nlinarith
      nlinarith

  calc
    D t = ((t : ℝ) * D t) / (t : ℝ) := by field_simp [ne_of_gt htpos]
    _ ≤ (4 * (D t + (∑ s ∈ Finset.Icc 1 (t-1), D s))) / (t : ℝ) := by
      apply div_le_div_of_nonneg_right htarget
      positivity
    _ = 4 * ((1 / (t : ℝ)) * (D t + (∑ s ∈ Finset.Icc 1 (t-1), D s))) := by ring

exact hgoal
```

```
theorem mc_cesaro_blocks (t : ℕ) (D : ℕ → ℝ) (hnn : ∀ s, 0 ≤ D s)
    (hsub : ∀ r s, Real.sqrt (D (r + s)) ≤ Real.sqrt (D r) + Real.sqrt (D s)) (j r : ℕ) :
    D (j * t + r) ≤ 2 * (j : ℝ) ^ 2 * D t + 2 * D r := by

  have h_sqrt_bound : Real.sqrt (D (j * t + r)) ≤ (j : ℝ) * Real.sqrt (D t) + Real.sqrt (D r) := by
    induction' j with k ih
    · simp
    · have h_sub := hsub (k * t + r) t
      have h_eq : (k.succ * t + r) = ((k * t + r) + t) := by
        calc
          k.succ * t + r = (k * t + t) + r := by rw [Nat.succ_mul]
          _ = (k * t + r) + t := by omega
      have h_target : Real.sqrt (D (k.succ * t + r)) ≤ Real.sqrt (D (k * t + r)) + Real.sqrt (D t) := by
        simpa [h_eq] using h_sub
      have h_comb : Real.sqrt (D (k.succ * t + r)) ≤ ((k : ℝ) * Real.sqrt (D t) + Real.sqrt (D r)) +
↪  Real.sqrt (D t) := by
        linarith

      simpa [Nat.cast_succ, add_comm, add_left_comm, add_assoc, mul_add, add_mul] using h_comb

  have h_sq : D (j * t + r) ≤ ((j : ℝ) * Real.sqrt (D t) + Real.sqrt (D r)) ^ 2 := by
    have h_nonneg_sqrt : 0 ≤ Real.sqrt (D (j * t + r)) := Real.sqrt_nonneg _
    have h := mul_self_le_mul_self h_nonneg_sqrt h_sqrt_bound

    rw [Real.mul_self_sqrt (hnn (j * t + r))] at h

    simpa [sq] using h

  have h_cross : 2 * ((j : ℝ) * Real.sqrt (D t)) * Real.sqrt (D r) ≤
      ((j : ℝ) * Real.sqrt (D t)) ^ 2 + (Real.sqrt (D r)) ^ 2 := by
    have h_sq_nonneg : 0 ≤ ((j : ℝ) * Real.sqrt (D t) - Real.sqrt (D r)) ^ 2 := by
      apply pow_two_nonneg
    nlinarith
  calc
    D (j * t + r) ≤ ((j : ℝ) * Real.sqrt (D t) + Real.sqrt (D r)) ^ 2 := h_sq
    _ = ((j : ℝ) * Real.sqrt (D t)) ^ 2 + (Real.sqrt (D r)) ^ 2 + 2 * ((j : ℝ) * Real.sqrt (D t)) *
↪  Real.sqrt (D r) := by ring
    _ ≤ ((j : ℝ) * Real.sqrt (D t)) ^ 2 + (Real.sqrt (D r)) ^ 2 + (((j : ℝ) * Real.sqrt (D t)) ^ 2 +
↪  (Real.sqrt (D r)) ^ 2) := by
      nlinarith
    _ = 2 * (((j : ℝ) * Real.sqrt (D t)) ^ 2 + (Real.sqrt (D r)) ^ 2) := by ring
    _ = 2 * (((j : ℝ) ^ 2 * (Real.sqrt (D t)) ^ 2) + (Real.sqrt (D r)) ^ 2) := by ring
    _ = 2 * (((j : ℝ) ^ 2 * D t) + D r) := by simp [Real.sq_sqrt (hnn t), Real.sq_sqrt (hnn r)]
    _ = 2 * (j : ℝ) ^ 2 * D t + 2 * D r := by ring

theorem mc_hasSum_sq_geom (r : ℝ) (h0 : 0 ≤ r) (h1 : r < 1) :
    HasSum (fun s : ℕ => (s : ℝ) ^ 2 * r ^ s) (r * (1 + r) / (1 - r) ^ 3) := by
  have hr_norm : ‖r‖ < 1 := by
    rw [Real.norm_eq_abs]
    exact abs_lt.mpr ⟨by linarith, h1⟩
  have h_choose2 : HasSum (fun n : ℕ => ((n + 2 : ℕ).choose 2 : ℝ) * r ^ n) ((1 : ℝ) / (1 - r) ^ 3) := by
    have h := hasSum_choose_mul_geometric_of_norm_lt_one (𝕜 := ℝ) 2 hr_norm
    simpa [show (2:ℕ)+1 = 3 by decide] using h
  have h_n_mul : HasSum (fun n : ℕ => (n : ℝ) * r ^ n) (r / (1 - r) ^ 2) :=
    hasSum_coe_mul_geometric_of_norm_lt_one hr_norm
  have h_geom : HasSum (fun n : ℕ => r ^ n) ((1 - r)⁻¹) :=
    hasSum_geometric_of_lt_one h0 h1

  have h_sq_eq : ∀ n : ℕ, (n : ℝ) ^ 2 = (2 : ℝ) * (((n : ℕ) + 2).choose 2 : ℝ) - (3 : ℝ) * (n : ℝ) - 2 := by
```

```
    intro n
    have h_choose_real : (((n : ℕ) + 2).choose 2 : ℝ) = ((n : ℝ) + 2) * ((n : ℝ) + 1) / 2 := by
      have h := Nat.cast_choose_two (a := n+2) (K := ℝ)
      push_cast at h
      have h' : ((n : ℝ) + 2) - 1 = (n : ℝ) + 1 := by ring
      simpa [h'] using h
    rw [h_choose_real]
    ring

  have h_sum : HasSum (fun s : ℕ => (s : ℝ) ^ 2 * r ^ s) (r * (1 + r) / (1 - r) ^ 3) := by
    have h1 : HasSum (fun s : ℕ => (2 : ℝ) * (((s : ℕ) + 2).choose 2 : ℝ) * r ^ s)
        ((2 : ℝ) * ((1 : ℝ) / (1 - r) ^ 3)) := by
      simpa [mul_assoc] using h_choose2.mul_left (2 : ℝ)
    have h2 : HasSum (fun s : ℕ => (3 : ℝ) * ((s : ℝ) * r ^ s))
        ((3 : ℝ) * (r / (1 - r) ^ 2)) := by
      simpa [mul_assoc] using h_n_mul.mul_left (3 : ℝ)
    have h3 : HasSum (fun s : ℕ => (2 : ℝ) * (r ^ s))
        ((2 : ℝ) * ((1 - r)⁻¹)) :=
      h_geom.mul_left (2 : ℝ)
    have h_total : HasSum (fun s : ℕ =>
        ((2 : ℝ) * (((s : ℕ) + 2).choose 2 : ℝ) * r ^ s)
        - ((3 : ℝ) * ((s : ℝ) * r ^ s))
        - ((2 : ℝ) * (r ^ s)))
        (((2 : ℝ) * ((1 : ℝ) / (1 - r) ^ 3))
        - ((3 : ℝ) * (r / (1 - r) ^ 2))
        - ((2 : ℝ) * ((1 - r)⁻¹))) := by
      simpa using ((h1.sub h2).sub h3)

    have h_fun_eq : (fun s : ℕ => ((2 : ℝ) * (((s : ℕ) + 2).choose 2 : ℝ) * r ^ s)
        - ((3 : ℝ) * ((s : ℝ) * r ^ s))
        - ((2 : ℝ) * (r ^ s))) = (fun s : ℕ => (s : ℝ) ^ 2 * r ^ s) := by
      ext s
      calc
        ((2 : ℝ) * (((s : ℕ) + 2).choose 2 : ℝ) * r ^ s)
            - ((3 : ℝ) * ((s : ℝ) * r ^ s))
            - ((2 : ℝ) * (r ^ s))
          = ((2 : ℝ) * (((s : ℕ) + 2).choose 2 : ℝ) - (3 : ℝ) * ((s : ℝ) - 2) * r ^ s := by ring
        _ = (s : ℝ) ^ 2 * r ^ s := by rw [h_sq_eq s]
    have h_total' : HasSum (fun s : ℕ => (s : ℝ) ^ 2 * r ^ s)
        (((2 : ℝ) * ((1 : ℝ) / (1 - r) ^ 3))
        - ((3 : ℝ) * (r / (1 - r) ^ 2))
        - ((2 : ℝ) * ((1 - r)⁻¹))) := by
      simpa [h_fun_eq] using h_total

    have h_sum_eq : ((2 : ℝ) * ((1 : ℝ) / (1 - r) ^ 3))
        - ((3 : ℝ) * (r / (1 - r) ^ 2))
        - ((2 : ℝ) * ((1 - r)⁻¹)) = r * (1 + r) / (1 - r) ^ 3 := by
      field_simp [show (1 : ℝ) - r ≠ 0 by linarith]
      ring
    rw [h_sum_eq] at h_total'
    exact h_total'
  exact h_sum

theorem mc_floor_div_sq_le (t s : ℕ) (ht : 1 ≤ t) :
    (((s / t : ℕ) : ℝ)) ^ 2 ≤ (s : ℝ) ^ 2 / (t : ℝ) ^ 2 := by
  have ht_pos : 0 < (t : ℝ) := by
    have : 0 < t := Nat.lt_of_lt_of_le (by decide : 0 < 1) ht
    exact Nat.cast_pos.mpr this
  have hdiv : ((s / t : ℕ) : ℝ) ≤ (s : ℝ) / (t : ℝ) := by
```

```
    have hle : ((s / t : ℕ) : ℝ) * (t : ℝ) ≤ (s : ℝ) := by
      have := Nat.div_mul_le_self s t
      exact mod_cast this
    calc
      ((s / t : ℕ) : ℝ) = (((s / t : ℕ) : ℝ) * (t : ℝ)) / (t : ℝ) := by
        field_simp [ne_of_gt ht_pos]
      _ ≤ (s : ℝ) / (t : ℝ) := by
        gcongr
  have hsq : (((s / t : ℕ) : ℝ)) ^ 2 ≤ ((s : ℝ) / (t : ℝ)) ^ 2 := by
    have hnonneg_left : 0 ≤ ((s / t : ℕ) : ℝ) := by exact mod_cast Nat.zero_le _
    have hnonneg_right : 0 ≤ (s : ℝ) / (t : ℝ) :=
      div_nonneg (by exact mod_cast Nat.zero_le s) (by exact mod_cast Nat.zero_le t)
    nlinarith
  calc
    (((s / t : ℕ) : ℝ)) ^ 2 ≤ ((s : ℝ) / (t : ℝ)) ^ 2 := hsq
    _ = (s : ℝ) ^ 2 / (t : ℝ) ^ 2 := by ring

theorem mc_geom_sq_div (t : ℕ) (ht : 1 ≤ t) :
    Summable (fun s : ℕ => (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * (((s / t : ℕ) : ℝ)) ^ 2) ∧
    ∑' s : ℕ, (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * (((s / t : ℕ) : ℝ)) ^ 2 ≤ 3 := by
  have htpos : (0:ℝ) < t := by exact_mod_cast ht
  have ht0 : (t:ℝ) ≠ 0 := ne_of_gt htpos
  have ht1 : (t:ℝ) + 1 ≠ 0 := by positivity
  set r : ℝ := (t : ℝ) / ((t : ℝ) + 1) with hr
  have hr0 : 0 ≤ r := by rw [hr]; positivity
  have hr1 : r < 1 := by
    rw [hr, div_lt_one (by positivity)]; linarith
  have hS := mc_hasSum_sq_geom r hr0 hr1
  have hg : HasSum (fun s : ℕ => (1 / ((t : ℝ) + 1)) / (t:ℝ)^2 * ((s : ℝ) ^ 2 * r ^ s))
      ((1 / ((t : ℝ) + 1)) / (t:ℝ)^2 * (r * (1 + r) / (1 - r) ^ 3)) := hS.mul_left _
  have hp0 : (0:ℝ) ≤ 1 / ((t : ℝ) + 1) := by positivity
  have hle : ∀ s : ℕ, (1 / ((t : ℝ) + 1)) * r ^ s * (((s / t : ℕ) : ℝ)) ^ 2
      ≤ (1 / ((t : ℝ) + 1)) / (t:ℝ)^2 * ((s : ℝ) ^ 2 * r ^ s) := by
    intro s
    have h1 := mc_floor_div_sq_le t s ht
    have hp : 0 ≤ (1 / ((t : ℝ) + 1)) * r ^ s := mul_nonneg hp0 (pow_nonneg hr0 s)
    calc (1 / ((t : ℝ) + 1)) * r ^ s * (((s / t : ℕ) : ℝ)) ^ 2
        ≤ (1 / ((t : ℝ) + 1)) * r ^ s * ((s : ℝ) ^ 2 / (t : ℝ) ^ 2) :=
          mul_le_mul_of_nonneg_left h1 hp
      _ = (1 / ((t : ℝ) + 1)) / (t:ℝ)^2 * ((s : ℝ) ^ 2 * r ^ s) := by ring
  have hnn : ∀ s : ℕ, 0 ≤ (1 / ((t : ℝ) + 1)) * r ^ s * (((s / t : ℕ) : ℝ)) ^ 2 :=
    fun s => mul_nonneg (mul_nonneg hp0 (pow_nonneg hr0 s)) (sq_nonneg _)
  have hsum : Summable (fun s : ℕ => (1 / ((t : ℝ) + 1)) * r ^ s * (((s / t : ℕ) : ℝ)) ^ 2) :=
    Summable.of_nonneg_of_le hnn hle hg.summable
  have h1r : 1 - r = 1 / ((t:ℝ) + 1) := by
    rw [hr, eq_div_iff ht1, sub_mul, div_mul_cancel₀ _ ht1]; ring
  have key : (1 / ((t : ℝ) + 1)) / (t:ℝ)^2 * (r * (1 + r) / (1 - r) ^ 3)
      = (2 * (t:ℝ) + 1) / t := by
    rw [h1r, hr]
    field_simp
    ring
  refine ⟨hsum, ?_⟩
  calc ∑' s : ℕ, (1 / ((t : ℝ) + 1)) * r ^ s * (((s / t : ℕ) : ℝ)) ^ 2
      ≤ ∑' s : ℕ, (1 / ((t : ℝ) + 1)) / (t:ℝ)^2 * ((s : ℝ) ^ 2 * r ^ s) :=
        Summable.tsum_le_tsum hle hsum hg.summable
    _ = (1 / ((t : ℝ) + 1)) / (t:ℝ)^2 * (r * (1 + r) / (1 - r) ^ 3) := hg.tsum_eq
    _ = (2 * (t:ℝ) + 1) / t := key
    _ ≤ 3 := by
      rw [div_le_iff₀ htpos]
```

```
      have : (1:ℝ) ≤ t := by exact_mod_cast ht
      linarith

theorem mc_geom_mod_coeff_geom (M : ℕ) :
    ∑ k ∈ Finset.range M, (1/2:ℝ)^k ≤ 2 - 2 * (1/2:ℝ)^M := by
  induction M with
  | zero => norm_num
  | succ M ih =>
    rw [Finset.sum_range_succ, pow_succ]
    linarith

theorem mc_geom_mod_coeff (t : ℕ) (ht : 1 ≤ t) (r N : ℕ) :
    ∑ s ∈ (Finset.range N).filter (fun s => s % t = r),
      (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s ≤ 2 / (t : ℝ) := by
  have ht_pos : (0:ℝ) < (t:ℝ) := Nat.cast_pos.mpr (by omega)
  have ht0 : (t:ℝ) ≠ 0 := ht_pos.ne'
  have ht1 : (t:ℝ) + 1 ≠ 0 := (by linarith : (0:ℝ) < (t:ℝ) + 1).ne'
  obtain ⟨q, hq⟩ : ∃ q : ℝ, q = (t:ℝ) / ((t:ℝ) + 1) := ⟨_, rfl⟩
  rw [← hq]
  have hq0 : 0 ≤ q := by rw [hq]; positivity
  have hq1 : q ≤ 1 := by
    rw [hq, div_le_one (by positivity : (0:ℝ) < (t:ℝ) + 1)]
    linarith
  have hqmul : q * (1 + 1 / (t:ℝ)) = 1 := by
    rw [hq]
    field_simp
    try ring
  have hbern : (2:ℝ) ≤ (1 + 1 / (t:ℝ)) ^ t := by
    have ha : (-2:ℝ) ≤ 1 / (t:ℝ) := by
      have : (0:ℝ) ≤ 1 / (t:ℝ) := by positivity
      linarith
    have h := one_add_mul_le_pow ha t
    rw [mul_one_div_cancel ht0] at h
    linarith
  have hqt : q ^ t ≤ 1 / 2 := by
    have h1 : q ^ t * (1 + 1 / (t:ℝ)) ^ t = 1 := by
      rw [← mul_pow, hqmul, one_pow]
    have h2 : q ^ t * 2 ≤ q ^ t * (1 + 1 / (t:ℝ)) ^ t :=
      mul_le_mul_of_nonneg_left hbern (pow_nonneg hq0 t)
    linarith
  have hterm : ∀ s ∈ (Finset.range N).filter (fun s => s % t = r),
      1 / ((t:ℝ) + 1) * q ^ s ≤ 1 / ((t:ℝ) + 1) * (1/2:ℝ) ^ (s / t) := by
    intro s _
    refine mul_le_mul_of_nonneg_left ?_ (by positivity)
    have hsplit : q ^ s = q ^ (s % t) * (q ^ t) ^ (s / t) := by
      rw [← pow_mul, ← pow_add, Nat.mod_add_div]
    rw [hsplit]
    have h1 : q ^ (s % t) ≤ 1 := pow_le_one₀ hq0 hq1
    have h2 : (q ^ t) ^ (s / t) ≤ (1/2:ℝ) ^ (s / t) := by
      first
        | exact pow_le_pow_left₀ (pow_nonneg hq0 t) hqt (s / t)
        | exact pow_le_pow_left (pow_nonneg hq0 t) hqt (s / t)
    have h3 : 0 ≤ (q ^ t) ^ (s / t) := pow_nonneg (pow_nonneg hq0 t) _
    calc q ^ (s % t) * (q ^ t) ^ (s / t) ≤ 1 * (q ^ t) ^ (s / t) :=
          mul_le_mul_of_nonneg_right h1 h3
      _ = (q ^ t) ^ (s / t) := one_mul _
      _ ≤ (1/2:ℝ) ^ (s / t) := h2
  have hinj : Set.InjOn (fun s : ℕ => s / t)
      ↑((Finset.range N).filter (fun s => s % t = r)) := by
```

```
    intro x hx y hy hxy
    have hxm : x % t = r := (Finset.mem_filter.mp (Finset.mem_coe.mp hx)).2
    have hym : y % t = r := (Finset.mem_filter.mp (Finset.mem_coe.mp hy)).2
    have hxy' : x / t = y / t := hxy
    calc x = x % t + t * (x / t) := (Nat.mod_add_div x t).symm
      _ = y % t + t * (y / t) := by rw [hxm, hym, hxy']
      _ = y := Nat.mod_add_div y t
  have hsub : ((Finset.range N).filter (fun s => s % t = r)).image (fun s => s / t)
      ⊆ Finset.range N := by
    intro k hk
    obtain ⟨s, hs, rfl⟩ := Finset.mem_image.mp hk
    exact Finset.mem_range.mpr (lt_of_le_of_lt (Nat.div_le_self s t)
      (Finset.mem_range.mp (Finset.mem_filter.mp hs).1))
  have hgeom : ∑ k ∈ Finset.range N, (1/2:ℝ) ^ k ≤ 2 := by
    have h := mc_geom_mod_coeff_geom N
    have : (0:ℝ) ≤ (1/2:ℝ) ^ N := by positivity
    linarith
  calc ∑ s ∈ (Finset.range N).filter (fun s => s % t = r), 1 / ((t:ℝ) + 1) * q ^ s
      ≤ ∑ s ∈ (Finset.range N).filter (fun s => s % t = r),
          1 / ((t:ℝ) + 1) * (1/2:ℝ) ^ (s / t) := Finset.sum_le_sum hterm
    _ = 1 / ((t:ℝ) + 1) * ∑ s ∈ (Finset.range N).filter (fun s => s % t = r),
          (1/2:ℝ) ^ (s / t) := by rw [Finset.mul_sum]
    _ = 1 / ((t:ℝ) + 1) * ∑ k ∈ ((Finset.range N).filter (fun s => s % t = r)).image
          (fun s => s / t), (1/2:ℝ) ^ k := by rw [Finset.sum_image hinj]
    _ ≤ 1 / ((t:ℝ) + 1) * ∑ k ∈ Finset.range N, (1/2:ℝ) ^ k :=
        mul_le_mul_of_nonneg_left
          (Finset.sum_le_sum_of_subset_of_nonneg hsub (fun k _ _ => by positivity))
          (by positivity)
    _ ≤ 1 / ((t:ℝ) + 1) * 2 := mul_le_mul_of_nonneg_left hgeom (by positivity)
    _ ≤ 1 / (t:ℝ) * 2 :=
        mul_le_mul_of_nonneg_right (one_div_le_one_div_of_le ht_pos (by linarith)) (by norm_num)
    _ = 2 / (t:ℝ) := by ring

theorem mc_geom_mod (t : ℕ) (ht : 1 ≤ t) (f : ℕ → ℝ) (hf : ∀ r, 0 ≤ f r) :
    Summable (fun s : ℕ => (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * f (s % t)) ∧
    ∑' s : ℕ, (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * f (s % t)
      ≤ (2 / (t : ℝ)) * ∑ r ∈ Finset.range t, f r := by
  have ht0 : 0 < t := ht
  have htR : (0:ℝ) < t := by exact_mod_cast ht0
  have hq0 : (0:ℝ) ≤ (t : ℝ) / ((t : ℝ) + 1) := by positivity
  have hq1 : (t : ℝ) / ((t : ℝ) + 1) < 1 := by
    rw [div_lt_one (by positivity)]; linarith
  have hnn : ∀ s : ℕ, 0 ≤ (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * f (s % t) := by
    intro s
    exact mul_nonneg (by positivity) (hf _)
  have hfle : ∀ s : ℕ, f (s % t) ≤ ∑ r ∈ Finset.range t, f r := fun s =>
    Finset.single_le_sum (fun i _ => hf i) (Finset.mem_range.mpr (Nat.mod_lt s ht0))
  have hpart : ∀ N : ℕ, ∑ s ∈ Finset.range N,
      (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * f (s % t)
      ≤ (2 / (t : ℝ)) * ∑ r ∈ Finset.range t, f r := by
    intro N
    rw [← Finset.sum_fiberwise_of_maps_to (g := fun s => s % t)
      (fun s _ => Finset.mem_range.mpr (Nat.mod_lt s ht0))]
    rw [Finset.mul_sum]
    refine Finset.sum_le_sum (fun r _ => ?_)
    have hc := mc_geom_mod_coeff t ht r N
    calc ∑ s ∈ (Finset.range N).filter (fun s => s % t = r),
          (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * f (s % t)
        = (∑ s ∈ (Finset.range N).filter (fun s => s % t = r),
```

```
            (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s) * f r := by
          rw [Finset.sum_mul]
          refine Finset.sum_congr rfl (fun s hs => ?_)
          rw [(Finset.mem_filter.mp hs).2]
      _ ≤ 2 / (t : ℝ) * f r := mul_le_mul_of_nonneg_right hc (hf r)
  have hsum : Summable (fun s : ℕ =>
      (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * f (s % t)) := by
    refine Summable.of_nonneg_of_le hnn (fun s => ?_)
      ((summable_geometric_of_lt_one hq0 hq1).mul_left
        ((1 / ((t : ℝ) + 1)) * ∑ r ∈ Finset.range t, f r))
    show _ ≤ (1 / ((t : ℝ) + 1)) * (∑ r ∈ Finset.range t, f r) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s
    have h1 := hfle s
    have h2 : 0 ≤ (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s := by positivity
    calc (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * f (s % t)
        ≤ (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * ∑ r ∈ Finset.range t, f r :=
          mul_le_mul_of_nonneg_left h1 h2
      _ = (1 / ((t : ℝ) + 1)) * (∑ r ∈ Finset.range t, f r) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s := by
          ring
  exact ⟨hsum, Real.tsum_le_of_sum_range_le hnn hpart⟩

theorem mc_geom_summable_bdd (t : ℕ) (f : ℕ → ℝ) (hnn : ∀ s, 0 ≤ f s)
    (hbdd : ∃ C, ∀ s, f s ≤ C) :
    Summable (fun s : ℕ => (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * f s) := by
  rcases hbdd with ⟨C, hC⟩
  have h_nonneg : ∀ s, 0 ≤ (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * f s := by
    intro s
    have hpos1 : 0 ≤ 1 / ((t : ℝ) + 1) := div_nonneg (by norm_num) (by positivity)
    have hpos2 : 0 ≤ ((t : ℝ) / ((t : ℝ) + 1)) ^ s := pow_nonneg (div_nonneg (Nat.cast_nonneg _) (by
↪  positivity)) s
    have hpos3 : 0 ≤ f s := hnn s
    positivity
  have h_bound : ∀ s, (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * f s ≤ (1 / ((t : ℝ) + 1)) *
↪  ((t : ℝ) / ((t : ℝ) + 1)) ^ s * C := by
    intro s
    gcongr
    exact hC s
  have h_geometric_summable : Summable (fun s : ℕ => ((t : ℝ) / ((t : ℝ) + 1)) ^ s) := by
    have h_ratio_nonneg : 0 ≤ (t : ℝ) / ((t : ℝ) + 1) := div_nonneg (Nat.cast_nonneg _) (by positivity)
    by_cases ht0 : (t : ℝ) = 0
    · have h : Summable (fun s : ℕ => (0 : ℝ) ^ s) := by
        simp
      simpa [ht0] using h
    · have hpos : 0 < (t : ℝ) := by
        apply Nat.cast_pos.mpr
        exact Nat.pos_of_ne_zero (by
          intro hzero
          apply ht0
          exact_mod_cast hzero)
      have h_ratio_lt_one : (t : ℝ) / ((t : ℝ) + 1) < 1 := by
        refine (div_lt_one ?_).mpr ?_
        · positivity
        · linarith
      exact summable_geometric_of_lt_one h_ratio_nonneg h_ratio_lt_one
  have hC_nonneg : 0 ≤ C := by
    have h0 := hC 0
    have h0nn := hnn 0
    linarith
  have h_geometric_summable_mul : Summable (fun s : ℕ => (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^
↪  s * C) := by
```

```
    have htemp := h_geometric_summable.mul_left C
    have htemp2 := htemp.mul_right (1 / ((t : ℝ) + 1))
    simpa [mul_assoc, mul_comm, mul_left_comm] using htemp2
  exact Summable.of_nonneg_of_le h_nonneg h_bound h_geometric_summable_mul

theorem mc_D_divmod (t : ℕ) (D : ℕ → ℝ) (hnn : ∀ s, 0 ≤ D s)
    (hsub : ∀ r s, Real.sqrt (D (r + s)) ≤ Real.sqrt (D r) + Real.sqrt (D s)) (s : ℕ) :
    D s ≤ 2 * (((s / t : ℕ) : ℝ)) ^ 2 * D t + 2 * D (s % t) := by
  rcases Nat.eq_zero_or_pos t with (htz | htp)
  · subst htz
    simp
    have hDs : 0 ≤ D s := hnn s
    nlinarith
  · have h_sq_subadd : ∀ r s : ℕ, D (r + s) ≤ 2 * D r + 2 * D s := by
      intro r s
      have h_sqrt := hsub r s
      have h_nonneg_r : 0 ≤ Real.sqrt (D r) := Real.sqrt_nonneg _
      have h_nonneg_s : 0 ≤ Real.sqrt (D s) := Real.sqrt_nonneg _
      have h_nonneg_Drs : 0 ≤ D (r + s) := hnn (r + s)
      have h_sq_sqrt : (Real.sqrt (D (r + s))) ^ 2 = D (r + s) := Real.sq_sqrt (hnn (r + s))
      have h_nonneg_sqrt_sum : 0 ≤ Real.sqrt (D r) + Real.sqrt (D s) :=
        add_nonneg h_nonneg_r h_nonneg_s
      have hsq : (Real.sqrt (D (r + s))) ^ 2 ≤ (Real.sqrt (D r) + Real.sqrt (D s)) ^ 2 := by
        have : Real.sqrt (D (r + s)) ≤ Real.sqrt (D r) + Real.sqrt (D s) := h_sqrt
        have hpos1 : 0 ≤ Real.sqrt (D (r + s)) := Real.sqrt_nonneg _
        have hpos2 : 0 ≤ Real.sqrt (D r) + Real.sqrt (D s) := h_nonneg_sqrt_sum
        nlinarith
      rw [h_sq_sqrt] at hsq
      have h_expand : (Real.sqrt (D r) + Real.sqrt (D s)) ^ 2 =
          D r + D s + 2 * Real.sqrt (D r) * Real.sqrt (D s) := by
        ring
        simp [Real.sq_sqrt (hnn r), Real.sq_sqrt (hnn s)]
      have h_amgm : 2 * Real.sqrt (D r) * Real.sqrt (D s) ≤ D r + D s := by
        have h_sq_nonneg : (Real.sqrt (D r) - Real.sqrt (D s)) ^ 2 ≥ 0 := by positivity
        have h_expand' : (Real.sqrt (D r) - Real.sqrt (D s)) ^ 2 =
            D r + D s - 2 * Real.sqrt (D r) * Real.sqrt (D s) := by
          ring
          simp [Real.sq_sqrt (hnn r), Real.sq_sqrt (hnn s)]
        nlinarith
      nlinarith
    set q := s / t with hq_def
    set r' := s % t with hr'_def
    have h_divmod : s = q * t + r' := by
      rw [hq_def, hr'_def]
      rw [mul_comm]
      exact (Nat.div_add_mod s t).symm
    by_cases hqz : q = 0
    ·
      have hs_eq_r' : s = r' := by
        rw [hqz] at h_divmod
        simpa using h_divmod
      rw [hs_eq_r']

      have hq0 : (q : ℝ) = 0 := by exact_mod_cast hqz
      simp [hq0]
      have hDr' : 0 ≤ D r' := hnn r'
      nlinarith
    ·
      have hq_pos : 1 ≤ q := Nat.one_le_of_lt (Nat.pos_of_ne_zero hqz)
```

```
      have h_D_qt : D (q * t) ≤ ((q : ℕ) : ℝ) ^ 2 * D t := by
        have h_sqrt_qt : Real.sqrt (D (q * t)) ≤ (q : ℝ) * Real.sqrt (D t) := by
          refine Nat.le_induction ?_ (fun k hk ih => ?_) q hq_pos
          .
            simp
          .
            have h_eq : (k.succ : ℕ) * t = k * t + t := by
              rw [Nat.succ_mul]
            rw [h_eq]
            calc
              Real.sqrt (D (k * t + t)) ≤ Real.sqrt (D (k * t)) + Real.sqrt (D t) := hsub (k * t) t
              _ ≤ (k : ℝ) * Real.sqrt (D t) + Real.sqrt (D t) := by nlinarith
              _ = ((k.succ : ℕ) : ℝ) * Real.sqrt (D t) := by push_cast; ring
        have h_sq : D (q * t) ≤ ((q : ℝ) * Real.sqrt (D t)) ^ 2 := by
          have h_sq_sqrt : (Real.sqrt (D (q * t))) ^ 2 = D (q * t) := Real.sq_sqrt (hnn (q * t))
          have h_nonneg_sqrt : 0 ≤ Real.sqrt (D (q * t)) := Real.sqrt_nonneg _
          have hsq' : (Real.sqrt (D (q * t))) ^ 2 ≤ ((q : ℝ) * Real.sqrt (D t)) ^ 2 := by
            have : Real.sqrt (D (q * t)) ≤ (q : ℝ) * Real.sqrt (D t) := h_sqrt_qt
            have : 0 ≤ Real.sqrt (D (q * t)) := h_nonneg_sqrt
            have : 0 ≤ (q : ℝ) * Real.sqrt (D t) :=
              mul_nonneg (by exact_mod_cast Nat.zero_le _) (Real.sqrt_nonneg _)
            nlinarith
          rw [h_sq_sqrt] at hsq'
          exact hsq'
        calc
          D (q * t) ≤ ((q : ℝ) * Real.sqrt (D t)) ^ 2 := h_sq
          _ = ((q : ℝ) ^ 2) * ((Real.sqrt (D t)) ^ 2) := by ring
          _ = ((q : ℝ) ^ 2) * D t := by simp [Real.sq_sqrt (hnn t)]
          _ = ((q : ℕ) : ℝ) ^ 2 * D t := by simp
      rw [h_divmod]
      calc
        D (q * t + r') ≤ 2 * D (q * t) + 2 * D r' := h_sq_subadd (q * t) r'
        _ ≤ 2 * (((q : ℕ) : ℝ) ^ 2 * D t) + 2 * D r' := by nlinarith
        _ = 2 * (((q : ℕ) : ℝ) ^ 2) * D t + 2 * D r' := by ring
        _ = 2 * (((s / t : ℕ) : ℝ)) ^ 2 * D t + 2 * D (s % t) := by simp [hq_def, hr'_def]

theorem mc_geom_cesaro_pre (t : ℕ) (ht : 1 ≤ t) (D : ℕ → ℝ)
    (hnn : ∀ s, 0 ≤ D s) (hbdd : ∃ C, ∀ s, D s ≤ C)
    (hsub : ∀ r s, Real.sqrt (D (r + s)) ≤ Real.sqrt (D r) + Real.sqrt (D s)) :
    ∑' s : ℕ, (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * D s
      ≤ 2 * D t * ∑' s : ℕ, (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * (((s / t : ℕ) : ℝ)) ^ 2
        + 2 * ∑' s : ℕ, (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * D (s % t) := by
  set p := 1 / ((t : ℝ) + 1) with hp_def
  set q := (t : ℝ) / ((t : ℝ) + 1) with hq_def
  have ht_nonneg' : 0 ≤ (t : ℝ) := Nat.cast_nonneg _
  have hp_nonneg : 0 ≤ p := by
    rw [hp_def]
    exact div_nonneg (by norm_num) (by nlinarith)
  have hq_nonneg : 0 ≤ q := by
    rw [hq_def]
    exact div_nonneg ht_nonneg' (by nlinarith)
  have hq_lt_one : q < 1 := by
    rw [hq_def]
    apply (div_lt_one (by nlinarith)).mpr
    linarith
  have hq_norm_lt_one : ‖q‖ < 1 := by
    rwa [Real.norm_of_nonneg hq_nonneg]
  have hq_summable : Summable (fun s : ℕ => q ^ s) :=
    summable_geometric_of_lt_one hq_nonneg hq_lt_one
```

```
have hq_pow_summable (k : ℕ) : Summable (fun s : ℕ => (s : ℝ) ^ k * q ^ s) := by
  apply summable_pow_mul_geometric_of_norm_lt_one k hq_norm_lt_one
have h_divmod (s : ℕ) : D s ≤ 2 * (((s / t : ℕ) : ℝ)) ^ 2 * D t + 2 * D (s % t) :=
  mc_D_divmod t D hnn hsub s

have h_div_sq_le_sq (s : ℕ) : ((s / t : ℕ) : ℝ) ^ 2 ≤ (s : ℝ) ^ 2 := by
  have hdiv : (s / t : ℕ) ≤ s := Nat.div_le_self s t
  have hcast : ((s / t : ℕ) : ℝ) ≤ (s : ℝ) := Nat.cast_le.mpr hdiv
  nlinarith

have h_summable_div_sq_mul_q : Summable (fun s : ℕ => ((s / t : ℕ) : ℝ) ^ 2 * q ^ s) := by
  have h_summable_sq_mul_q : Summable (fun s : ℕ => (s : ℝ) ^ 2 * q ^ s) := hq_pow_summable 2
  refine h_summable_sq_mul_q.of_nonneg_of_le
    (fun s => by
      have hsq : 0 ≤ ((s / t : ℕ) : ℝ) ^ 2 := by positivity
      have hqpow : 0 ≤ q ^ s := pow_nonneg hq_nonneg s
      positivity)
    (fun s => mul_le_mul_of_nonneg_right (h_div_sq_le_sq s) (pow_nonneg hq_nonneg s))

have h_summable_sq_term : Summable (fun s : ℕ => p * q ^ s * (((s / t : ℕ) : ℝ)) ^ 2) := by
  simpa [mul_comm, mul_left_comm, mul_assoc] using h_summable_div_sq_mul_q.mul_left p
have h_summable_mod_term : Summable (fun s : ℕ => p * q ^ s * D (s % t)) := by
  rcases hbdd with ⟨C, hC⟩
  have h_nonneg (s : ℕ) : 0 ≤ p * q ^ s * D (s % t) := by
    have hq_pow : 0 ≤ q ^ s := pow_nonneg hq_nonneg s
    have hDmod_nonneg : 0 ≤ D (s % t) := hnn _
    apply mul_nonneg
    · apply mul_nonneg hp_nonneg hq_pow
    · exact hDmod_nonneg
  have h_bound (s : ℕ) : p * q ^ s * D (s % t) ≤ p * q ^ s * C := by
    have hD : D (s % t) ≤ C := hC (s % t)
    have hq_pow : 0 ≤ q ^ s := pow_nonneg hq_nonneg s
    have h_nonneg_prod : 0 ≤ p * q ^ s := mul_nonneg hp_nonneg hq_pow
    nlinarith
  have h_summable_C : Summable (fun s : ℕ => p * q ^ s * C) := by
    have := hq_summable.mul_left p |>.mul_right C
    simpa [mul_comm, mul_left_comm, mul_assoc] using this
  exact h_summable_C.of_nonneg_of_le h_nonneg h_bound
have h_summable_rhs_first : Summable (fun s : ℕ => 2 * D t * (p * q ^ s * (((s / t : ℕ) : ℝ)) ^ 2)) := by
  simpa [mul_comm, mul_left_comm, mul_assoc] using h_summable_sq_term.mul_left (2 * D t)
have h_summable_rhs_second : Summable (fun s : ℕ => 2 * (p * q ^ s * D (s % t))) := by
  simpa [mul_comm, mul_left_comm, mul_assoc] using h_summable_mod_term.mul_left 2
have h_summable_rhs : Summable (fun s : ℕ =>
    2 * D t * (p * q ^ s * (((s / t : ℕ) : ℝ)) ^ 2) + 2 * (p * q ^ s * D (s % t))) :=
  h_summable_rhs_first.add h_summable_rhs_second

have h_summable_lhs : Summable (fun s : ℕ => p * q ^ s * D s) := by
  rcases hbdd with ⟨C, hC⟩
  have h_nonneg (s : ℕ) : 0 ≤ p * q ^ s * D s := by
    have hq_pow : 0 ≤ q ^ s := pow_nonneg hq_nonneg s
    have hD_nonneg : 0 ≤ D s := hnn s
    apply mul_nonneg
    · apply mul_nonneg hp_nonneg hq_pow
    · exact hD_nonneg
  have h_bound (s : ℕ) : p * q ^ s * D s ≤ p * q ^ s * C := by
    have hD : D s ≤ C := hC s
    have h_nonneg_prod : 0 ≤ p * q ^ s := mul_nonneg hp_nonneg (pow_nonneg hq_nonneg s)
    nlinarith
  have h_summable_C : Summable (fun s : ℕ => p * q ^ s * C) := by
```

```
      have := hq_summable.mul_left p |>.mul_right C
      simpa [mul_comm, mul_left_comm, mul_assoc] using this
    exact h_summable_C.of_nonneg_of_le h_nonneg h_bound

  have h_term_bound (s : ℕ) : p * q ^ s * D s ≤
      p * q ^ s * (2 * (((s / t : ℕ) : ℝ)) ^ 2 * D t + 2 * D (s % t)) := by
    have hpos : 0 ≤ p * q ^ s := mul_nonneg hp_nonneg (pow_nonneg hq_nonneg s)
    have hDmod_nonneg : 0 ≤ 2 * (((s / t : ℕ) : ℝ)) ^ 2 * D t + 2 * D (s % t) := by
      have hsq : 0 ≤ ((s / t : ℕ) : ℝ) ^ 2 := by positivity
      have hD_nonneg : 0 ≤ D t := hnn t
      have hDmod_nonneg' : 0 ≤ D (s % t) := hnn _
      nlinarith
    have hdivmod := h_divmod s
    nlinarith

  have h_summable_rhs' : Summable (fun s : ℕ => p * q ^ s * (2 * (((s / t : ℕ) : ℝ)) ^ 2 * D t + 2 * D (s %
↪  t))) := by
    simpa [mul_add, add_mul, mul_comm, mul_left_comm, mul_assoc] using h_summable_rhs
  calc
    ∑' s : ℕ, p * q ^ s * D s
    _ ≤ ∑' s : ℕ, p * q ^ s * (2 * (((s / t : ℕ) : ℝ)) ^ 2 * D t + 2 * D (s % t)) :=
      h_summable_lhs.tsum_le_tsum h_term_bound h_summable_rhs'
    _ = ∑' s : ℕ, (2 * D t * (p * q ^ s * (((s / t : ℕ) : ℝ)) ^ 2) + 2 * (p * q ^ s * D (s % t))) := by
      refine tsum_congr (fun s => ?_)
      ring
    _ = ∑' s : ℕ, 2 * D t * (p * q ^ s * (((s / t : ℕ) : ℝ)) ^ 2)
        + ∑' s : ℕ, 2 * (p * q ^ s * D (s % t)) :=
      h_summable_rhs_first.tsum_add h_summable_rhs_second
    _ = 2 * D t * (∑' s : ℕ, p * q ^ s * (((s / t : ℕ) : ℝ)) ^ 2)
        + 2 * (∑' s : ℕ, p * q ^ s * D (s % t)) := by
      simp [tsum_mul_left]
    _ = 2 * D t * ∑' s : ℕ, (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * (((s / t : ℕ) : ℝ)) ^ 2
        + 2 * ∑' s : ℕ, (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * D (s % t) := by
      simp [hp_def, hq_def]

theorem mc_range_le_Icc (t : ℕ) (D : ℕ → ℝ) (hD0 : D 0 = 0) (hnn : ∀ s, 0 ≤ D s) :
    ∑ r ∈ Finset.range t, D r ≤ ∑ s ∈ Finset.Icc 1 t, D s := by
  by_cases ht : t = 0
  · subst ht; simp
  · have htpos : 1 ≤ t := by omega
    have h0_mem_range : (0 : ℕ) ∈ Finset.range t := by
      simpa [htpos] using Finset.mem_range.2 (by omega)
    have ht_mem_Icc : t ∈ Finset.Icc (1 : ℕ) t := by
      simp [htpos]
    have h_erase_eq : (Finset.range t).erase 0 = (Finset.Icc (1 : ℕ) t).erase t := by
      ext x
      simp [Finset.mem_range]
      omega
    have hsum_range_erase : ∑ r ∈ Finset.range t, D r = D 0 + ∑ r ∈ (Finset.range t).erase 0, D r := by
      have := Finset.sum_erase_add (Finset.range t) D h0_mem_range

      linarith [this]
    have hsum_Icc_erase : D t + ∑ r ∈ (Finset.Icc (1 : ℕ) t).erase t, D r = ∑ s ∈ Finset.Icc (1 : ℕ) t,
↪  D s := by
      have := Finset.sum_erase_add (Finset.Icc (1 : ℕ) t) D ht_mem_Icc
      linarith [this]
    calc
      ∑ r ∈ Finset.range t, D r = D 0 + ∑ r ∈ (Finset.range t).erase 0, D r := hsum_range_erase
      _ = (0 : ℝ) + ∑ r ∈ (Finset.range t).erase 0, D r := by rw [hD0]
```

```
      _ = ∑ r ∈ (Finset.range t).erase 0, D r := by simp
      _ = ∑ r ∈ (Finset.Icc (1 : ℕ) t).erase t, D r := by rw [h_erase_eq]
      _ ≤ D t + ∑ r ∈ (Finset.Icc (1 : ℕ) t).erase t, D r := by
        nlinarith [hnn t]
      _ = ∑ s ∈ Finset.Icc (1 : ℕ) t, D s := by rw [hsum_Icc_erase]

theorem mc_geom_cesaro (t : ℕ) (ht : 1 ≤ t) (D : ℕ → ℝ) (hD0 : D 0 = 0)
    (hnn : ∀ s, 0 ≤ D s) (hbdd : ∃ C, ∀ s, D s ≤ C)
    (hsub : ∀ r s, Real.sqrt (D (r + s)) ≤ Real.sqrt (D r) + Real.sqrt (D s)) :
    ∑' s : ℕ, (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * D s
      ≤ 28 * ((1 / (t : ℝ)) * ∑ s ∈ Finset.Icc 1 t, D s) := by
  have hpre := mc_geom_cesaro_pre t ht D hnn hbdd hsub
  obtain ⟨_, hS2⟩ := mc_geom_sq_div t ht
  obtain ⟨_, hmod⟩ := mc_geom_mod t ht D hnn
  have hDt := mc_cesaro_Dt t ht D hD0 hnn hsub
  have hrange := mc_range_le_Icc t D hD0 hnn
  have htpos : (0:ℝ) < t := by exact_mod_cast ht
  have hDt0 := hnn t
  have h1 : (2 / (t:ℝ)) * ∑ r ∈ Finset.range t, D r
      ≤ 2 * ((1 / (t:ℝ)) * ∑ s ∈ Finset.Icc 1 t, D s) := by
    rw [show (2 / (t:ℝ)) * ∑ r ∈ Finset.range t, D r
        = 2 * ((1 / (t:ℝ)) * ∑ r ∈ Finset.range t, D r) by ring]
    exact mul_le_mul_of_nonneg_left
      (mul_le_mul_of_nonneg_left hrange (by positivity)) (by norm_num)
  have h2 : 2 * D t * ∑' s : ℕ, (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * (((s / t : ℕ) : ℝ))
↪  ^ 2
      ≤ 2 * D t * 3 :=
    mul_le_mul_of_nonneg_left hS2 (by linarith)
  linarith [hpre, h2, hmod, h1, hDt]

theorem mc_resolvent_D_eq {n : ℕ} (t : ℕ) (π : Fin n → ℝ)
    (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochastic A) (x : Fin n → L1)
    (G : Matrix (Fin n) (Fin n) ℝ)
    (hG : ∀ i j, G i j = ∑' s : ℕ, (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * (A ^ s) i j)
    (D : ℕ → ℝ) (hD : ∀ s, D s = ∑ i, ∑ j, π i * (A ^ s) i j * ‖x i - x j‖ ^ 2) :
    ∑ i, ∑ j, π i * G i j * ‖x i - x j‖ ^ 2
      = ∑' s : ℕ, (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * D s := by
  classical
  have hsum := mc_resolvent_summable t A hA
  have h1 : ∀ i j, HasSum (fun s : ℕ => π i * ((1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * (A ^
↪  s) i j) * ‖x i - x j‖ ^ 2)
      (π i * G i j * ‖x i - x j‖ ^ 2) := by
    intro i j
    rw [hG i j]
    exact ((hsum i j).hasSum.mul_left (π i)).mul_right _
  have h2 := hasSum_sum (s := Finset.univ) (fun i _ => hasSum_sum (s := Finset.univ) (fun j _ => h1 i j))
  have key : HasSum (fun s : ℕ => (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * D s)
      (∑ i, ∑ j, π i * G i j * ‖x i - x j‖ ^ 2) := by
    convert h2 using 1
    funext s
    rw [hD s, Finset.mul_sum]
    refine Finset.sum_congr rfl (fun i _ => ?_)
    rw [Finset.mul_sum]
    refine Finset.sum_congr rfl (fun j _ => ?_)
    ring
  exact key.tsum_eq.symm

open MarkovCotype in
lemma pow_nonneg_entry {n : ℕ} (A : Matrix (Fin n) (Fin n) ℝ) (hA : ∀ i j, 0 ≤ A i j) :
```

```
    ∀ s i j, 0 ≤ (A ^ s) i j := by
  intro s i j
  induction s generalizing i j with
  | zero =>
    simp [Matrix.one_apply]
    split <;> positivity
  | succ k ih =>
    rw [pow_succ', Matrix.mul_apply]
    refine Finset.sum_nonneg (fun k' _ => ?_)
    have h1 : 0 ≤ A i k' := hA i k'
    have h2 : 0 ≤ (A ^ k) k' j := ih k' j
    exact mul_nonneg h1 h2

open MarkovCotype in
lemma pow_stochastic_row_sum {n : ℕ} (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochastic A) (s : ℕ) (i : Fin
↪  n) :
    ∑ j, (A ^ s) i j = 1 := by
  have hA_row_sum : ∀ i, ∑ j, A i j = 1 := hA.2
  induction s with
  | zero => simp [Matrix.one_apply, Finset.sum_ite_eq, Finset.mem_univ]
  | succ k ih =>
    rw [pow_succ]
    calc
      ∑ j, (A ^ k * A) i j = ∑ j, ∑ l, (A ^ k) i l * A l j := by
        simp [Matrix.mul_apply]
      _ = ∑ l, ∑ j, (A ^ k) i l * A l j := by rw [Finset.sum_comm]
      _ = ∑ l, (A ^ k) i l * (∑ j, A l j) := by simp [Finset.mul_sum]
      _ = ∑ l, (A ^ k) i l * 1 := by simp [hA_row_sum]
      _ = ∑ l, (A ^ k) i l := by simp
      _ = 1 := ih

open MarkovCotype in
lemma pow_stochastic_entry_le_one {n : ℕ} (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochastic A)
    (s : ℕ) (i j : Fin n) : (A ^ s) i j ≤ 1 := by
  have h_entry_nonneg : ∀ i j, 0 ≤ (A ^ s) i j := fun i j =>
    pow_nonneg_entry A hA.1 s i j
  have h_row_sum := pow_stochastic_row_sum A hA s i
  have h_single_le_sum : (A ^ s) i j ≤ ∑ j', (A ^ s) i j' :=
    Finset.single_le_sum (fun k _ => h_entry_nonneg i k) (Finset.mem_univ j)
  rw [h_row_sum] at h_single_le_sum
  exact h_single_le_sum

open MarkovCotype in
lemma pow_stochastic {n : ℕ} (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochastic A) (s : ℕ) :
    IsStochastic (A ^ s) := by
  have hA_nonneg : ∀ i j, 0 ≤ A i j := hA.1
  have h_entry_nonneg : ∀ i j, 0 ≤ (A ^ s) i j := fun i j =>
    pow_nonneg_entry A hA_nonneg s i j
  have h_row_sum : ∀ i, ∑ j, (A ^ s) i j = 1 :=
    pow_stochastic_row_sum A hA s
  exact ⟨h_entry_nonneg, h_row_sum⟩

theorem mc_D_props {n : ℕ} (π : Fin n → ℝ) (hπ : IsProbVec π)
    (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochastic A) (x : Fin n → L1) (D : ℕ → ℝ)
    (hD : ∀ s, D s = ∑ i, ∑ j, π i * (A ^ s) i j * ‖x i - x j‖ ^ 2) :
    D 0 = 0 ∧ (∀ s, 0 ≤ D s) ∧ ∃ C, ∀ s, D s ≤ C := by
  have hπ_nonneg : ∀ i, 0 ≤ π i := hπ.1
  have hπ_sum : ∑ i, π i = 1 := hπ.2
  have hA_nonneg : ∀ i j, 0 ≤ A i j := hA.1
```

```
  have hpow0 : (A ^ 0 : Matrix (Fin n) (Fin n) ℝ) = 1 := by
    simpa using pow_zero A
  have hD0 : D 0 = 0 := by
    rw [hD 0, hpow0]
    simp [Matrix.one_apply]
  have hnonneg : ∀ s, 0 ≤ D s := by
    intro s
    rw [hD s]
    refine Finset.sum_nonneg (fun i _ => Finset.sum_nonneg (fun j _ => ?_))
    have hA_nonneg_s : 0 ≤ (A ^ s) i j := pow_nonneg_entry A hA_nonneg s i j
    have h_norm_sq_nonneg : 0 ≤ ‖x i - x j‖ ^ 2 := pow_two_nonneg _
    apply mul_nonneg
    · apply mul_nonneg (hπ_nonneg i) hA_nonneg_s
    · exact h_norm_sq_nonneg
  have hbdd : ∃ C, ∀ s, D s ≤ C := by
    by_cases h_ne : Nonempty (Fin n)
    · obtain ⟨i₀⟩ := h_ne
      let B : ℝ := ∑ i, ∑ j, ‖x i - x j‖ ^ 2
      have hB : 0 ≤ B := Finset.sum_nonneg (fun i _ => Finset.sum_nonneg (fun j _ => by positivity))
      use B
      intro s
      rw [hD s]
      calc
        ∑ i, ∑ j, π i * (A ^ s) i j * ‖x i - x j‖ ^ 2
            ≤ ∑ i, ∑ j, π i * 1 * ‖x i - x j‖ ^ 2 := by
          refine Finset.sum_le_sum (fun i _ => ?_)
          refine Finset.sum_le_sum (fun j _ => ?_)
          have h_entry_le_one : (A ^ s) i j ≤ 1 :=
            pow_stochastic_entry_le_one A hA s i j
          have h_entry_nonneg : 0 ≤ (A ^ s) i j := pow_nonneg_entry A hA_nonneg s i j
          have hπ_nonneg_i : 0 ≤ π i := hπ_nonneg i
          have h_norm_sq_nonneg : 0 ≤ ‖x i - x j‖ ^ 2 := by positivity
          gcongr
        _ = ∑ i, ∑ j, π i * ‖x i - x j‖ ^ 2 := by simp
        _ ≤ ∑ i, ∑ j, 1 * ‖x i - x j‖ ^ 2 := by
          refine Finset.sum_le_sum (fun i _ => Finset.sum_le_sum (fun j _ => ?_))
          have hπi : π i ≤ 1 := by
            have : π i ≤ ∑ k, π k := Finset.single_le_sum (fun k _ => hπ_nonneg k) (Finset.mem_univ i)
            rw [hπ_sum] at this
            exact this
          gcongr
        _ = ∑ i, ∑ j, ‖x i - x j‖ ^ 2 := by simp
        _ = B := rfl
    ·
      have h_empty : IsEmpty (Fin n) := not_nonempty_iff.mp h_ne
      use 0
      intro s
      rw [hD s]
      simp
  exact ⟨hD0, hnonneg, hbdd⟩

theorem mc_cesaro_rhs_eq {n : ℕ} (t : ℕ) (π : Fin n → ℝ)
    (A : Matrix (Fin n) (Fin n) ℝ) (x : Fin n → L1) (D : ℕ → ℝ)
    (hD : ∀ s, D s = ∑ i, ∑ j, π i * (A ^ s) i j * ‖x i - x j‖ ^ 2) :
    ∑ i, ∑ j, π i * ((1 / (t : ℝ)) * ∑ s ∈ Finset.Icc 1 t, (A ^ s) i j) * ‖x i - x j‖ ^ 2
      = (1 / (t : ℝ)) * ∑ s ∈ Finset.Icc 1 t, D s := by
  have hsum : (∑ i, ∑ j, ∑ s ∈ Finset.Icc 1 t, π i * (A ^ s) i j * ‖x i - x j‖ ^ 2)
      = (∑ s ∈ Finset.Icc 1 t, ∑ i, ∑ j, π i * (A ^ s) i j * ‖x i - x j‖ ^ 2) := by
```

```
    calc
      (∑ i, ∑ j, ∑ s ∈ Finset.Icc 1 t, π i * (A ^ s) i j * ‖x i - x j‖ ^ 2)
          = (∑ i, ∑ s ∈ Finset.Icc 1 t, ∑ j, π i * (A ^ s) i j * ‖x i - x j‖ ^ 2) := by
        refine Finset.sum_congr rfl fun i _ => ?_
        rw [Finset.sum_comm]
      _ = (∑ s ∈ Finset.Icc 1 t, ∑ i, ∑ j, π i * (A ^ s) i j * ‖x i - x j‖ ^ 2) := by
        rw [Finset.sum_comm]
  calc
    ∑ i, ∑ j, π i * ((1 / (t : ℝ)) * ∑ s ∈ Finset.Icc 1 t, (A ^ s) i j) * ‖x i - x j‖ ^ 2
        = (1 / (t : ℝ)) * ∑ i, ∑ j, ∑ s ∈ Finset.Icc 1 t, π i * (A ^ s) i j * ‖x i - x j‖ ^ 2 := by
      simp only [Finset.mul_sum, mul_assoc, mul_comm, mul_left_comm]
    _ = (1 / (t : ℝ)) * (∑ i, ∑ j, ∑ s ∈ Finset.Icc 1 t, π i * (A ^ s) i j * ‖x i - x j‖ ^ 2) := rfl
    _ = (1 / (t : ℝ)) * (∑ s ∈ Finset.Icc 1 t, ∑ i, ∑ j, π i * (A ^ s) i j * ‖x i - x j‖ ^ 2) := by
      rw [hsum]
    _ = (1 / (t : ℝ)) * ∑ s ∈ Finset.Icc 1 t, D s := by
      simp only [hD]

set_option maxHeartbeats 1000000 in
theorem mc_comparison_aux {n : ℕ} (t : ℕ) (ht : 1 ≤ t) (π : Fin n → ℝ) (hπ : IsProbVec π)
    (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochastic A)
    (hstat : ∀ j, ∑ i, π i * A i j = π j) (x : Fin n → L1)
    (G : Matrix (Fin n) (Fin n) ℝ)
    (hG : ∀ i j, G i j = ∑' s : ℕ, (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * (A ^ s) i j)
    (D : ℕ → ℝ) (hD : ∀ s : ℕ, D s = ∑ i, ∑ j, π i * (A ^ s) i j * ‖x i - x j‖ ^ 2) :
    ∑ i, ∑ j, π i * G i j * ‖x i - x j‖ ^ 2
      ≤ 28 * ∑ i, ∑ j,
          π i * ((1 / (t : ℝ)) * ∑ s ∈ Finset.Icc 1 t, (A ^ s) i j) * ‖x i - x j‖ ^ 2 := by
  have h1 := mc_resolvent_D_eq t π A hA x G hG D hD
  have h2 := mc_cesaro_rhs_eq t π A x D hD
  have hprops := mc_D_props π hπ A hA x D hD
  have hsub : ∀ r s : ℕ, Real.sqrt (D (r + s)) ≤ Real.sqrt (D r) + Real.sqrt (D s) :=
    fun r s => mc_D_subadd π hπ A hA hstat x D hD r s
  have hgc := mc_geom_cesaro t ht D hprops.1 hprops.2.1 hprops.2.2 hsub
  first
    | (rw [h1, h2]; exact hgc)
    | linarith [h1, h2, hgc]

theorem mc_comparison {n : ℕ} (t : ℕ) (ht : 1 ≤ t) (π : Fin n → ℝ) (hπ : IsProbVec π)
    (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochastic A)
    (hstat : ∀ j, ∑ i, π i * A i j = π j) (x : Fin n → L1)
    (G : Matrix (Fin n) (Fin n) ℝ)
    (hG : ∀ i j, G i j = ∑' s : ℕ, (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * (A ^ s) i j) :
    ∑ i, ∑ j, π i * G i j * ‖x i - x j‖ ^ 2
      ≤ 28 * ∑ i, ∑ j,
          π i * ((1 / (t : ℝ)) * ∑ s ∈ Finset.Icc 1 t, (A ^ s) i j) * ‖x i - x j‖ ^ 2 := by
  set_option maxHeartbeats 1000000 in
  exact mc_comparison_aux t ht π hπ A hA hstat x G hG _ (fun s => rfl)

theorem solution
    (n t : ℕ) (ht : 1 ≤ t)
    (π : Fin n → ℝ) (hπ : IsProbVec π)
    (A : Matrix (Fin n) (Fin n) ℝ) (hA : IsStochastic A)
    (hrev : IsReversible π A)
    (x : Fin n → L1) :
    ∃ y : Fin n → L1,
      ∑ i, π i * ‖x i - y i‖ ^ 2
        + (t : ℝ) * ∑ i, ∑ j, π i * A i j * ‖y i - y j‖ ^ 2
      ≤ 3024 * ∑ i, ∑ j,
          π i * ((1 / (t : ℝ)) * ∑ s ∈ Finset.Icc 1 t, (A ^ s) i j)
```

```
          * ‖x i - x j‖ ^ 2 := by
  have hstat := mc_stationary π A hA hrev
  obtain ⟨w, T, hw, hT, hdist, hcontr⟩ := mc_cut_L1 x
  obtain ⟨G, hG⟩ : ∃ G : Matrix (Fin n) (Fin n) ℝ, ∀ i j, G i j = ∑' s : ℕ, (1 / ((t : ℝ) + 1)) * ((t :
↪  ℝ) / ((t : ℝ) + 1)) ^ s * (A ^ s) i j :=
    ⟨fun i j => ∑' s : ℕ, (1 / ((t : ℝ) + 1)) * ((t : ℝ) / ((t : ℝ) + 1)) ^ s * (A ^ s) i j, fun _ _ => rfl⟩
  obtain ⟨hG0, hGrow, hGπ⟩ := mc_resolvent_basic t π A hA hstat G hG
  obtain ⟨hAG, hGA⟩ := mc_resolvent_first_step t A hA G hG
  obtain ⟨φ, hφ⟩ : ∃ φ : ℝ → ℝ, ∀ r, φ r = 3 * r ^ 2 - 2 * r ^ 3 :=
    ⟨fun r => 3 * r ^ 2 - 2 * r ^ 3, fun _ => rfl⟩
  have hz : ∀ (i : Fin n) (B : Finset (Fin n)),
      (if i ∈ B then (1:ℝ) else 0) = 0 ∨ (if i ∈ B then (1:ℝ) else 0) = 1 := by
    intro i B
    by_cases h : i ∈ B <;> simp [h]
  have hcube := mc_cube_energy t π hπ A hA w hw (fun i B => if i ∈ B then (1:ℝ) else 0) hz φ hφ G hG0
↪  hGrow hAG hGA hGπ
  obtain ⟨y, hy⟩ := mc_transfer t π hπ.1 A hA.1 x w hw (fun i B => if i ∈ B then (1:ℝ) else 0) hz T hT
↪  hdist hcontr G
    (fun i b => φ (∑ j, G i j * (if j ∈ b then (1:ℝ) else 0))) hcube
  have hcomp := mc_comparison t ht π hπ A hA hstat x G hG
  exact ⟨y, by linarith⟩
```